



SAP Cloud Integration

Generated on: 2026-10-06 11:44:06 GMT+0000

SAP Cloud Integration | Cloud

Public

Original content: https://help.sap.com/docs/CLOUD_INTEGRATION/368c481cd6954bdfa5d0435479fd4eaf?locale=en-US&state=PRODUCTION&version=Cloud

Warning

This document has been generated from SAP Help Portal and is an incomplete version of the official SAP product documentation. The information included in custom documentation may not reflect the arrangement of topics in SAP Help Portal, and may be missing important aspects and/or correlations to other topics. For this reason, it is not for production use.

For more information, please visit <https://help.sap.com/docs/disclaimer>.

Getting Started with Integration Flow Development

Learn how to develop and run your first integration flow.

i Note

These exercises apply for both cases when you use SAP Cloud Integration in the Neo and in the Cloud Foundry environment.

Note that at certain steps there are specific things to consider depending on the environment. If this is the case, it's indicated in this documentation.

i Note

In addition to the exercises provided in this section, you can also check out the [Integration Flow Design Guidelines](#).

The integration flow design guidelines provide both documentation and predefined integration content. You can deploy and run the integration content out-of-the-box to learn basic concepts of Cloud Integration. You can also access the integration flows from SAP Business Accelerator Hub as described in [How to Work with the Example Integration Flows](#).

A key part of an SAP Cloud Integration project is to develop integration flows. An integration flow allows you to specify how SAP Cloud Integration is to process a message. SAP Cloud Integration provides a modeling environment that allows you to design the details of message processing (its senders and receivers as well as the individual processing steps) with a graphical user interface.

This section shows you step by step how to develop and run your first, simple integration flows. In other words, it gives you an introduction to the tasks of an integration developer. We show you the design of four integration flows, with increasing complexity.

i Note

The first three integration flows start with a timer and don't have a sender. This means that you can omit all tasks related to setting up a sender system to SAP Cloud Integration.

The fourth integration flow is initiated by a request from a sender system that is simulated by an HTTP client.

To complete the tasks, you use the SAP Cloud Integration Web UI.

Before designing any integration flow of this section, you need to create an integration package first and, within this integration package, create an integration flow. When you've created the integration flow, you add the steps as described for the specific integration flow exercise.

- The first exercise shows you how to perform a simple *smoke test* to check whether your tenant cluster is working correctly and that it processes messages in the expected way. A simple message is created with the text Hello World! in the message body. The integration flow has no receiver. To check if the message has been processed successfully, you can go to the monitoring application and check for the message content there.

More information: [Smoke Test Scenario](#)

- The second exercise shows you how to extend the *smoke test* scenario by adding an outbound call to an external data source. The integration flow requests data exposed by the external component through an OData application programming interface (API). The message body is created based on that data and, like in the first exercise, can be displayed by the monitoring application.

More information: [Smoke Test Scenario with External Data Source](#)

- The third exercise is a simple enhancement and modification of second integration flow. It has an e-mail receiver so that you receive the message (processed by SAP Cloud Integration) in an e-mail account of your choice.

More information: [Timer-Initiated Scenario with a Mail Receiver](#)

- The last exercise is a simple integration flow that also has a sender component. The sender in this example is simulated by an HTTP client installed on your computer.

More information: [Sender-Initiated Scenario \(with HTTPS Sender Adapter\)](#)

The exercises are designed so that you can do all four of them independently. All steps are described one-by-one. But you can also start with the first one and, successively, enhance it to derive the second and the third scenario out of the first one.

There are sample integration flows that show other simple modifications and additional features.

More information: [Further Sample Integration Flows](#)

i Note

Prerequisites:

- You have set up an e-mail account that you can use as the receiver system for the integration flow (only required for third exercise with the Mail adapter).

This is custom documentation. For more information, please visit [SAP Help Portal](#).

- You have opened the SAP Cloud Integration Web UI, which contains all the tools you need to set up, run, and monitor your integration scenario.

You can access the Web UI using the hyperlink that ends with `/itspaces`.

- You have been given access to an SAP Cloud Integration tenant and have the following permissions assigned to your user (in the Cloud Foundry environment):
 - Authorization group `PI_Integration_Developer` (to be authorized to perform integration developer tasks)
 - Authorization group `PI_Business_Expert` (to allow you to access message processing log attachments)
- You have been given access to an SAP Cloud Integration tenant and have the following permissions assigned to your user (in the Neo environment):
 - Role collection `AuthGroup.IntegrationDeveloper` (to be authorized to perform integration developer tasks)
 - Role collection `AuthGroup.BusinessExpert` (to allow you to access message processing log attachments)

Further Reading

For an introduction to the concept of integration flows, check out [Elements of an Integration Flow](#).

For an overview of the available connectivity options (adapter types), check out [Connectivity \(Adapters\)](#).

For an overview of the available message processing options (integration flow step types), check out [Integration Capabilities](#).

Related Information

[Overview of the SAP Cloud Integration Web UI](#)

[Creating an Integration Package](#)

[Creating an Integration Flow](#)

[Smoke Test Scenario](#)

[Smoke Test Scenario with External Data Source](#)

[Timer-Initiated Scenario with a Mail Receiver](#)

Overview of the SAP Cloud Integration Web UI

The SAP Cloud Integration Web UI is your one-stop shop for integration development.

Note that the URL to access the Web UI ends with `/itspaces`.

The Web UI comprises the following sections:

- Discover

Here, you can find predefined integration content provided by SAP that you can use out of the box and adapt to your requirements. As the Getting Started documentation focuses on how to design your own integration content, we do not go into any more detail on this section.

- Design

This is where you design your integration content. As you progress through the exercise in the Getting Started documentation, you will spend most of your time in this section. It contains the graphical integration flow modeling environment.

- Monitor

This is where you can monitor your integration flow. You also use this section to manage additional artifacts that you need to deploy on your tenant to complement your integration flows (for example, User Credential artifacts to configure connections using basic authentication).

Design Section

When you go to the **Design** section, you find a list of integration packages defined for the tenant.

When you select an integration package, you can find the integration flows (and other artifacts) defined for the package (on the **Artifacts** tab).

In this Getting Started documentation, we assume that you have not yet defined an integration package for your integration content. Therefore, the first step is to define an integration package.

Monitor Section

The **Monitor** section (also referred to as *Operations view*) has several subsections, each one containing several tiles. These subsections allow you to perform various tasks that are required for an integration project in addition to integration content design.

- [Monitor Message Processing](#)

When you select a tile in this section, you find all messages that have been processed by all integration flows deployed on the tenant. You can find out whether messages have been processed successfully for your integration flow and analyze the situation if not.

- [Manage Integration Content](#)

When you select a tile in this section, you find all deployed integration flows and can check whether deployment was successful. You can also find the endpoint address for your integration flow (which you need if you want to configure the endpoint in the connected sender system).

- [Manage Security](#)

- The [Security Material](#) tile contains security-related artifacts that are required on the tenant in addition to the security-relevant settings in your integration flow. In the course of the following exercise, you will create and deploy a [User Credentials](#) artifact to define the user name and password for the mail account that is addressed by the Mail receiver adapter of the integration flow.
- The [Keystore](#) tile shows the content of the tenant keystore, which contains key pairs and certificates required to set up connections that are protected using certificate-based authentication. In the exercise, we need to add certificates when setting up the connection to the e-mail receiver.
- The [Connectivity Tests](#) (only available in the Neo environment) tile allows you to test outbound connections (from SAP Cloud Integration to a receiver system). We use the connectivity test tool to retrieve the certificates of the e-mail receiver that need to be imported into the tenant keystore.

There are other sections and tiles that are required for additional tasks, but these are not required in the Getting Started exercise, so we will not look at them in any more detail here.

Related Information

[Developing Integration Content with SAP Cloud Integration](#)

[Monitor](#)

Create an Integration Package

Create an integration package that contains your integration flows.

An integration package is used like a folder for your integration content (integration flows, value mappings, and OData services). You can transport an integration package, for example, if you want to design your integration content on a test tenant first and then transport it to a production tenant.

1. Open the Web UI using the hyperlink provided to you in the mail from SAP (the links ends with `/itspaces`).
2. Go to the [Design](#) section of the Web UI.
3. Choose [Create](#).
4. Enter a name and description for your integration package and choose [Save](#).

[Design](#) / [New Package](#) / Save Cancel



My First Integration Package

[Header](#) [Overview](#) [Artifacts](#) [Documents](#) [Tags](#)

*Name:

*Technical Name: 

*Short Description:

My first integration package

Version:

Vendor:

Smoke Test Scenario

This is a very simple test to verify that your SAP Cloud Integration is working as expected. You do not need any receiver system to perform this test.

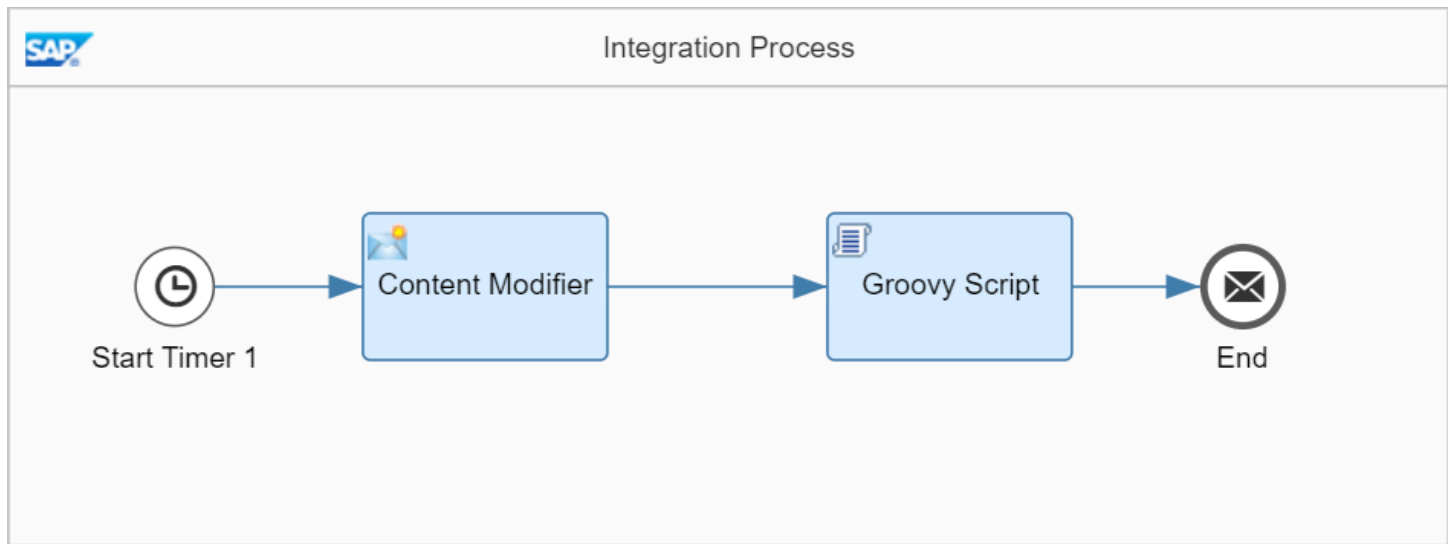
In this scenario, you create a *Hello World* text and write it into the message body (scheduled on deployment of the integration flow). The result is written into the message processing log which you can directly inspect with the message monitoring application.

Caution

This integration scenario is designed to show how to quickly (without much effort) set up and run an integration flow without the need to configure and connect to any receiver system. It uses a Script step to store the message payload in the message processing log (to enable you to easily check in the message monitoring application if the message was processed without any errors).

Note that this is not according to standard best practice. When designing productive scenarios, don't store the message payload in the message processing log. This can cause severe issues with memory consumption. The reason is that tasks such as message processing and message monitoring share the same memory and CPU which are available on your tenant.

In the course of this exercise, you develop the following integration flow.



To make it as easy as possible for you to develop this first integration flow, you don't need to configure any sender system. That saves the effort for you to set up a dedicated sender system and to connect it to SAP Cloud Integration. Instead of this, message processing is triggered by a Timer event, and the inbound message payload is created *within the integration flow*, in a dedicated Content Modifier step.

Furthermore, it is also not required that you set up any receiver system. To enable you to check if the message has been processed correctly, you will configure the integration so that the message payload is written into the message processing log (where you can easily inspect it using the **Monitor** application of the Web UI).

This is how the integration flow will process the message at runtime:

1. The Timer event triggers the processing of the message (according to the settings of the Timer's scheduler).
2. The Content Modifier step creates a message with a simple text content (*Hello World!*).
3. The Groovy Script step logs the payload of the message (that means, it writes the message content into the message processing log).

When you have finished the integration flow design, you save and deploy the integration flow.

You can then monitor message processing.

Related Information

[Create a Content Modifier to Define the Message Body](#)

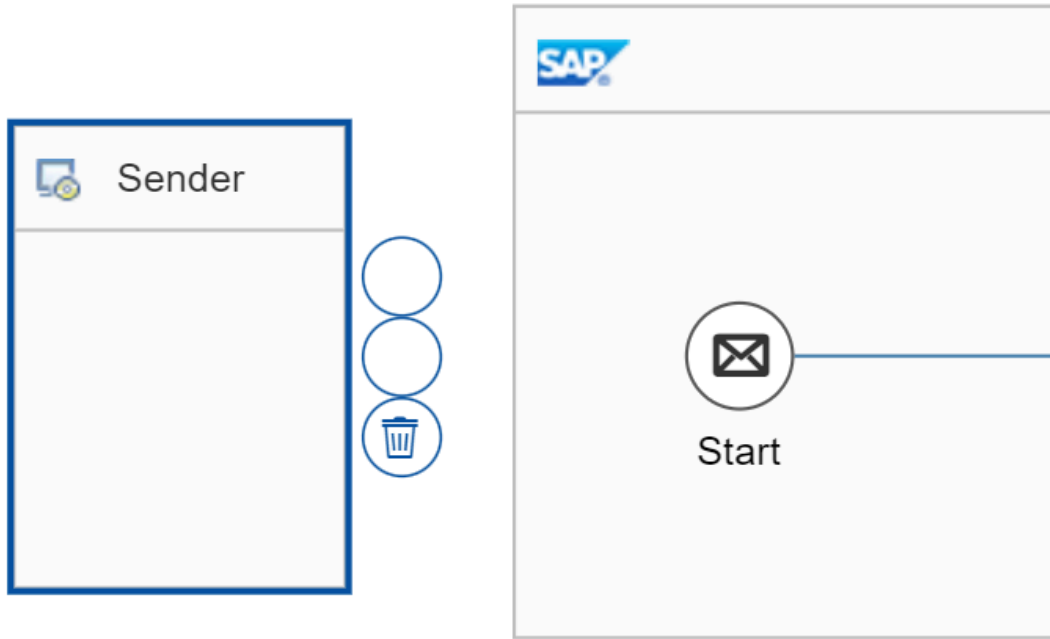
[Create a Script Step to Log the Payload](#)

[Save and Deploy the Integration Flow](#)

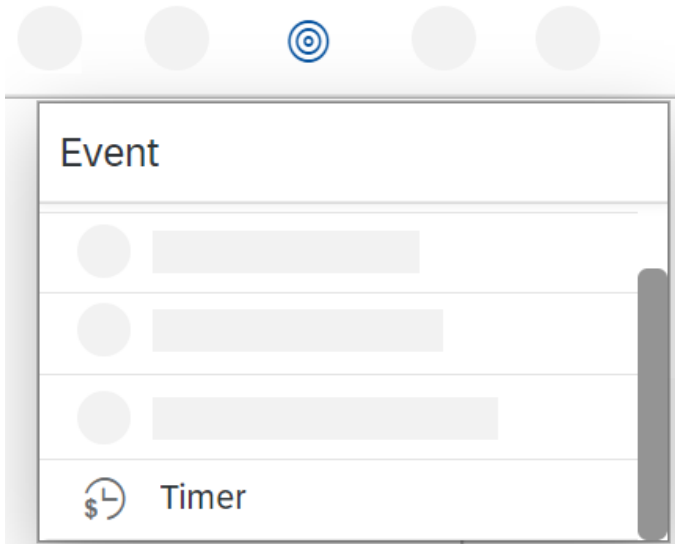
[Monitor Message Processing](#)

Add a Timer Start Event

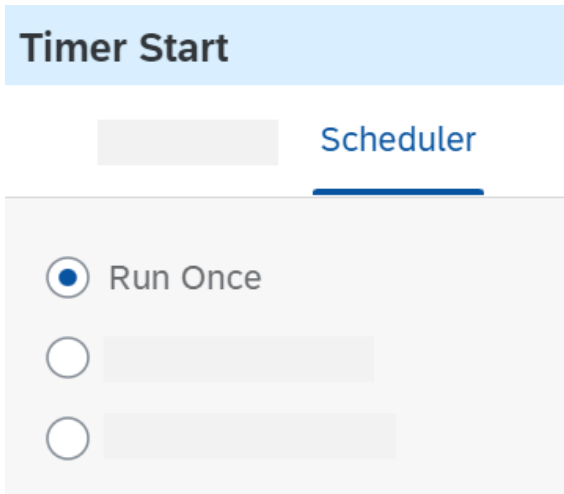
1. Open the integration flow model (Edit mode), select the Sender shape, and choose the recycle bin icon (to remove the Sender shape).



2. In the same way, remove the Start event.
3. In the palette, select the **Events** entry, and then select the Timer shape.



4. Place the Timer shape in the Integration Process shape (at the point where the Start event was previously located).
5. In the properties section of the Timer event (displayed below the integration flow model when the Timer event is selected in the model), go to the **Scheduler** tab.



6. Make sure that the option **Run Once** is selected.

i Note

You can, of course, try out the other settings, which enable the Timer to start message processing periodically. However, take care when selecting these options and you have added a receiver to your scenario. For example, in another demo scenario provided in this documentation, the receiver of the message is an e-mail account, and you don't want your e-mail account to be inundated with periodically generated e-mails.

Related Information

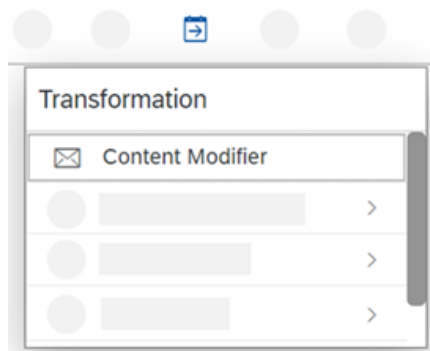
[Define a Timer Start Event](#)

Create a Content Modifier to Define the Message Body

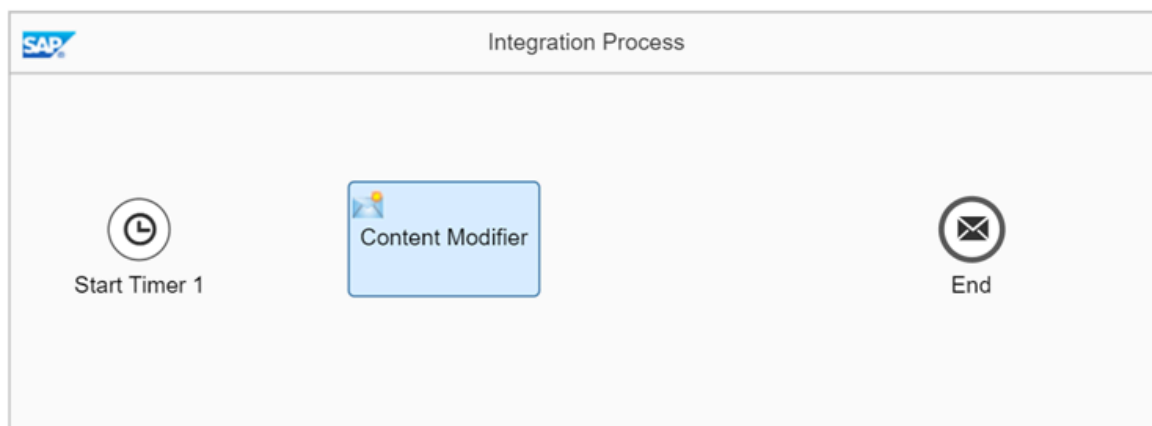
Add a Content Modifier step to create the message body.

As the integration flow has no sender, we use a Content Modifier to create a message from scratch.

1. To add a Content Modifier, go to the palette, choose the [Message Transformers](#) icon, and select the [Content Modifier](#) icon.



2. Place the Content Modifier in the model after the Timer Start event.



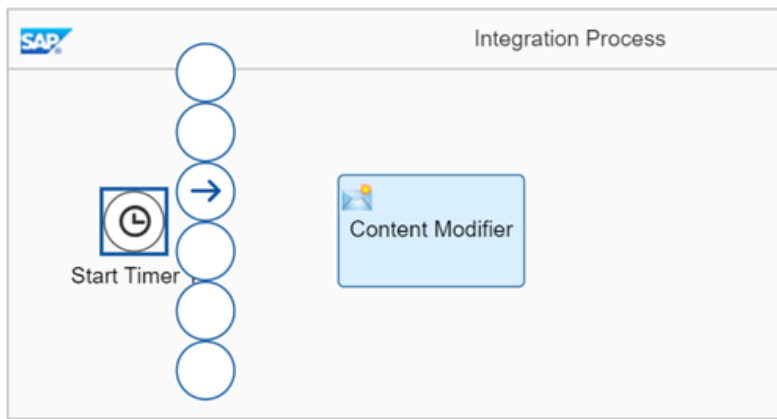
3. In the Content Modifier properties section, go to the [Message Body](#) tab and enter the following string sequence in the entry field:

Hello World!

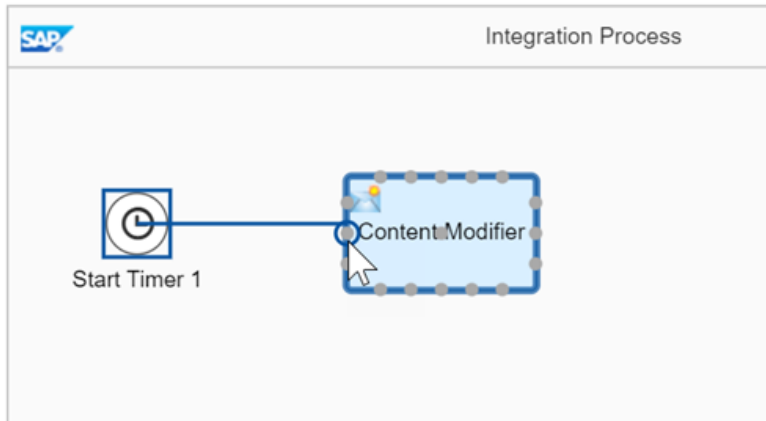
 A screenshot of the SAP Integration Suite Content Modifier properties section. The 'Message Body' tab is selected. The 'Type' dropdown is set to 'Expression'. The 'Body' field contains the text 'Hello World!'.

This simulates the inbound XML message.

4. Connect the Timer event with the Content Modifier. To do this, select the Timer event.



5. Click the arrow icon, and drag and drop the cursor to the Content Modifier.

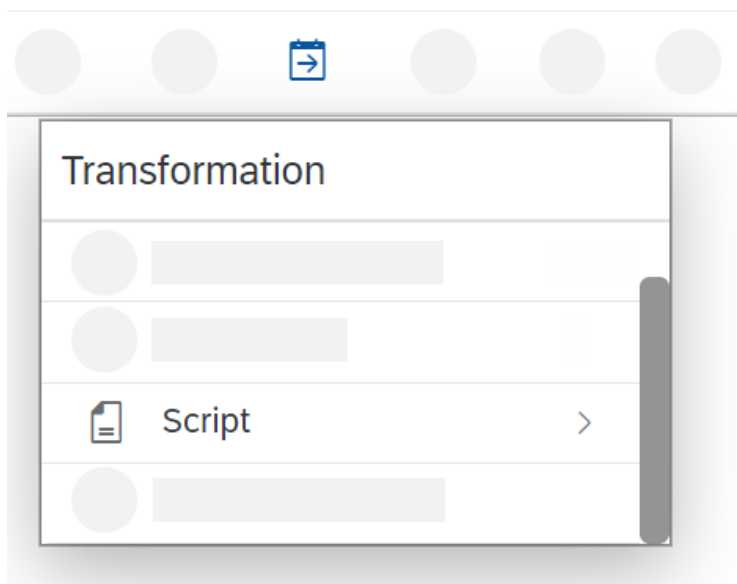


Create a Script Step to Log the Payload

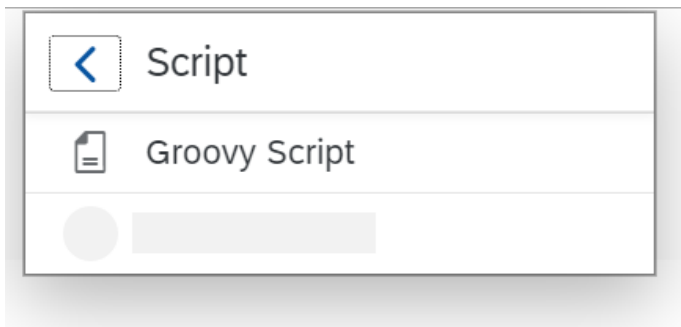
Add a Script step to log the message payload.

With a Groovy Script step, you can configure the integration in such a way that the payload of the message is written to the message processing log as attachment.

1. To add a Script step (containing a Groovy script), go to the palette and choose the **Message Transformers** icon and select the **Script** icon.



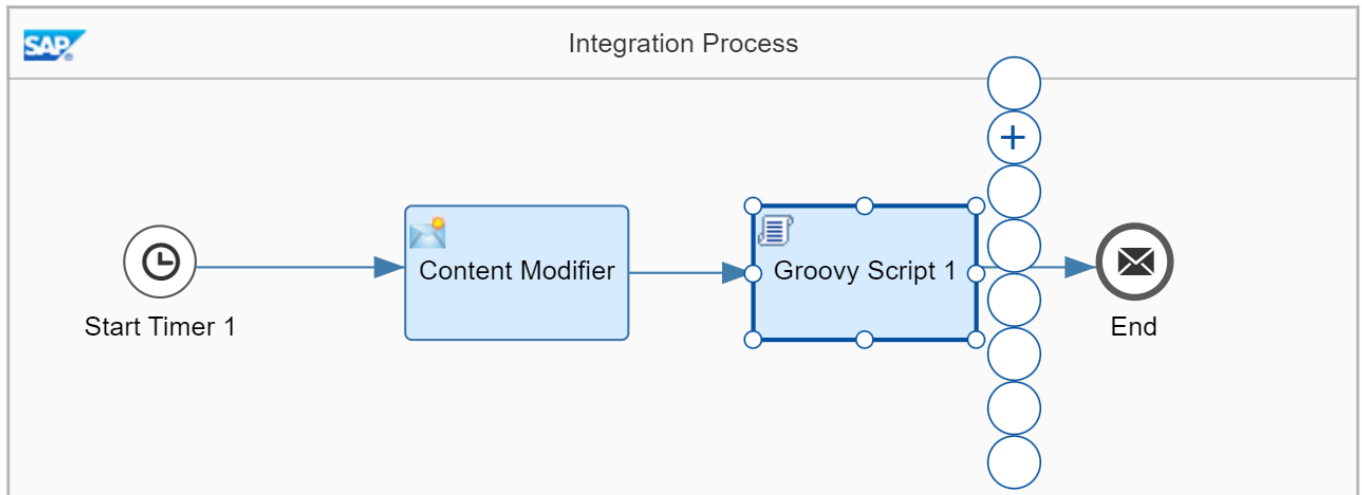
2. In the **Script** submenu, select **Groovy Script**.



3. Place the Script step shape after the Content Modifier step and connect both shapes.

4. Select the Script step.

The context icons are displayed.



5. Choose the  icon.

6. The default script coding of the step is displayed.

```

1  /* Refer the link below to learn more about the use cases of script.
2  https://help.sap.com/viewer/368c481cd6954bdfa5d0435479fd4eaf/Cloud/en-US/148851bf8192412cba1f9d2c17f4bd25.html
3
4  If you want to know more about the SCRIPT APIs, refer the link below
5  https://help.sap.com/doc/a56f52e1a58e4e2bac7f7adbf45b2e26/Cloud/en-US/index.html */
6  import com.sap.gateway.ip.core.customdev.util.Message;
7  import java.util.HashMap;
8  def Message processData(Message message) {
9      //Body
10     def body = message.getBody();
11     message.setBody(body + " Body is modified");
12     //Headers
13     def headers = message.getHeaders();
14     def value = headers.get("oldHeader");
15     message.setHeader("oldHeader", value + " modified");
16     message.setHeader("newHeader", "newHeader");
17     //Properties
18     def properties = message.getProperties();
19     value = properties.get("oldProperty");
20     message.setProperty("oldProperty", value + " modified");
21     message.setProperty("newProperty", "newProperty");
22     return message;
23 }
  
```

7. Replace this content by the script provided in the coding example below.

```

import com.sap.gateway.ip.core.customdev.util.Message;
import java.util.HashMap;
  
```

```
def Message processData(Message message)
{
    def body = message.getBody(java.lang.String) as String;
    def messageLog = messageLogFactory.getMessageLog(message);
    if(messageLog != null)
    {
        messageLog.addAttachmentAsString("Log current Payload:", body, "text/plain");
    }
    return message;
}
```

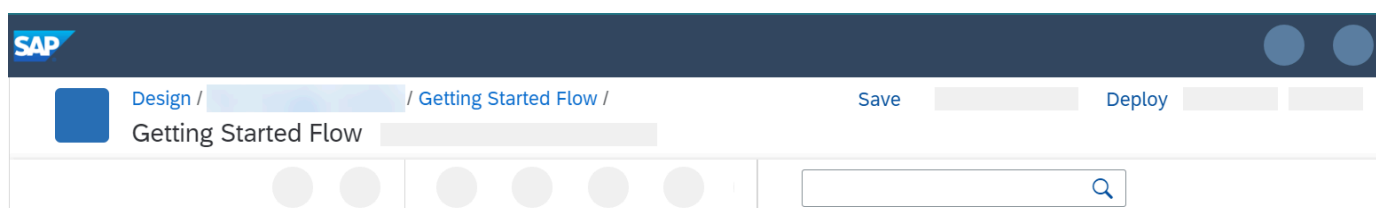
8. Choose **OK**.

The integration flow model is again displayed.

Save and Deploy the Integration Flow

Save and deploy the integration flow on the tenant to process it.

1. When you've finished modeling the integration flow, select **Save**.



Once saving has been successful, a corresponding status message is displayed.

2. Select **Deploy**.

A message is displayed that asks you to confirm this action.

3. Select **Yes**.

Another message is displayed when the validation has been performed and the integration flow deployment has been triggered.

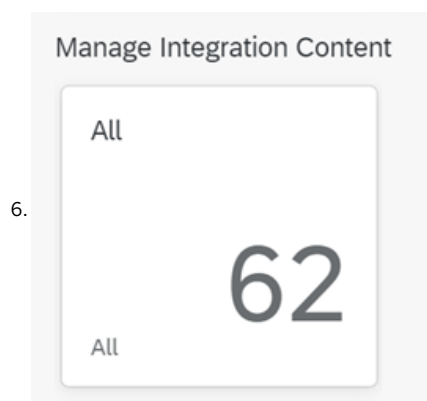
4. Select **OK**.

After successful deployment, a status message is displayed.

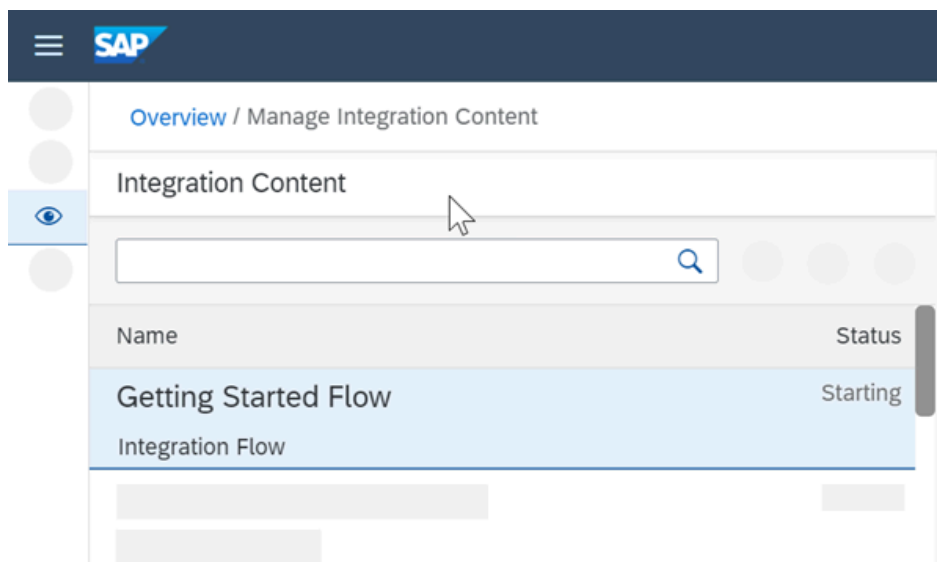
i Note

In case of a modeling error, instead of this message, a **Validation Failed** message is displayed. Only when you've fixed the error, the deployment of the integration flow is triggered.

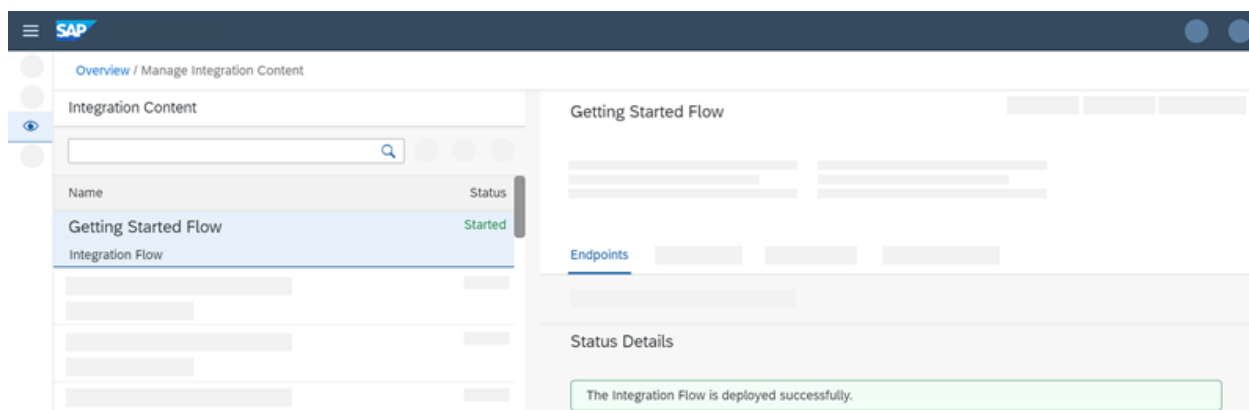
5. Choose the **Monitor > Integrations and APIs > Manage Integration Content** view to check the status of the deployment.



You can check the deployment status of your integration flow.



It changes from Starting to Started.



→ Tip

There's an alternative approach to deploy an integration flow:

Open the integration package that contains the integration flow. Go to the **Artifacts** tab, select the **Actions** button (next to the name of the integration flow that you like to deploy) and select **Deploy**.

Monitor Message Processing

Run the integration flow and check the result of message processing.

When the integration flow has been deployed successfully, the message is processed without any further trigger (based on the settings of the timer).

1. Go to the **Monitor > Integrations and APIs > Monitor Message Processing** view.

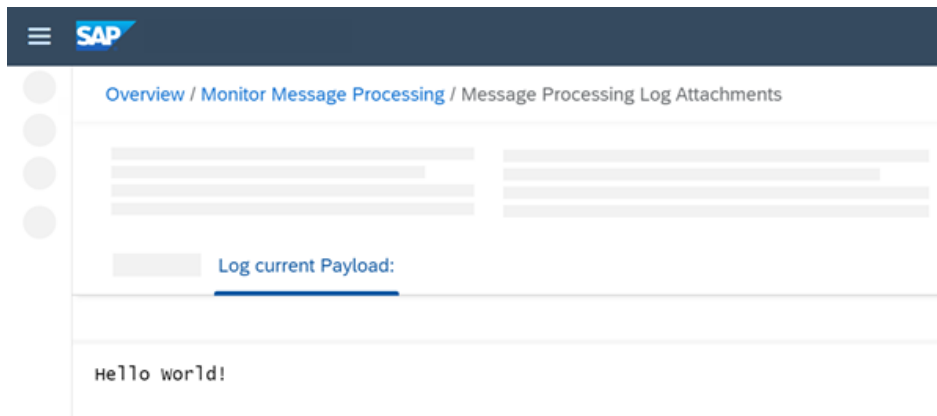
If your integration flow has been processed successfully, the status **Completed** should be shown.

2. Select the integration flow and analyze the details area to the right of the integration flow list.

3. Under **Attachments**, click **Log current Payload**.

You should see the message content, which consists of the following text:

Hello World!



This shows you that the message has been processed correctly.

Smoke Test Scenario with External Data Source

This is a very simple test to verify that your SAP Cloud Integration is working as expected. You do not need any receiver system to perform this test.

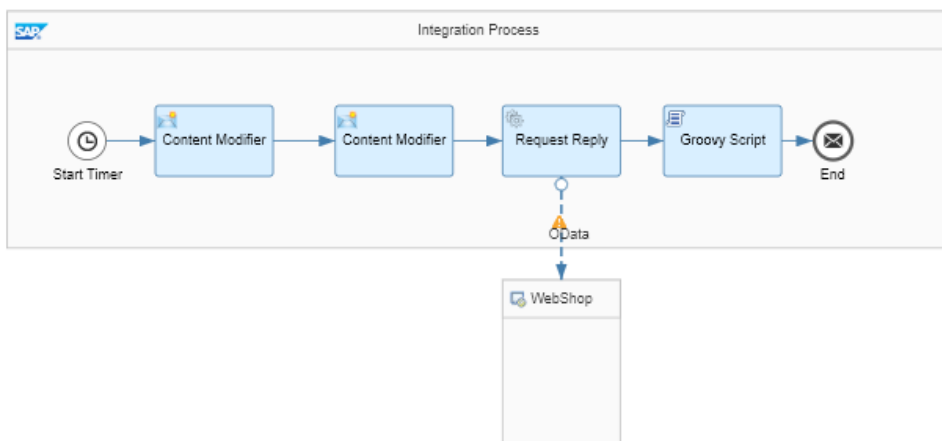
In this scenario, you access an OData API and get information about a product (for a specific product ID). The result is written into the message processing log which you can directly inspect with the message monitoring application.

⚠ Caution

This integration scenario is designed to show how to quickly (without much effort) set up and run an integration flow without the need to configure and connect to any receiver system. It uses a Script step to store the message payload in the message processing log (to enable you to easily check in the message monitoring application if the message was processed without any errors).

Note that this is not according to standard best practice. When designing productive scenarios, don't store the message payload in the message processing log. This can cause severe issues with memory consumption. The reason is that tasks such as message processing and message monitoring share the same memory and CPU which are available on your tenant.

In the course of this exercise, you develop the following integration flow.



To make it as easy as possible for you to develop this first integration flow, you don't need to configure any sender system. That saves the effort for you to set up a dedicated sender system and to connect it to SAP Cloud Integration. Instead of this, message processing is triggered by a Timer event, and the inbound message payload is created *within the integration flow*, in a dedicated Content Modifier step.

Furthermore, it is also not required that you set up any receiver system. To enable you to check if the message has been processed correctly, you will configure the integration so that the message payload is written into the message processing log (where you can easily inspect it using the **Monitor** application of the Web UI).

This is how the integration flow will process the message at runtime:

1. The Timer event triggers the processing of the message (according to the settings of the Timer's scheduler).
2. The first Content Modifier step creates a message with only one element, a `productIdentifier` (to identify a product from the product catalog).

The actual value of the `productIdentifier` is *hard-coded* in this step. If you like to process the integration flow with another product identifier, you need to change the value in this step and re-deploy the integration flow again. This is the drawback which results from abstaining from a dedicated sender system.

3. The second Content Modifier creates a message header (which we also call `productIdentifier`) and writes the actual value of the `productIdentifier` element into it. This header will be used in the subsequent step.

- The Request Reply step passes over the message to an external data source and retrieves data (about products) from there.

The external data source is represented by the lower WebShop shape.

The external data source supports the Open DataProtocol (OData). For our scenario, we use the ESPM WebShop, which is based on the Enterprise Sales and Procurement Model (ESPM) provided by SAP.

For the connection to the WebShop, an OData receiver channel is used. To query for exactly one product (for the product identifier provided with the inbound message), the header that has been created in the preceding Content Modifier is used.

- The OData API provides the details of one specific product (according to the product identifier provided with the inbound message).
- The Groovy Script step logs the payload of the message (that means, it writes the message content into the message processing log).

You can then run the integration flow and monitor message processing as described under: [Run the Integration Flow and Monitor the Message Processing](#).

Related Information

[Create the Script Step to Log the Payload](#)

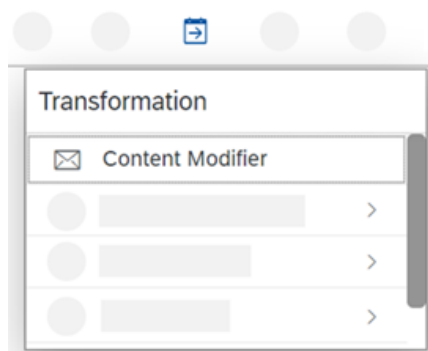
[Run the Integration Flow and Monitor the Message Processing](#)

Create a Content Modifier to Define the Message Body

Add a Content Modifier step to create the message body.

As the integration flow has no sender, we use a Content Modifier to create a message from scratch.

- To add a Content Modifier, go to the palette, choose the **Message Transformers** icon, and select the **Content Modifier** icon.

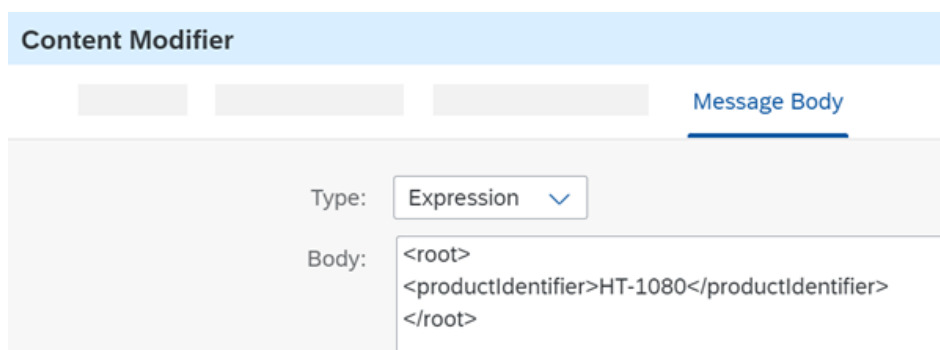


- Place the Content Modifier in the model after the Timer Start event.
- In the Content Modifier properties section, go to the **Message Body** tab and enter the following string sequence in the entry field:

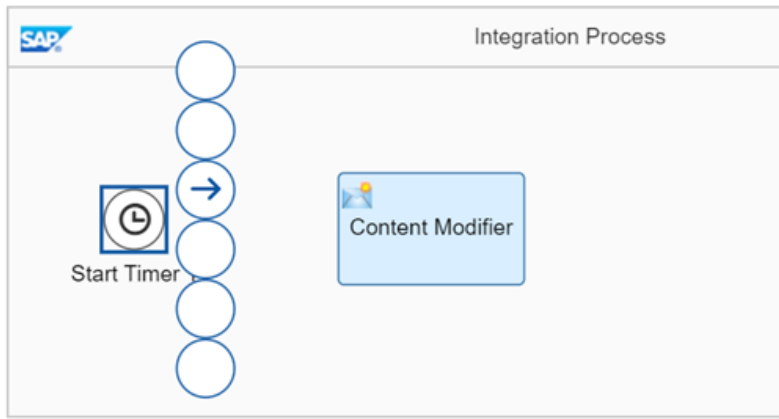
Sample Code

```
<root>
<productIdIdentifier>HT-1080</productIdIdentifier>
</root>
```

This simulates the inbound XML message.



- Connect the Timer event with the Content Modifier. To do this, select the Timer event, click the arrow icon, and drag and drop the cursor to the Content Modifier.



Related Information

[Define Content Modifier](#)

Create a Content Modifier to Add a Header

Add a Content Modifier to your model to define a header, which will be used in a later step to filter data from the external source.

If you remember, our input message has only one field: `productIdentifier`. This field will contain a product identifier that we want to use to filter the results from the WebShop application.

To make this number available to the integration framework during message processing, SAP Cloud Integration provides the option to store the value of `productIdentifier` from the incoming message either in the message header or in a data container referred to as an exchange property.

We use the first option, and to prepare the message accordingly we use a Content Modifier.

1. Add a second Content Modifier (after the first one) to the integration flow model.
2. In the properties section of the second Content Modifier, go to the **Message Header** tab and choose **Add**.
3. Specify the following parameters:
 - **Name:** Enter any name, for example, `productIdentifier`. This is the name of the header that will be created by the Content Modifier step.
 - **Type:** Select **XPath**.

→ Tip

In this example, you use an XML Path Language (XPath) expression to address a dedicated element of your inbound message. XPath allows you to address any element in an XML structure by using a well-defined syntax. The expression `//<element name>` addresses all elements with name `<element name>` in the XML document.

- **Data Type:** Enter `java.lang.String`.
- **Value:** Enter `//productIdentifier` (which is the XPath expression that points to the `productIdentifier` field in the inbound message).

The screenshot shows the "Content Modifier" configuration window with the "Message Header" tab selected. Under the "Headers" section, there is a table with columns: Action, Name, Source Type, Source Value, Data Type, and Default Value. The "Add" button is visible in the top right corner.

Action	Name	Source Type	Source Value	Data Type	Default Value
Create	productIdentifier	XPath	//productIdentifier	java.lang.String	

4. Connect the first Content Modifier (which defines the message body) with the second one.

In other words, the Content Modifier creates a header with the name `productIdentifier`, which will contain the value of the `productIdentifier` field of the incoming message.

Related Information

[Define Content Modifier](#)

Create the Outbound OData Channel

To call the external data source, add a Request Reply step to the integration flow model and connect this step with the external system using an OData channel.

i Note

Follow this procedure in case you use SAP Cloud Integration in the Neo environment.

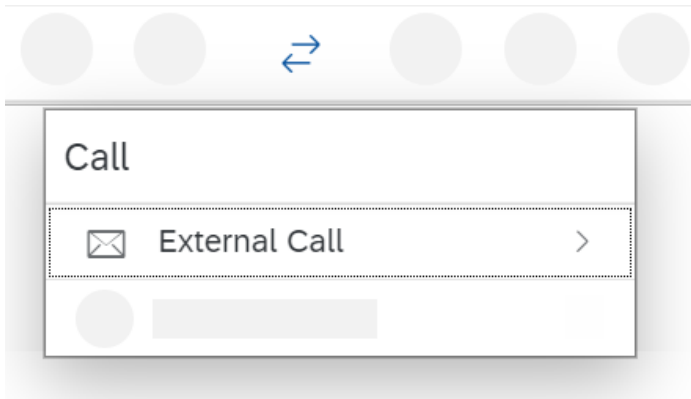
To configure SAP Cloud Integration to send a request message to the external OData API (to retrieve the required data), you need to do the following:

- Create a Request Reply step.
- Connect the Request Reply step to a Receiver shape and select the OData adapter type.
- Configure the OData adapter to specify how the OData API of the external service should be called (to define query options, for example).

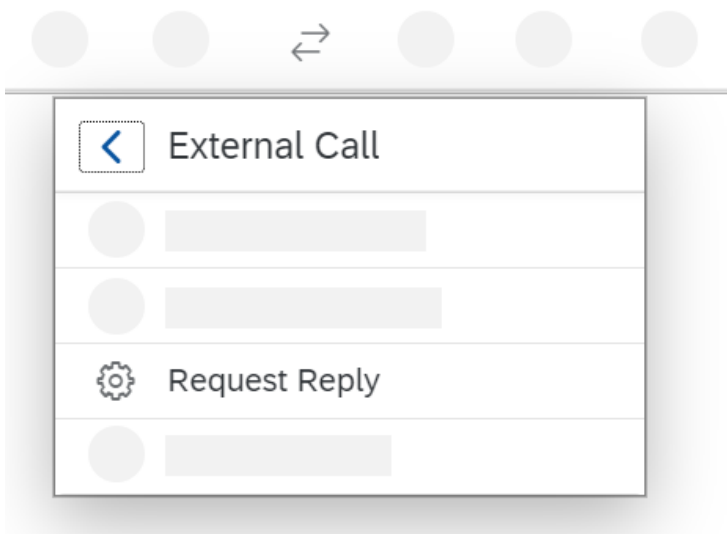
→ Remember

There are currently certain limitations when working in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#).

1. Go to the palette and select the **Call** entry.



2. Select **External Call** and in the submenu choose **Request Reply**.



3. Place the Request Reply shape between the second Content Modifier and the End event in the model.

Furthermore, connect the second Content Modifier with the Request Reply step and the Request Reply step with the End event.

4. Move the Receiver shape closer to the Request Reply shape (below the Request Reply shape but outside the Integration Process shape, as shown in the overall integration flow model under [Smoke Test Scenario with External Data Source](#)).
5. Connect the Request Reply shape to the Receiver shape (by selecting the Request Reply shape, clicking the arrow icon, and dragging and dropping the cursor on the Receiver shape).
6. In the next dialog, choose adapter type **▶ OData ▶ OData V2 ▶**.
7. In the next dialog, as **Message Protocol** select **OData V2**.

i Note

This adapter supports different versions of the OData protocol. Select version 2.0.

8. Go to the **Connection** tab of the OData adapter and enter the following as **Address**:

`https://refapp-espm-ui-cf.cfapps.eu10.hana.ondemand.com/espm-cloud-web/espm.svc`

→ **Tip**

This is the endpoint address of the ESPM WebShop's OData application programming interface.

9. Go to the **Processing** tab.

10. Next to **Resource Path**, choose **Select**.

The screenshot shows a web application window titled "OData". Below the title bar, there are two tabs: "Address" and "Processing", with "Processing" being the active tab. Under the "Processing" tab, the section "PROCESSING DETAILS" contains two fields: "Operation Details: *" with a dropdown menu showing "Query (GET)" and a downward arrow, and "Resource Path: *" with a text input field containing "Products" and a "Select" button to its right.

In the Query Editor opens, you can conveniently define the OData query.

The **Address** field is already populated with the value you just entered.

11. Make sure that **Remote** is selected as the **Connection Source**, and choose **Step 2**.

The system connects to the WebShop service and retrieves the metadata from its OData API.

12. Choose the search icon in the **Select Entity** field.

The screenshot shows a "Model Operation" dialog box. At the top, there is a progress bar with two steps: "1" and "2 Select Entity & Define Operation", with "2" being the active step. Below the progress bar, there are two fields: "Operation : *" with a dropdown menu showing "Query (GET)" and a downward arrow, and "Select Entity : *" with a text input field containing a search icon. Below the "Select Entity" field, a list of available elements is displayed, including "Products".

Select **Products**.

13. A list of the available elements is provided, using the information from the OData API.

Choose a set of elements for which you want to retrieve data (for example, all elements, as shown in the next figure) and choose **Step 3**.

14. Choose the copy icon in the **Filter By** field and select **ProductId**.

15. Select **Equal**.

16. In the third field, enter an expression that allows the integration framework to access the message header **productId** that you created in the preceding **Content Modifier** step: `${header.productId}`.

→ Tip

The dollar sign and the curled brackets indicate that we're dealing with Apache's *Simple Expression Language*, which is often used in SAP Cloud Integration. In particular, here, you see a *dynamic parameter*, which has the following effect: The value of the header **productId** (which is identical to the value of the **productId** field of the incoming message) is used dynamically at runtime to define the OData query.

17. Choose **Finish**.

Related Information

[Define Request Reply](#)

[Configure the OData V2 Receiver Adapter](#)

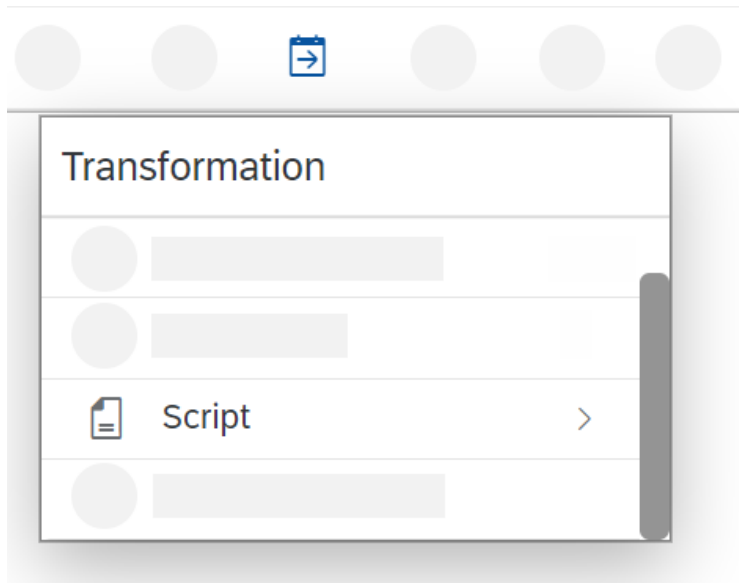
[Dynamically Configure Integration Flow Parameters](#)

Create the Script Step to Log the Payload

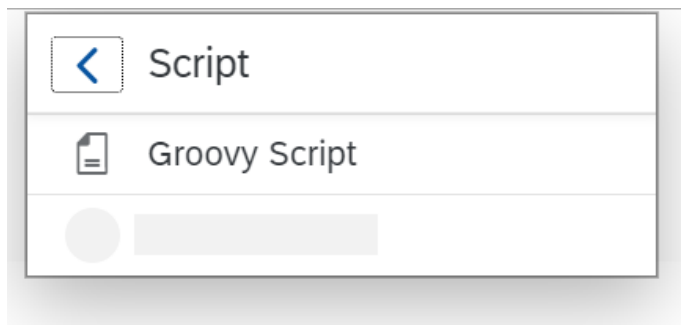
Add a Script step to log the message payload.

With a Groovy Script step, you can configure the integration in such a way that the payload of the message is written to the message processing log.

1. To add a Script step (containing a Groovy script), go to the palette and choose the **Message Transformers** icon and select the **Script** icon.

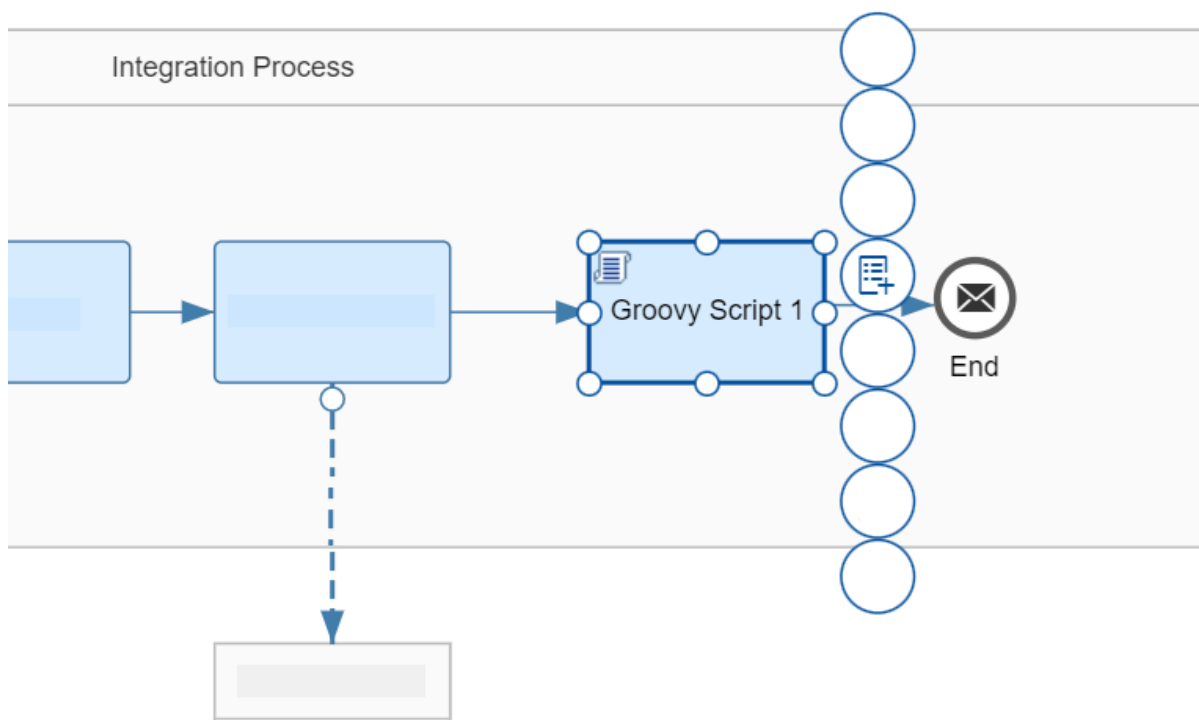


2. In the **Script** submenu, select **Groovy Script**.



3. Place the Script Step shape after the Request Reply step and connect both shapes.
4. Select the Script step.

The context icons are displayed.



5. Choose the **+** icon.
6. The default script coding of the step is displayed.

```

1 1 /* Refer the link below to learn more about the use cases of script.
2 2 https://help.sap.com/viewer/368c481cd6954bdfa5d0435479fd4eaf/Cloud/en-US/148851bf8192412cba1f9d2c17f4bd25.html
3 3
4 4 If you want to know more about the SCRIPT APIs, refer the link below
5 5 https://help.sap.com/doc/a56f52e1a58e4e2bac7f7adbf45b2e26/Cloud/en-US/index.html */
6 6 import com.sap.gateway.ip.core.customdev.util.Message;
7 7 import java.util.HashMap;
8 8 def Message processData(Message message) {
9 9     //Body
10 10     def body = message.getBody();
11 11     message.setBody(body + " Body is modified");
12 12     //Headers
13 13     def headers = message.getHeaders();
14 14     def value = headers.get("oldHeader");
15 15     message.setHeader("oldHeader", value + " modified");
16 16     message.setHeader("newHeader", "newHeader");
17 17     //Properties
18 18     def properties = message.getProperties();
19 19     value = properties.get("oldProperty");
20 20     message.setProperty("oldProperty", value + " modified");
21 21     message.setProperty("newProperty", "newProperty");
22 22     return message;
23 23 }

```

7. Replace this content by the script provided in the coding example below.

```

import com.sap.gateway.ip.core.customdev.util.Message;
import java.util.HashMap;

def Message processData(Message message)
{
    def body = message.getBody(java.lang.String) as String;
    def messageLog = messageLogFactory.getMessageLog(message);
    if(messageLog != null)
    {
        messageLog.addAttachmentAsString("Log current Payload:", body, "text/plain");
    }
    return message;
}

```

8. Choose **OK**.

The integration flow model is again displayed.

9. Save and deploy the integration flow.

Run the Integration Flow and Monitor the Message Processing

Run the integration flow and check the result of message processing.

When the integration flow has been deployed successfully, the message is processed without any further trigger (based on the settings of the timer).

1. When you have saved and deployed your integration flow, check the deployment status. Go to the **Monitor** section of the Web UI and select a tile under **Manage Integration Content**.

2. As soon as deployment has finished, the status **Started** should be displayed for your integration flow.

3. Go back to the overview page of the Web UI **Monitor** section and select a tile under **Monitor Message Processing**.

If your integration flow has been processed successfully, the status **Completed** should be shown.

4. Select the integration flow and analyze the details area to the right of the integration flow list.

5. Under **Attachments**, click **Log current Payload**.

You should see the message content, which consists of the details of the product associated with the value of `productIdentifier` entered in the first Content Modifier.

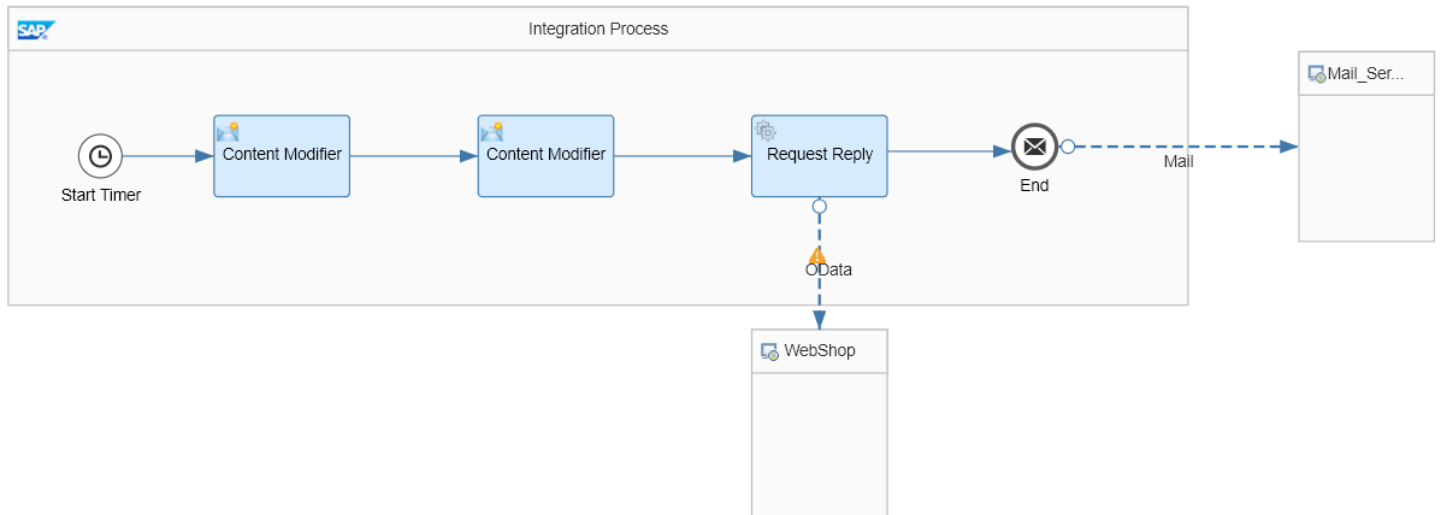
This shows you that the message has been processed correctly.

Timer-Initiated Scenario with a Mail Receiver

Create a simple integration scenario that is initiated by a timer, retrieves data from an external source, and sends the result to an e-mail account (as the receiver system).

A typical challenge addressed by an integration scenario is to retrieve data from a certain source (for example, product details from a product catalog on a vendor's site) using certain filter criteria. We use the integration flow described in this section to address such a use case.

In the course of this exercise, you develop the following integration flow:



To make it as easy as possible for you to develop this integration flow, you don't need to configure a sender system. This saves you the effort of setting up a dedicated sender system and connecting it to SAP Cloud Integration. Instead, message processing is triggered by a Timer event, and the inbound message is created *within the integration flow*, in a dedicated Content Modifier step.

As a prerequisite to use the Mail adapter, you need to take care of the following things.

- Update the tenant keystore with the certificates required by the mail server.
- Create and deploy a **User Credentials** artifact that contains the credentials of the mail account.

This is how the integration flow processes the message at runtime:

1. The Timer event triggers the processing of the message (according to the settings of the Timer's scheduler).
2. The first Content Modifier step creates a message with only one element: a `productId` (to identify a product from the product catalog).

The actual value of the `productId` is *hard-coded* in this step. If you want to process the integration flow with another product identifier, you need to change the value in this step and redeploy the integration flow. This is the drawback of not having a dedicated sender system.

3. The second Content Modifier step creates a message header (which we also call `productId`) and writes the actual value of the `productId` element into it. This header is used in the subsequent step.

4. The Request Reply step passes the message to an external data source from which it retrieves data (about products).

The external data source is represented by the lower **WebShop** shape.

The external data source supports the Open Data Protocol (OData). For our scenario, we use the ESPM WebShop, which is based on the Enterprise Sales and Procurement Model (ESPM) provided by SAP.

An OData receiver channel is used to connect to the WebShop. The header that was created in the preceding Content Modifier is used to query exactly one product (using the product identifier provided with the inbound message).

5. The OData API provides the details of this product.
6. Finally, the result of the request is forwarded to an e-mail account using the Mail receiver adapter (the e-mail server is represented by the **Mail_Ser ...** shape on the right in the integration flow model).

When you have finished integration flow design, you can monitor message processing.

This integration flow introduces you to a number of important aspects of integration development, such as defining an OData query and using a message header to dynamically query an OData source.

Related Information

[Update the Tenant Keystore with the Certificates Required by the Mail Server](#)

[Create and Deploy a User Credentials Artifact for the E-Mail Account](#)

[Create the Mail Receiver Channel](#)

[Monitor Message Processing](#)

Update the Tenant Keystore with the Certificates Required by the Mail Server

Add the required server root certificates (required by the e-mail provider) to the tenant keystore.

The tenant keystore contains the key pairs and certificates that are required (on the tenant side) to establish trusted communication with the connected systems.

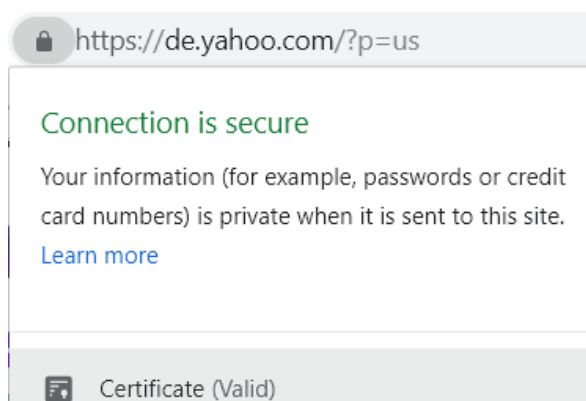
When establishing the connection to the SAP Cloud Integration tenant, the e-mail server needs to authenticate itself against SAP Cloud Integration using a digital server certificate. For this purpose, the tenant keystore must contain a root certificate that is also trusted by the e-mail server.

You can download the required certificates usually on a dedicated section of the email provider's website. You can search for *server certificate* to get more information. Note that the procedure can differ depending on the email provider.

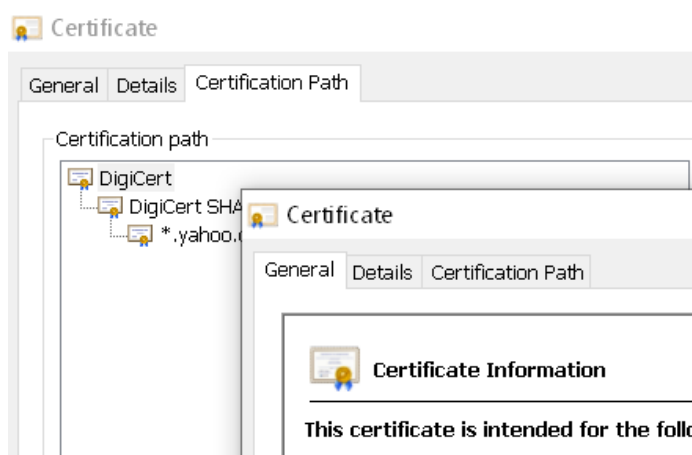
Get the E-Mail Servers' Certificate

To get the e-mail servers' root certificate, you can do the following:

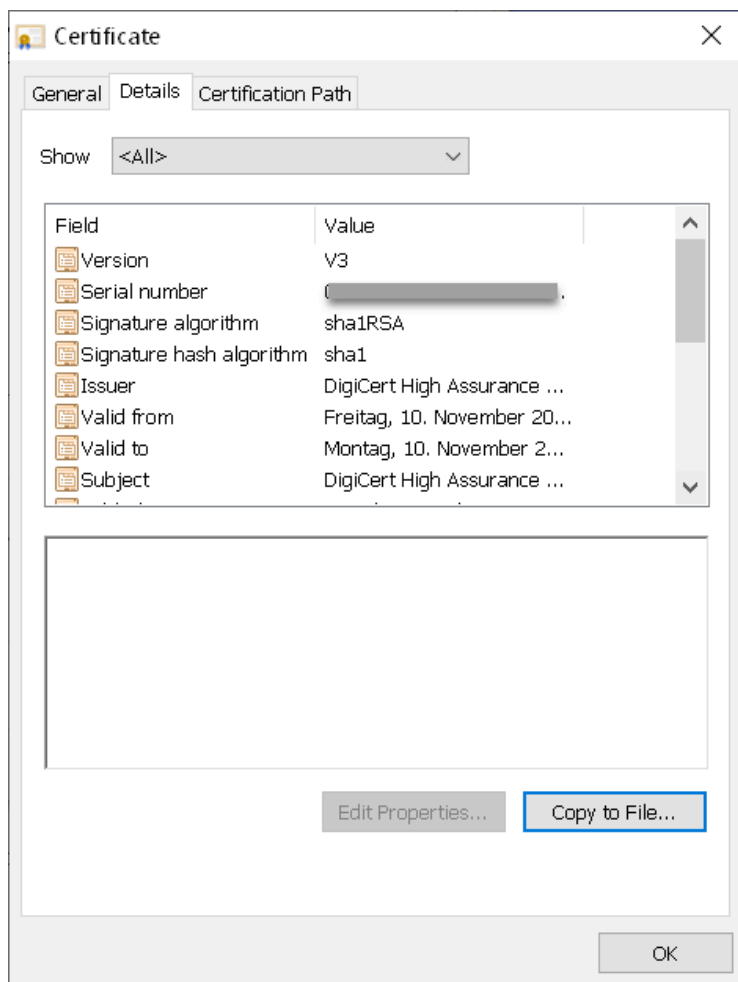
1. Open the website that hosts the mail account you like to address with the Mail adapter.
2. In the browser address field, click the lock icon and select **Certificate (Valid)** (example for using Google Chrome).



3. In tab **Certification Path**, doubleclick the uppermost node (which is the root certificate).



4. Click **Details** and select **Copy to File**



5. In the wizard, click **Next**.
6. On the next screen, keep the setting **DER encoded binary X.509 (.CER)** and click **Next**.
7. Choose a directory on your computer where to store the certificate file and a name.
8. Click **Next** and then **Finish**.

The root certificate is stored as file with extension **.cer** on your computer.

Get the E-Mail Servers' Certificate

A smart way to get the required certificates is by using the Connectivity Test application provided as part of SAP Cloud Integration.

→ Remember

There are currently certain limitations when working in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#).

1. Go to the Monitor section of the Web UI and select the **Connectivity Tests** tile under **Manage Security**.
2. Choose **SMTP**.
3. Specify the following parameters.
 - **Host**: Enter the address of the e-mail provider (for the SMTP protocol). This is the same address that you specified in the Mail receiver adapter (without the port), for example, `smtp.mail.yahoo.com`.
 - **Proxy Type**: Select **Internet**.
 - **Port**: Select the port that you specified in the Mail receiver adapter (for example, **465 (SMTPS)**).
 - **Authentication**: Select **None**.
 - Deselect the option **Validate Server Certificate**.
4. Choose **Send**.

The resulting response shows details about the certificates required by the e-mail server.

The screenshot shows the SAP Web UI Test Connectivity tool interface. The 'Request' tab is active, displaying the following configuration:

- Host: smtp.mail.yahoo.com
- Port: 465 (SMTPS)
- Proxy Type: Internet
- Protection: SMTPS
- Authentication: None (selected)

The 'Response' tab shows a successful connection message: 'Successfully reached host at [redacted]'. Below this, the status of various protocols is listed:

- STARTTLS: Not Supported
- Authentication: Unsuccessful
- Server Certificate: Invalid

A 'Server Certificate Chain' section is visible with a download icon.

5. Choose **Download**.

The certificates are downloaded as a compressed file on your computer.

6. Extract the certificates on your computer from the compressed (.zip) file.

Finally, you need to import the downloaded certificates to the tenant keystore. To do this, open the Keystore monitor.

1. Go to the Monitor section of the Web UI and select the **Keystore** tile under **Manage Security**.

All certificates that are already included in the keystore are displayed. These certificates are initially provided by SAP.

2. Choose **Add > Certificate**.

The screenshot shows the SAP Web UI Manage Keystore tool. The 'Current' tab is active, displaying a table of certificate entries. An 'Add' button is visible, and a 'Certificate' dialog box is open, showing fields for alias and certificate data.

3. Browse to the certificate stored on your computer, provide an alias (to identify the certificate in the keystore) and choose **Deploy**.

4. Repeat this task for all certificates that you have downloaded.

Note

You also need to change the settings of your e-mail account so that the mail server accepts connections to remote applications with a lower security level (for example, for Yahoo mail, this is the Less Secure Apps setting). If you don't do this, the integration flow raises an error during processing.

Note

If you don't upload the required root certificate to the tenant keystore and try to execute the integration flow (when having finished its design), message processing fails with the following error message starting with:

Sample Code

```
javax.mail.MessagingException: Could not connect to SMTP host: smtp.mail.yahoo.com, port: 465;
nested exception is:
javax.net.ssl.SSLHandshakeException ...
```

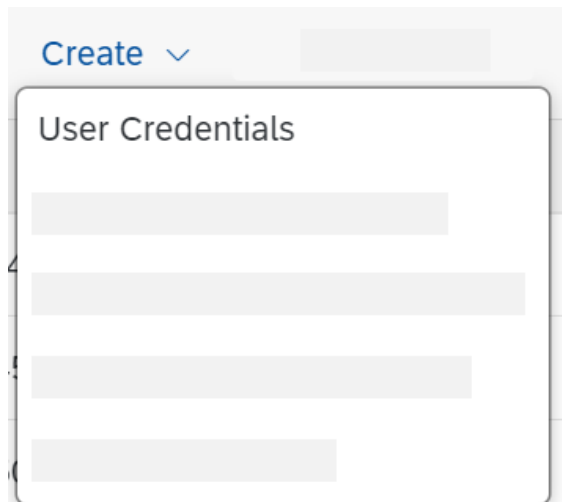
Related Information

[SMTP Connectivity Tests](#)

Create and Deploy a User Credentials Artifact for the E-Mail Account

Deploy a User Credentials artifact that contains the user name and password for your receiver mail account.

1. Go to the **Monitor** section, and select the **Security Material** tile under **Manage Security**.
2. Choose **Create** > **User Credentials**



3. As **Name**, enter the **User Credentials** name that you specified in the Mail receiver adapter, and as **User** enter the e-mail account user name (also specified in the fields **From** and **To** in the Mail receiver adapter).

Also provide the password of the mail account.

 A screenshot of a web form titled 'Create User Credentials'. The form contains the following fields:

- Name:** * mycredentials
- Description:** (empty text box)
- Type:** * User Credentials (dropdown menu with a blue arrow)
- User:** * myname@yahoo.de
- Password:** (text box with 7 dots)
- Repeat Password:** (text box with 7 dots)

 At the bottom right of the form is a blue **Deploy** button.

i Note

Storing the user name and password in a separate artifact increases the security level of integration development.

4. Choose **Deploy**.

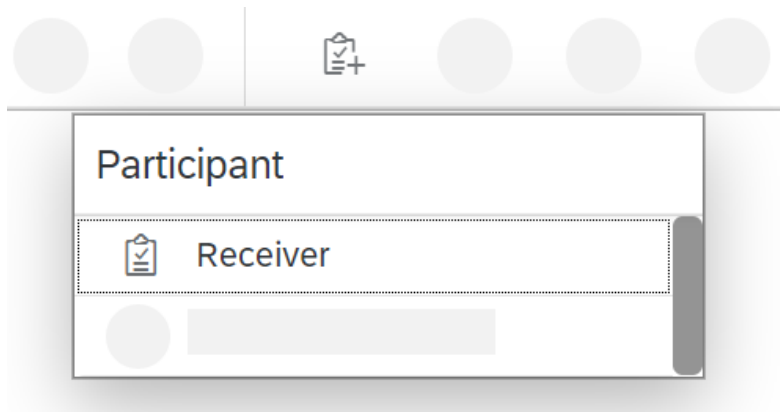
Related Information

[Deploying a User Credentials Artifact](#)

Create the Mail Receiver Channel

Add a Mail receiver channel to enable the integration flow to send messages to an e-mail account.

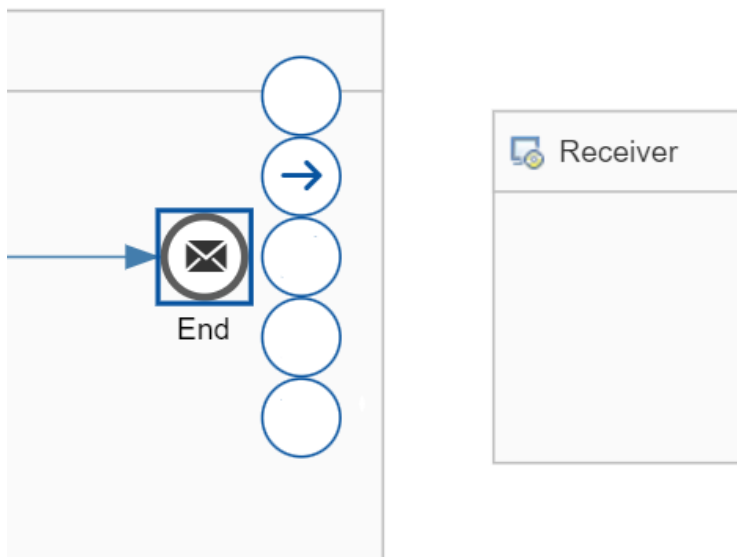
1. First, add a second receiver to represent the e-mail account. In the integration flow model (in **Edit mode**), select the **Participants** entry from the palette and select **Receiver**.



2. Place the Receiver shape on the right side of the model, outside the Integration Process shape.

You can rename the shape to `Mail_Receiver` (for example).

3. Select the End event, click the arrow button and drag and drop the cursor on the Receiver shape.



4. Select **Mail** as the **Adapter Type**.
5. In the Mail adapter properties section (below the model), go to the **Connection** tab and specify the following Mail adapter parameters.

The figure shows example settings, which are explained further below.

Mail

Connection

Address: * smtp.mail.yahoo.com:465

Proxy Type: Internet

Timeout (in ms): * 30000

Protection: SMTPS

Authentication: Plain User/Password

Credential Name: * mycredentials

- **Address:** Enter the address of your mail provider, followed by a colon and the port number. The example in the figure shows the address for Yahoo Mail (smtp.mail.yahoo.com:465) and SMTP protocol.
- **Proxy Type:** Select **Internet**.
- **Protection:** Select **SMTPS**.
- **Authentication:** Select **Plain User/Password**.
- **Credential Name:** Enter a name for a User Credentials artifact that you will create in a subsequent step, for example, MyCredentials.

The User Credentials artifact will contain the user name and password for the e-mail account to which the message is to be sent.

Keep the default settings for the other parameters.

6. In the Mail adapter properties section (below the model), go to the **Processing** tab and specify the following Mail adapter parameters.

Mail

Processing

From: * abcde@yahoo.de

To: * abcde@yahoo.de

Subject: Product Details from Webshop

Mail Body: \${in.body}

- **From** and **To:** Enter the mail address of the e-mail account that should receive the message.
- **Subject:** Enter a meaningful text.

7. Save and deploy the integration flow.

Related Information

[Mail Adapter](#)

Monitor Message Processing

As you use a Timer event to trigger the message processing, the integration flow is processed as soon as it is deployed.

1. To check whether the processing has been executed correctly, go to your e-mail account. You should find a mail with the following content:



@yahoo.com
An: 9@yahoo.com



1. Aug. um 14:27 ★

```
<Products>
<Product>
  <CategoryName>Scanners</CategoryName>
  <DimensionHeight>0.05</DimensionHeight>
  <DimensionUnit>m</DimensionUnit>
  <CurrencyCode>EUR</CurrencyCode>
  <ProductId>HT-1080</ProductId>
  <ShortDescription>Flatbed scanner - 1200 dpi x 1000 dpi - 216 x 297 mm - Hi-Speed USB - Bluetooth Ver. 1.2</ShortDescription>
  <Name>Photo Scan</Name>
  <Category>Scanners</Category>
  <LongDescription>Flatbed scanner - 1200 dpi x 1000 dpi - 216 x 297 mm - Hi-Speed USB - Bluetooth Ver. 1.2</LongDescription>
  <QuantityUnit>EA</QuantityUnit>
  <Weight>2.3</Weight>
  <DimensionWidth>0.34</DimensionWidth>
  <Price>51.0</Price>
  <PictureUrl>HT-1080.jpg</PictureUrl>
  <DimensionDepth>0.48</DimensionDepth>
  <SupplierId>100000042</SupplierId>
</Product>
</Products>
```

2. Finally, check how the message was processed by opening the **Monitor** section of the Web UI.

3. Choose a tile under **Monitor Message Processing** and you should find your message with the integration flow name.

4. Open the integration flow in Edit mode, click the first Content Modifier and on the **Message Body** tab change the value of the productIdentifier to HT-2001 and redeploy the integration flow.

5. Once the integration flow has been deployed successfully, you should receive an e-mail with details about another product.

Related Information

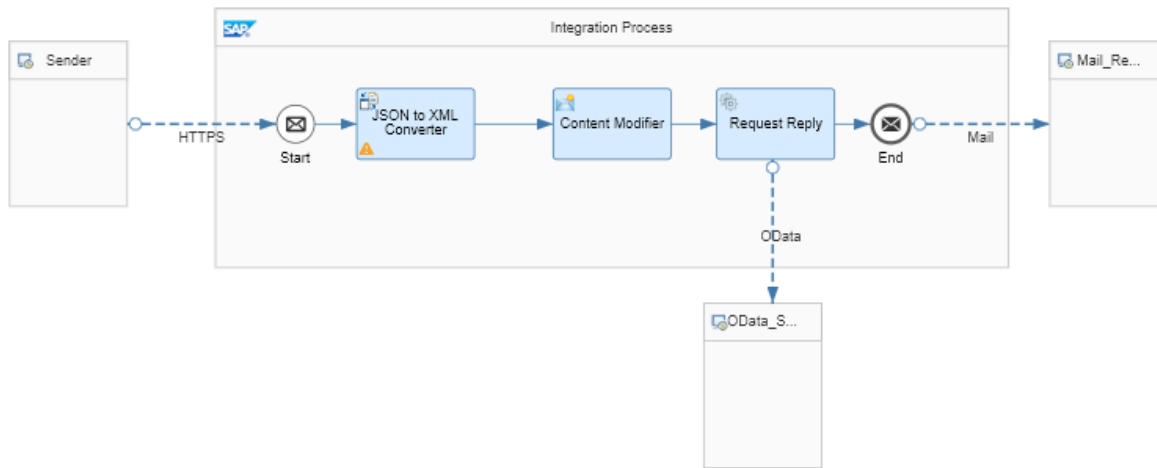
[Monitor Message Processing](#)

Sender-Initiated Scenario (with HTTPS Sender Adapter)

Create a simple integration scenario that is initiated by a sender (using the HTTPS sender adapter).

With the following steps, you can easily modify and extend the previously built integration flow with the email receiver (*Timer-Initiated Scenario with a Mail Receiver*).

The figure shows the integration flow model that you get as a result of this exercise.



In the modified integration flow, an HTTP client instead of a Timer event triggers message processing.

Furthermore (to simplify the design), we have merged the steps processed by two different Content Modifier steps in the previously built integration flow into one Content Modifier step.

i Note

As a prerequisite to execute this integration flow in the Cloud Foundry environment, you need to authorize the sender system (HTTP client) to call the integration flow endpoint. For that purpose, you create a service instance on SAP BTP and generate service key credentials (which can then be used by the HTTP client to call the integration flow endpoint).

This is how the integration flow processes the message at runtime:

1. The HTTP client (represented by the Sender shape) sends an HTTP request to SAP Cloud Integration through an HTTPS sender channel. The HTTPS request body, which is in JavaScript Object Notation (JSON) format, contains a product identifier.
2. The JSON-to-XML converter transforms the request body into XML format (which can be processed in the following step, the Content Modifier).
3. The Content Modifier creates a message header (which we also call `productIdentifier`) and writes the actual value of the `productIdentifier` element into it. This header is used in the subsequent step.

In this exercise, you use **one** Content Modifier to create the header and to write the message body.

4. The Request Reply step passes the message to an external data source and retrieves data (about orders) from there.

The external data source supports the Open DataProtocol (OData). For our scenario, we use the ESPM WebShop, which is based on the Enterprise Sales and Procurement Model (ESPM) provided by SAP.

5. An OData receiver channel is used for the connection to the OData source. To query for exactly one product (for the product identifier provided with the inbound message), the header that was created in the preceding Content Modifier is used.
6. The OData API provides the details of one specific product, which is identified by the actual value of the `productIdentifier` field (provided with the inbound HTTP request).
7. Finally, the result of the request is forwarded to an e-mail account using the Mail receiver adapter (the e-mail server is represented by the right Mail ... shape in the integration flow model).

When you have finished the integration flow design, you can send the message through the HTTP client.

Related Information

- [Timer-Initiated Scenario with a Mail Receiver](#)
- [Authorize the HTTP Client to Call the Integration Flow Endpoint, Cloud Foundry Environment](#)
- [Create the HTTPS Sender Channel](#)
- [Add the JSON to XML Converter](#)
- [Create the Mail Receiver Channel](#)
- [Send the HTTP Request and Process the Integration Flow](#)

Authorize the HTTP Client to Call the Integration Flow Endpoint, Cloud Foundry Environment

You perform these steps to authorize the sender (HTTP client) to call the SAP Cloud Integration integration flow endpoint.

i Note

This information is relevant only when you use SAP Cloud Integration in the Cloud Foundry environment.

i Note

You need to perform these steps only in case you use SAP Cloud Integration in the Cloud Foundry environment.

These steps imply that you create a service instance on SAP BTP and generate a service key for it. The credentials which you get as a result can be used by the HTTP client to call the integration flow endpoint.

Create Service Instance

A service instance defines how an application on SAP BTP can be called by a sender (in our scenario, the HTTP client). In technical terms, it is an OAuth client.

Go to SAP BTP cockpit and perform the steps described at [Creating Service Instance and Service Key for Inbound Authentication](#).

When defining the service instance, enter the following JSON content:

Sample Code

```
{
  "roles": [
    "ESBMessaging.send"
  ]
}
```

Create Service Key

With this step, you generate credentials to communicate with a service instance.

Perform the steps described at [Creating Service Instance and Service Key for Inbound Authentication](#).

The sender application (HTTP client) uses these credentials (`clientId` and `clientSecret`) to access the SAP Cloud Integration integration flow endpoint.

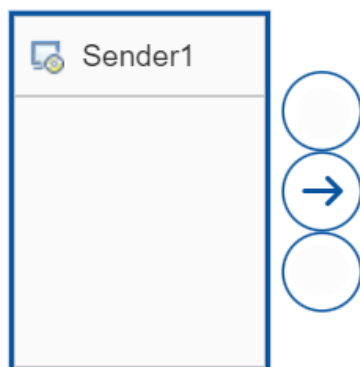
You need to copy the values of `clientId` and `clientSecret` to your clipboard or to a text editor for later reference.

These values specify the credentials of the user associated with the sending application.

Create the HTTPS Sender Channel

Add an HTTPS sender channel to enable the integration flow to receive HTTP requests.

1. Select the integration flow and choose [Edit](#).
2. Click the Sender shape. The context icons for the Sender appear.

**i Note**

If you choose the information icon, the version of the integration flow component is displayed.

Do not confuse the version of an individual integration flow component with the software version of SAP Cloud Integration. An integration flow component gets a new version each time a new feature is added to it by SAP. Let's imagine a situation where you started modeling an integration flow some time ago and now want to continue working on it. Let's assume that SAP has updated the software in the meantime. A new version of an integration flow step or shape that you have used is now available, containing a new feature. You can continue to use the old component version, but if you want to use the new feature you need to update to the new version.

- Click the arrow icon and drag and drop the cursor on the Start event.

The list of available adapter types is displayed in a dialog.

- Choose adapter type **HTTPS**.

The properties of the adapter are shown below the model.

- Go to the **Connection** tab.

The screenshot shows the configuration window for the HTTPS adapter. The 'Connection' tab is active. The 'REQUEST PROCESSING' section contains the following fields:

- Address:** /FirstFlow
- Authorization:** User Role (dropdown menu)
- User Role:** ESBMessaging.send (text field)
- CSRF Protected:** ☒

Specify the following parameters:

- Address:** This parameter defines the endpoint under which the integration flow can be called from the sender. Start the address with a slash, for example, /FirstFlow.
- Authorization** and **User Role:** You want to configure your first integration flow so that the inbound request is authenticated using basic authentication (based on user credentials). To keep it simple, you use your dialog user. During onboarding, you also made sure that the role `ESBMessaging.send` was assigned to your user (and this setting was also used when configuring the HTTP client).

You can select other roles for inbound authentication (if you have defined these roles for the runtime node in Neo), but you don't use this option in this scenario.
- CSRF Protected:** Keep this option selected (default setting). It ensures that your integration flow is protected against Cross-Site-Request-Forgery, a kind of attack where a malicious party can perform harmful actions by masquerading as the logged in user (the user specified for the HTTP client in our case).

Related Information

[HTTPS Sender Adapter](#)

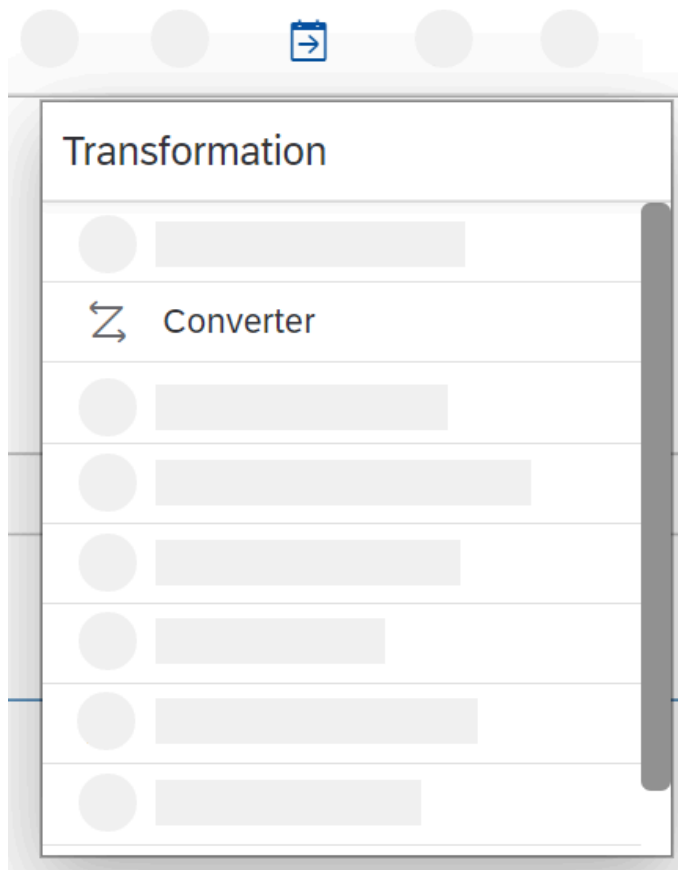
[Authentication and Authorization Options \(Inbound\)](#)

Add the JSON to XML Converter

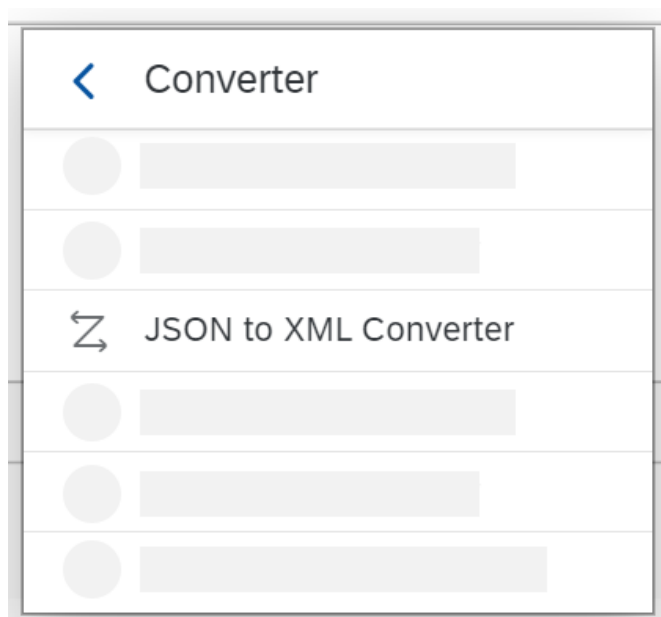
Add a JSON-to-XML converter to convert the HTTP request, which is in JavaScript Object Notation (JSON) format, to XML for further processing.

With the HTTP client, we send a POST request with a request body in JSON format. To enable the subsequent steps to process the message, it needs to be converted to XML first. To perform the required conversion, you can use the JSON-to-XML converter.

- In the palette, select the **Message Transformers** entry and then choose **Converter**.

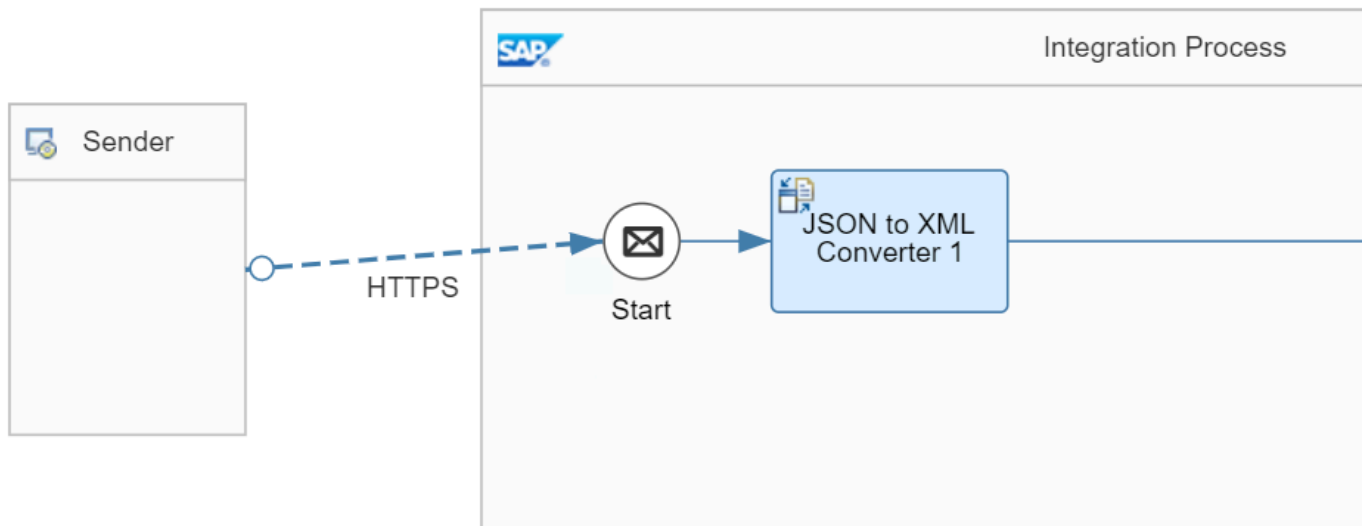


2. In the submenu, choose **JSON to XML Converter**.



3. In the integration flow model, place the shape inside the Integration Process shape after the Start event.

The model should now look like this:



Related Information

Define JSON to XML Converter

Send the HTTP Request and Process the Integration Flow

Set up an HTTP client using Postman and send the HTTP request.

1. Install an HTTP client on your computer.
2. Since the HTTPS sender adapter is configured to expect **User Role** authorization, the authentication mode used is basic authentication (with user credentials).

Select the corresponding authentication mode for your HTTP client.

Depending on whether you work in the Neo or the Cloud Foundry environment, the way how to get the required credentials differs.

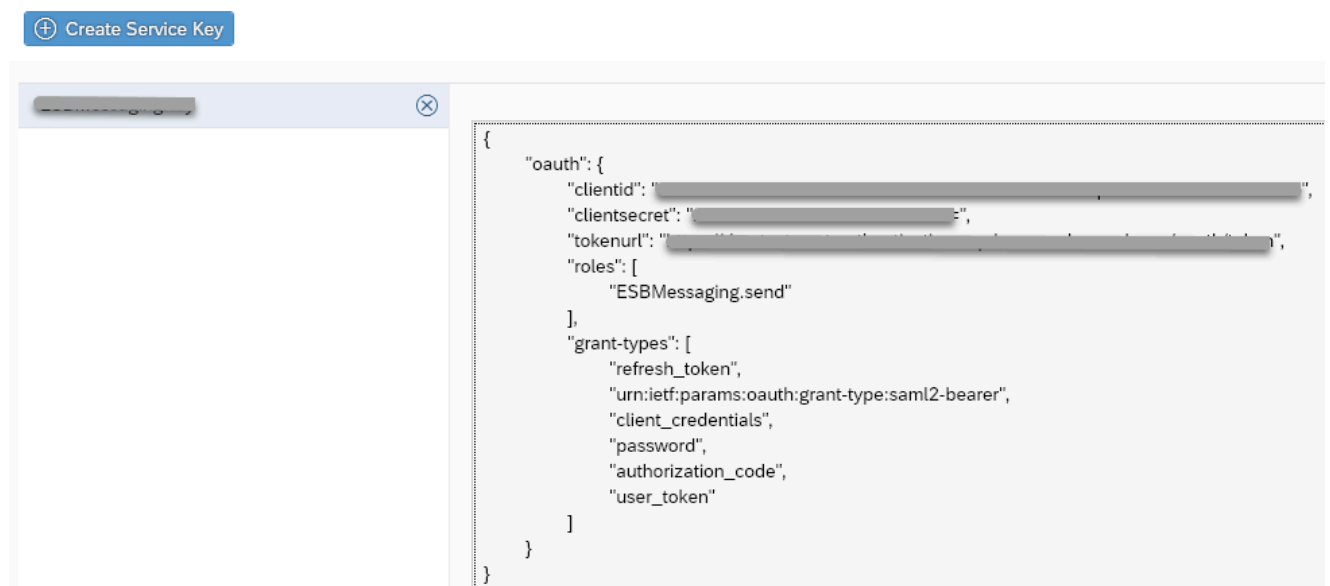
- o Neo environment:

Specify the credentials of the user that is associated with the inbound HTTP request and that has been assigned the role `ESBMessaging_send` in SAP BTP cockpit.

- Cloud Foundry environment:

As credentials, enter the values of `clientid` and `clientsecret` that have been generated when you created the service key during onboarding.

The following figure shows the related dialog from SAP BTP cockpit.

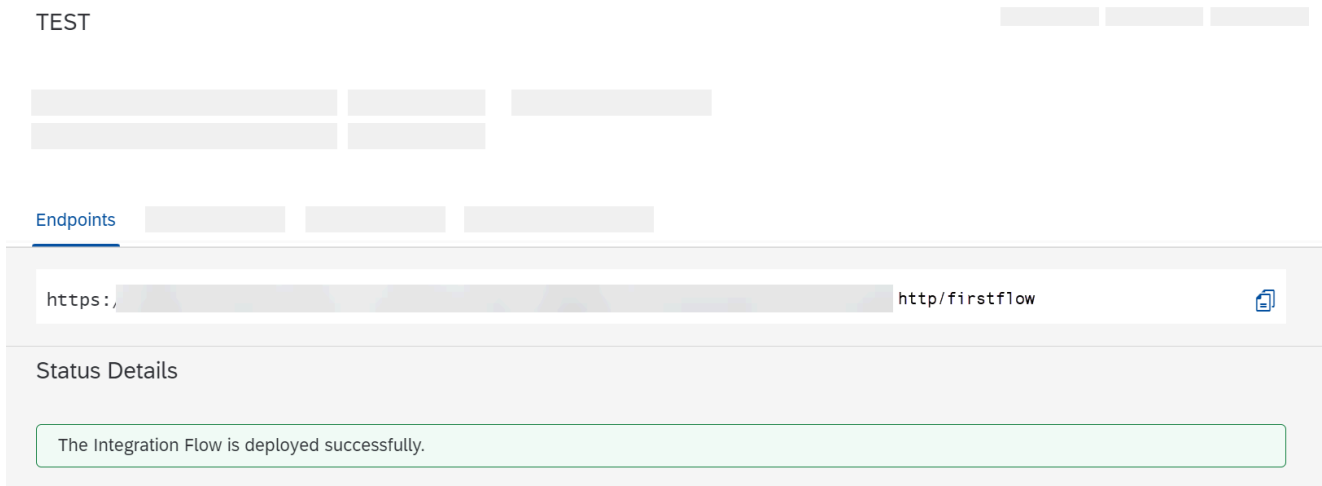


3. As you have selected **CSRF Protected** in the HTTPS adapter, you need to fetch a CSRF token.

The CSRF token is then used to place the POST request for your integration flow.

- a. Find out the endpoint address of the integration flow. To do this, go to the **Monitor** section of the Web UI.

- b. Choose a tile under [Manage Integration Content](#).
- c. Select your integration flow. It should be displayed in the list of deployed artifacts with status **Started**.



- d. Copy the endpoint URL to the clipboard.

The URL should end with `/http/<Address specified in the HTTPS adapter>`.

- 4. Using your HTTP client, send a GET request to the endpoint address.

Make sure that you send a header with the key `X-CSRF-token` and the value `fetch` with the request.

- 5. You should receive the CSRF token.

You can now send the POST request to the integration flow.

- 1. Specify the same authentication settings as for the GET request above.
- 2. Specify the following HTTP request body (JSON format):

Sample Code

```
{  
  
  "productIdentifier": "HT-1080"  
  
}
```

- 3. Copy the value of the CSRF token (obtained from the GET request above) to the clipboard.
- 4. Add a header to the request.

In the **Key** field, enter `X-CSRF-Token` and in the **Value** field, enter the value of the CSRF token from your clipboard.

- 5. Send the request.

You should get the details of the product with productIdentifier HT-1080.

- 6. Go to the e-mail account specified in the Mail adapter. You should have received an e-mail like this:



```
<Products>
<Product>
  <CategoryName>Scanners</CategoryName>
  <DimensionHeight>0.05</DimensionHeight>
  <DimensionUnit>m</DimensionUnit>
  <CurrencyCode>EUR</CurrencyCode>
  <ProductId>HT-1080</ProductId>
  <ShortDescription>Flatbed scanner - 1200 dpi x 1000 dpi - 216 x 297 mm - Hi-Speed USB - Bluetooth Ver. 1.2</ShortDescription>
  <Name>Photo Scan</Name>
  <Category>Scanners</Category>
  <LongDescription>Flatbed scanner - 1200 dpi x 1000 dpi - 216 x 297 mm - Hi-Speed USB - Bluetooth Ver. 1.2</LongDescription>
  <QuantityUnit>EA</QuantityUnit>
  <Weight>2.3</Weight>
  <DimensionWidth>0.34</DimensionWidth>
  <Price>51.0</Price>
  <PictureUrl>HT-1080.jpg</PictureUrl>
  <DimensionDepth>0.48</DimensionDepth>
  <SupplierId>100000042</SupplierId>
</Product>
</Products>
```

7. Place another POST request with a body containing productIdentifier HT-2001, and you receive details of another product.

8. Finally, check how the message was processed by opening the **Monitor** section of the Web UI.

Choose a tile under **Monitor Message Processing** and you should find your message with the integration flow name.

Related Information

Further Sample Integration Flows

Check out the following links to find out how to extend the Getting Started integration flow with additional features.

It's easy to modify the Getting Started scenario to demonstrate specific features.

Related Information

[SOAP Sender Adapter: Example Integration Flow](#)

[Define Data Store Select Operations](#)

[Examples](#)

Packaging Integration Content in Cloud Integration

Cloud Integration allows you to assemble integration contents into packages so that integration developers can use these packages in their integration scenarios.

As an integration developer, you can now create integration packages for your specific domain or organization. You can also view different packages published by other integration developers and consume them for your integration purposes. You can modify these packages based on your requirements and upload them through the web application.

Creating an Integration Package

Create an integration package that is specific to your integration scenario.

Procedure

1. Choose **Design**.
2. Choose **Create**.
3. In the **Header** tab, provide a **Name**, a **Technical Name**, and a **Short Description** for the package.
4. Choose **Save** to keep your package.
5. Navigate to other tabs of the package to explore what else you can do with the package.

Related Information

Importing Integration Packages

Upload your existing integration packages that suits your integration scenario to reduce redundancy and duplication efforts of creating a package from scratch.

Procedure

1. Choose  **Design** > **Import** .

i Note

Importing a new version of a package (zip import) over an existing package doesn't overwrite the configured values of the externalized parameters in the existing package.

Merge scenario – A package exists in the target tenant. If you change the package in its source tenant and try to import with delta changes, the import action overwrites the package in the target tenant with the delta changes. Additionally, if you had added new artifacts to the package in the target tenant, the import action retains those artifacts as is. That is, the import action merges the delta changes and unique artifacts of the package in the target tenant.

2. Browse and select the integration package for uploading.
3. Choose **OK**.

Import of a package fails with **Uniqueness Conflict** when the source package contains one or more artifacts that already exist in other packages of the target tenant.

A package is created based on the metadata of the uploaded file.

Working with an Integration Package

Perform various tasks with the artifacts and the integration package.

Procedure

1. Choose **Design** to view the list of integration packages.
2. If you want to view the details of an integration package, choose **<integration package name>**.
3. Navigate to the **Artifacts** tab, to work with the available artifacts in the package.

- a. Choose  > **View metadata**  to view the metadata of the artifact.

- b. Choose  > **Deploy**  to deploy the artifact.

You can only deploy data flows, integration flows, and value mappings. You can deploy multiple value mappings at a time.

- c. Choose  > **Configure**  to configure the artifact.

You can configure only integration flows. For more information, see [Configure Externalized Parameters of an Integration Flow](#).

- d. Choose  > **Download**  to download an artifact. The download feature allows users to download the integration flow artifact with two options:

- **Default Values Only:** Downloads the integration flow with default values only.

i Note

When the downloaded integration flow imported into the system with default values, these values are seen to the user under externalized parameter view.

- **Merged Configured and Default Values:** Downloads the integration flow with values that consists of configured and default values. This option would replace the default value with the configured value and accept it as a new default value.

i Note

When you import the integration flow in the tenant, the configured values become the default value. For more information, see [Externalize Parameters of an Integration Flow](#)

You can only download integration flows, files, and value mappings. Your local file system stores the downloaded artifacts with name as <artifact name>.zip file. If you want to use the artifact in Eclipse, you need to import it in eclipse after extracting it from the zip file.

Editing an Integration Package

Edit your integration package to revise the header and overview sections, add or remove content to the artifacts, and delete the package.

Procedure

1. Choose **Design** to view the list of integration packages.
2. Select the required integration package.
3. In the integration package editor, choose **Edit**.

You can edit the existing information in the following tabs:

- o Header
- o Overview
- o Artifacts
- o Documents
- o Tags

4. In the **Header** tab, you can maintain the name of the package, a short description, a version, and the vendor of the package.

All this data is displayed on top of the integration package. Furthermore, in the list of integration packages within your design workspace, the name, the short description, and the version are displayed.

i Note

Although the technical name of the integration package is displayed in the **Header** tab, you can't change it. The technical name can only be defined either when creating a new package or when copying it from the **Discover** area.

5. In the **Overview** tab, you can maintain a more detailed description of your integration package.

You can also add hyperlinks.

Furthermore, you've the option to upload a package image. The image is placed on top of the overview page.

6. In the **Artifacts** tab, you can add various artifacts, see [Integration Artifact Types](#). To easily categorize the artifacts, choose  to group the artifacts by type. For each artifact, choose  to get the following functions.

The list of functions varies for different artifact types.

- o Delete
- o Copy
- o View metadata
- o Download
- o Configure (only for integration flows)
- o Deploy

7. To edit the metadata of an artifact, perform the following steps:

- a. Choose **View metadata**.

The **<Artifact Type> Details** dialog box comes up with the metadata of the artifact. For certain prepackaged, standard SAP artifacts you see an additional metadata **Integration: SAP-to-SAP**. This metadata identifies the standard SAP integration artifacts that connect two SAP systems.

- b. In the **<Artifact Type> Details** dialog box, choose **Edit**.

i Note

The name of integration flow must begin with an underscore '_' or alphabets. It can contain numbers '0–9', space ' ', dot '.', and hyphen '-'.

- c. Choose **Save** to keep the changes.

- d. Choose **Save as version** to retain a copy of the current artifact. For more information, see [Versioning of Artifacts](#).

You can view the version history of an artifact by choosing the current version mentioned along with it. You can save versions of a single or multiple artifacts, or an integration package. You can also view the list of existing versions of the required artifacts or integration packages. You can choose to make an existing version as the latest one with or without saving a draft of it.

- e. Choose **Cancel** to revert the changes.

8. In the **Documents** tab, you can upload files, such as configuration guides, and add URLs.

9. In the **Tags** tab, you can maintain standard tags such as Product, Country/Region, Lines of Business and Industry.

Furthermore, you can add custom tags to identify and track your packages based on your custom-defined attributes. For more information, see [Creating Custom Tags](#).

10. Choose **Save** to keep the changes made to the integration package.

11. Choose **Cancel** to discard the changes.

12. Choose **Delete Package** to permanently delete the integration package.

Provide your consent to delete the package.

→ Remember

You can't recover the package and its content after you delete them.

Updates for SAP's Integration Packages

SAP provides pre-shipped content to address various integration scenarios. You can copy these integration packages from the **Discover** view to the **Design** space. SAP also publishes updates with new enhancements and bug fixes.

Whenever there's an update (planned or available) for the content that you've copied into your workspace, you're informed by an **Update Available** label that is displayed your corresponding artifact along this documentation.

i Note

You see a notification for all updates performed on all standard content packages that have been copied from the **Discover** view to the **Design** view of your tenant. This is even the case if the content isn't deployed on your tenant.

The update mode is defined by the content package publisher. There are two modes of update:

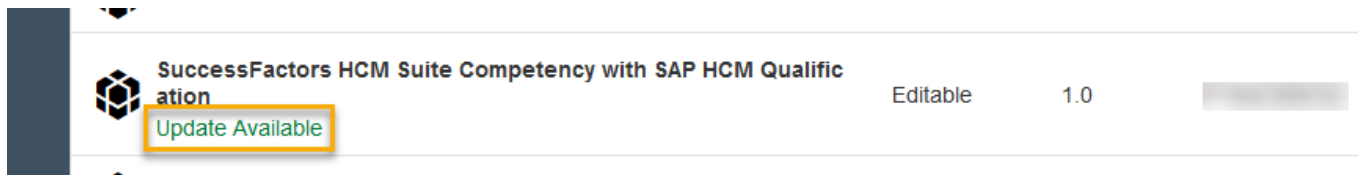
- **Manual:** You've the option to not implement the change.
- **Automatic:** The update is always applied. It is of two types:
 - Immediate
 - Scheduled

i Note

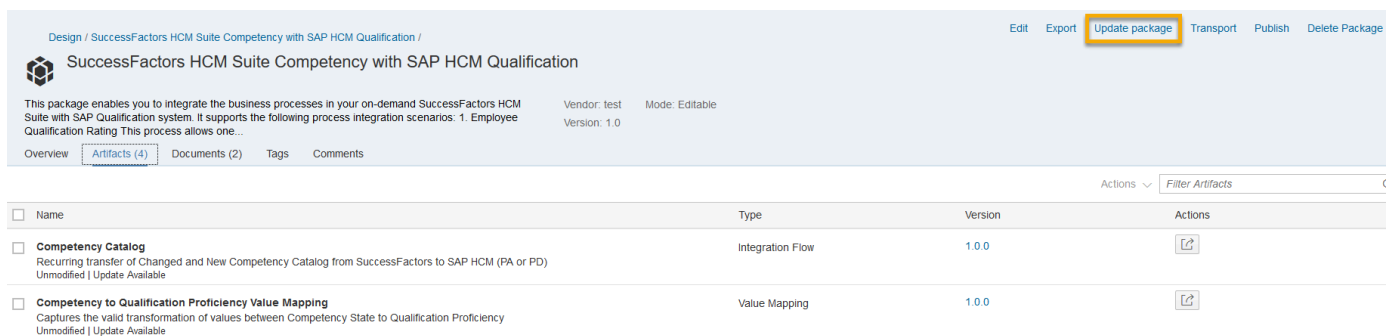
For an editable package, you can revert the underlying artifacts to an older version, regardless of the update mode. However, these artifacts are then marked as Modified and will no longer receive updates from the original content.

Manual

1. If you've copied a package into your **Design** space and an update is available, only then **Update Available** label appears next to the package name.



2. Open the package and choose **Update Package**.



i Note

If an artifact is already deployed on the tenant, and you trigger a manual update for that content package, the manual update process also updates the deployed instance of the artifact in addition to the stored instance in the **Design** view.

Automatic

If the content is set to update automatically, the update of the packages in the Design space happens in one of two ways.

Automatic Update - Immediate

The package gets updated within 12 hours. Updates are pushed immediately when the update includes important bug fixes. These packages have the label **Will be updated within 12 hrs** next to their names.

Name	Mode	Version	Created By
 SuccessFactors HCM Suite Competency with SA P HCM Qualification Will be updated within 12 hrs	Editable	1.0	

Automatic Update - Scheduled

The content of the package is set to be updated on a particular date as configured during publication. Packages that are scheduled to update automatically have the label **Will be updated on <date>**.

Name	Mode	Version	Created By
 SuccessFactors HCM Suite Competency with SA P HCM Qualification Will be updated on 21/09/2019	Editable	1.0	

You can always update the package manually even if the package is set to be updated automatically.

i Note

If an artifact is already deployed on the tenant, and an automatic update is released for that content package, the automatic update process also updates the deployed instance of the artifact in addition to the stored instance in the **Design** view.

Post Update

After successful update, the updated packages display the last modified details alongside the package names, and the design-time artifacts are replaced with the new updated content.

If the artifacts contain previously deployed content, the update procedure triggers the deployment of the new design-time content. If the deployment of updated design-time content fails, then the system retries the deployment three times with an interval of 1 hour. With every iteration, if the deployment fails, the artifact is rolled back to the previously deployed runtime version.

If this happens, your tenant contains content with a newer version of the design time, as there's no rollback of the design time content to an older version.

You need to manually deploy the newly updated design-time content. If the manual deployment also fails, contact your tenant administrator with the relevant log details of the issue.

Updates for Edited Content

We recommend to only configure the standard integration content.

In some scenarios, you can customize the content according to your needs. But once the artifact is modified, it wont receive any updates. Such artifacts appear with the grayed-out label of **Update Available**.

☐ Employee Qualification Rating One time upload of Employee Skills (PA or PD) from SAP HCM to SFSF HCM Suite - Modified the properties Modified Update Available	Integration Flow
☐ User Competency Rating Recurring transfer of Changed and New Competency Rating from SuccessFactors to SAP HCM (PA or PD) Unmodified Update Available	Integration Flow

i Note

- Package update only updates the content of the package. All attributes and configured externalized parameters of the package maintained by the user are retained.
- Updates are available even if the packages are renamed.

- A renamed artifact does no longer receive any updates.

Exporting Integration Packages

Download your integration packages to your local system, so that you can provide it to your content management team to upload and publish them. Publishing integration packages makes them available on SAP Business Accelerator Hub.

Procedure

1. Choose **Design** to view the list of integration packages.
2. Select the required integration package.
3. Choose **Export**.

By default, the current active version of the artifact gets downloaded. The downloaded file takes the name of the integration package. In case the name contains special characters, the browser changes the file name by appending them. The destination of export depends on the default browser setting.

i Note

Configure-only content can't be downloaded.

Backup and Restore Your Integration Artifacts

Context

Download all the artifacts of a package in one go, using which you can create a local backup of all artifacts. Later, you can upload the downloaded resources in another package or tenant based on your requirements.

Procedure

1. Open your integration package and choose **Edit**.

Download all artifacts

2. Choose the **Artifacts** tab.
3. Select the checkbox besides the column header **Name** to select all the integration artifacts in the package. Alternatively, you can select only the artifacts that you would like to download.
4. Choose **Actions** > **Download**.

A .zip file with filename same as the integration package's name is ready for download.

5. Save the file to your choice of destination.

→ Remember

The folder structure of the mass download file is different from the structure of an individually downloaded artifact. In a mass download, the .zip file contains individual folders for each artifact. In turn, each folder contains the resources of an artifact. But, in an individual download, the downloaded .zip file directly contains the resources of the artifact.

This information is critical to know because when you attempt to upload the downloaded resources in another package, you can't directly upload the mass-downloaded .zip file.

To upload the downloaded artifacts in another package or tenant, continue with the below steps.

6. Choose **Design**.
7. Choose **Import**.
8. Browse and select a .zip file.

→ Remember

Upload a .zip file that contains multiple .zip files for each artifact. Each .zip file of the artifact must contain the resources directly in it and not within a folder.

Develop API-Based Integration Artifacts

Develop OData API-based integration scenarios, deploy, and manage your artifacts in one place.

You can create integration artifacts of types OData APIs. The API-based artifacts generate a default integration design that you can further extend to meet your requirements. After you complete your integration design, you can save and deploy the artifacts. Finally, you can create your own tiles in the monitoring section to view your API-based artifacts that are deployed. For more information, see [SAP Cloud Integration – Step towards Building API Based Integrations](#) .

[Create and Deploy an OData API Artifact](#)

Use this procedure to create and deploy an OData API artifact.

Create and Deploy an OData API Artifact

Use this procedure to create and deploy an OData API artifact.

Context

An OData API artifact creates an OData API-based integration scenario with the OData V2 sender adapter.

Create an OData API Artifact

Procedure

1. Choose **Design**.
2. Select the integration package where you want to create an SOAP API artifact and choose **Edit**.
3. In the **Artifacts** tab, choose **Add > API**.
The **Create API** wizard opens.
4. Select **Cloud Integration** from the list of available runtime profiles. For more information, see [Runtime Profiles](#). Choose **Next**.
5. Choose **OData API > Next**.
6. In the **Select A Method** tab, choose **Create Using Template**.
By choosing **Create Using Wizard** or **Upload**, you start creating an OData API project. See [Developing an OData API Project](#).
7. Enter the name and description for the OData API.
ID and Runtime Profile are uneditable fields.
8. Choose **Create**.
The artifact is added to the integration package and the API artifact opens up in the editor.
9. Choose **Edit**.
A palette appears on top of the integration flow model that provides access to all integration flow steps that you can add to the model. See [Overview of Integration Flow Editor](#).
10. Make your changes and choose **Save**.
11. Choose **Deploy** when your artifact is ready for execution.

i Note

The URL of the OData API endpoint is derived from the name of the OData API artifact.

Next Steps

You can check the status of all your integration artifacts in the **Monitor** view. By default, API-based artifacts don't appear in the **Monitor**. Perform the steps that follow to view the deployed artifact.

View the deployed OData API Artifact

Procedure

1. Choose **Monitor**.
2. Under **Manage Integration Content**, add a new tile.
3. In the **Tile Settings** dialog, do the following:
 - a. Select **All** from the **Status** list.
 - b. Select OData API from the **Type** list.

c. Choose [OK](#) to create a tile.

4. Open the tile to view your artifact and its status.

Next Steps

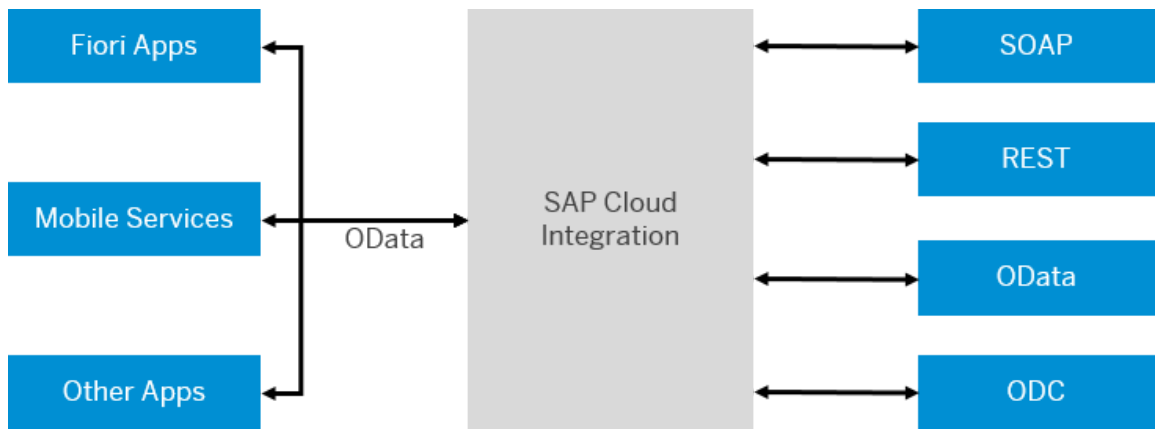
Repeat the same process to create a tile under [Monitor Message Processing](#) to view the status of your processed messages.

Developing an OData API Project

→ Remember

This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#) .

You can develop OData APIs that expose existing data sources, such as SOAP, as OData endpoints. These OData APIs can be consumed by SAP Fiori apps, SAP BTP Mobile Services, or any other custom app, to implement user-centric scenarios.



Differences Between OData API and Integration Projects

When deciding whether to use OData APIs or integration projects to implement your business scenario, keep the following differences in mind:

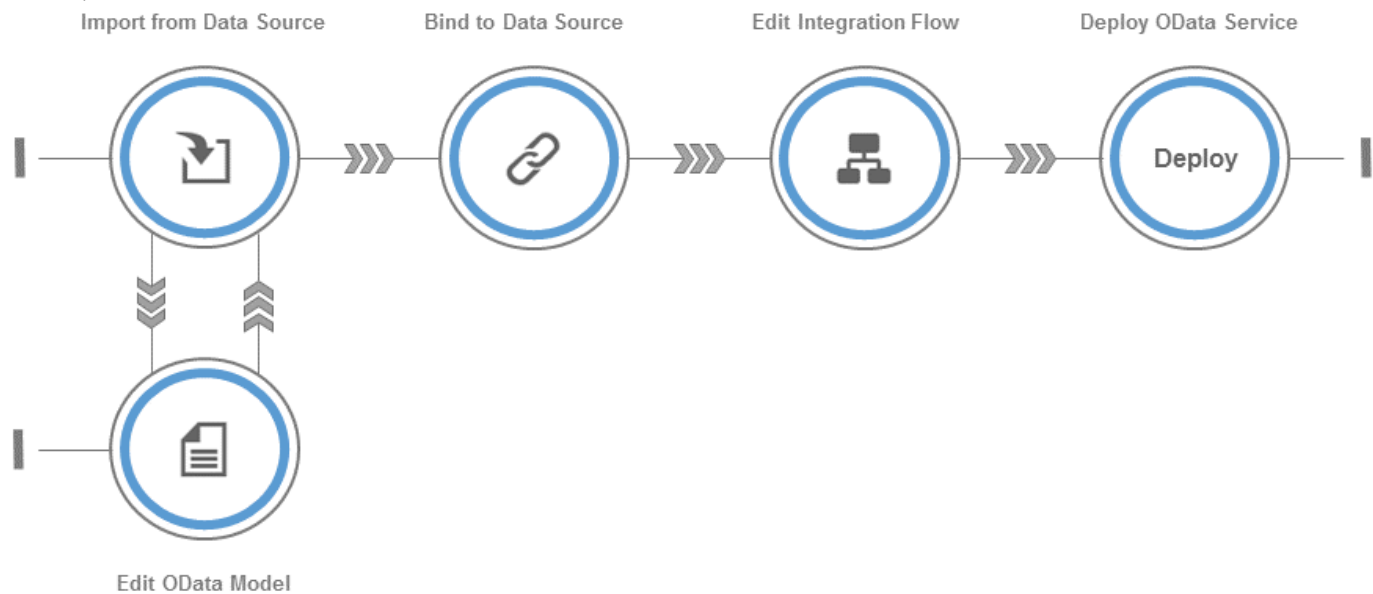
OData APIs	Integration Projects
An OData API can expose multiple entities and multiple associated operations at the same time.	An integration project can expose only one entity and one associated operation at a time.
You can configure any application to consume an OData API. As a result, any number of users with access to these applications can consume the OData API.	In an integration project, the credentials of a limited set of users are used to authenticate the integration scenario between two systems.

Service Development Process

As a service developer, you can create and deploy an OData API without going through the time-consuming process of writing code. You start off the process by creating an OData API artifact in an integration package. For more information, see [Creating an OData API](#). Cloud Integration enables you to further develop the OData API with the following steps:

1. Import an OData model using simple modeling steps. If you prefer writing code, Cloud Integration also lets you handcraft an OData model or edit an existing one using an editor.
2. Connect the OData objects to a data source through integration flows.
3. Tweak the integration flows to enhance business logic.
4. Deploy and monitor the OData API.

This image is interactive. Hover over each area for a description. Click highlighted areas for more information.



Please note that image maps are not interactive in PDF outputs.

i Note

- You can deploy OData API artifact for [SAP Cloud Integration](#) and [SAP Cloud Integration OEM](#) runtime profiles only.
- The OData API artifacts cannot be Externalized.

Authorization

[User Role](#) is the default authorization used in OData APIs. Assign the `ESBMessaging.send` role to users in order for them to access the OData APIs.

Related Information

[Supported OData Features](#)

Supported OData Features

→ **Remember**

This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#) .

The following operations are supported when developing an OData API in the Cloud Integration web application:

OData Operation	Description
Create	Maps to HTTP POST method
Read	Maps to HTTP GET method
Update	Maps to HTTP PUT method
Delete	Maps to HTTP DELETE method
Query	Maps to HTTP GET method
Function Imports	Custom operations that can be invoked through HTTP GET or POST method. i Note Currently, Cloud Integration supports only function imports that can be invoked through HTTP GET method.

The OData API supports the data types for the uploaded metadata file (EDMX) defined [here](#)  except:

- Null
- Edm.Single

The following system query options are supported when calling an OData API developed in the Cloud Integration web application:

OData System Query Option	Description
\$select	Specifies a subset of properties to return.
\$top	Determines the maximum number of records to return.
\$skip	Sets the number of records to skip before it retrieves records in a collection.
\$filter	Specifies an expression or function that must evaluate to true for a record to be returned in the collection.
\$count	Returns the number of records in a collection.
\$orderby	Determines which values are used to order a collection of records.
\$expand	Specifies that related records must be retrieved in line with the record or collection being retrieved. For example, you can use \$expand to retrieve a customer and all orders placed by that customer in a single query.
\$inlinecount	Specifies that the response must contain a number of records in the collection of entries identified by the resource path section of the URI. Message property must be an Integer value. This value is returned in the response.
\$format	Specifies that a response to the request must be in the format specified by this query option.

i Note
The system query options \$select and \$count are supported out of the box. You do not have to write a script to enable functionality for these system query options.

i Note
Server side pagination support:

To enable serverside pagination with next link in response, set the message header skipToken with a value for the skiptoken.

🔗 Example
skipToken : 20 adds the next link in response as <link href="Products?\$skiptoken=20" rel="next"/>

Related Information

[Developing an OData API Project](#)

Creating an OData API

Use this procedure to create an integration package that contains an OData API artifact.

Context

➔ **Remember**
This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#) .

Procedure

1. Choose **Design**.
2. Choose **Create**.
3. Enter the required data for the integration package. See [Creating an Integration Package](#).
4. In the **Artifacts** tab, choose  **Add**  **OData API** .
5. In the **Add OData API** dialog box, choose one of the following options:

- [Create Using Wizard](#) – to create a new OData API artifact that you can further develop based on your requirements.
- [Upload](#) – to create a new OData API artifact by uploading the necessary resources from your local file directory.

i Note

The [Create Using Template](#) option is targeted for low code API development.

6. Enter the OData API details as listed in the following table.

Field	Description
Name	Name of the OData API.
Namespace (for Create Using Wizard)	Logical grouping of the OData API. Default is <i>SAP</i> .
ID (for Create Using Wizard)	Uniquely identifies the OData API and is a combination of the name and namespace of the OData API in the format <name>_<namespace>_1. This field isn't editable.
Description	Description of the OData API and its purpose.
OData API (only for uploading the artifact)	Upload a ZIP file containing the OData API artifact.
OData Version	Currently, OData V2 is the only supported OData version.

i Note

You can edit some of the OData API details such as Name, Description, and Namespace after the service is created. For more information, see [.](#)

7. Choose [Create](#).

You can further develop the OData API by:

- [Creating an OData mode](#)
- [Binding to a data source](#)
- [Editing an Integration Flow](#)

8. Deploy the OData API. For more information, see [Deploying an OData API](#).

9. Save the integration package.

After the iFlow template generation, ensure that the Sender channel is always of type *OData*; for the other types, the tracing functionality won't capture the details of the sender.

Related Information

[Packaging Integration Content in Cloud Integration](#)

Create an OData Model

→ Remember

This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#) .

You can create an OData model in SAP Cloud Integration by:

- Importing from a data source
- Editing a model from the ground up

Cloud Integration gives you the flexibility to create an OData model in an iterative fashion. For example, you can create an OData model by importing the model definition from a SOAP WSDL file. You can add to this OData model by editing it. Or, you can add to the model definition by importing from a different WSDL file or even a different data source, for example, an EDMX file.

Related Information

[Import from Data Source](#)

[Editing an OData Model](#)

Import from Data Source

Cloud Integration allows you to create an OData model by importing the model definition from an existing data source, such as SOAP and OData.

i Note

When an OData API is generated not all attributes are copied from source metadata. However, once the OData API is generated, you can configure these attributes as needed in the OData Model Editor.

An import wizard guides you in this process with the following steps :

1. Select a data source type.
2. Select a structure.
3. Verify and finalize the EDMX structure.

Related Information

[Importing from SOAP](#)

[Importing from OData](#)

[Importing from ODC](#)

Importing from SOAP

Context

You have created an OData API artifact in an integration package. You can use this procedure to create an OData model or build up an existing one by importing the model definition from SOAP web services.

Procedure

1. If you are creating a new OData API artifact, proceed to the next step. If you are editing an existing OData API artifact, choose [Edit](#) on the Service Designer page.
2. Choose [Import Model Wizard](#) (🔗).
3. In the [Select Data Source Type](#) page, select [SOAP](#) as data source type.
4. Choose [Browse](#) and select the WSDL file from your file system. You can also select the WSDL from the list of existing WSDL files (if any) used in the OData API artifact.

i Note

Composite WSDLs that are flattened are supported. However, multipart WSDLs and other file types are not supported.

5. Choose [Step 2](#). In the [Select Structure](#) page, you can see the elements from the WSDL listed in a table.
6. Select elements that you want to add to the OData model from the [WSDL Hierarchy](#) column. You can expand or collapse WSDL elements to view the hierarchy better.
7. Deselect any child element that you do not want to include in the OData model. You can see that the corresponding parent elements are in a mixed state (neither checked nor unchecked), if they have other child elements that are selected.

i Note

While importing from any data source using the [Import Wizard](#), if the element is of complex type containing a single child element of simple type, the option to deselect only the child element is not supported. In such a scenario, you can select [Flatten](#) from the [Import As](#) dropdown for the parent WSDL element. After completing the [Import Wizard](#), navigate to the [OData Model Editor](#) and modify the structure as required.

8. Select the OData type that each WSDL element has to be converted to in the OData model. To see options available for various types of WSDL elements, refer the following table:

WSDL Type	OData Type
Operation	Complex Type
	Entity Type
	Function Import
Complex Type	Complex Type
Input Message	Complex Type

WSDL Type	OData Type
Output Message	Complex Type
Part	Complex Type

i Note

You can select **Flatten** from the **Import As** dropdown for all WSDL types other than **Operation**. **Flatten** is not an OData type that a WSDL element can be converted to. It is an instruction to the system to skip importing the specified WSDL element into the OData model without affecting the import of its children in the hierarchy.

9. Choose **Step 3**. You can review the mapping of WSDL elements to OData objects in this step.
10. In the **Review and Finish EDMX Structure** page, select a primary key from the properties of entity types.
11. Verify and perform the following additional changes, if required, to the OData model definition:
 - Rename the OData objects listed under **EDM Name**.
 - Select a different EDM type.
 - Update the **Documentation** column.
 - Restore default definition of the OData model by choosing **Restore Default Values**.
12. Once you have completed all the changes, choose **Finish**. The Service Designer page displays the entity sets and function imports from the OData model in a table.
13. Choose **Save** to save your changes to the OData API artifact.

i Note

To retain a copy of the current artifact, choose **Save as version**.

Importing from OData

Context

You have created an OData API artifact in an integration package. You can use this procedure to create an OData model or build up an existing one by importing the model definition from an existing OData API.

Procedure

1. If you are creating a new OData API artifact, proceed to the next step. If you are editing an existing OData API artifact, choose **Edit** on the Service Designer page.
2. Choose  (*Import Model Wizard*).
3. In the **Select Data Source Type** page, select **ODATA** as data source type.
4. Choose **Browse** and select the EDMX file from your file system. You can also select the EDMX from the list of existing EDMX files (if any) used in the OData API artifact.

metadata.edmx is the file used internally in Cloud Integration Web application to store the OData model definition of the OData API artifact.

i Note

- The OData Model Editor supports only OData SAP V2 annotations.
- Only OData models up to 300 KB in size are supported.

! Restriction

Importing of EDMX files with navigation entities are not supported and also there is size restrictions on EDMX files with large number of complex entities. For more information, see the KBA [2844323](#)  and read the blog on [When to use an OData API](#) .

5. Choose **Step 2**. In the **Select Structure** page, you can see entity types and function imports from the EDMX listed in a table. The table also displays the EDM type, primary key and documentation information associated with the OData objects.

i Note

While importing from any data source using the **Import Wizard**, if the element is of complex type containing a single child element of simple type, the option to deselect only the child element is not supported. In such a scenario, you can select **Flatten** from the **Import As** dropdown for the parent WSDL element. After completing the **Import Wizard**, navigate to the **OData Model Editor** and modify the structure as required.

- 6. Select OData objects that you need to add to the model from the [OData Hierarchy](#) column. You can expand or collapse OData objects to view its properties better.
- 7. Choose [Step 3](#).
- 8. In the [Review and Finish EDMX Structure](#) page, verify and perform the following additional changes, if required, to the OData model definition:
 - Rename the OData objects listed under [EDM Name](#).
 - Select another primary key.
 - Select a different EDM type.
 - Update the [Documentation](#) column.
 - Restore default definition of the OData model by choosing [Restore Default Values](#).
- 9. Once you have completed all the changes, choose [Finish](#). The Service Designer page displays the entity sets and function imports from the OData model in a table.
- 10. Choose [Save](#) to save your changes to the OData API artifact.

i Note

To retain a copy of the current artifact, choose [Save as version](#).

Importing from ODC

Prerequisites

You have created an OData API artifact in an integration package.

Context

SAP Cloud Integration helps you import a model definition from an OData API, which is created on an on-premise SAP Gateway system. SAP Gateway has a component called Backend Provisioning Component or IW_BEP. This component resides on SAP Business Suite and is used to build OData APIs from data sources like RFCs, BAPIs, and so on. SAP Cloud Integration communicates with the IW_BEP component in SAP Gateway through OData Channel or ODC protocol, which is an SAP proprietary protocol.

i Note

SAP Cloud Integration does not support connecting to an SAP Gateway system based on SAP NetWeaver 7.02 where the IW_BEP component version is SP 06 or lower.

SAP Cloud Integration also does not support connecting to an SAP Gateway system based on SAP NetWeaver 7.40 where the SAP_GWFND component is SP 02 or lower.

You can use this procedure to create an OData model or build up an existing one by importing the model definition from an OData API created in IW_BEP (ODC).

Procedure

- 1. If you are creating a new OData API artifact, proceed to the next step. If you are editing an existing OData API artifact, choose [Edit](#) on the Service Designer page.
- 2. Choose [Import Model Wizard](#) (.
- 3. In the [Select Data Source Type](#) page, select [ODC](#) as data source type.
- 4. Enter the connection details as listed in the following table.

Connection Details	Description
Address	Endpoint URI of the target system
Client	Client of the target system
Namespace	Logical grouping of the service implementation
Service Name	Name of the service implementation
Version	Version of the service implementation
Proxy Type	The type of proxy that you are using to connect to the target system:

Connection Details	Description
	Select On-Premise if you are connecting to an on-premise system using the SAP Cloud Connector. This option is only available if you have chosen Cloud Integration as the runtime profile. If you select this option, the Address field of the adapter refers to a virtual address that has to be configured in the SAP Cloud Connector settings.
Authentication	Select Basic if you want to use user credentials to connect to the target system.
Credential Name	Name of the user credentials artifact deployed on the tenant

5. Choose **Step 2**.

i Note

While importing from any data source using the **Import Wizard**, if the element is of complex type containing a single child element of simple type, the option to deselect only the child element is not supported. In such a scenario, you can select **Flatten** from the **Import As** dropdown for the parent WSDL element. After completing the **Import Wizard**, navigate to the **OData Model Editor** and modify the structure as required.

In the **Select Structure** page, you can see entity types and function imports from the EDMX listed in a table. The table also displays the associated EDM type, primary key, and documentation information.

6. Select entity types and function imports that you need to add to the model from the **OData Hierarchy** column. You can expand or collapse entity types and function imports to view its properties better.

7. Choose **Step 3**.

8. In the **Review and Finish EDMX Structure** page, verify and perform the following additional changes, if required, to the OData model definition:

- o Rename the OData objects listed under **EDM Name**.
- o Select a different EDM type.
- o Select another primary key.
- o Update the **Documentation** column.
- o Restore default definition of the OData model by choosing **Restore Default Values**.

9. Once you have completed all the changes, choose **Finish**. The Service Designer page displays the entity sets and function imports from the OData model in a table.

10. Choose **Save** to save your changes to the OData API artifact.

i Note

To retain a copy of the current artifact, choose **Save as version**.

Editing an OData Model

Context

You have created an integration package and added an OData API artifact to the package.

You can use this procedure to create an OData model from the ground up. You can also use it to edit an existing OData model.

Procedure

1. If you are creating a new OData API artifact, proceed to the next step. If you are editing an existing OData API artifact, choose **Edit** on the Service Designer page.
2. Choose **OData Model Editor** (🔗) on the Service Designer page.

If you are creating an OData model from the ground up, the OData Model Editor opens a predefined structure based on the selected OData version. The root is a **Schema** element. Below the root, subelements such as **EntityType**, **ComplexType**, **Association**, **EntityContainer**, and so on, can be added if required.

i Note

- o In the case of an existing OData API artifact, you can only view the OData model in the OData Model Editor unless you explicitly choose **Edit** on the Service Designer page.

For more information on the features of the OData Model Editor, see [OData Model Editor Features](#).

- o Only OData models up to 300 KB in size are supported.

3. Once you have finished editing the OData model, choose **OK**.
4. Choose **Back** (↩) to get back to the Service Designer page.
5. Choose **Save** to save your changes to the OData API artifact.

i Note

To retain a copy of the current artifact, choose **Save as version**.

OData Model Editor Features

The OData Model Editor in the Cloud Integration Web application enables you to write an Entity Data Model XML (EDMX). This model serves as metadata for your OData API. OData Model Editor provides you with the following basic features:

- **Code Assist:** Provides you with code suggestions that help speed up your development time. There are two types of code assist:
 - o Schema-based
 - o Model-based
- **Autocomplete:** Predicts the code element you are entering based on the pattern of letters entered.

i Note

You can use the OData Model Editor to edit and fine-tune your OData model, after you have imported it from a data source. However, we recommend that you keep the overall end-to-end scenario in mind when you edit the model using this tool. The OData Model Editor can only provide **semantic** validations. It cannot validate the correctness of the OData model in the context of the OData provisioning scenario that you are working in.

Related Information

[Using Schema-Based Code Assist](#)

[Using Model-Based Code Assist](#)

[Using Autocomplete](#)

[Validation in the OData Model Editor](#)

Using Schema-Based Code Assist

Context

The schema-based code assist feature provides you with code suggestions that adhere to the Conceptual Schema Definition Language (CSDL) specification. These code suggestions are provided based on the current cursor position.

Follow this procedure to use schema-based code assist in the OData Model Editor.

Procedure

1. Choose **OData Model Editor** (🔗) on the Service Designer page. The OData Model Editor page opens.
2. Place your cursor in the context where you require assistance, and press **Ctrl** + **Spacebar**.

Based on the cursor location, a list of the elements that can be used appears.

Each element in the dropdown list is represented by an icon. The following image shows the list of elements and their corresponding icons:

Icons	Elements
	Property Ref, Return Type, On Delete, Association Set
	Association
	Complex Type
	Entity Container
	Entity Type
	Entity Set
	Function
	Using
	Documentation
	End
	Referential Constraint
	Parameter
	Navigation Property
	Key
	Function Import
	Property
	Dependent
	Default image

3. Scroll through the list, select the required element, and press **Enter**. The OData Model Editor inserts a full snippet of code in the desired location.

Using Model-Based Code Assist

Context

The model-based code assist feature provides code suggestions based on the OData model you are working on.

Follow this procedure to use model-based code assist in the OData Model Editor.

Procedure

1. Choose **OData Model Editor** () on the Service Designer page. The OData Model Editor page opens.
2. Place the cursor between the quotes of the attribute of an element such as PropertyRef, EntitySet, or AssociationSet and press **Ctrl** + **Spacebar** to see the relevant suggestions.
3. Scroll through the list and select the required suggestion.

Using Autocomplete

Context

The autocomplete feature predicts the word you are entering based on the pattern of letters entered and provides you with suggestions. The suggestions are context-sensitive. You can get suggestions for elements, attributes, attribute types, properties, and property values.

Follow this procedure to use the autocomplete feature in the OData Model Editor.

Procedure

1. Choose **OData Model Editor** () on the Service Designer page. The OData Model Editor page opens.
2. Place the cursor in the context where you need assistance and start entering letters. Autocomplete suggestions appear next to the letters in context.
3. Use the **Up arrow** and **Down arrow** keys on the keyboard to select the required suggestions and press **Enter**.

i Note

Use the arrow keys if there is more than one suggestion. If there is only one suggestion, simply press **Enter**.

Validation in the OData Model Editor

Validation in the OData Model Editor ensures that your OData model adheres to the CSDL schema and does not deviate from logical or semantic specifications.

After validation, lines of code with errors are highlighted with a red icon in the code editor.

Let us look at an example of how validation works.

Example

In any OData model, remove the value corresponding to the **Name** attribute in an **EntityContainer** element.

Since **Name** is a mandatory attribute, the corresponding line of code is highlighted with a red icon in the code editor. You can see the error message in a tooltip when you hover the mouse over the red icon.

Viewing the OData Model Layout

Context

→ Remember

This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#)



You have created an OData model either by importing from a data source or by editing the model manually using the Cloud Integration Web application.

You can use this procedure to view the overall layout of an OData model in the Graphical Model Viewer. This helps you verify the OData model and determine if any additional changes are necessary.

Procedure

1. Choose **Graphical Model Viewer** (🔍) on the Service Designer page. The OData model is rendered in a graphical format.
You can use the navigation control and viewing options to customize your viewing preferences. For more information, see [Graphical Model Viewer Features](#).
2. Choose **Back** (⬅) to get back to the Service Designer page.

Graphical Model Viewer Features

The Graphical Model Viewer in the Cloud Integration Web application provides you with the following navigation controls and viewing options to customize your viewing preferences:

Fit to Screen

There might be instances where the OData model objects of a big model extend beyond the scope of the Graphical Model Viewer space. In such cases, you can choose **Fit to Screen** to view the entire model within the boundaries of the Graphical Model Viewer. This feature automatically aligns the model to fit the Graphical Model Viewer space.

Expanding and Collapsing OData Objects

The Graphical Model Viewer lets you expand or collapse all the section headers of the objects in the OData model. You can also expand and collapse individual OData model objects.

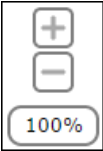
Collapsed State

- To collapse all OData objects in the Graphical Model Viewer, choose the **Collapse All** button on the toolbar. The **Collapse All** button toggles to **Expand All** once the model is collapsed.
- If you want to collapse an individual OData object, choose the arrowhead button in the expanded object.

Expanded State

- To expand all OData objects in the Graphical Model Viewer, choose the **Expand All** button on the toolbar. The **Expand All** button toggles to **Collapse All** once the model is expanded.
- If you want to expand an individual OData object, choose the arrowhead button in the collapsed object.

Depending on the number of OData objects in the model, objects might appear too small or be too big for the Graphical Model Viewer to accommodate. In such cases, the navigation controls help you view the model better.

Button	Usage
	Use the compass to control the perspective of the model: - Choose  to move right. - Choose  to move left. - Choose  to move down. - Choose  to move up. - Choose  to zoom the model to 100%.
	- Choose  to zoom in to the model. - Choose  to zoom out of the model. - The zoom level indicator  shows the zoom percentage.

Bind to Data Source

→ Remember

This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#) .

You have created an OData model in the Cloud Integration Web application. As a result, you can see OData objects associated with the model listed in a table on the Service Designer page.

Next, you need to bind or connect function imports and operations of entity sets to a data source. When you bind an OData object to a data source, a predefined integration flow is autogenerated and set up for you. You can modify this integration flow to suit your scenario.

i Note

The externalise option is not visible for these autogenerated integration flows.

You can bind one or all five CRUDQ operations of an entity set to a data source. This means that each entity set can have multiple integration flows connecting to the same data source. You can also bind entity sets and function imports in an OData API artifact to different data sources.

The following data sources are currently supported:

- SOAP
- OData
- REST
- ODC

Binding Status

To identify the graphical representations of the OData objects and their binding status on the Service Designer page, refer to the following table:

OData Object	Description
	Represents an entity set in an OData model.
	Represents an operation associated with an entity set in an OData model.
	Represents an operation bound to a data source with errors in the associated integration flow. You cannot deploy the OData API unless the errors are corrected.

OData Object	Description
	Represents an operation bound to a data source with warnings in the associated integration flow. We recommend that you verify the warnings and take corrective measures where necessary before deploying the OData API.
	Represents an operation bound to a data source with no errors or warnings.
	Represents a function import in an OData model.
	Represents a function import bound to a data source with errors in the associated integration flow. You cannot deploy the OData API unless the errors are corrected.
	Represents a function import bound to a data source with warnings in the associated integration flow. We recommend that you verify the warnings and take corrective measures where necessary before deploying the OData API.
	Represents a function import bound to a data source with no errors or warnings.

Unused Bindings

After binding an entity set or function import to a data source, if you rename or delete it in the OData Model Editor, the binding is no longer associated with it and becomes unused. You cannot deploy the OData API unless you update the binding. For more information, see [Managing Unused Bindings](#).

Related Information

- [Binding to SOAP](#)
- [Binding to OData](#)
- [Binding to REST](#)
- [Binding to ODC](#)

Binding to SOAP

Prerequisites

You are editing an OData API artifact and you have created an OData model. OData objects in the model are listed in a table on the Service Designer page.

Context

You can use this procedure to bind function imports and operations of entity sets to the corresponding SOAP Web service operations.

Procedure

1. Select **SOAP** as the data source for an entity set or function import.
2. Choose **Bind** () for the operation associated with the entity set or function import that you need to bind. The **Configure SOAP Data Source** dialog box appears.
3. Choose **Browse** and select the WSDL file from your file location. If the OData model was imported from a WSDL file, the corresponding file name appears by default. You can change this to any other file name as required.

i Note

Composite WSDLs that are flattened are supported. However, multipart WSDLs and other file types are not supported.

4. Select an operation. If the OData model was imported from a WSDL file, operations defined in the WSDL appear in the dropdown list.
5. Enter the Endpoint URI where the operation is located. If the OData model was imported from a WSDL file and an operation was selected from this WSDL, the corresponding Endpoint URI is auto-populated.
6. Choose **OK**. The Service Designer page appears and the **Action** column for this operation now displays **Navigate to Integration Flow Editor** () and **Delete Binding** () buttons.

A predefined integration flow with mapping steps is generated for the operation. For more information, see [Predefined Integration Flows for SOAP](#).

You can update the integration flow to better suit your business scenario by choosing **Navigate to Integration Flow Editor** (). For more information, see [Editing an Integration Flow](#).

In case you need to update the binding information, you can do it in one of the following ways:

- Choose **Navigate to Integration Flow Editor** () and update the Endpoint URI information in the receiver channel.
- Choose **Delete Binding** () and repeat the binding procedure.

7. Choose **Save** to save your changes to the OData API artifact.

i Note

To retain a copy of the current artifact, choose **Save as version**.

Binding to OData

Prerequisites

You are editing an OData API artifact and you have created an OData model. OData objects in the model are listed in a table on the Service Designer page.

Context

You can use this procedure to bind function imports and operations of entity sets to the corresponding OData APIs.

Procedure

1. Select **ODATA** as the data source for an entity set or function import.
2. Choose **Bind** (🔗) for the function import or operation associated with the entity set that you need to bind. The **Configure OData Data Source** dialog box appears.
3. Choose **Browse** and select the EDMX file from your file location. You can also select the file from the **Existing Files** list. The **Existing Files** list is populated with EDMX file names used in other bindings in the same OData API artifact.
4. Select an entity set from the **Entity Sets** dropdown list. This list displays all entity sets from the selected EDMX file.
5. Enter the Endpoint URI corresponding to the OData API that you are binding to.
6. Choose **OK**. The Service Designer page appears and the **Action** column for this operation now displays **Navigate to Integration Flow Editor** (🔗) and **Delete Binding** (🗑️) buttons.

A predefined integration flow with mapping steps is generated for the operation or function import. For more information, see [Predefined Integration Flows for OData](#).

You can update the integration flow to better suit your business scenario by choosing **Navigate to Integration Flow Editor** (🔗). For more information, see [Editing an Integration Flow](#).

In case you need to update the binding information, you can do it in one of the following ways:

- Choose **Navigate to Integration Flow Editor** (🔗) and update the Endpoint URI information in the receiver channel.
 - Choose **Delete Binding** (🗑️) and repeat the binding procedure.
7. Choose **Save** to save your changes to the OData API artifact.

i Note

To retain a copy of the current artifact, choose **Save as version**.

Binding to REST

Prerequisites

You are editing an OData API artifact and you have created an OData model. OData objects in the model are listed in a table on the Service Designer page.

Context

You can use this procedure to bind function imports and operations of entity sets to the corresponding REST services.

Procedure

1. Select **REST** as the data source for an entity set or function import.
2. Choose **Bind** (🔗) for the function import or operation associated with the entity set that you need to bind. The **Configure REST Data Source** dialog box appears.
3. Enter the Endpoint URI corresponding to the REST service that you are binding to.
4. Choose **OK**. The Service Designer page appears and the **Action** column for this operation or function import now displays **Navigate to Integration Flow Editor** (🔗) and **Delete Binding** (🗑️) buttons.

A predefined integration flow with mapping steps is generated for the operation or function import. For more information, see [Predefined Integration Flows for REST](#).

You can update the integration flow to better suit your business scenario by choosing [Navigate to Integration Flow Editor](#) (🔗). For more information, see [Editing an Integration Flow](#).

In case you need to update the binding information, you can do it in one of the following ways:

- Choose [Navigate to Integration Flow Editor](#) (🔗) and update the Endpoint URI information in the receiver channel.
- Choose [Delete Binding](#) (🗑️) and repeat the binding procedure.

5. Choose [Save](#) to save your changes to the OData API artifact.

i Note

To retain a copy of the current artifact, choose [Save as version](#).

Binding to ODC

Prerequisites

You are editing an OData API artifact and you have created an OData model. OData objects in the model are listed in a table on the Service Designer page.

Context

Cloud Integration helps you bind or connect to an OData API OData API, which is created on an on-premise SAP Gateway system. SAP Gateway has a component called Backend Provisioning Component or IW_BEP. This component resides on SAP Business Suite and is used to build OData APIs from data sources like RFCs, BAPIs, and so on. Cloud Integration communicates with the IW_BEP component in SAP Gateway through OData Channel or ODC protocol, which is an SAP proprietary protocol.

i Note

Cloud Integration does not support connecting to an SAP Gateway system based on SAP NetWeaver 7.02 where the IW_BEP component version is SP 06 or lower.

Cloud Integration also does not support connecting to an SAP Gateway system based on SAP NetWeaver 7.40 where the SAP_GWFND component is SP 02 or lower.

You can use this procedure to bind function imports and operations of entity sets to the corresponding OData APIs created in IW_BEP (ODC).

Procedure

1. Select **ODC** as the data source for an entity set or function import.
2. Choose [Bind](#) (🔗) for the function import or operation associated with the entity set that you want to bind.
The [Configure ODC Data Source](#) dialog box appears. The entity set and CRUDQ operation information is prepopulated.
3. Enter the following information related to the ODC destination.

Connection Details	Description
Entity Sets	A list of entity sets that is populated from the ODC source when importing the OData model. If you have coded the OData model, then the list is populated directly from your model. Select an entity set that you want to bind to.
Address	Endpoint URI of the target system
Client	Client of the target system
Namespace	Logical grouping of the service implementation
Service Name	Name of the service implementation
Version	Version of the service implementation
Proxy Type	The type of proxy that you are using to connect to the target system: ◦ Select On-Premise if you are connecting to an on-premise system using the SAP Cloud Connector.

Connection Details	Description
	This option is only available if you have chosen <i>HANA Cloud Integration</i> as the product profile. If you select this option, the Address field of the adapter refers to a virtual address, which has to be configured in the SAP Cloud Connector settings.
Authentication	Select Basic if you want to use user credentials to connect to the target system.
Credential Name	Name of the user credentials artifact deployed on the tenant

4. Choose **OK**. The Service Designer page appears and the **Action** column for this operation now displays **Navigate to Integration Flow Editor** (🔗) and **Delete Binding** (🗑️) buttons.

A predefined integration flow with mapping steps is generated for the operation or function import. For more information, see [Predefined Integration Flows for ODC](#).

You can update the integration flow to better suit your business scenario by choosing **Navigate to Integration Flow Editor** (🔗). For more information, see [Editing an Integration Flow](#).

If you want to update the binding information, you can do it in one of the following ways:

- Choose **Navigate to Integration Flow Editor** (🔗) and update the Endpoint URI information in the receiver channel.
- Choose **Delete Binding** (🗑️) and repeat the binding procedure.

5. Choose **Save** to save your changes to the OData API artifact.

i Note

To retain a copy of the current artifact, choose **Save as version**.

Managing Unused Bindings

Context

→ Remember

This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#).



You are editing an OData API artifact and you have bound a function import or operations of an entity set to a data source. At this point, let us assume you changed the name of the function import or entity set, or even deleted it in the OData Model Editor. The binding associated with the renamed or deleted entity set or function import is now unused.

The unused bindings appear in a separate table on the Service Designer page. You cannot deploy the OData API unless you update the unused bindings.

Cloud Integration Web application helps you manage unused bindings by providing you with the following options:

Reconfigure Binding

Context

You can use this procedure to reconfigure the binding from the earlier entity set or function import to one in the updated OData model.

Procedure

- Choose **Bind** (🔗) for the entity set or function import associated with the unused binding. The **Reconfigure Binding** dialog box appears.
- Select the entity set or function import in the updated OData model to which the unused binding needs to be associated.
- Choose **OK**. The selected entity or function import in the updated OData model is now associated with the binding. The **Unused Bindings** table disappears if there are no other unused bindings.

SAP Cloud Integration updates the sender channel with the new binding information. However, you have to update other integration flow components that references the earlier entity set or function import, such as mapping and scripts.

- Choose **Save** to save your changes to the OData API artifact.

i Note

Delete Binding

Context

You can use this procedure to delete the unused binding so that it is no longer associated with any entity set or function import in the OData model.

Procedure

1. Choose **Delete Binding** (🗑️) for the associated entity set or function import.
2. Confirm the deletion. The **Unused Bindings** table disappears if there are no other unused bindings.
3. Choose **Save** to save your changes to the OData API artifact.

i Note

To retain a copy of the current artifact, choose [Save as version](#).

Editing an Integration Flow

Context

→ Remember

This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#)



You are editing an OData service artifact and you have created an OData model. You have provided binding information for at least one OData object in the model.

Once a function import or operation of an entity set is bound to a data source, a predefined integration flow is set up for you. Within this integration flow, the OData adapter is selected as the default for the sender channel. The data source to which the operation is bound is selected as the default adapter for the receiver channel.

You can use this procedure to update the predefined integration flow to suit your business scenario.

Procedure

1. Choose **Navigate to Integration Flow Editor** (🔗) for an operation or function import that is bound to a data source. The associated integration flow opens in the Integration Flow Editor.
2. Edit the integration flow according to your requirements.

i Note

The following points need to be kept in mind when editing integration flows in an OData service artifact:

- The OData sender channel is not editable.
- The OData sender channel, participant and start event cannot be deleted.
- A new sender participant cannot be dragged and dropped into the integration flow.
- If you modify the default name of the receiver adapter, you need to make sure it is unique across all integration flows in an OData service artifact.
- If your service requires [Session Handling](#), ensure that session handling is enabled for each integration flow in your OData service project.
- You need to ensure that the Namespace Mapping entered in the runtime configuration, is consistent across all integration flows in an OData service artifact.

For example, you can enter the namespace-prefix pair `xmlns:test=http://sapcd.com/testABC` in multiple integration flows in an OData service artifact. You cannot enter `xmlns:test=http://sapcd.com/testABC` in one integration flow and enter `xmlns:test=http://sapab.com` in another integration flow in the same OData service artifact. However, it is perfectly valid to enter `xmlns:test=http://sapcd.com/testABC` in one integration flow and `xmlns:test2=http://sapab.com` in another.

- You can add other receiver/outbound adapters in the integration flow, but ensure that one of the following outbound adapters is present in the integration flow:
 - OData V2
 - ODC

- HTTPS
- SOAP

For more information about configuring integration flow components, see [Configure Integration Flow Components](#).

- Once you have finished editing the integration flow, choose **OK**.
- Choose **Back** (↩) to get back to the Service Designer page.
- Choose **Save** to save your changes to the OData service artifact.

i Note

To retain a copy of the current artifact, choose **Save as version**.

Related Information

[Predefined Integration Flows](#)

Predefined Integration Flows

The Cloud Integration Web application sets up predefined integration flows when OData objects are bound to a data source. These integration flows have the sender channel preconfigured to the OData adapter type.

A predefined integration flow has the minimum setup required for an integration scenario. You have to configure this integration flow to suit the specific requirements of your scenario.

Related Information

[Predefined Integration Flows for SOAP](#)

[Predefined Integration Flows for OData](#)

[Predefined Integration Flows for REST](#)

[Predefined Integration Flows for ODC](#)

Predefined Integration Flows for SOAP

The Cloud Integration Web application sets up predefined integration flows when the CRUDQ (Create, Read, Update, Delete, and Query) operations of entity sets and function imports are bound to a SOAP Web service.

Related Information

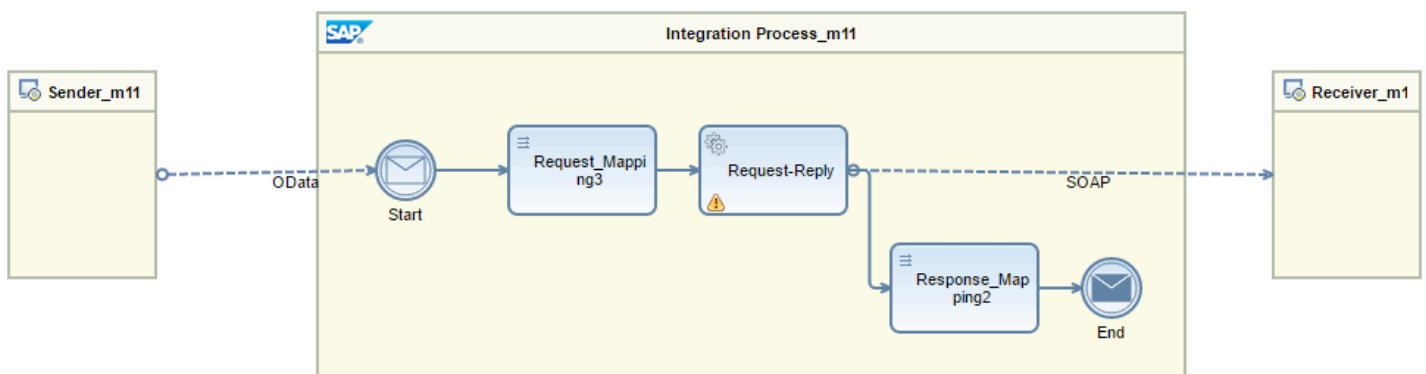
[Predefined Integration Flows for Create and Read Operations](#)

[Predefined Integration Flows for Update and Delete Operations](#)

[Predefined Integration Flow for Query Operations](#)

[Predefined Integration Flow for Function Imports](#)

Predefined Integration Flows for Create and Read Operations



This integration flow is autogenerated for Create and Read operations. In order to complete the binding to the data source, you have to edit at least the following elements:

- Request Mapping
- Response Mapping

To understand this integration flow, let's look at an example for the Create operation.

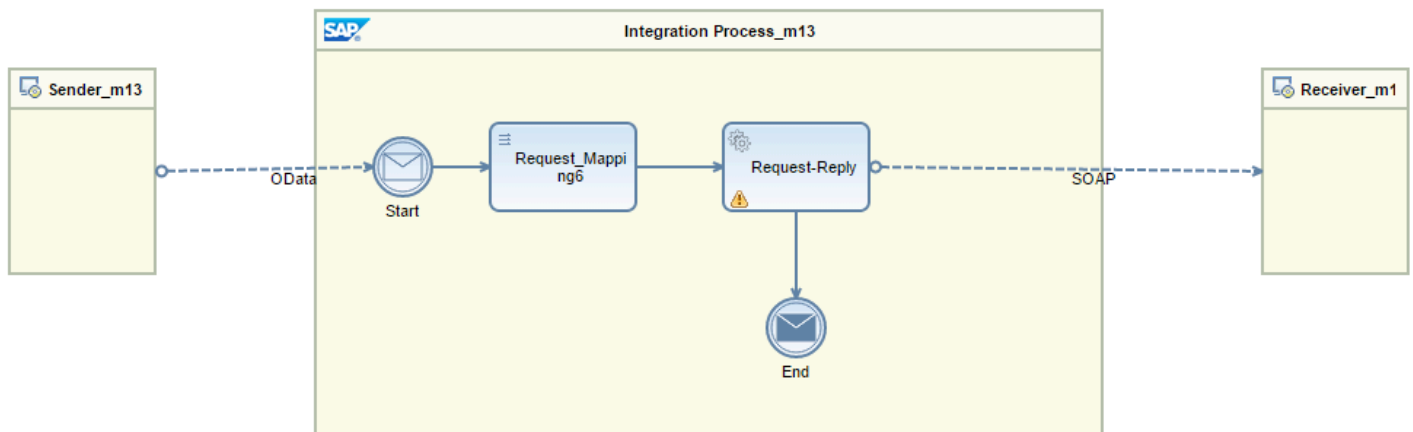
Example

You open an app on your smartphone to book a hotel room in the city of New York. This application sends the booking information to a hotel booking system. The hotel booking system receives this data with the help of a SOAP Web service, stores it, and sends the booking status back to you.

The mobile application communicates with the hotel booking system by calling a URI to create a booking record and passes the booking information as the payload. When the OData sender receives the request, it passes it to the Request Mapping element. This mapping element contains information that maps the fields from the OData structure to the SOAP structure and passes the create request to the SOAP receiver.

The SOAP receiver processes the request and creates a booking record in the back end. If the operation is successful, it returns the status to the Response Mapping element. The Response Mapping element contains information that maps the fields from the SOAP structure to the OData structure. It passes the booking status and other mapped details such as the confirmation number to the OData sender. OData sender in turn passes the response to the mobile application.

Predefined Integration Flows for Update and Delete Operations



This integration flow is autogenerated for Update and Delete operations. In order to complete the binding to the data source, you have to edit at least the Request Mapping element.

To understand this integration flow, let's look at an example of an Update operation.

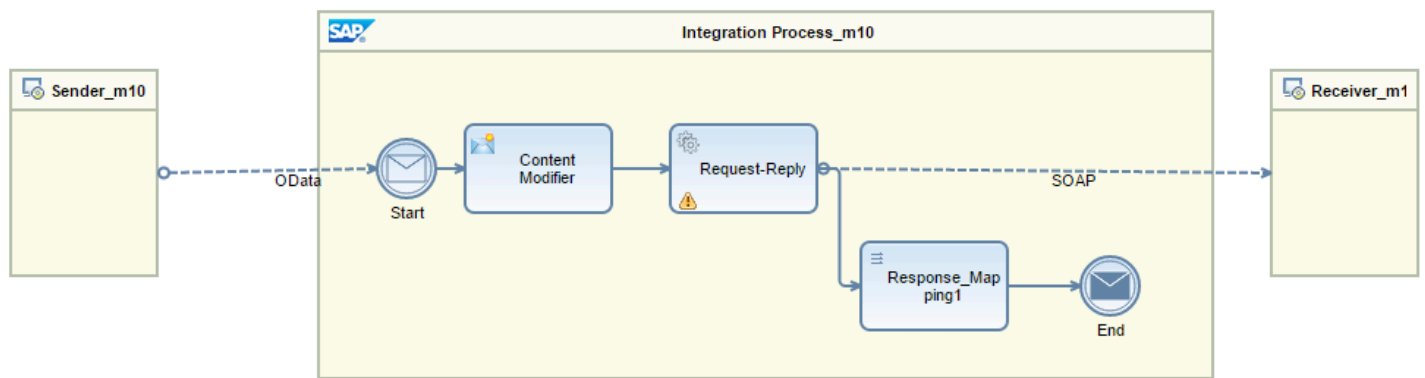
Example

You open an app on your smartphone to extend your stay in a hotel room in the city of New York. This application sends the booking information to a hotel booking system. The hotel booking system receives this data with the help of a SOAP Web service and updates it.

The mobile application communicates with the hotel booking system by calling a URI to update a booking record and passes the booking information as the payload. When the OData sender receives the request, it passes it to the Request Mapping element. This mapping element contains information that maps the fields from the OData structure to the SOAP structure and passes the update request to the SOAP receiver.

The SOAP receiver processes the request and updates the booking record in the back-end. The status of the update operation is then sent to the OData sender. The OData sender in turn sends it to the mobile application.

Predefined Integration Flow for Query Operations



This integration flow is autogenerated for Query operations. In order to complete the binding to the data source, you have to edit at least the following elements:

- Content Modifier
- Response Mapping

Note

If you want to enable system query options such as \$top, \$skip, and \$filter, you can replace the Content Modifier element with a Script element.

To understand this integration flow, let's look at an example.

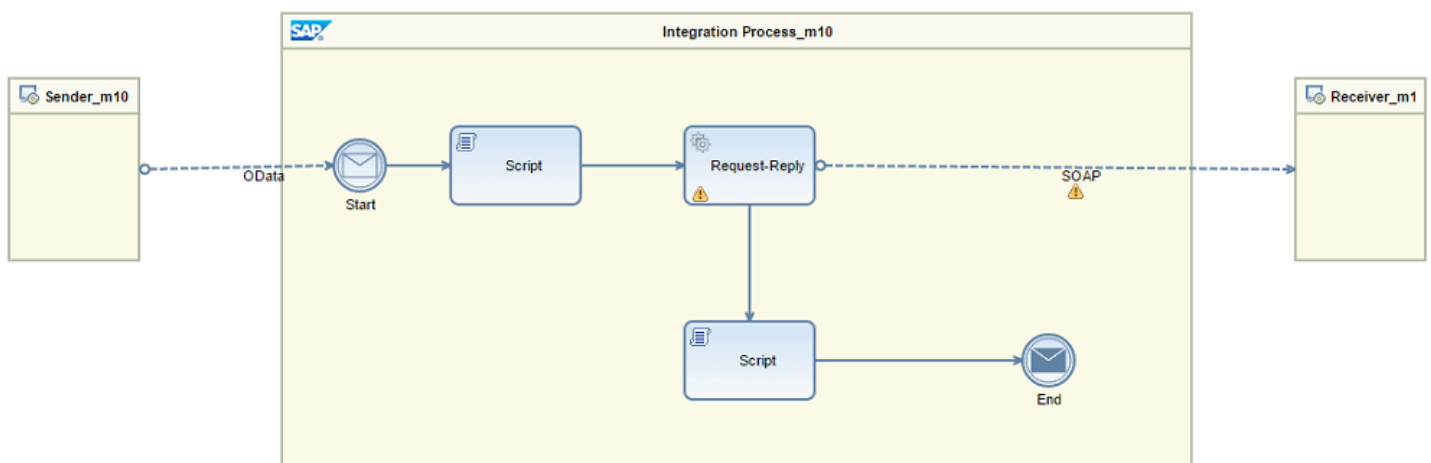
Example

A mobile application needs to get a list of hotels available in the city of New York from a hotel booking system. The hotel booking system publishes this data in the form of a SOAP Web service. The mobile application places this request in the form of a URI, where the city of New York is passed as a parameter.

When the OData sender channel receives the URI request, it passes it to the Content Modifier element. The Content Modifier element adds additional information to the head and body of the incoming message so that the SOAP receiver can understand it.

The SOAP receiver processes the request and, if there are no issues with the request, returns a list of hotels as the response. The Response Mapping element contains information that maps the fields from the SOAP structure to the OData structure. The mapped data is then sent to the mobile application through the OData sender channel.

Predefined Integration Flow for Function Imports



This integration flow is autogenerated for function imports. In order to complete the binding to the SOAP Web service, you have to edit at least the Script elements for request and response mapping.

To understand this integration flow, let's look at an example.

Example

You open an app on your smartphone to check the most frequently visited hotel in the city of New York and the number of guests that visited this hotel in the previous year. The hotel booking system publishes this data in the form of a SOAP Web service. The mobile application places this request in the form of a URI, where the city of New York is passed as a parameter.

When the OData sender channel receives the URI request, it passes it to the Script element. The Script element maps the request so that the SOAP receiver understands it.

The SOAP receiver processes the request and, if there are no issues with the request, returns the most frequently visited hotel and the guest count as the response. The Script element contains information that maps the fields from the SOAP structure to the return type defined in the OData model. The mapped data is then sent to the mobile application through the OData sender channel.

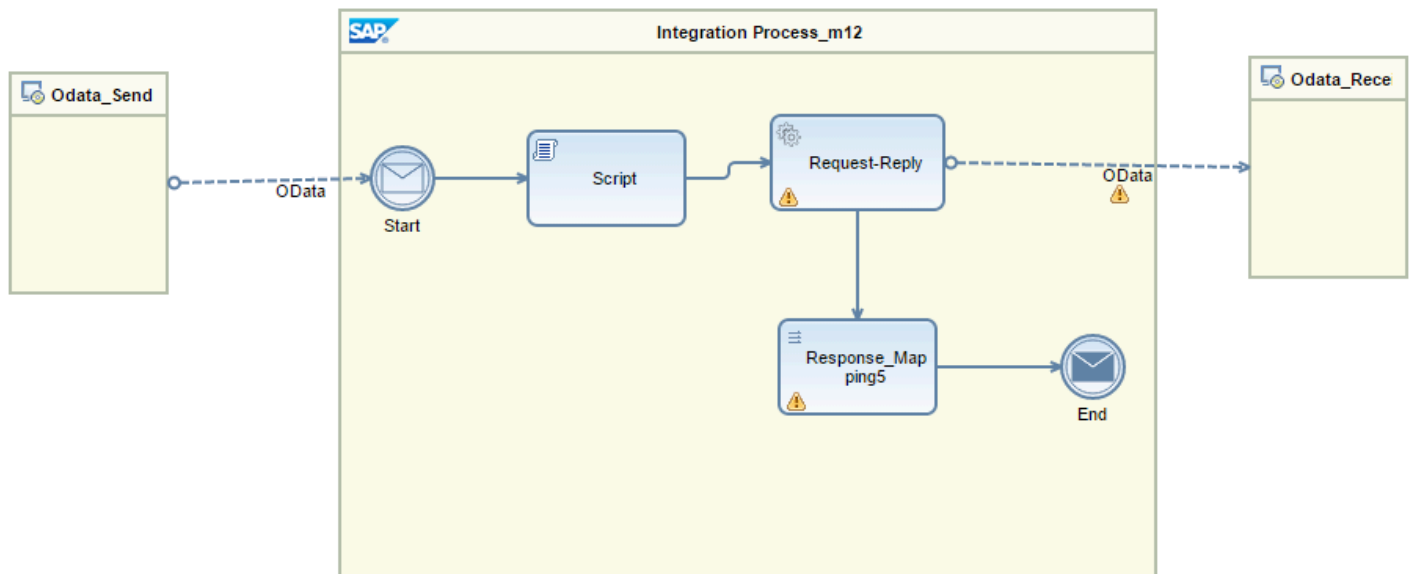
Predefined Integration Flows for OData

The Cloud Integration Web application sets up predefined integration flows when the Create, Read, Update, and Query operations of entity sets are bound to an OData API.

i Note

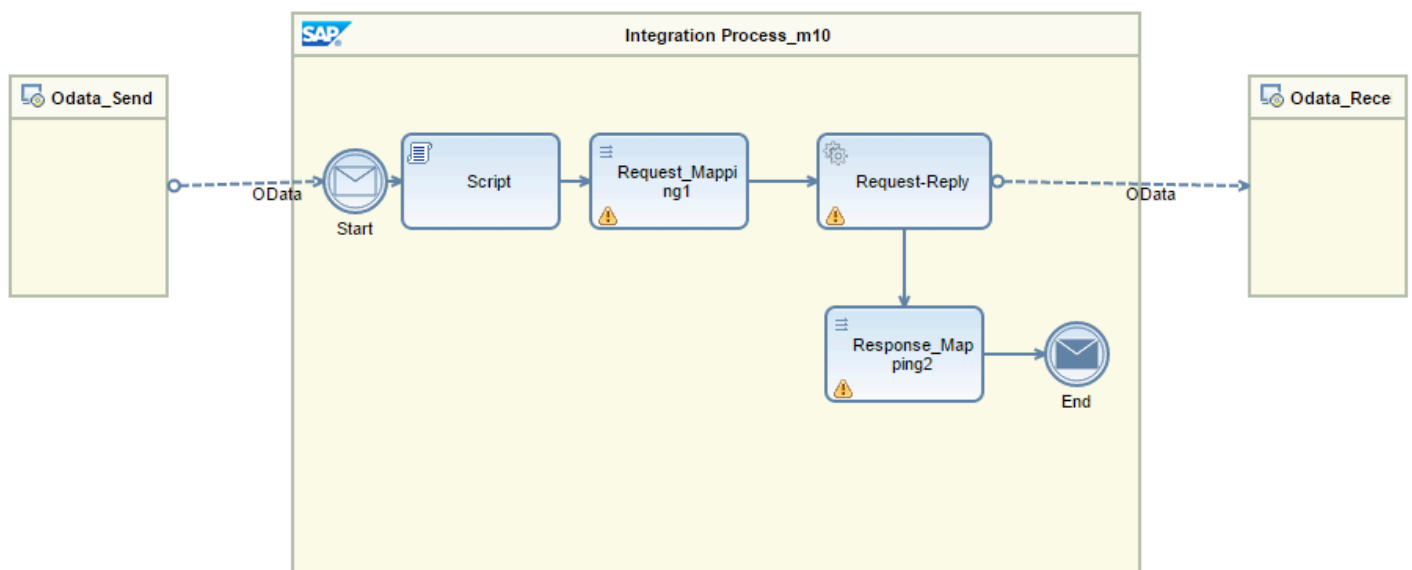
Function imports are not available in the OData receiver channel. We recommend that you use another adapter like HTTP for it.

Predefined Integration Flows for Query and Read Operations



This integration flow is autogenerated for Query and Read operations. Script step in this integration flow takes the query options from the incoming URI and sets it into an exchange header `<odataURI>`. In the case of Query operations, you can use this exchange header to dynamically pass the query options to the OData receiver channel. And in the case of Read operations, you can use this exchange header to dynamically pass the key fields of the entity set to the OData receiver channel.

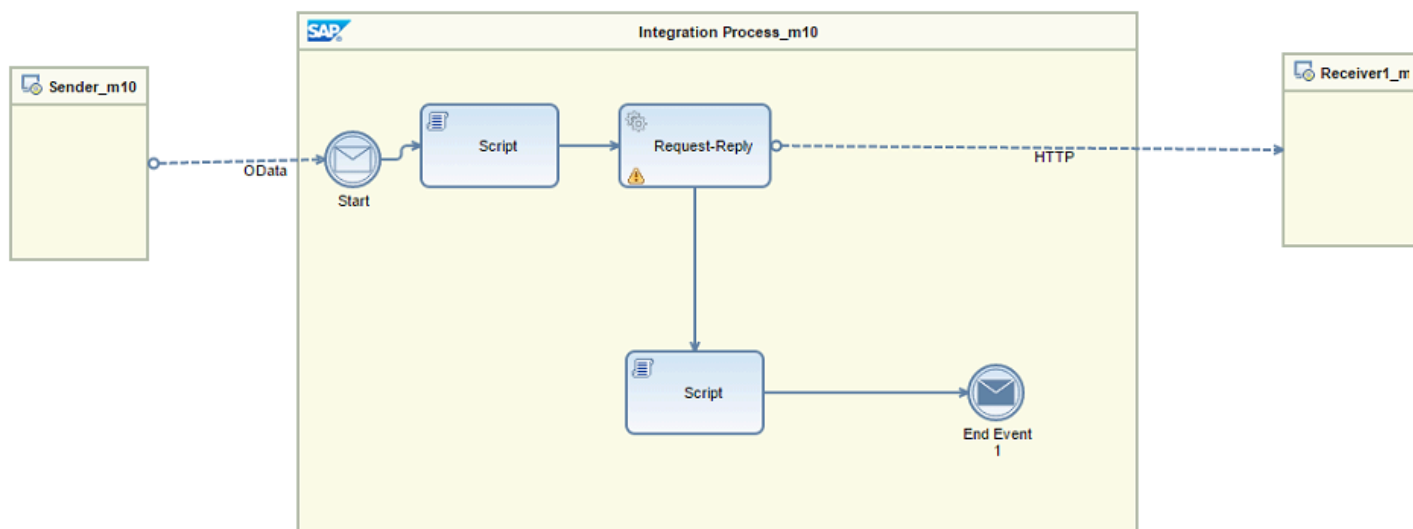
Predefined Integration Flows for Create and Update Operations



This integration flow is autogenerated for Create and Update operations. Script step in this integration flow takes the query options from the incoming URI and sets it into an exchange header `<odataURI>`. In the case of Update operations, you can use this exchange header to dynamically pass the key fields of the entity set to the

Predefined Integration Flows for REST

The Cloud Integration Web application sets up predefined integration flows when the CRUDQ (Create, Read, Update, Delete, and Query) operations of entity sets and function imports are bound to a REST web service.

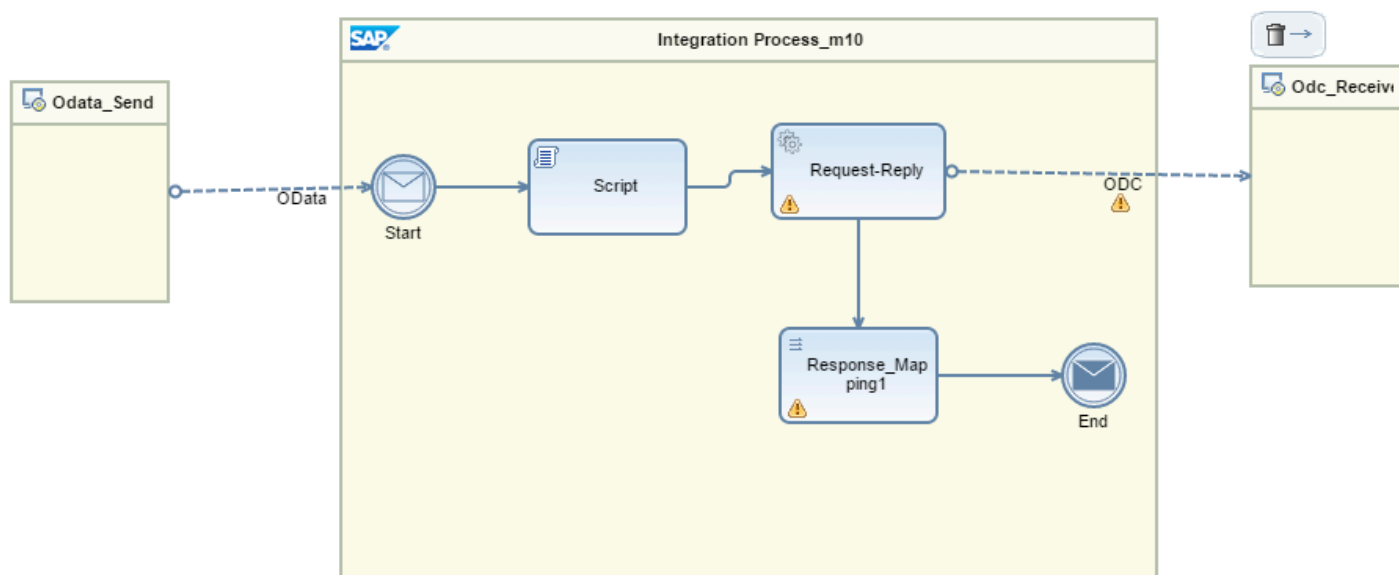


This integration flow is autogenerated for all CRUDQ operations when the receiver is a REST service. You can edit the Script elements in order to define the request and response mappings.

Predefined Integration Flows for ODC

The Cloud Integration Web application sets up predefined integration flows when the CRUDQ (Create, Read, Update, Delete, and Query) operations of entity sets and function imports are bound to an ODC service.

Predefined Integration Flow for Query and Read Operations



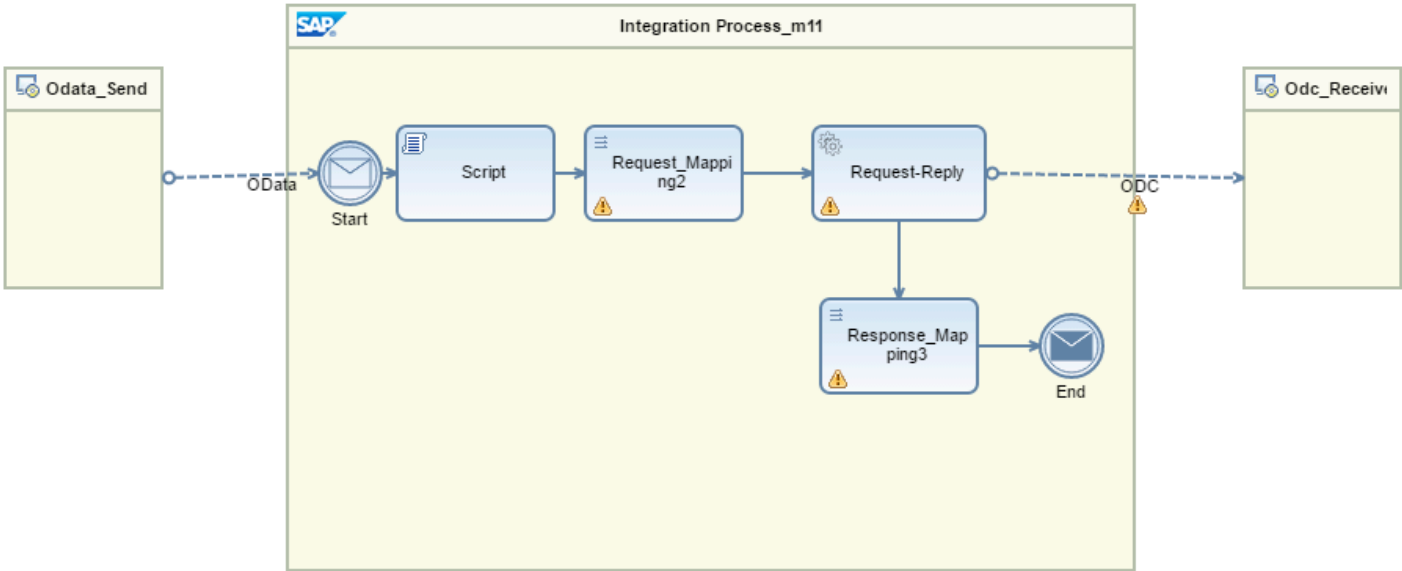
This integration flow is autogenerated for Query and Read operations. Script step in this integration flow takes the query options and key fields of entity sets from the incoming URI and sets it into an exchange header `<odataURI>`.

i Note

Currently, the ODC receiver channel does not support any query options or key fields of entity sets that are dynamically passed using the exchange header.

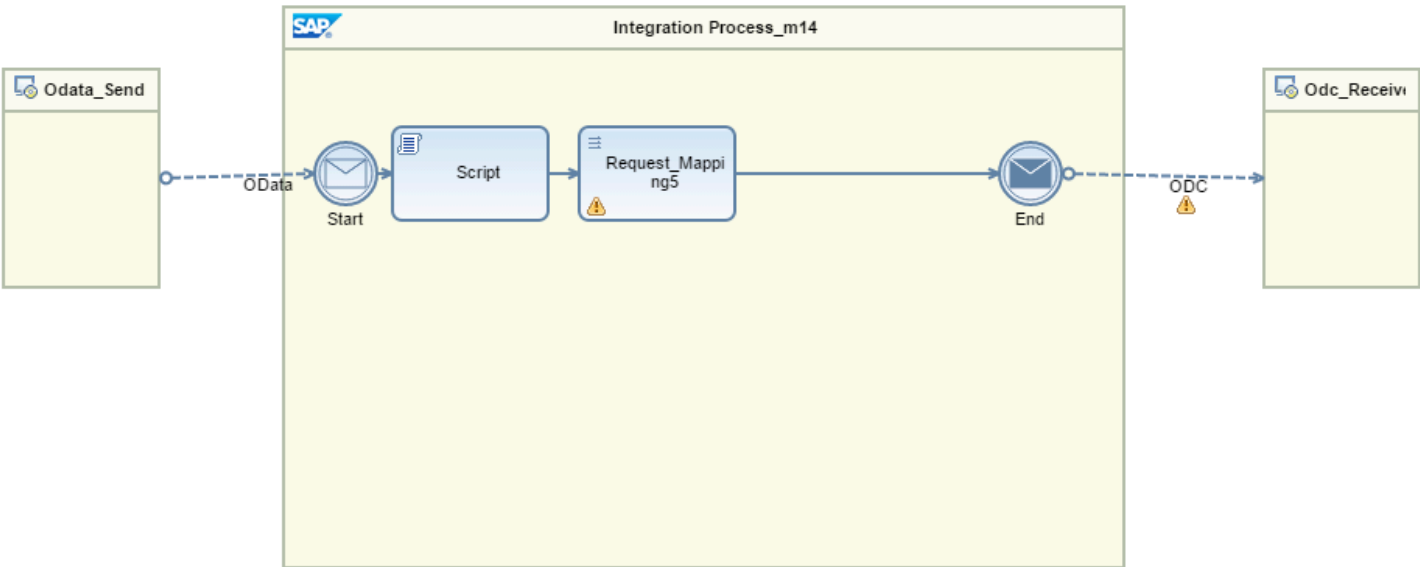
You can map the fields from the ODC structure to the OData structure in the Response Mapping element.

Predefined Integration Flow for Create Operation



This integration flow is autogenerated for Create operation. You can map the fields from the OData structure to the ODC structure in the Request Mapping element. Similarly, map the fields from the ODC structure to the OData structure in the Response Mapping element.

Predefined Integration Flow for Update Operation



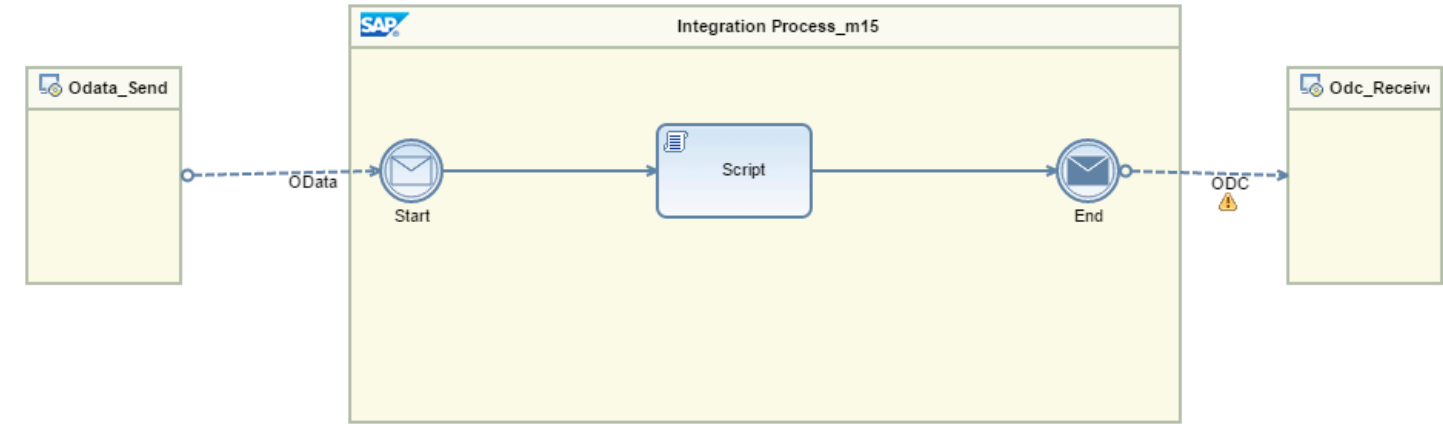
This integration flow is autogenerated for Update operation. Script step in this integration flow takes the key fields of entity sets from the incoming URI and sets it into an exchange header <odataURI>.

i Note

Currently, the ODC receiver channel does not support any key fields of entity sets that are dynamically passed using the exchange header.

You can map the fields from the OData structure to the ODC structure in the Request Mapping element.

Predefined Integration Flow for Delete Operation



This integration flow is autogenerated for Delete operation. Script step in this integration flow takes the key fields of entity sets from the incoming URI and sets it into an exchange header `<odataURI>`.

i Note

Currently, the ODC receiver channel does not support any key fields of entity sets that are dynamically passed using the exchange header.

Configuring an OData API

Context

→ **Remember**

This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#).

You have created an OData API artifact.

You can use this procedure to modify the parameters that you entered when creating the artifact. These parameters identify the OData API.

Procedure

- 1. Choose **Design**.
- 2. Choose the integration package that contains the OData artifact for which the details need to be changed.
- 3. Choose and then choose **View metadata** to view or modify the metadata of the OData API.
- 4. Edit or modify the parameters listed in the following table, as required.

Field	Description
Name	Name of the OData API.
Description	Description of the OData API and its purpose.
Namespace	Logical grouping of the OData API.

- 5. Choose **Save** to save your changes to the OData API artifact.

i Note

To retain a copy of the current artifact, choose **Save as version**.

Deploying an OData API

Context

→ **Remember**

This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#).

You've created an OData API artifact with a valid OData model. You've provided binding information for at least one OData object in the model. You've verified that all integration flows are valid.

You can use this procedure to deploy the OData API.

Procedure

1. Choose **Deploy** on the Service Designer page.
2. Confirm that you do want to deploy. If there are no validation errors, a message is displayed confirming that the service is triggered for deployment.
3. Choose **OK**. The OData API is now deployed.

The OData API starts on the tenant, after deployment. You can confirm that the status of the OData API is **Started** in the Integration Content Monitor. For more information, see [Manage Integration Content](#).

It can take a few moments for the OData API to get active. Once it's active, the OData API is ready for consumption.

i Note

The URL of the OData API endpoint is derived from the name of the OData API artifact.

Invoking an OData API

Context

→ Remember

This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#).



You have deployed an OData API successfully; the service is in active status ready to be consumed.

Procedure

1. Choose **Monitor**.
2. On the Overview page, in the section **Manage Integration Content**, choose the tile that shows the services with the status *Started*.
3. Search for and open the service that you deployed.
4. On the **Endpoints** tab, you can view the endpoint URI. This endpoint URI is the OData API Root URI.
5. You can copy this URI by choosing the copy icon and invoke it from the calling application.

You can invoke an OData API in *\$batch* mode by appending the term */ \$batch* to the service root URI and invoking the same using the relevant payload.

- Currently only one operation is supported for each *changeset*.
- The Message Processing Log (MPL) is generated for each *changeset*.

i Note

If your MPL shows an error message **FAILED** without any error in the trace, then it means that your Response does not match the Metadata defined in the adapter.

- To obtain a JSON response, add **Accept HTTP header** with application/json value.
- All the headers available in the integration flow pipeline at the time of message processing, will be returned as response headers when the OData API is invoked.
- POST calls support deep structure responses.

Developing Custom Adapters

Develop adapters tailored for your specific requirements.

Though Cloud Integration provides a set of adapters to meet standard connectivity needs, you can develop and import your own custom adapters for your specific needs.

Adapter Development Prerequisites

You can contribute to an Apache Camel component as Cloud Integration adapter. To integrate camel component into Cloud Integration framework, specify the detailed set of adapter attributes as component metadata. Component metadata defines the adapter attributes, dependencies between them and the structure of configuration interface for a specific adapter, which is used for user interface generation.

Following are the guidelines that you need to understand and follow while developing camel component:

- **Protocol and Exchange Headers** - While you are specifying the protocol headers as camel headers or vice-versa from your custom integration adapter, we strongly recommend to consider these rules for security reasons:
 - Propagate only compliant headers or the headers required by adapter protocol.
 - Camel-specific arbitrary headers should not be used. You may still propagate the headers compliant with the adapter protocol following a naming convention conforming to adapter protocol guidelines.
 - Provide the configuration in the custom integration adapter to allow the non-compliant exchange headers to be propagated.
 - Include the propagated headers in the adapter documentation.
- **Camel Component Scheme** – Make sure that you use a unique camel component scheme to avoid runtime collisions with other registered schemes. For example, the camel component scheme can be sap-dropbox.
- This is the new content.
- **Secure Programming** – Store secure parameters in runtime secure store and accessed through alias configured in adapter.
- **Security Aspects** –
 - Always scan for virus in the Adapter workspace / project before deployment.
 - The Adapter design time content bundle shouldn't contain any confidential information (For example: hard-coded secrets, passwords, users, or documents).
 - Perform ftify and java static code checks.
- **Logging** – Use Slf4j logging
- **API Consumption**
 - To access platform-specific features, you can use public APIs that are available with the Adapter Development Kit (ADK). Public APIs for the ADK are available in Cloud Integration tools page.
 - The two maven coordinates available in the central maven for Public API consumption are:

`com.sap.cloud.adk:generic.api`

`com.sap.cloud.adk:adapter.api`

Refer the central maven repository to obtain the latest version number.
 - To consume the APIs, you need to use the code snippet `ITApiFactory.getApi(apiType, context)`; wherein, `apiType` is the class of the requested API interface. The camel context can be null if it isn't required for a particular API. For Example: For accessing `KeystoreService`, use `ITApiFactory.getApi(KeystoreService.class, context)`

i Note

The Java standard libraries of Java 8 can be used.

Cloud Integration supports the XML Document Object Model (DOM) to process XML documents.

- **Manifest.MF + Allowlisting** – Only ADK APIs, Camel core and Logging are accessible at runtime. However, if necessary you can import packages for dependent libraries bundled along with project.
- If you want to read a resource in integration flow project at runtime, use the following code snippet:

≡, Sample Code

```
InputStream is = ResourceHelper.resolveMandatoryResourceAsStream(getCamelContext().getClassResolver(), "/"
```

Adapter Development Process

Understand the steps involved in developing custom adapters.

[Develop Adapters Using Archetype](#)

You can develop custom adapters using Adapter SDK in Cloud Integration.

[Consuming Non-OSGi Jar](#)

OSGi bundles are normal jar components with extra manifest headers, which you use to develop adapters. In OSGi environment, every service we use must be an OSGi bundle.

[Component Metadata](#)

Component metadata specifies the attributes of sender and receiver adapters used to define the UI. Each tab can contain multiple attributes. You can group the attributes into multiple attribute groups within the tabs. You can use the same attribute across the sender and receiver adapters by using attribute reference.

[Blueprint Metadata](#)[Enabling Endpoint Visualization for Custom Adapter](#)

Configure a custom adapter with endpoint visualization to view the adapter endpoint URL.

[Managing Cluster Lock in Custom Adapters](#)

Custom Integration Adapters provide a mechanism to acquire cluster-level locks, ensuring that specific actions (e.g., message processing) are executed by only one worker or node instance at a time. This prevents duplicate processing and maintains data consistency across distributed environments.

[Enabling ADK Persistency](#)[Enabling Authentication for Servlet Based Adapters](#)[Enabling Tracing for Custom Adapter](#)

Configure a custom adapter with a tracing feature that records the processing of the incoming and outgoing messages in an adapter.

[Enabling Scheduler Support for ADK Sender Adapter](#)[Enabling Connection Status for Integration Flow](#)[Accessing User Credentials](#)[Accessing Trust and Key Managers](#)[Accessing On-Premise Application using Cloud Connector](#)

Follow the steps here to extend the connectivity of Cloud Integration with remote on-premise application.

[Versioning Rules for Custom Adapters](#)

Follow the guidelines mentioned below to version a custom adapter.

Develop Adapters Using Archetype

You can develop custom adapters using Adapter SDK in Cloud Integration.

Prerequisites

- You are aware of the technical details of component/domain that you have to connect with Cloud Integration through the newly developed adapter type.
- You have Java 8 version.
- You are familiar with using Apache Camel and Java.
- You have the Camel component and its dependencies as OSGi bundles.
- You have the camel core runtime version 3.14.
- You have the simple logging facade for java (SLF4J).

Context

You use an adapter in an integration flow for outbound and inbound communication with different applications during runtime. In such scenario if an adapter does not support your communication requirements you use SAP ADK framework to create an adapter and deploy it in an integration flow. .

There are two types of adapter project template to create a new adapter project:

- **Adapter Project**

Use this template when you have a requirement to create a new SAP adapter and a new Camel component or customize an existing runtime component.

- **Component reuse Adapter Project**

Use this template if you want to re-use an existing runtime component without any customization.

Here we describe how to build and deploy an adapter during runtime using these two templates.

Procedure

1. Adapter Project Template

- a. Use any IDE for creating a project from the archetype or use the generate command and enter the following:

Use the below maven archetype artifacts:

```
com.sap.cloud.adk:com.sap.cloud.adk.archetype-adapter
```

Refer to the central maven to obtain the latest version number.

≡ Sample Code

```
mvn archetype:generate -B -DarchetypeGroupId=com.sap.cloud.adk \
-DarchetypeArtifactId=com.sap.cloud.adk.archetype-adapter \
-DarchetypeVersion=1.0.1 \
-DcomponentName=SampleComponentName \
-DadapterName=AdapterName \
-DadapterId=AdapterNameId \
-DadapterVendor=VendorName \
-DadapterVersion=1.0.0 \
-Dscheme=scheme \
-DgroupId=project.group \
-DartifactId=project.artifact \
-Dversion=1.0-SNAPSHOT
```

Component Description	
Component	Description
archetypeGroupId archetypeArtifactId archetypeVersion	Artifacts of maven archetype
componentName scheme	Artifacts of your runtime component componentName is the name of your camel component. scheme is the scheme name of the component.
adapterName adapterId adapterVendor adapterVersion	Artifacts of your integration adapter
groupId artifactId version	Artifacts of your maven project

b. The following table describes different components created for the custom adapter project using maven:

Component	Description
*Component.java	Sample runtime component.
*Consumer.java	Sample sender component.
*Endpoint.java	Logger information is found.
*Producer.java	Sample receiver component.
metadata.xml	UI values are coded here.
*ComponentTest.java	Sample JUnit test
pom.xml	Contains configuration details and dependencies.
*config.adk	Contains the adapter configuration parameters.
sap-sample	Represents the endpoint scheme. i Note If you deploy multiple adapters make sure the filenames are unique. When you change the filename do the following: ■ In the endpoint class, change scheme value. For example, @UriEndpoint(scheme = "sap-sample", syntax = "", title = "").

Component	Description
	■ In metadata.xml change the attribute value. For example, <code>gen:RuntimeComponentBaseUri="sap-sample"</code>.

i Note

You can update to the latest ADK version by manually changing the version in `com.sap.cloud.adk.build.archive` artifact found in the `pom.xml` file. Refer the central maven repository to get the latest ADK version.

2. **Component reuse Adapter Project Template**

a. Use any IDE for creating a project from the archetype or use the generate command and enter the following:

Use the below maven archetype artifacts:

`com.sap.cloud.adk:com.sap.cloud.adk.archetype-adapter-component-reuse`

Refer to the central maven to obtain the latest version number.

i Note

You can reuse an existing camel component in your adapter project using this template.

 Sample Code

```
mvn archetype:generate -B -DarchetypeGroupId=com.sap.cloud.adk \
-DarchetypeArtifactId=com.sap.cloud.adk.archetype-adapter-component-reuse \
-DarchetypeVersion=1.0.1 \
-DadapterName=AdapterName \
-DadapterId=AdapterNameId \
-DadapterVendor=VendorName \
-DadapterVersion=1.0.0 \
-DgroupId=project.group \
-DartifactId=project.reuse.artifact \
-Dversion=1.0-SNAPSHOT
```

Component Description

Component	Description
archetypeGroupId archetypeArtifactId archetypeVersion	Artifacts of maven archetype
adapterName adapterId adapterVendor adapterVersion	Artifacts of your integration adapter
groupId artifactId version	Artifacts of your maven project

b. The following table describes different components created for the custom adapter project using maven:

Component	Description
component	A folder where runtime component JAR must be placed.
libs	A folder where runtime component dependencies JARs must be placed.
metadata	A folder where component and blueprint (optional) metadata will be generated.
pom.xml	Contains configuration details and dependencies.

i **Note**

You can update to the latest ADK version by manually changing the version in com.sap.cloud.adk.build.archive artifact found in the pom.xml file. Refer the central maven repository to get the latest ADK version.

c. Copy runtime component **JAR** into **component** folder and their dependencies into the **libs** folder.

3. Run the command `mvn clean install` to build the adapter. This command will generate runtime component metadata and ESA with a component.

Blueprint metadata is not generated by default with this command. To generate this metadata, refer the next step.

4. Other maven goals that you can use are

o **Generate runtime component metadata**

This will generate component metadata using the camel component.

```
mvn com.sap.cloud.adk:com.sap.cloud.adk.build.archive:generate-cmd
```

o **Generate blueprint metadata**

This will generate the blueprint metadata for custom generation.

```
mvn com.sap.cloud.adk:com.sap.cloud.adk.build.archive:generate-bmd
```

o **Check adapter project**

This is used explicitly for validation. This step is included by default in the project build step.

```
mvn com.sap.cloud.adk:com.sap.cloud.adk.build.archive:check
```

Consuming Non-OSGi Jar

OSGi bundles are normal jar components with extra manifest headers, which you use to develop adapters. In OSGi environment, every service we use must be an OSGi bundle.

Prerequisites

- You have created a simple adapter.
- The adapter is an OSGi esa file with type as Composite. The dependencies are bundled within this esa.
- You are aware of Java development.

Context

To integrate camel component into Cloud Integration framework, camel component and its dependencies should be in a OSGi bundle. To convert the jar file to OSGi bundle, refer to the blog, [here](#)👉.

Results

The consumed non-osgi jar is available inside the component jar as bundle classpath.

Component Metadata

Component metadata specifies the attributes of sender and receiver adapters used to define the UI. Each tab can contain multiple attributes. You can group the attributes into multiple attribute groups within the tabs. You can use the same attribute across the sender and receiver adapters by using attribute reference.

Component Metadata

This set of metadata allows you to define components details.

Component Metadata

Property	Datatype	Cardinality	Description
ComponentId	xsd:string	Attribute	Specifies a unique Id for the component. The following are technical identifiers and should not contain any space or : Example: ctype::Adapter/cname::sap:dropbox/version::1.0.0 ctype is type of component, which is adapter in this particular example

Property	Datatype	Cardinality	Description
			cname is component name, which is sap:dropbox in this example. Here, sap is vendor and dropbox is name
ComponentDisplayName	xsd:string	Attribute	Specifies display name of the component.
Variant		Child Element	Different variants for a specific component type - for example: Sender and Receiver variants of an adapter
gen:RuntimeComponentBaseUri	xsd:string	Attribute	This attribute specifies the name with which the component is identified. If not added, then the baseUri value is take TP (Transport Protocol). Example for specifying attribute in the CMD: ⌘ Sample Code <pre><ComponentMetadata ComponentId="ctype::Adapter/cname::sap:sampleadapter/version::1.0.0 ComponentName="sap:sampleadapter" UIElementType="Adapter" IsDefaultGenerator="true" hci:RuntimeComponentBaseUri="dropbox"></pre> Example for generated bean.xml containing camel:from ⌘ Sample Code <pre><camel:from id="MessageFlow_1_1435812482630" uri="dropbox:foo?path=/input/test.txt& method=get&sendEmptyMessageWhenIdle=false& initialDelay=1000&greedy=false&startScheduler=false&timeUnit=MILLISECONDS& useFixedDelay=false&delay=500&runLoggingLevel=TRACE"/></pre>
IsRequestResponse	xsd:string	Attribute	This attribute is used for enabling the component to allow the service call for Request Reply. Default value is false. Henceforth, you should define it at variants level only.
hci:FirstUriPart	xsd:string	Attribute	Specifies the initial part of the URI. Customer can provide custom values for the first part of the URI string. Default v. 'foo'.
adapterInstanceID	xsd:string	Attribute	Use cluster lock to prevent same message being processed by different runtime nodes simultaneously. i Note You must define the following property at the endpoint. ⌘ Sample Code <pre><Metadata> <AdditionalMetadata> <Name>requiredIntegrationFlowProperties</Name> <Value>adapterInstanceID</Value> <GuiLabels> <Label language="en"/> </GuiLabels> </AdditionalMetadata> </Metadata></pre> Use the above parameter as the lock name in the cluster lock API.

i Note

IsDefaultGenerator and Metadata Version must always be true and should never be set to false. Metadata Version attribute is generated with default value as 2.0.

Variants

This set of metadata allows you to define variants for a component.

Variants			
Property	Datatype	Cardinality	Description
Id	xsd:string		Provides a unique Id for the component variant. Example: ctype::AdapterVariant/cname::sap:dropbox/tp::dropbox/mp::dropbox/direction::Send tp stands for transport protocol

Property	Datatype	Cardinality	Description						
			mp stands for message protocol direction refers to Sender or Receiver system						
Groups		Child Element	Specifies collection of attributes defining metadata.						
VariantDisplayName	xsd:string	Attribute	Specifies display name of the variant.						
Input Content Tag used <InputContent>		Child Element	Input content specifies the type of data the adapter processes. <table><tr><th>Properties</th><th>Attributes/Child Elements</th><th>Description</th></tr><tr><td>ContentType</td><td>Child Element</td><td>Actual content type Supported content types: Xml, S Any, Json, Binary, Zip, GZip, Integer, NonXml, None and Text. <ManifestEntries> <ManifestEntr Package</Name> <Value>com.sap.gateway.core.service.ljs.registry:resolution:=optional,com.sap.gateway.core.l </ManifestEntry> </ManifestEntries></td></tr></table>	Properties	Attributes/Child Elements	Description	ContentType	Child Element	Actual content type Supported content types: Xml, S Any, Json, Binary, Zip, GZip, Integer, NonXml, None and Text. <ManifestEntries> <ManifestEntr Package</Name> <Value>com.sap.gateway.core.service.ljs.registry:resolution:=optional,com.sap.gateway.core.l </ManifestEntry> </ManifestEntries>
Properties	Attributes/Child Elements	Description							
ContentType	Child Element	Actual content type Supported content types: Xml, S Any, Json, Binary, Zip, GZip, Integer, NonXml, None and Text. <ManifestEntries> <ManifestEntr Package</Name> <Value>com.sap.gateway.core.service.ljs.registry:resolution:=optional,com.sap.gateway.core.l </ManifestEntry> </ManifestEntries>							
Output Content		Child Element	Output content specifies the type of data the adapter produces. Example: XML. <table><tr><th>Properties</th><th>Attributes/Child Elements</th><th>Description</th></tr><tr><td>ContentType</td><td>Child Element</td><td>Actual content type <ManifestEntries> <ManifestEntry> <Name>Import-PackageSupported content types: Xml, S Any, Json, Binary, Zip, GZip, Integer, NonXml, None and Text. <ManifestEntries> <ManifestEntr Package</Name> <Value>com.sap.gateway.core.service.ljs.registry:resolution:=optional,com.sap.gateway.core.l </ManifestEntry> </ManifestEntries></td></tr></table>	Properties	Attributes/Child Elements	Description	ContentType	Child Element	Actual content type <ManifestEntries> <ManifestEntry> <Name>Import-PackageSupported content types: Xml, S Any, Json, Binary, Zip, GZip, Integer, NonXml, None and Text. <ManifestEntries> <ManifestEntr Package</Name> <Value>com.sap.gateway.core.service.ljs.registry:resolution:=optional,com.sap.gateway.core.l </ManifestEntry> </ManifestEntries>
Properties	Attributes/Child Elements	Description							
ContentType	Child Element	Actual content type <ManifestEntries> <ManifestEntry> <Name>Import-PackageSupported content types: Xml, S Any, Json, Binary, Zip, GZip, Integer, NonXml, None and Text. <ManifestEntries> <ManifestEntr Package</Name> <Value>com.sap.gateway.core.service.ljs.registry:resolution:=optional,com.sap.gateway.core.l </ManifestEntry> </ManifestEntries>							
Metadata		Child Element	Helps to add additional metadata in the component. Specifies the display name of transport protocol and message protocol. Example for specifying attribute in the CMD:  Sample Code <pre><Metadata> <AdditionalMetadata> <Name>mp</Name> <GuiLabels guid="e7fa4f1d-ec6a-4248-9752-920ea9616bc9" > <Label language="EN" >AS2</Label> <Label language="DE" >AS2</Label> </GuiLabels> </AdditionalMetadata> <AdditionalMetadata> <Name>tp</Name> <GuiLabels guid="e7fa4f1d-ec6a-4248-9752-920ea9616bc9" > <Label language="EN" >HTTPS</Label> <Label language="DE" >HTTPS</Label> </GuiLabels> </AdditionalMetadata> </Metadata></pre> i Note You should refer message protocol as mp and transport protocol as tp .						
ManifestEntries		Child Element	Specifies which manifest entries you can add in manifest file. You can add or update Import-Package entries in manifest file. Example for specifying attribute in the CMD:  Sample Code						
Allowed Headers		Child Element	Specifies which headers can be passed in the integration flow pipeline from sender component. Example for specifying attribute in the CMD:  Sample Code						

Property	Datatype	Cardinality	Description
			<code><AllowedHeaders></code> <code>Header1,Header2</code> <code></AllowedHeaders></code>
hci:FirstUriPart	xsd:string	Attribute	Specifies the initial part of the URI. Customer can provide custom values for the first part of the URI string.
IsRequestResponse	xsd:string	Attribute	This attribute is used for enabling the variant to allow the service call for Request Reply
supportsPolling	xsd:boolean	Attribute	This attribute depicts if the component has the polling behavior. (supportsPolling). It also helps in populating the timer icon decorated. Default value is false. It is applicable for sender channel only.
AttachmentBehavior	xsd:string	Attribute	This attribute helps to identify the component behavior towards attachment. • Modify - Create or update an attachment • Preserve - Pass through for the attachments • Drop - Attachment will be lost If attachment behavior is not specified; system considers that the component lost the attachment.
ReferencedComponents		Child Element	Specifies the reused component to be referred by the adapter.

i Note

If you want to rename a variant id or delete a variant from adapter, first undeploy the adapter with old variants and then deploy updated adapter.

You can still view the older variants if you do not undeploy the adapter with old variants.

Tabs

This set of metadata allows you to specify tab pages for the adapter configuration user interface.

Tabs

Property	Datatype	Cardinality	Description
Id	xsd:string		Specifies the tab identifier.
GuiLabels	xsd:string		Specifies a language-specific GUILabel. Sample GUILabel is shown below: <pre><GuiLabels> <Label language="EN">Address</Label> </GuiLabels></pre> Here, Label language = "EN" represents that the text of guilabel is in english. Address is the text for the attribute name. Currently, guilabel supports only English language.

Attribute Group

This set of metadata allows you to group adapter attributes.

Attribute Group

Property	Datatype	Cardinality	Description
Name	xsd:string		
Id	xsd:string		Specifies the attribute group identifier. Examples: Dropbox Endpoint Dropbox Security Settings
AttributeReference		Child Element	Specifies ReferenceName, ErrorMessage, Restriction, and Description
GuiLabels			Specifies a language-specific GUI label.

Property	Datatype	Cardinality	Description
Value - Datatype		Child Element	Datatype for value xsd:string xsd:integer xsd:boolean xsd:long xsd:password xsd:id xsd:idref i Note If an attribute is marked with xsd:id in metadata, then its value should be unique across elements of integration flow, during design time. If an attribute is marked with xsd:idref then it specifies a reference to another resource. If resource path starts with /<folder_name>/<resource_name>, then checks will validate if specific resource path exists in src\main\resource folder of project or not.

The sample attribute group is shown below:

```
<AttributeGroup id = "Connection Details">
  <GuiLabels>
    <Label language="EN">Connection Details</Label>
  </GuiLabels>

  <AttributeReference>
    <ReferenceName>address</ReferenceName>
    <!-- Restriction tag defines java regular expression for field value -->
    <Restriction>Constraint.isValidRegex([A-Za-z]+[0-9_~\-\.\.]*)</Restriction>
    <!-- This tag defines message to be displayed if field value is invalid -->
    <ErrorMessage>Enter a valid address. E.g. /file</ErrorMessage>
    <!-- Description tag defines tool tip for field value -->
    <description>Relative endpoint address on which.</description>
  </AttributeReference>

  <AttributeReference>
    <ReferenceName>wsdlURL</ReferenceName>
    <Restriction>Constraint.isValidRegex:^(https://){1}(.)+</Restriction>
    <ErrorMessage>Enter a valid URL. E.g. /wsdl/abc.wsdl;</ErrorMessage>
    <description>URL to the WSDL.</description>
  </AttributeReference>

  <AttributeReference>
    <ReferenceName>cxfoNewayRobust</ReferenceName>
    <description>Used for reliable one-way message exchanges.</description>
  </AttributeReference>
</AttributeGroup>
```

Attribute Reference

Attribute Reference Table

Property	Data Type	Cardinality	Description
ReferenceName	xsd:string		Specifies that in the CMD there is an attribute definition associated with this reference name.
description	xsd:string		Brief description of the attribute, which is shown as the tool tip.
Restriction	xsd:string		Details given below in the table.
ErrorMessage	xsd:string		If the constraint specified in the restriction tag fails, the system shows the error message set in this tag. If this field is not set, then the system shows a default error message at the time of check failure.

List of Constraints that are supported for Restriction tag

Constraint	Description
------------	-------------

Constraint	Description
isValidRegex	Method will check a value against a passed regex. Returns true in case value matches regex ⌚ Sample Code <AttributeReference> <ReferenceName>stepid</ReferenceName> <Restriction>Constraint.isValidRegex([a-zA-Z0-9]+)</Restriction> <ErrorMessage>StepID not defined for Persist Message step</ErrorMessage> </AttributeReference>
isAlphaNumeric	Checks whether the pass value is alphanumeric
isStartsWithLetter	Checks the passed value starts with Letter
isValidURIString	Returns true if it is valid URI String. It can start with http(s), ftp, file or ldap
isValidXMLString	Returns true if passed string is a valid XML String
isValidNCName	Return true if passed string value is valid NCName
isValidXpath	Returns true if passed string value is valid xpath expression

Attribute Metadata

This set of metadata allows you to define the details of all adapter attributes. This includes the definition of fixed values for input help, and the kind of user interface element used for an adapter attribute (for example if it is a checkbox or a dropdown list).

The sample attribute metadata should be as shown below:

```
<AttributeMetadata>
  <Name>address</Name>
  <Usage>true</Usage>
  <Default>/BSN/paymentOrder</Default>
  <DataType>xsd:string</DataType>
  <isparameterized>true</isparameterized>
  <Length>200</Length>
  <FixedValues>
    <FixedValue>
      <Value>Atleast Once</Value>
      <GuiLabels>
        <Label language="EN">Atleast Once</Label>
      </GuiLabels>
    </FixedValue>
  </FixedValues>
  <GuiLabels>
    <Label language="EN">Address</Label>
  </GuiLabels>
</AttributeMetadata>
```

Attribute Metadata

Property	Datatype	Cardinality	Description
Name	xsd:string		Specifies the parameter id. Example: address
Usage	xsd:boolean		Specifies if the field is mandatory or not. If the value is true then usage of field is mandatory. If the value is false then usage of field is not mandatory.
Default	xsd:string		Specifies a default value. Example: /BSN/paymentOrder
Datatype			Specifies the datatype for the value. Examples: xsd:string xsd:integer

Property	Datatype	Cardinality	Description												
Length	xsd:integer		Specifies the maximum character length for field values.												
Minlength	xsd:integer	Child Element	Specifies the minimum character length for field values.												
HelpService		Child Element	Helps to configure specified resources in adapters. Example for specifying attribute in the CMD: ≡ Sample Code <pre><HelpService> <HelpServiceName> resourceLookupService </HelpServiceName> <HelpServiceQueryAttributes> <HelpServiceQueryAttribute> <Name> Type </Name> <Value> xml </Value> </HelpServiceQueryAttribute> <HelpServiceQueryAttribute> <Name> ResourceLocation </Name> <Value> edmx </Value> </HelpServiceQueryAttribute> </HelpServiceQueryAttributes> </HelpService></pre> Once you add the above code, then a Browse button gets enabled. You can click on the button and select resource file from the project of file system. ResourceType specifies resource extension. Currently, only one extension can be specified. ResourceLocation specifies relative path to project from where files are read or written.												
GUILabel		Child Element	Specifies a language-specific GUI label.												
FixedValue(s)		Child Element	Specifies a fixed list of allowed values. <table><tr><th>Properties</th><th>Attributes/Child Elements</th><th>Description</th></tr><tr><td>FixedValue</td><td>Child Element</td><td>Specifies fixed list of allowed values. Multiple FixedValue tags together are called a combo.</td></tr><tr><td>isEditable</td><td>Child Element</td><td>If the element is set to 'true', then the dropdown and combo fields are editable.</td></tr><tr><td>Sort</td><td>Child Element</td><td> If the element is set to 'true', then the values are arranged either in ascending or descending order. Use the following formats to arrange the values: - To arrange the values in ascending order, use <Sort order="ascending">true</Sort> - To arrange the values in descending order, use <Sort order="descending">true</Sort> i Note If the element is set to 'false', then the values are arranged as per the order defined in the cmd. </td></tr></table>	Properties	Attributes/Child Elements	Description	FixedValue	Child Element	Specifies fixed list of allowed values. Multiple FixedValue tags together are called a combo.	isEditable	Child Element	If the element is set to 'true', then the dropdown and combo fields are editable.	Sort	Child Element	If the element is set to 'true', then the values are arranged either in ascending or descending order. Use the following formats to arrange the values: - To arrange the values in ascending order, use <Sort order="ascending">true</Sort> - To arrange the values in descending order, use <Sort order="descending">true</Sort> i Note If the element is set to 'false', then the values are arranged as per the order defined in the cmd.
Properties	Attributes/Child Elements	Description													
FixedValue	Child Element	Specifies fixed list of allowed values. Multiple FixedValue tags together are called a combo.													
isEditable	Child Element	If the element is set to 'true', then the dropdown and combo fields are editable.													
Sort	Child Element	If the element is set to 'true', then the values are arranged either in ascending or descending order. Use the following formats to arrange the values: - To arrange the values in ascending order, use <Sort order="ascending">true</Sort> - To arrange the values in descending order, use <Sort order="descending">true</Sort> i Note If the element is set to 'false', then the values are arranged as per the order defined in the cmd.													

Attribute Reference Table

Property	Data Type	Cardinality	Description
----------	-----------	-------------	-------------

Property	Data Type	Cardinality	Description
AttributeName		Child Element	Name of the attribute
AttributeValue		Child Element	Value of the attribute
AndCondition		Child Element	Attribute Reference Property This is complex constraint that is used when there are two or more complex or basic constraints that need to be satisfied to achieve certain criteria. Example of Complex condition ☰ Sample Code <pre><AndCondition> <EditCondition> <AttributeName>reference1</AttributeName> <AttributeValue>reference1value</AttributeValue> </EditCondition> <EditCondition> <AttributeName>reference2</AttributeName> <AttributeValue>reference2value</AttributeValue> </EditCondition> </AndCondition></pre> As you can see in the above example AndCondition is complex condition that contains two or more EditConditions(basic conditions) and for the condition to be satisfied both of these basic conditions is satisfied.
OrCondition		Child Element	Attribute Reference Property This is complex constraint that is used when satisfaction of any one of the nested complex or basic constraint is required to achieve certain criteria The structure of OrCondition is similar to And condition, with only change from <AndCondition> to <OrCondition>.
NotCondition		Child Element	Attribute Reference Property This is complex constraint that is mostly used with other complex constraints, this constraint reverses the result returned by the nested complex or basic condition Not condition can contain only one edit condition but multiple complex conditions. These complex conditions can be used with each other. ☰ Sample Code <pre><AndCondition> <OrCondition> <EditCondition></EditCondition> <EditCondition></EditCondition> </OrCondition> <NotCondition> <EditCondition></EditCondition> </NotCondition> </AndCondition></pre>
EditCondition		Child Element	Attribute Reference Property Constraint to be executed, this is on top of base constraint defined at Attribute Level Environment Variables: If the Attribute Name starts with \$ then next part is considered as an Environment variable. There can be two types of variables: a) \$PROFILE.NAME - environment variables starting with profile are properties fetched on profile configured for project b) \$ENVVARIABLE - environment variables starting with env are system properties for vm
AttributeBehavior	xsd:string	Child Element	If you assign AttributeBehavior as SecureAlias for a particular field, then the field refers to alias of security parameter artifact.

Attribute Table Metadata

Property	Data Type	Cardinality	Description
Name	xsd:string		

Property	Data Type	Cardinality	Description
Usage	xsd:boolean		
HelpService		Child Element	Helps to configure certificate in adapters. Example for specifying attribute in the CMD: ≡ Sample Code <pre><HelpService> <HelpServiceName> certificateLookupService </HelpServiceName> </HelpService></pre> Once you add the above code, then a Browse button gets enabled. You can click on the button and select certificate file. This attribute supports *.crt and *.cer types of certificates only.
AttributeReference			Specifies reference name that is header of the column. i Note Each reference name constitutes to one column in the table.
GuiLabels	xsd:string		Specifies the label of the table.

The sample attribute table metadata should be as shown below:

≡

Sample Code

```
<AttributeTableMetadata>
  <Name>clientCertificates</Name>
  <Usage>true</Usage>
  <GuiLabels guid="01c2bd59-8dd1-4efb-b795-869851eb979b">
    <Label language="EN">Client Certificate Authorization</Label>
    <Label language="DE">Client Certificate Authorization</Label>
  </GuiLabels>
  <AttributeReference>
    <ReferenceName>clientCertificate.subjectDN</ReferenceName>
  </AttributeReference>
  <AttributeReference>
    <ReferenceName>clientCertificate.issuerDN</ReferenceName>
  </AttributeReference>
</AttributeTableMetadata>
```

i

Note

- You must not use any special character in the table fields.
- You can reload component metadata in the following three ways:
 - You can reopen eclipse tool.
 - You can reconnect to server.
 - You can use  icon (on eclipse toolbar) to manually download it.

Blueprint Metadata

Blueprint metadata defines the attributes of the sender and receiver adapters that contain multiple tags.This document mainly describes all the tags and attributes apart from CMD and camel blueprint tags for providing additional generation injection for ADK development.

This set of tags and attributes allows you to define the metadata.

Attributes	Datatype	Cardinality	Description
createOnce	xsd:string		If you set the string value as "true", then the bean instantiation in the blueprint generation is done only once. By default the value is 'false'.
createCondition	xsd:string		This attribute can be used to conditionally instantiate the bean. The syntax followed is similar to XSLT test attribute. For example: <pre><=, Sample Code</pre> <pre><bean id="123" class="com.example.Someclass.java" createCondition="\$.cmd.attribute(proxyType)='onPremise'" /></pre> Here Someclass.java is generated in the blueprint only if the UI variable proxyType contains the value onPremise.

Tag	Description
<TransformationTags xmlns:bp="http://www.adk.gen/xmlns/blueprint/">	This is the parent tag for the <blueprint> tag. This tag does not contain any generation specific information as part of attribute. The <blueprint> tag must be included within this tag.
<bp:blueprint>	This tag can be used to specify additional namespaces if needed.
<bp:bean>	Beans are declared using <bean> element.
<bp:reference>	To generate References this tag can be used. This tag does not contain any new attributes. It is similar to the camel blueprint's reference tag. Refer the XSD above for all supported attributes.
<cxfc:bus xmlns:cxfc="http://cxf.apache.org/blueprint/core">	Refer http://cxf.apache.org/schemas/blueprint/core.xsd for all attributes supported.
<cxf:cxfEndpoint xmlns:cxf="http://camel.apache.org/schema/blueprint/cxf">	Refer https://camel.apache.org/schema/cxf/camel-cxf.xsd for all the attributes supported.
<camel:camelContext xmlns:camel="http://camel.apache.org/schema/blueprint">	This is the root camel context tag just for the reference. This contains all the properties a final blueprint's camelContext contains as part of g&b generation. The user is not allowed to make any change to this tag. Any changes/additions will simply be ignored.
<http-conf:conduit xmlns:http-conf="http://cxf.apache.org/transport/http/configuration" >	This is same as that of Camel Blueprint specification for conduit tag. Refer http://cxf.apache.org/schemas/configuration/http-conf.xsd.
<camel:route xmlns:camel="http://camel.apache.org/schema/blueprint">	This is same as that of Camel blueprint specification <route>. Must appear within <camelContext>.. Refer the XSD for the attributes. Multiple routes are supported.
<properties>	Can be contained in the DSL just for reference. This tag must be same as g&b's generated tag. It has no significance. i.e. any modification to this tag can simply be ignored by the DSL processor.
<streamCaching>	Provided just for the reference.

Enabling Endpoint Visualization for Custom Adapter

Configure a custom adapter with endpoint visualization to view the adapter endpoint URL.

The following sections describe the guidelines that you need to understand and follow:

Adding Libraries

You can find the latest ADK API in the [Adapter Development Kit](#) section. After downloading the *.zip files, add the following libraries to the  **src**  **main**  **resources**  folder that is in your ADK project:

- com.sap.it.public.adapter.api.jar
- com.sap.it.public.generic.api.jar

For successful code compilation, right-click the *.jar file and choose  **Build Path**  **Add to Build Path**  to add the jar files to the Referenced Libraries of the adapter project.

Adding Dependencies

Open the `pom.xml` file and add the dependencies shown here:

Source Code

```
<dependency>
<groupId>com.sap.it.public</groupId>
<artifactId>adapter.api</artifactId>
<version>...</version>
<scope>system</scope>
<systemPath>${project.basedir}/src/main/resources/adapter.api.<version of the downloaded file>.jar</systemPath>
</dependency>
<dependency>
<groupId>com.sap.it.public</groupId>
<artifactId>generic.api</artifactId>
<version>...</version>
<scope>system</scope>
<systemPath>${project.basedir}/src/main/resources/generic.api.<version of the downloaded file>.jar</systemPath>
</dependency>
```

Note

Make sure that you use the latest version of the *.jar files.

After adding the dependency, add the `com.sap.it.public` group ID to the `<excludeGroupIds>` tag.

Adding Endpoint Information Service

You must create a **Class** and use the following method to retrieve the list of all endpoints for all integration flows:

Source Code

```
import java.util.List;

import com.sap.it.api.adapter.monitoring.AdapterEndpointInformation;
import com.sap.it.api.adapter.monitoring.AdapterEndpointInformationService;

public class AdapterEndpointInfo implements AdapterEndpointInformationService{
    /**
     * Gets the list of all endpoints for all IntegrationFlows used by the
     * adapter
     *
     * @return the list of endpoints
     */
    @Override
    public List<AdapterEndpointInformation> getAdapterEndpointInformation() {
        // TODO Auto-generated method stub
        return null;
    }

    @Override
    public List<AdapterEndpointInformation> getAdapterEndpointInformationByIFlow(String arg0) {
        // TODO Auto-generated method stub
        return null;
    }
}
```

The service implementation must contain the following method, which supports the retrieval of all endpoints exposed across all integration flows:

Source Code

```
package com.sap.it.api.adapter.monitoring;

import java.util.List;
```

```

/**
 * Interface, which adapter developers have to implement to return the endpoints
 * (URLs) used by their adapter
 */
public interface AdapterEndpointInformationService {

    /**
     * Gets the list of all endpoints for all IntegrationFlows used by the
     * adapter
     *
     * @return the list of endpoints
     */
    public List<AdapterEndpointInformation> getAdapterEndpointInformationList();

    /**
     * Gets the list of endpoints for a given IntegrationFlow. If the given
     * Integration Flows does not use the adapter, it has to return an empty
     * instance list.
     *
     * @param integrationFlowId
     *         the id (bundle symbolic name) of the Integration Flow
     * @return the endpoint information for the given Integration Flow
     */
    public AdapterEndpointInformation getAdapterEndpointInformation(String integrationFlowId);
}

```

The class describing the **adapter endpoint information** is mentioned here:

Source Code

```

package com.sap.it.api.adapter.monitoring;

import java.util.List;

/**
 * Class representing endpoint information for an Integration Flow
 */
public class AdapterEndpointInformation {
    [...]

    /**
     * Get the endpoint instance list for this Integration Flow.
     *
     * @return the endpoint instance list
     */
    public List<AdapterEndpointInstance> getAdapterEndpointInstanceList()
    [...]

    /**
     * Set the endpoint instance list for this object.
     *
     * @param adapterEndpointInstanceList
     *         the endpoint instance list
     */
    public void setAdapterEndpointInstanceList(List<AdapterEndpointInstance> adapterEndpointInstanceList)
    [...]

    /**
     *
     * @return the integration flow id (bundle symbolic name)
     */
    public String getIntegrationFlowId()
    [...]

    /**
     * sets the integration flow id
     */
}

```

```

    * @param integrationFlowId
    *         the integration flow id (bundle symbolic name)
    */
    public void setIntegrationFlowId(String integrationFlowId)
    {
    }

```

The class describing the **adapter endpoint instance** is mentioned here:

Source Code

```

package com.sap.it.api.adapter.monitoring;

/**
 * The endpoint instance consists of an <code>endpointCategory</code> which
 * describes the kind of endpoint and an URL relative to the worker node. It is
 * mainly about the distinction between endpoints to which messages can be sent
 * and API definition endpoints from which metadata about the API can be
 * retrieved.<br/>
 * The consumer of the API is responsible for retrieving the correct dispatcher
 * URL for the worker node (group) on which the Integration Flow is
 * deployed.<br/>
 * There is also an optional attribute available to provide additional info
 * about the character of the endpoint in case the category is not sufficient.
 * For CXF based endpoints, the WSDL download is offered in several flavors, for
 * instance, so the adapter can provide this information in that attribute.
 */
public class AdapterEndpointInstance {
    [...]

    /**
     * instantiates the object
     *
     * @param endpointCategory
     *         the endpoints category, see also {@link EndpointCategory}.
     * @param relativeEndpointUrl
     *         the URL relative to the worker nodes dispatcher URI starting
     *         with a forward slash ('/')
     */
    public AdapterEndpointInstance(EndpointCategory endpointCategory, String relativeEndpointUrl)
    {
    [...]

    /**
     * instantiates the object
     *
     * @param endpointCategory
     *         the endpoints category, see also {@link EndpointCategory}.
     * @param relativeEndpointUrl
     *         the URL relative to the worker nodes dispatcher URI starting
     *         with a forward slash ('/')
     * @param additionalInfo
     *         additional information (optional) in order to provide more
     *         information if the category is not self explanatory. For example, in
     *         case the WSDLs are offered in different flavors for different
     *         consumers
     */
    public AdapterEndpointInstance(EndpointCategory endpointCategory, String relativeEndpointUrl, String additionalInfo)
    {
    [...]

    /**
     *
     * @return the endpoints category
     */
    public EndpointCategory getEndpointCategory()
    {
    [...]

    /**
     *
     * @return the URL relative to the worker nodes dispatcher URI starting with

```

```

*          a forward slash ('/')
*/
public String getRelativeEndpointUrl()
    [...]

/**
 *
 * @return additional information (optional) in order to provide more
 *         information if the category is not self explanatory. For example, in case
 *         the WSDLs are offered in different flavors for different
 *         consumers. Can be null
 */
public String getAdditionalInfo()
    [...]
}

```

The *EndpointCategory* can have the following values:

Source Code

```

package com.sap.it.api.adapter.monitoring;

public enum EndpointCategory {
    ENTRY_POINT, DEFINITION_OAS_JSON, DEFINITION_OAS_YAML, DEFINITION_RAML_JSON, DEFINITION_RAML_YAML, DEFINITION_WSDL, DEF
}

```

Binding the Service with the Implementation

Next, you link the implementation to the service by creating a `beans.xml` file in the `src/main/resources/OSGI-INF/blueprint/` path and adding the following snippet:

Source Code

```

<blueprint xmlns="http://www.osgi.org/xmlns/blueprint/v1.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.osgi.org/xmlns/blueprint/v1.0.0 http://www.osgi.org/xmlns/blueprint/v1.0.0/blueprint.
    <bean id="iAdapterEndpointInformationService" class="<add the implementation class path>"
        activation="eager"></bean>
    <service id="IAdapterEndpointInformationService" ref="iAdapterEndpointInformationService"
        interface="com.sap.it.api.adapter.monitoring.AdapterEndpointInformationService">
    </service>
</blueprint>

```

To support the binding snippet, add the following dependency in the `pom.xml` file:

Source Code

```

<dependency>
<artifactId>camel-blueprint</artifactId>
<groupId>org.apache.camel</groupId>
<version>${camel.version}</version>
</dependency>

```

Updating the Dependency to Export the Service

Ensure that the following dependency version is always greater than or equal to **1.25.0**. to export this service from your subsystem.

Source Code

```

<dependency>
<groupId>com.sap.cloud.adk</groupId>
<artifactId>com.sap.cloud.adk.build.archive</artifactId>
<version>1.25.0</version>
</dependency>

```

Deploying the Adapter

After completing the process, deploy the adapter and use the Cloud Integration Web application to consume the adapter in an integration flow. Now, go to the [Manage Integration Content](#) section to verify the endpoint information.

Related Information

[Endpoint Visualization](#)

Managing Cluster Lock in Custom Adapters

Custom Integration Adapters provide a mechanism to acquire cluster-level locks, ensuring that specific actions (e.g., message processing) are executed by only one worker or node instance at a time. This prevents duplicate processing and maintains data consistency across distributed environments.

Prerequisites

- You have the necessary permissions to utilize the public APIs. See [SDK API](#).
- Your adapter should be configured to support cluster lock functionalities. See [Adapter Development Prerequisites](#).

Context

The LockManager interface offers following methods to acquire, release, and manage cluster-level locks.

- **newLock:** Creates a new lock with a specified expiration duration.

Source Code

```
Lock lock = lockManager.newLock("vendor", "component", "lockName", Duration.ofMinutes(30));
```

- **getAllLocksByVendor:** Retrieves all locks associated with a specific vendor.

Source Code

```
Set<LockInfo> locks = lockManager.getAllLocksByVendor("vendor");
```

- **getAllLocks:** Retrieves all locks for the current tenant.

Source Code

```
Set<LockInfo> locks = lockManager.getAllLocks();
```

- **releaseLocks:** Releases a set of locks identified by their lock IDs.

Source Code

```
int releasedCount = lockManager.releaseLocks(lockIds);
```

- **isLockAcquired:** Checks if a lock with a specific ID is currently held.

Source Code

```
boolean isAcquired = lockManager.isLockAcquired("lockId");
```

Procedure

1. Obtain LockManager Service.

Source Code

```
LockManager lockManager = ITApiFactory.getService(LockManager.class, null);
```

2. In your adapter's metadata, define the adapterInstanceID property:

Source Code

```

<Metadata>
  <AdditionalMetadata>
    <Name>requiredIntegrationFlowProperties</Name>
    <Value>adapterInstanceId</Value>
    <GuiLabels>
      <Label language="en"/>
    </GuiLabels>
  </AdditionalMetadata>
</Metadata>

```

This adapterInstanceId now can be accessed in your adapter runtime using an endpoint field. You can now use this value as lockname.

3. Use the newLock or newRuntimeLock method to create a lock.

Source Code

```
Lock lock = lockManager.newLock("vendor", "component", "lockName", Duration.ofMinutes(30));
```

4. Use the tryLock method to attempt acquiring the lock.

Source Code

```
boolean lockStatus = lock.tryLock();
```

5. Once the task is completed, release the lock.

Source Code

```
lock.unlock();
```

API Interface:

Sample Code

```

package com.sap.it.api.clusterlock;

import com.sap.it.api.ITApi;
import com.sap.it.api.clusterlock.types.LockInfo;

import java.time.Duration;
import java.util.Set;
import java.util.concurrent.locks.Lock;

/**
 * LockManager helps adapter developers to acquire, release and get information related to cluster level locks
 *
 */
public interface LockManager extends ITApi {

    /**
     * Gets a new {@link Lock} object which will be used for acquiring the lock
     * @param vendor - this should be the vendor name of the adapter. 'SAP' vendor name is reserved for internal usage
     * @param component - this should be the adapter name which is used while creating the adapter. Length restricted
     * @param lockName - a unique lockName (suggestion: Use either bundle symbolic name or the channelId)
     * @param expireAfter - time period after which lock expires
     * @return {@link Lock}
     */
    Lock newLock(String vendor, String component, String lockName, Duration expireAfter);

    /**
     * Gets a new {@link Lock} object which will be used for acquiring the lock with Runtime Scope.
     * This lock will be automatically released by the cleanup job if the node/worker/instance restarts due to a crash
     * @param vendor - this should be the vendor name of the adapter. 'SAP' vendor name is reserved for internal usage
     * @param component - this should be the adapter name which is used while creating the adapter. Length restricted
     * @param lockName - a unique lockName (suggestion: Use either bundle symbolic name or the channelId)
     * @param expireAfter - time period after which lock expires
     * @return {@link Lock}
     */
    Lock newRuntimeLock(String vendor, String component, String lockName, Duration expireAfter);

}

```

```
* Gets all {@link LockInfo}s by vendor name for the current tenant
* @param vendor - this should be the vendor name of the adapter. 'SAP' vendor name is reserved for internal usage
* @return {@link Set} of {@link LockInfo}s
*/
Set<LockInfo> getAllLocksByVendor(String vendor);

/**
 * Gets all {@link LockInfo}s for the current tenant
 * @return {@link Set} of {@link LockInfo}s
 */
Set<LockInfo> getAllLocks();

/**
 * Releases all the locks held with lockIds
 * @param lockIds - lock identifiers
 * @throws com.sap.it.api.clusterlock.types.LockException (is a RuntimeException) in case of failures
 * @return number of released locks
 */
int releaseLocks(Set<String> lockIds);

/**
 * Returns true if lock with lockId is held else false
 * @param lockId - lock identifier
 * @return true or false
 */
boolean isLockAcquired(String lockId);
```

Troubleshooting

Issue	Impact	Mitigation
Database connection unavailability	Inability to create or release locks.	Implement retry mechanisms in the adapter.
Lock deletion job failure	Locks may persist beyond their expiration, preventing new lock creation.	Implement retry mechanisms in the adapter.
Deployment inconsistencies	Potential for overload on a single node during deployment.	Implement random wait times for lock acquisition to distribute load evenly.
Unreleased locks due to system failures	Message processing downtime.	Release the lock from monitoring manually. See .
Inconsistent shutdown	Potential downtime if old worker nodes fail to shut down properly.	Release the lock from monitoring manually. See .

Results

Your custom adapters can now execute critical tasks by a single node with cluster-level locks, preventing duplicate processing and preserving system integrity.

Enabling ADK Persistency

As an integration developer you can now utilize an adapter API to store and retrieve critical information ensuring data persistence even after node restarts or crashes. This also ensures flexibility and reliability in data management.

You can also integrate this feature into your workflows to enhance system resilience and maintain seamless operations. It has following benefits:

- Reduces event duplication, ensuring efficient and streamlined processing.
- Avoids unnecessary costs by maintaining continuity.

To utilize this feature,

- Add an additional metadata entry in the metadata.xml file for your adapter:

```
<Metadata>
<AdditionalMetadata>
  <Name>ADKStoreRequired</Name>
  <Value>true</Value>
  <GuiLabels>
```

```

        <Label language="en"/>
    </GuiLabels>
</AdditionalMetadata>
</Metadata>

```

- Ensure to refer the adapter.api dependency in the pom.xml file. See [SDK API](#)
- In the endpoint class, the ADK store can be accessed by using the following getter and setter methods:

```

private ADKStore adkStore;

public ADKStore getAdkStore() {
    return adkStore;
}

public void setADKStore(ADKStore adkStore) {
    this.adkStore = adkStore;
}

```

i Note

This API should not be used to store any personal or sensitive information such as passwords and keys.

Frequent calls may impact the performance of the worker.

Enabling Authentication for Servlet Based Adapters

Context

Both Basic and Certificate based authentication can be enabled by performing the steps described here:

Procedure

1. Open the web.xml file and add the filter definition given below:

Source Code

```

<filter>
    <filter-name>ADKAuthenticationFilter</filter-name>
    <filter-class>com.sap.it.api.asdk.cloud.authentication.ADKAuthenticationFilter</filter-class>
</filter>
<filter-mapping>
    <filter-name>ADKAuthenticationFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>

```

2. Open the pom.xml file and include the below import package instruction in the maven-bundle-plugin:

Source Code

```

<Import-Package>com.sap.it.api.asdk.cloud.authentication,*</Import-Package>

```

3. Save the changes.

Results

Now you can connect to the adapter's endpoint either using Basic or Certificate based authentication.

Enabling Tracing for Custom Adapter

Configure a custom adapter with a tracing feature that records the processing of the incoming and outgoing messages in an adapter.

In the integration flow model, the logs generated by the adapter tracing help you to do the following:

- Monitor the overall status of the incoming or outgoing messages
- Troubleshoot specific errors encountered by the adapter during runtime

i Note

The incoming or outgoing messages in an adapter are only recorded if you have set the log level to **Trace** in the Cloud Integration Web application.

Adding Libraries

You can find the latest ADK API in the [Adapter Development Kit](#) section. After downloading the *.zip files, add the following libraries to the **src > main > resources** folder that is in your ADK project:

- com.sap.it.public.adapter.api.jar
- com.sap.it.public.generic.api.jar

For successful code compilation, right-click the *.jar file and choose **Build Path > Add to Build Path** to add the jar file to the Referenced Libraries of the adapter project.

Adding Dependencies

Open the pom.xml file and add the following dependencies to enable the tracing feature:

 Source Code

```
<dependency>
  <groupId>com.sap.it.public</groupId>
  <artifactId>generic.api</artifactId>
  <version>2.12.0</version>
  <scope>system</scope>
  <systemPath>${project.basedir}/src/main/resources/com.sap.it.public.generic.api-2.12.0.jar</systemPath>
</dependency>
<dependency>
  <groupId>com.sap.it.public</groupId>
  <artifactId>adapter.api</artifactId>
  <version>2.12.0</version>
  <scope>system</scope>
  <systemPath>${project.basedir}/src/main/resources/com.sap.it.public.adapter.api-2.12.0.jar</systemPath>
</dependency>
```

i Note

Make sure that the version of the *.jar files is greater than **2.12.0**.

After adding the dependency, add the com.sap.it.public group ID to the `<excludeGroupIds>` tag.

Enabling Tracing

The tracing feature is only relevant for adapters that transform the message either before sending or upon reception.

→ Recommendation

- Inbound messages are traced before the transformation takes place, which means that the user can see the incoming message with its original payload and headers. Outbound messages (headers and payload) are traced after the transformation has taken place.
- The trace data is written even if errors occur. This can help the user to analyze transformation errors for inbound messages or connection errors for outbound messages.
- The inbound message is traced after authentication and authorization have taken place. This is to prevent denial-of-service attacks that flood the trace database table.
- Security-relevant header values must be obfuscated.
- Trace data is only written if the adapter trace is enabled for the integration flow.
- If possible, also set the character encoding for the trace payload.

The following procedure helps you to implement adapter tracing in either Consumer or Producer component:

1. Use the following code snippet to retrieve the *Adapter Message Log Factory* from your endpoint:

Source Code

```
/** get Adapter Message Log Factory */
AdapterManagerLogFactory msgLogFactory = (AdapterManagerLogFactory) this.endpoint.getCamelContext().getRegistry()
    .lookupByName(com.sap.it.api.msglog.adapter.AdapterMessageLogFactory.class.getName());
```

2. Use the following code snippet to create the *Adapter Message Log* instance from the *Adapter Message Log Factory*.

Source Code

```
AdapterManagerLog mplLog = adapterMessageLogFactory.getMessageLog(exchange, description, "<CMD component ID of the ada
    UUID.randomUUID().toString());
```

3. Use the snippet to create the trace message:

Source Code

```
AdapterTraceMessage traceMessage = mplLog.createTraceMessage(type, body, truncated );
```

4. Use the snippet to write the trace message to the MPL log:

Source Code

```
mplLog.writeTrace(traceMessage);
```

The interfaces available for *Adapter Message Log* are:

```
/**
 * Returns <code>true</code> if the adapter tracing for the integration flow to
 * which this message log belongs is active, otherwise <code>false</code>.
 */
boolean isTraceActive();

/**
 * Creates a trace message which can be used in the method
 * {@link #writeTrace(AdapterTraceMessage)}.
 *
 * The supplied message body shall not exceed 25 MB = 25 * 1024 * 1024 bytes =
 * 26214400 bytes. We strongly recommend to supply a truncated copy of the
 * original message body if the original message body exceeds 25 MB. If, despite
 * of this rule, the supplied message body still exceeds 25 MB, then the body
 * will be truncated internally and the property <tt>isBodyTruncated</tt> will
 * be set to <code>true</code>.
 *
 * @param traceMessageType
 *         type of the trace message
 * @param body
 *         message body, shall not exceed 25 MB, truncate the body if
 *         necessary
 * @param isBodyTruncated
 *         indicator whether the supplied message body is truncated
 * @throws IllegalArgumentException
 *         if <tt>traceMessageType</tt> or <tt>body</tt> is
 *         <code>null</code>
 */
AdapterManagerLog createTraceMessage(AdapterTraceMessageType traceMessageType, byte[] body, boolean isBodyTruncated)

/**
 * Writes a trace message into the adapter trace if the trace for the
 * integration flow of the adapter is active.
 *
 * <p>
 * <b>Attention:</b>Camel endpoints, which are used in several from elements of
 * different Camel routes cannot write traces. This method will not have any
 * effect for such kind of endpoints.
 * <p>
 * Messages incoming to the adapter (inbound messages) shall be traced, if the
 * adapter transforms the message before it is handled over to the Camel route;
 * the trace shall be written <b>before</b> the transformation takes place.
 * Transformation here means changes to the message body or headers.
 * <p>
 * Messages outgoing from the adapter (outbound messages) shall be traced if the
 * adapter transforms the message before it is sent out; the trace shall be
 * written <b>after</b> the transformation.
 * <p>
 * Typical usage:
 *
 * <pre>
 * {
 *     if (this.isTraceActive()){
 *         byte[] body = getTraceBody(originalMessage);
 *         boolean isBodyTruncated = isBodyTruncated(body,originalMessage);
```

```

*          // specify the correct adapter message type, in this example we use AdapterTraceMessageType.SENDER_INBOUND
*          AdapterTraceMessage traceMessage = this.createTraceMessage(AdapterTraceMessageType.SENDER_INBOUND, body, :
*          // fill the optional properties of the trace message: encoding and headers
*          traceMessage.setEncoding(getEncoding());
*          traceMessage.setHeaders(getHeaders());
*          this.writeTrace(traceMessage);
*      }
*  </pre>
*
*  @param traceMessage trace message
*/
void writeTrace(AdapterTraceMessage traceMessage);

```

Write the trace data. You can use the following example for reference:

❖ Example

≡ Sample Code

```

import org.apache.camel.Exchange;
import com.sap.it.api.msglog.adapter.AdapterMessageLog;
import com.sap.it.api.msglog.adapter.AdapterMessageLogFactory;
import com.sap.it.api.msglog.adapter.AdapterTraceMessage;
import com.sap.it.api.msglog.adapter.AdapterTraceMessageType;

// OSGi service
private final AdapterMessageLogFactory adapterMessageLogFactory;

void writeTrace(Exchange exchange, byte[] traceData, boolean isOutbound) {
    // replace "<adapter type>" by your adapter type!
    String text = isOutbound ? "Sending <adapter type> message" : "Receiving <adapter type> message";
    // replace "<CMD component ID of the adapter>" by your CMD component ID
    AdapterMessageLog mplLog = adapterMessageLogFactory.getMessageLog(exchange, text, "<CMD component ID of the adapte
        UUID.randomUUID().toString());
    if (!mplLog.isTraceActive()) {
        return;
    }
    // if you have a fault inbound message then specify AdapterTraceMessageType.RECEIVER_INBOUND_FAULT,
    // if you have a fault outbound message then specify AdapterTraceMessageType.SENDER_OUTBOUND_FAULT
    // for synchronous adapters you may also need AdapterTraceMessageType.SENDER_OUTBOUND and AdapterTraceMessageType.
    AdapterTraceMessageType type = isOutbound ? AdapterTraceMessageType.RECEIVER_OUTBOUND : AdapterTraceMessageType.SE

    AdapterTraceMessage traceMessage = mplLog.createTraceMessage(type, traceData, false );//Setting isTruncated as fal
    // Encoding is optional, but should be set if available.
    traceMessage.setEncoding("UTF-8");
    // Headers are optional and do not forget to obfuscate security relevant header values.
    mplLog.writeTrace(traceMessage);
}

```

5. Build and deploy the adapter.

Viewing the Trace Data

To show the trace information details, select an integration flow from the overview list, go to the [Log](#) section, and open the link by clicking the [Trace](#). For more information, see [Message Processing Log - Adapter Tracing](#).

Logs

[Open Text View](#)

Trace data is removed after the configured retention time, typically 1 hour.

Log Level: **Trace**

Process ID:

Related Information

[Setting Log Levels](#)

Enabling Scheduler Support for ADK Sender Adapter

Context

Referenced scheduler component is equipped with an in-built locking feature for sender integration adapter. Also, a scheduler component comes with a standard User Interface. Follow these steps to use scheduler component in your custom adapter:

Procedure

1. Implement the polling sender by extending `<ScheduledPollingConsumer>`. Other consumers aren't supported.
2. Add the referenced component for the Sender Variant.

Source Code

```
<ReferencedComponents>
  <ReferencedComponent>
    <ReferencedComponentId>ctype::ExtensionVariant/cname::sap:Scheduler/version::1.0</ReferencedComponentId>
    <AttributeMetadataConfiguration>
      <Name>scheduleKey</Name>
      <AttributeBehavior>Scheduler_ScheduleOnDay,Scheduler_ScheduleToRecur</AttributeBehavior>
    </AttributeMetadataConfiguration>
  </ReferencedComponent>
</ReferencedComponents>
```

Note

Currently, only the above-listed behavior options `<Scheduler_ScheduleOnDay>` and `<Scheduler_ScheduleToRecur>` are supported.

3. Set the variant level attribute `<useDefaultScheduler>` as FALSE or skip it.

Source Code

```
<Variant VariantName="VariantName" VariantId="ctype::AdapterVariant/cname::GreetingsDs/vendor::sap/tp::sap-sample/mp::
  MetadataVersion="2.0" gen:RuntimeComponentBaseUri="sap-sample" AttachmentBehavior="Preserve"
  useDefaultScheduler="false">
  ...
</Variant>
```

4. Ensure that all the scheduler-related references (UI) aren't present in CMD.

Enabling Connection Status for Integration Flow

Prerequisites

This is custom documentation. For more information, please visit [SAP Help Portal](#).

You must have the access to Public APIs. For more information, see [SDK API](#).

Context

You can use the public API to publish the status of the connection to the integration flow monitoring for ADK based integration adapters. For example, this can provide you more insight on consumption of integration flow connection status.

Use the below interface, which is available for the ADK based integration adapters:

Source Code

```
package com.sap.it.api.adapter.iflowmonitoring;

import com.sap.it.api.ITApi;
import org.apache.camel.Endpoint;

/**
 * Service to submit an event on the status of the Connection as and when necessary.
 * <p>
 * The class can be initialized via the following statement:
 * </p>
 * <code>
 * IFlowMonitorService monitorService = ITApiFactory.getService(IFlowMonitorService.class, null);
 * </code>
 * <p>
 * EndPoint and EventDetails need to be passed.<br><br>
 * </p>
 * This interface must
 * neither be extended nor implemented by clients.
 * * @since 2.28.0
 */
public interface IFlowMonitorService extends ITApi {
    /**
     * Publishes event for the integration flow
     * @param endpoint The camel endpoint
     * @param eventDetails {@link EventDetails } Contains the details of the event
     * @throws IFlowMonitorServiceException The exception is thrown whenever endpoint/camelContext/EventStatus is invalid
     */
    void publishEvent(Endpoint endpoint, EventDetails eventDetails) throws IFlowMonitorServiceException;
}
```

Procedure

1. Invoke the `getService` method to get the integration flow monitor service API.

Source Code

```
IFlowMonitorService monitorService = ITApiFactory.getService(IFlowMonitorService.class, null);
```

2. Based on your requirement, invoke the `publishEvent` based on the event status type. You should use this API to submit an event on the integration flow.

Sample Code

Below example shows event submission in success state.

```
EventDetails eventDetails =new EventDetails();
eventDetails.setEventStatus(EventStatus.OK);
monitorService.publishEvent(endpoint, eventDetails);
```

Sample Code

Below example shows event submission in error state.

```
EventDetails eventDetails =new EventDetails();
eventDetails.setEventStatus(EventStatus.ERROR);
eventDetails.setException(exception);
```

```
monitorService.publishEvent(endpoint, eventDetails);
```

Accessing User Credentials

Context

Follow the procedure below to get user credentials in an ADK project.

Procedure

1. Download the latest ADK API from the [Adapter Development Kit](#) section.
2. Add the `com.sap.it.public.generic.api.jar` file to the `resources` folder that is in your ADK project.
3. Open the `pom.xml` file and add the dependency below:

Source Code

```
<dependency>
  <groupId>com.sap.it.public</groupId>
  <artifactId>api</artifactId>
  <version>2.8.0</version>
  <scope>system</scope>
  <systemPath>${project.basedir}/src/main/resources/com.sap.it.public.generic.api-2.8.0.jar</systemPath>
</dependency>
```

4. Add the `com.sap.it.public` group ID to the `<excludeGroupIds>`.
5. To import user credentials, use the code below:

Source Code

```
SecureStoreService secureStoreService = IApiFactory.getService(SecureStoreService.class, null);
UserCredential userCredential = secureStoreService.getUserCredential();

/**
 *Provide an alias name in getUserCredential()
 */
```

Add the following import declarations:

Source Code

```
import com.sap.it.api.securestore.SecureStoreService;
import com.sap.it.api.securestore.UserCredential;
import com.sap.it.api.securestore.exception.SecureStoreException;
import com.sap.it.api.IApiFactory;
```

Accessing Trust and Key Managers

Context

Follow the procedure below to get trust and key managers in an ADK project.

Procedure

1. Download the latest ADK API from the [Adapter Development Kit](#) section.
2. Add the `com.sap.it.public.generic.api.jar` file to the `resources` folder that is in your ADK project.
3. Open the `pom.xml` file and add the dependency below:

Source Code

```
<dependency>
  <groupId>com.sap.it.public</groupId>
  <artifactId>api</artifactId>
  <version>2.8.0</version>
```

```
<scope>system</scope>
<systemPath>${project.basedir}/src/main/resources/com.sap.it.public.generic.api-2.8.0.jar</systemPath>
</dependency>
```

4. Add the `com.sap.it.public` group ID to the `<excludeGroupIds>`.

5. Depending on your requirements, add the code below to either the `*Producer.java` (receiver adapter) or `*Consumer.java` (sender adapter) file. This allows you to import all the trust managers.

Source Code

```
KeystoreService keyStoreService = ITApiFactory.getService(KeystoreService.class, null);
TrustManager[] trustmanager = keyStoreService.getTrustManagers();
/**
 *Provide an alias name in getTrustManagers()
 */
```

Add the following import declarations:

Source Code

```
import javax.net.ssl.TrustManager;
import com.sap.it.api.ITApiFactory;
import com.sap.it.api.exception.InvalidContextException;
import com.sap.it.api.keystore.KeystoreService;
import com.sap.it.api.keystore.exception.KeystoreException;
```

6. To import all the key managers, use the code below:

Source Code

```
KeystoreService keyStoreService = ITApiFactory.getService(KeystoreService.class, null);
KeyManager[] keymanagers = keyStoreService.getKeyManagers();
/**
 *Provide an alias name in getKeyManagers()
 */
```

Add the following declaration:

Source Code

```
import javax.net.ssl.KeyManager;
```

Accessing On-Premise Application using Cloud Connector

Follow the steps here to extend the connectivity of Cloud Integration with remote on-premise application.

Prerequisites

You must have the access to Public APIs. For more information, see [SDK API](#).

Context

You can prepare an Adapter Development Kit (ADK) project to access any on-premise application using Cloud Connector.

Procedure

1. Depending on your requirements, add the code below to either the `*Producer.java` (receiver adapter) or `*Consumer.java` (sender adapter) file. This is for connecting to on-premise application.

a. For HTTP

Source Code

Use the below code to fetch the proxy details and additional headers.

```
CloudConnectorContext context = new CloudConnectorContext();
context.setConnectionType(ConnectionType.HTTP);
CloudConnectorProperties props = ITApiFactory.getService(CloudConnectorProperties.class, context);
String proxyHost = props.getProxyHost();
```

```
String proxyPort = props.getProxyPort();
Map<String, String> additionalHeaders = props.getAdditionalHeaders();
```

Add the following import declarations:

Source Code

```
import com.sap.it.api.ITApiFactory;
import com.sap.it.api.ccs.adapter.CloudConnectorContext;
import com.sap.it.api.ccs.adapter.CloudConnectorProperties;
import com.sap.it.api.ccs.adapter.ConnectionType;
```

Sample Code

The example here shows how you can use the above proxy details in HTTP client. Ensure to provide the location ID of the Cloud Connector using the header `SAP-Connectivity-SCC-Location_ID`.

```
DefaultHttpClient httpClient = new DefaultHttpClient();
HttpGet httpGet = new HttpGet(address);
HttpHost proxy = new HttpHost(proxyHost, proxyPort);
httpClient.getParams().setParameter(ConnRoutePNames.DEFAULT_PROXY, proxy);
httpGet.setHeader("SAP-Connectivity-SCC-Location_ID", locationId);
if (additionalHeaders != null && !additionalHeaders.isEmpty()) {
    for (Entry<String, String> header : additionalHeaders.entrySet()) {
        httpGet.setHeader(header.getKey(), header.getValue());
    }
}
HttpResponse response = httpClient.execute(httpGet);
```

b. For TCP

Source Code

Depending on the environment type there are two types of proxy authentication; via JWT access token and via Username and password. The simplest way to determine the type to be used is to look at cloud connector properties.

```
CloudConnectorContext context = new CloudConnectorContext();
context.setConnectionType(ConnectionType.TCP);
```

```
CloudConnectorProperties cloudConnectorProperties = ITApiFactory.getService(CloudConnectorProperties.class, context);
AuthType authType = cloudConnectorProperties.getAuthType();
```

Source Code

Proxy host and proxy port of the cloud connector can be accessed using the below API:

```
cloudConnectorProperties.getProxyPort();
cloudConnectorProperties.getProxyHost();
```

The following are intended to use with appropriate authentication methods only.

Source Code

For use with `AuthType.PasswordAuthentication`

```
cloudConnectorProperties.getUserName(locationId)
```

For more information, see [Username Password Authentication](#)

Source Code

For use with `AuthType.JWTTokenAuthentication`

```
cloudConnectorProperties.getJWTToken()
```

For more information, see [JWT Access Token Authentication](#)

2. Save the changes.

Versioning Rules for Custom Adapters

Follow the guidelines mentioned below to version a custom adapter.

i Note

This versioning feature is available from ADK version 1.38.0.

Versioning Rules

Version Type	Applicable For	Rules
Micro Version	Small changes such as label and tooltip updates	- Existing metadata variant should be updated with new micro version - For example, if the current version is 1.0.0, then the version has to be updated to 1.0.1 both at the component and variant level. - Relevant metadata changes has to be done in the same file.
Minor Version	Adding a new feature i Note It is not recommended to add mandatory fields. If needed, ensure that the runtime is backward compatible.	- Each variant should be in a single metadata file. If both sender and receiver version has to be updated, two separate metadata files has to be created for each. - Both the component and variant version has to be updated. - Necessary metadata changes has to be done in the newly created metadata file. - The existing version remains unchanged. So, the existing scenarios will have old user interface and it should continue to work as before. Only new scenarios will reflect the new version changes.
Major Version	Incompatible Changes i Note This is not supported.	

The step by step procedure to perform a micro and minor version update is explained below.

Micro Version Update

1. Ensure that the necessary changes has been made to the existing metadata variant.
2. Increment the micro version of the existing metadata both at component level and variant level.
3. Increment the version in `config.adk`.

Minor Version Update

1. Copy the variant of latest version to a new metadata file. Only one variant should exist within a metadata file for any new version.

i Note

The same metadata directory will contain multiple metadata files. Any naming convention can be followed for versioned files.

2. Make the minor version increment to the new metadata both at component and variant level.
3. Make the necessary changes to the metadata.
4. Make the necessary runtime changes.

i Note

Make sure the runtime is compatible with earlier version and the existing scenarios should continue to work.

5. Increment the version in `config.adk`.

Deploying a Secure Parameter Artifact

Use the secure parameter artifact to deploy confidential data for custom adapters.

Prerequisites

In the Node Explorer you have selected a tenant, in the context menu you have selected [Deploy Artifacts](#), and you have specified [Secure Parameter](#) as the artifact type.

Context

You can store secure data like passwords in a secure store and use an alias name to access this data in an integration flow. The secure parameter artifact contains this alias name and confidential data.

You can deploy the artifact on the tenant by following the procedure below.

Procedure

1. Specify the attributes on the wizard page.

Option	Description
Name	Provide a name for the artifact. The artifact name is used as an alias for the confidential data assigned by this parameter.
Description	Provide a more detailed description of the artifact.
Secure Parameter	Enter the confidential value of the attribute. i Note The possible length of the secure parameter varies: - for Cloud Foundry: 4096 characters max. (including spaces) - for Neo: 255 characters max. (including spaces)
Repeat Secure Parameter	Repeat the confidential value of the attribute.

2. Choose [Finish](#).

i Note

If correct alias is configured in the integration flow, then the runtime framework passes the secured artifact value to the component at runtime from the secure store.

Parameterizing Integration Flows Using the Partner Directory

The Partner Directory is a tenant-specific storage option that allows you to store information on business partners that are connected to the tenant in the context of a larger business network.

The Partner Directory facilitates the management of integration scenarios encompassing communication networks with numerous partners. In these scenarios, integration flows are parameterized with partner-specific parameters that are centrally stored in the Partner Directory. At runtime, integration flows can read information from the Partner Directory, based on the partner-specific values contained in the incoming request or the payload of the inbound message. This *dynamic* usage of integration flow parameters provides the advantage of adding new partners to the network without the need to edit and redeploy any integration flow, allowing for the extension of the business partner network without any downtime of the scenario.

i Note

Partner-specific information comprises information such as endpoints, XSLT mappings, XSD definitions, or certificates (to mention some examples).

You can access (read/write) the content of the partner directory through the OData API as well as during message processing through certain (parameterized) integration flow steps.

To display and maintain Partner Directory entries, there are the following options:

Option	Description
User interface access	In the Monitor section, you can access the Partner Directory, display existing entries and create new ones. See: Managing Partner Directory Entries
API access	You can access (read/write) its content through the OData API as well as during message processing through certain (parameterized) integration flow steps. See: Partner Directory

Want to Learn More

For an introduction into the concepts, see: [Partner Directory Concepts](#).

For information on the OData API access to the Partner Directory, see: [Partner Directory](#).

For information on the integration flow components that can be parameterized with Partner Directory content, see [Parameterizable Integration Flow Components](#).

Dedicated integration flow design guidelines show you how to use the Partner Directory. For more information, see [Use the Partner Directory Appropriately](#).

The following SAP Community blogs introduce you step-by-step to the topic:

- [Cloud Integration – Partner Directory – Step-by-Step Example](#) 
- [Cloud Integration – Partner Directory – Partner Dependent XML Structures and IDs](#) 
- [Cloud Integration – Partner Directory – Sender Partner Connecting with Client Certificate Authentication](#) 
- [Cloud Integration – Partner Directory – Partner Dependent User Credential Selection](#) 
- [Cloud Integration – Partner Directory – Mass Configuration](#) 

Caution

Limitations

Be aware of the following limitations when working with the Partner Directory:

Size restrictions for the different entity types:

- Maximum number of StringParameters overall: 3,000,000 (corresponds to 10,000 partners each using 300 StringParameters)
- Maximum number of BinaryParameters overall: 400,000 (corresponds to 10,000 partners each using 40 BinaryParameters)
- Maximum number of AlternativePartners overall: 1,000,000 (corresponds to 10,000 partners each using 100 AlternativePartners)
- Maximum number of AuthorizedUsers overall: 500,000 (corresponds to 10,000 partners each using 50 AuthorizedUsers)

Limit for certificate-to-user mapping (when using the Neo environment): 2 MB (corresponds to about 2000 X.509 certificates)

This component stores data on your tenant. Note that the tenant space is limited and shared with other tenant data like message processing logs.

i Note

The maximum size of a keystore is 2 MB (when using the Neo environment).

- The 2 MB limit corresponds to around 2000 X.509 certificates.
- A key pair with a chain of three X.509 certificates consumes about 3 KB, so if the keystore only contains key pairs of this type, then you can store around 600 key pairs in the keystore.

The maximum size of a keystore is 6 MB (when using the Cloud Foundry environment).

- The 6 MB limit corresponds to around 6000 X.509 certificates.
- A key pair with a chain of three X.509 certificates consumes about 3 KB, so if the keystore only contains key pairs of this type, then you can store around 1800 key pairs in the keystore.

If you upload a whole keystore (.jks file) to the tenant, the maximum keystore size is limited to 2 MB.

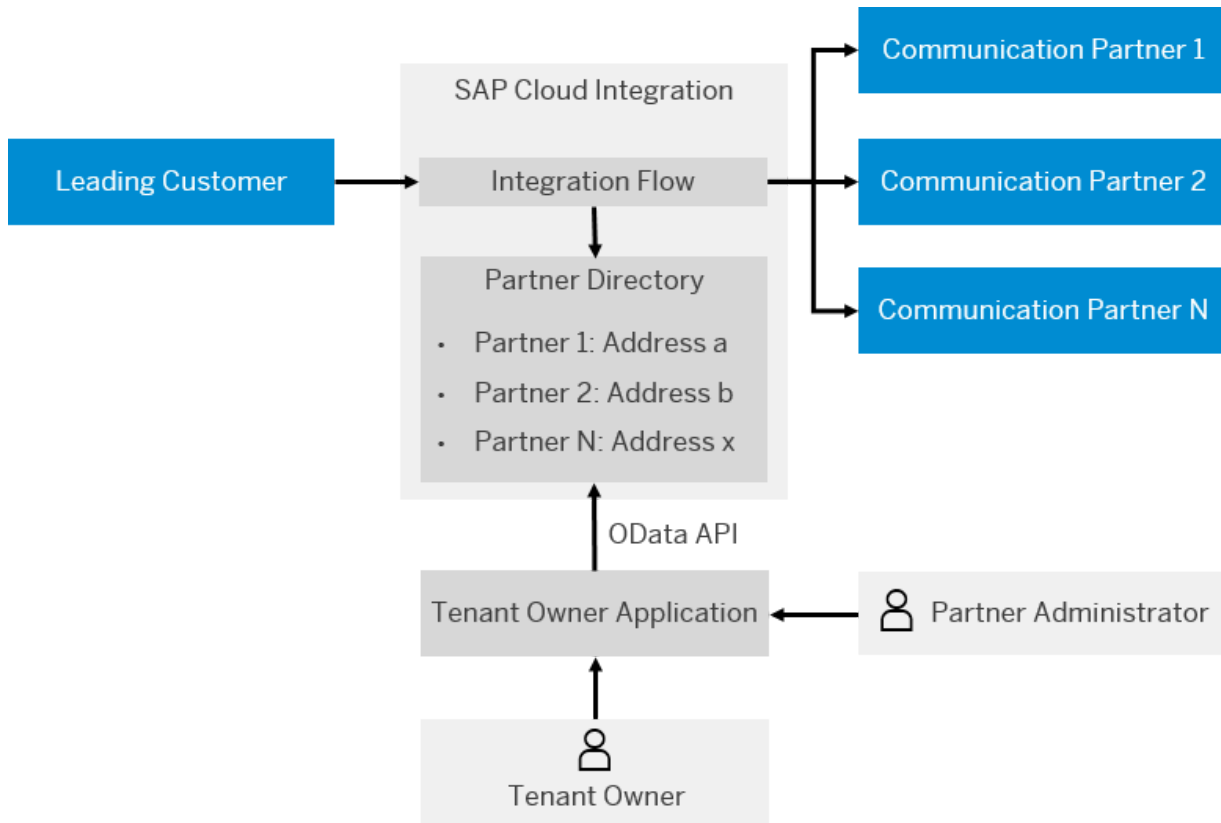
Partner Directory Concepts

In the context of a business-to-business (B2B) scenario involving a partner network, the person or organization that is responsible for the B2B scenario as a whole is also the owner of the SAP Cloud Integration tenant.

Administrators at the side of each business partner use a dedicated application (referred to as tenant owner application) to maintain entities in the Partner Directory (through an OData API). The tenant owner application is provided to the partners by the tenant owner.

Note that such a tenant owner application is not part of the feature set of SAP Cloud Integration. However, SAP Cloud Integration comes with a set of OData APIs that allow access to the Partner Directory (and can be used to implement such a tenant owner application).

As illustrated in the figure, the Partner Directory is embedded in the system landscape in the following way.



Partner Directory Entities

The Partner Directory contains the following entities (which can be accessed using an OData API):

- Partner ID (PID)

A partner has an ID (PID) that is unique within the Partner Directory. The uniqueness of the PID is ensured by the tenant owner application.

- Alternative Partner

A partner can have several alternative identifiers (Alternative Partner). The same concepts are applied to the Alternative Partner as to party identifiers in SAP Process Integration: Each Alternative Partner has three string fields: Agency, Scheme, and ID.

- Agency

Name of the organization that defines the identification scheme (or schema) and issues names for the objects to be identified.

- Scheme

The reference framework within which objects are uniquely identified by names.

- ID

An ID or name that identifies the object within the given scheme.

For more information on the alternative partner ID and how it is related to the partner ID, check out the following SAP Community blog: [Cloud Integration – Partner Directory – Partner Dependent XML Structures and IDs](#).

For more information on the usage of the fields Agency, Scheme, and ID, see the documentation of SAP Process Integration at <http://help.sap.com>.

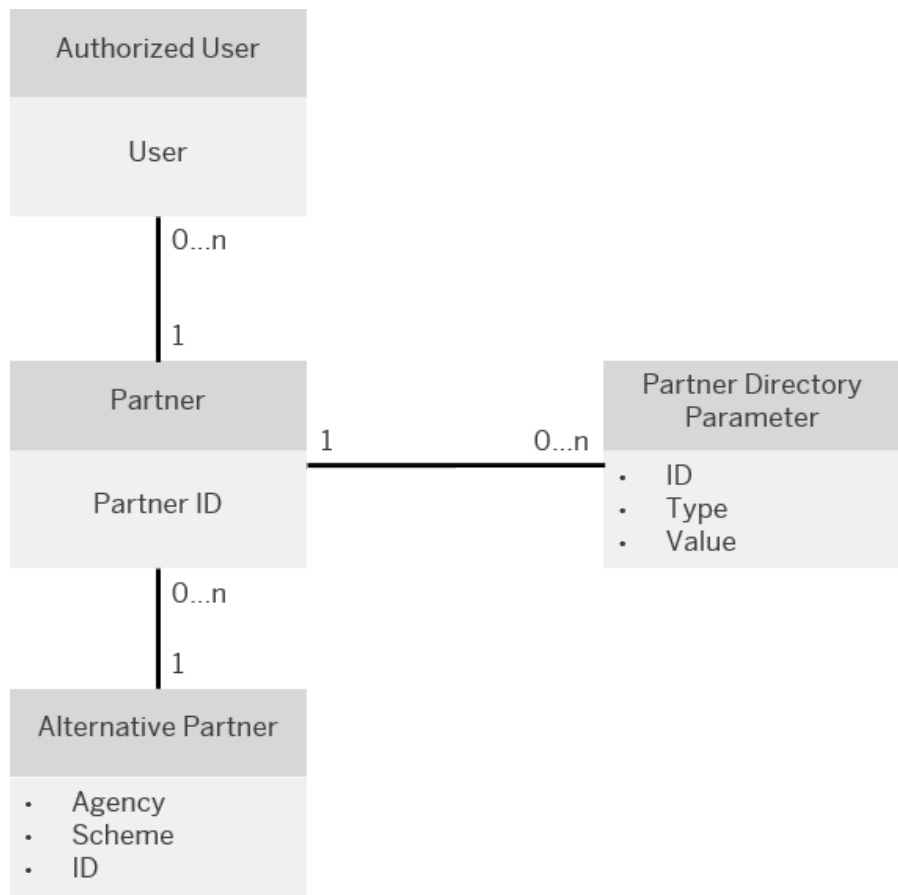
- Authorized User

This user authorizes a sending partner system to log in to SAP Cloud Integration (inbound communication).

If the partner uses HTTPS with client certificate authentication to connect to SAP Cloud Integration, certificate-to-user mappings are applied. One or more Authorized Users can be defined for each partner.

- Partner Directory Parameters for the parameterization of integration flows

The following figure provides an overview of the Partner Directory entities.



The Partner Directory entities related to a partner are maintained by an administrator at the partner organization using the tenant owner application. The Partner Directory entities are accessed using an OData application interface.

Related Information

- [Partner Authorization \(Inbound\)](#)
- [Dynamically Reading XSLT Mappings from the Partner Directory](#)
- [Dynamically Reading XSD Files from the Partner Directory](#)
- [Partner Directory Cache](#)

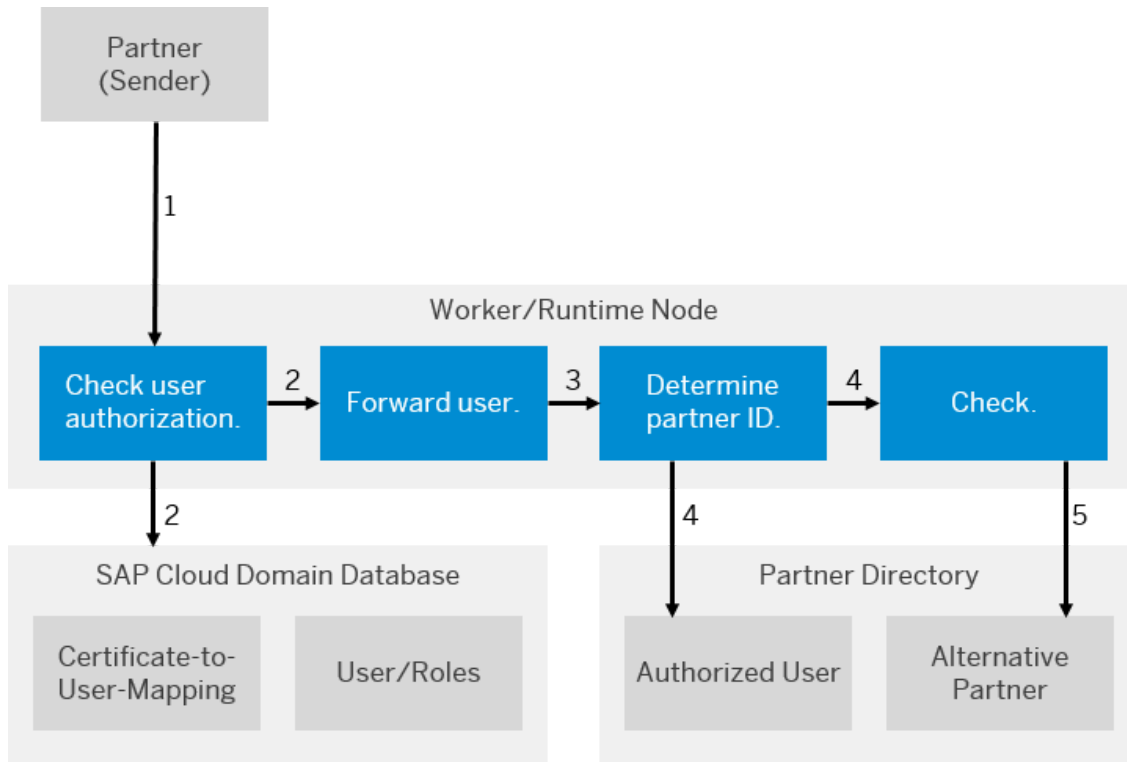
Partner Authorization (Inbound)

For inbound calls (when a partner sends a message to the integration platform), a partner authorization check evaluates whether the logged-in user has permission to execute messages with a particular sender partner ID.

Basic authentication and client certificate authentication using certificate-to-user mappings are both supported.

Note that the authorization check is implemented based on a script contained in the deployed integration flow (referred to as the *authorization check script* in the following).

The following figure shows how the partner authorization check works (the numbers in the figure correspond to those in the numbered list below).



1. The partner sends a message to the integration platform (Cloud Integration) and the user associated with the sender system is logged in.

The sender calls the tenant either by basic authentication, client certificate authentication, or OAuth authentication with client certificate grant.

- If basic authentication is configured, the user is a user from the SAP ID service.
- If client certificate authentication is configured, it is a user defined in a certificate-to-user mapping.
- If OAuth authentication is configured, the user name has the form `oauth_client_<client ID>` where `<client ID>` is the client ID of the OAuth client.

2. Cloud Integration checks whether the role `ESBMessaging.send` is assigned to the logged-in user.

3. The sender adapter forwards the logged-in user as the subject in the Camel header `SapAuthenticatedUserName`.

4. From the user (contained in the Camel header `SapAuthenticatedUserName`, which is read using a [Content Modifier](#) step), the authorization check script determines the Partner IDs by checking the list of Authorized Users in the Partner Directory.

5. The authorization check script reads the Partner ID or the Alternative Partner from the message content. If an Alternative Partner is read from the message content, it is mapped to the Partner ID stored in the Partner Directory.

The authorization check script compares the Partner ID determined from the Authorized User table with the Partner ID derived from the message. If they are identical, the message has been processed.

Note that for each Authorized User (provided by a partner), a user-to-role assignment (to role `ESBMessaging.send`) is automatically generated by the system (using a REST API from SAP BTP) prior to the execution of the scenario.

Dynamically Reading XSLT Mappings from the Partner Directory

You can configure the XSLT Mapping step in an integration flow to dynamically read XSLT mappings from the Partner Directory. The Mapping step will then point to an XSLT mapping defined in the Partner Directory.

To do this, in the XSLT Mapping step, specify the name of a header that points to the XSLT mapping in the Partner Directory.

Select [Header](#) as the [Source](#) in the XSLT Mapping step.

In order for the scenario to work, there must be an additional integration flow step (either a Script step or a Content Modifier step) that sets the header so that the XSLT Mapping step can refer to it.

In this additional step, the header must be specified as a URI in the following format:

`pd:<Partner ID>:<Parameter ID>:Binary`

This URI specifies a `Binary` `Parameter` from the Partner Directory. At runtime, the XSLT Engine resolves this URI.

i Note

XSL documents that are stored in the Partner Directory can be referenced in other XSL and XML mappings by specifying it in the Partner Directory URI (using the `xsl:import` or "document" feature).

For detailed information, check out the [blog post](#) about partner dependent XML structures and IDs.

Example

The integration flow contains a Content Modifier step followed by an XSLT Mapping step. You can configure both steps so that an XSLT mapping is dynamically read from the Partner Directory.

In the **XSLT Mapping** step, specify the following settings:

In the **Source** field, select **Header**, and in the **Header Name** field, specify `XSLT_FROM_PD`. This is the name of the header, which must be filled in a previous step (in our example, a Content Modifier).

In the **Content Modifier** step, specify the following settings (under **Message Header**):

Attribute	Specify With the Following Entry ...
Name	Enter <code>XSLT_FROM_PD</code> .
Type	Select Expression .
Data Type	Enter <code>java.lang.string</code> .
Value	Enter <code>pd:\${property.SENDER}:xslt:Binary</code>. The expression <code>\${property.SENDER}</code> makes sure that the Partner ID is dynamically read from the Partner Directory, the entry <code>xslt</code> is the name of the parameter for the XSLT mapping in the Partner Directory, and the expression <code>Binary</code> classifies it as a binary parameter.

Dynamically Reading XSD Files from the Partner Directory

You can configure the XML Validator step in an integration flow to dynamically read XML schema (XSD) files from the Partner Directory. The XML Validator step will then point to an XSD file in the Partner Directory.

To do this, in the XML Validator step, specify the name of a header that points to the XSD file in the Partner Directory.

In the **XML Schema** field, enter the Camel simple expression for the header that should point to the corresponding Partner Directory entity.

In order for the scenario to work, there must be an additional integration flow step (either a Script step or a Content Modifier step) that sets the header so that the XML Validator step can refer to it.

In the additional step, the header must be specified as a URI in the following format:

`pd:<Partner ID>:<Parameter ID>:Binary`

This URI specifies a `Binary Parameter` from the Partner Directory. At runtime, the XSLT Engine resolves this URI.

i Note

XSL documents that are stored in the Partner Directory can be referenced in other XSL and XML mappings by specifying it in the Partner Directory URI (using the `xsl:import` or "document" feature).

For detailed information, check out the [blog post](#) about partner dependent XML structures and IDs.

Example

The integration flow contains a Content Modifier step followed by an XML Validator step. You can configure both steps so that an XSD file is dynamically read from the Partner Directory.

In the **XML Validator** step, enter the following simple expression as the **XML Schema**: `${header.XSD_FROM_PD}`.

`XSD_FROM_PD` is the name of the header, which must be filled in a previous step (in our example, a Content Modifier).

In the **Content Modifier** step, specify the following settings (under **Message Header**):

Attribute	Specify With the Following Entry ...
Name	Enter XSD_FROM_PD.
Type	Select Expression.
Data Type	Enter java.lang.string.
Value	Enter pd:\${property.SENDER}:xsd:Binary. The expression \${property.SENDER} makes sure that the Partner ID is dynamically read from the Partner Directory, the entry xsd is the name of the parameter for the XSD file in the Partner Directory, and the expression Binary classifies it as a binary parameter.

Partner Directory Cache

To improve performance, Partner Directory is cached.

Entities from the Partner Directory are cached on runtime nodes. From there, they can be accessed with good performance from Script steps in integration flows deployed on the same runtime node.

The following Partner Directory entities are cached:

- StringParameter
- BinaryParameter
- AuthorizedUser

If an object (for a requested key) is not found in the cache, it is loaded from the Partner Directory.

Maximum cache size is restricted to 5% of the total heap size. Elements are evicted from the cache according to the Least Recently Used order.

An entry in the cache is invalidated when the entry is updated or deleted. This invalidation can take a few minutes. The invalidation time depends on the number of changes that are done during design time. The more changes are done at the same time, the longer the invalidation takes.

Parameterizable Integration Flow Components

The following integration flow components are parameterizable so that partner-specific information (such as partner endpoint address, specific mapping, client certificates for inbound calls) can be used at runtime.

- XSLT: XSLT files are specified by each partner
- XML Schema Validator: XSD files are specified by each partner
- HTTP receiver adapter: receiver address (however, one common user or client certificate is used to call the partner system)
- HTTP receiver adapter: user credential
- HTTP sender adapter: sender partner-specific user or client certificate
- SOAP receiver adapter: user credential
- Script: for accessing partner-specific information and performing partner-specific operations

Partner Directory Parameters for AS2 Adapter

The AS2 sender reads the following parameters from the Partner Directory:

Field	Parameter	Value
Decrypt Message	SAP_AS2_Inbound_Decrypt_Message	- true - false
Private Key Alias	pd:<pd parameter name>	Parameter name of private key alias in Partner Directory.
Verify Signature	SAP_AS2_Inbound_Verify_Signature	- notRequired - trustedCertificate

Field	Parameter	Value
		trustedRootCertificate
Public Key Alias	pd:<pd parameter name>	Parameter name of public key in Partner Directory.

AS2 MDN sender reads the following parameters from Partner Directory:

Field	Parameter	Value
Verify Signature	SAP_AS2_Inbound_Mdn_Verify_Signature	- true - false
Verify MIC	SAP_AS2_Inbound_Mdn_Verify_Mic	- true - false
Public Key Alias	pd:<pd parameter name>	Parameter name of public key in Partner Directory.

i Note

PD parameters are shown in the MPL log as MPL properties.

API Documentation

Cloud Integration provides various application programming interfaces (APIs) with which you can access data.

Overview

Cloud Integration provides SDK APIs (or Java APIs) and OData APIs for accessing various aspects of the platform. Furthermore, OData APIs can be easily found and tested in the SAP Business Accelerator Hub. For further information, simply click on the tiles below:

This image is interactive. Hover over each area for a description. Click highlighted areas for more information.

SDK API

OData API

SAP Business Accelerator Hub

Please note that image maps are not interactive in PDF outputs.

SDK API

APIs can be accessed using either Groovy Script or Java Script as the programming language. Besides, they can also be used when developing SAP Cloud Integration adapter using the Software Development Kit (short: SDK). For more information, see [SDK API](#).

OData API

OData APIs are implemeted as REST API and use the Open Data Protocol as a technical protocol. For more information, see [OData API](#).

OData APIs on SAP Business Accelerator Hub

Cloud Integration OData APIs can be tested and explored on SAP Business Accelerator Hub under <https://api.sap.com/package/CloudIntegrationAPI/odata>. They are structured around entity types or resources. Click on the tiles below to find more information on the entity types and aspects.

This image is interactive. Hover over each area for a description. Click highlighted areas for more information.

Integration Content

Log Files

Message Processing Logs

Message Store

Partner Directory

Security Content

Please note that image maps are not interactive in PDF outputs.

The OData APIs are protected by basic authentication (username and password). For more information, click on the tile below:

This image is interactive. Hover over each area for a description. Click highlighted areas for more information.

Authentication

Please note that image maps are not interactive in PDF outputs.

Additionally, the OData APIs make use of common query options:

This image is interactive. Hover over each area for a description. Click highlighted areas for more information.

Query Options

Please note that image maps are not interactive in PDF outputs.

SDK API

SDK APIs refer to the overall set of APIs like the Generic API, Adapter API, Script API.

Application programming interfaces (APIs) are available for the following use cases:

- Using a programming language (Java Script or Groovy), you can control how a message is being processed on a tenant.
- Using an API, you can build your own custom adapter in the context of the Adapter Development Kit.

i Note

The Java standard libraries of Java 8 can be used.

Cloud Integration supports the XML Document Object Model (DOM) to process XML documents.

You can implement logging by using Simple Logging Façade for Java (SLF4J) API package. We recommend using SLF4J framework over using your own logging frameworks.

Public APIs in Maven Central Repository

Some of the public APIs are available in the Maven Central Repository. Look out for the specific APIs with their associated coordinates. Also, for detailed information about these APIs, refer to their corresponding JavaDocs mentioned in the next paragraphs.

Adapter API

Use Adapter APIs if you want to develop custom adapters.

To access the Adapter API from [Maven Central Repository](#) , do the following steps:

1. Look out for the latest public API in Maven Central Repository. The group ID is `com.sap.cloud.adk` and artifact ID is `adapter.api`.
2. Add the maven dependency for the API in your adapter pom file. For example, the sample that follows is a way to consume the adapter API.

Sample Code

```
<dependency>
  <groupId>com.sap.cloud.adk</groupId>
  <artifactId>adapter.api</artifactId>
  <version>XX.YY.ZZ</version>
  <!-- replace XX.YY.ZZ with the latest version -->
</dependency>
```

3. Add `com.sap.cloud.adk` to the `excludeGroupIds` section in the **Configuration Settings** of the **Maven Dependency Plugin**. For example:

Sample Code

```
<excludeGroupIds>com.sap.cloud.adk,org.apache.camel,org.slf4j,log4j</excludeGroupIds>
```

From the [versions](#) ➦ tab on Maven central repository, choose [Browse](#) for the respective version to access the [javadoc](#) ➦ . The javadoc for Adapter API contains the complete set of packages and classes.

i Note

On how to consume cluster lock feature from the package, see [Component Metadata](#).

Script API

Use the Script APIs (version 1 or 2) if you want to use the Script step in your integration flow.

Look out for latest public script API [version 1](#) ➦ and [version 2](#) ➦ from Maven Central Repository. Use the maven coordinates that follow in your project to consume the script API.

For Script version 1 (v1):

≡, Sample Code

```
<dependency>
  <groupId>com.sap.cloud.script</groupId>
  <artifactId>script.api</artifactId>
  <version>XX.YY.ZZ</version>
  <!-- replace XX.YY.ZZ with the latest version -->
</dependency>
```

The [javadoc for Script v1 API](#) contains the complete set of packages and classes.

For Script version 2 (v2):

≡, Sample Code

```
<dependency>
  <groupId>com.sap.cloud.script</groupId>
  <artifactId>script.v2.api</artifactId>
  <version>XX.YY.ZZ</version>
  <!-- replace XX.YY.ZZ with the latest version -->
</dependency>
```

The [javadoc for Script v2 API](#) ➦ contains the complete set of packages and classes.

Generic API

Generic APIs are common for Script step and developing custom adapters.

To access the Generic API from [Maven Central Repository](#) ➦ , do the following steps:

1. Look out for the latest public API in Maven Central Repository. The group ID is `com.sap.cloud.adk` and artifact ID is `generic.api`.
2. Add the maven dependency for the API in your adapter pom file. For example, the sample that follows is a way to consume the generic API.

≡, Sample Code

```
<dependency>
  <groupId>com.sap.cloud.adk</groupId>
  <artifactId>generic.api</artifactId>
  <version>XX.YY.ZZ</version>
  <!-- replace XX.YY.ZZ with the latest version -->
</dependency>
```

3. Add `com.sap.cloud.adk` to the `excludeGroupIds` section in the [Configuration Settings](#) of the [Maven Dependency Plugin](#). For example:

≡, Sample Code

```
<excludeGroupIds>com.sap.cloud.adk,org.apache.camel,org.slf4j,log4j</excludeGroupIds>
```

From the [versions](#)  tab on Maven central repository, choose [Browse](#) for the respective version to access the javadoc. The [javadoc](#)  for Generic API contains the complete set of packages and classes.

Mapping API

The [javadoc for Mapping API](#) contains the complete set of packages and classes.

Related Information

[Developing Custom Adapters](#)

Setting and Getting SOAP Headers

Understand how SOAP headers can be accessed, set, and read through an API.

A SOAP message consists of SOAP headers and a SOAP body. SOAP Headers need to be accessed by an API for further processing.

SOAP Header Attributes

The attributes that can be used in the SOAP header are conform to the SOAP specifications. You can set two attributes for the SOAP header:

- **MustUnderstand** defines whether the header needs to be processed by the recipient or not. If the value is set to [true], the processing of the header becomes mandatory and you need to specify the header in the WSDL file. Otherwise, the header will be rejected. If the value is set to [false], the header does not have to be processed.
- **Actor** defines who of the processors has to react to the message. It can be used for your own purposes as it is not used at runtime. If you specify more than one actor, it is possible to route the message to multiple recipients with only the specific information that is relevant to each of the recipient.

How to Set and Get SOAP Headers

There are three methods which can be used to process SOAP headers:

- **GetSoapHeaders**: returns a list (ArrayList) of SOAP headers represented by objects of type SOAPHeader.
- **SetSoapHeaders**: sets SOAP headers in a request or response message before the message is sent out. This method can be used in an integration flow as often as required.
- **ClearSoapHeaders**: removes headers that were received, so that they won't be propagated.

You can create a SOAP header instance using a DOM tree element or a Java string class instance.

Check out the related blog if you want to know how to process SOAP headers in an integration flow. The blog describes a detailed integration flow example using a script step: [Cloud Integration - Accessing and Setting SOAP Headers in an Integration Flow](#) 

Using Script API Methods in Groovy Scripts

Prerequisites

- You have installed Groovy plugins for Eclipse.

Context

If you are creating a script based on the [Message](#) class from SAP, you can use a jar file that contains the definition of the [Message](#) class in your script project to make your life easier. By using this jar file, Eclipse provides content assist while writing scripts for the [Message](#) class. This enables you to complete the script easily.

Here's how you can import this jar into your script project:

Procedure

1. Download the JAR from the [Using Script API](#) section of the [Cloud Integration Tools Page](#).
2. Create a new [Groovy Project](#).
3. In the [Build Settings](#), choose [Add External JARs](#) to add the downloaded JAR file to your groovy project.

4. When you create a new groovy class, reference the groovy script project where you have added the JAR.

Results

When you are creating the script, Eclipse automatically suggests options when you are inserting methods from [Message](#) class.

Creating an Exception Response for SOAP-Based Adapters

Understand how to populate the fields in the exception response for SOAP-based adapters via API by populating/overwriting the generated exception.

i Note

This topic is related to SOAP-based adapters. This includes the following adapter types:

- SOAP SOAP 1.x
- SOAP SAP RM
- IDoc
- XI

If an error is thrown during processing, the user receives a detailed fault description. Overwrite that generated exception with exception responses adapted to the workflow's needs.

The `com.sap.gateway.ip.core.customdev.processor.SoapFaultFactory` class provides the following methods:

Method	Description
<code>throwSoapFault(SoapFaultParam param)</code>	Use this method for SOAP sender channels. It allows the setting of all attributes that the SOAP specification describes, especially the SOAP version. For details, see the SOAP specification. Be aware that there are some differences between SOAP 1.1 and SOAP 1.2. In particular, SOAP 1.2 only allows certain fault codes. • <code>@param param</code>: Object to set all parameters according to the SOAP specification. The parameter detail is expected to be an Element object of a DOM tree. The parameter namespaces can be used to add additional namespace declarations for parameter detail if required. • <code>@throws: UnknownSoapVersionException</code> • <code>@throws UnknownFaultCodeException</code>
<code>throwSoapFault(String faultstring, Element detail, Map<String, String> namespaces)</code>	Use this method for SOAP sender channels. It provides a minimum set of parameters to only set text parameters. All other parameters are set to their defaults. The SOAP version is set to SOAP 1.1. • <code>@param faultstring</code>: See also the SOAP specification. This is a simple error text. • <code>@param detail</code>: See also the SOAP specification. Here you can set additional information. The SOAP fault exception object of the SOAP adapter required here is an Element object of a DOM tree. • <code>@param namespaces</code>: If the detail parameter requires additional namespace declarations, then they can be set here.
<code>throwXiSoapFault(XiSoapFaultParam param)</code>	Use this method for XI sender channels. It throws a SOAP fault exception that fulfills the XI protocol specification. This specification basically restricts the detail attribute of the SOAP fault to a certain format. For the SOAP adapter, this can be freely chosen. In the XI adapter, the detail has to follow the XI error header specification. In addition, the SAP category is always set to 'XI Server' and the SAP code is set to 'INTERNAL.MESSAGE'. All other codes and categories are reserved. P1 to P4, additional text, and stack can be freely chosen. • <code>@param param</code>: Object to set all possible attributes of an XI SOAP fault. • <code>@throws XMLStreamException</code>

Check out the related blog if you want to know how to process SOAP headers in an integration flow. The blog describes a detailed integration flow example using a script step: [Cloud Integration - Accessing and Setting SOAP Headers in an Integration Flow](#) 

Related Information

[Creating Scripts in a Script Collection](#)

[XI Adapter](#)

OData API

The Cloud Integration application programming interface (API) allows you to access Cloud Integration resources, for example, monitoring data.

The OData API is implemented as a REST API and the technical protocol is Open Data Protocol (OData). This means that you can use standard HTTP methods (for example, the GET method) to call the API.

You can find the OData API on SAP Business Accelerator Hub at:

[SAP Cloud Integration](#) 

A prerequisite for using the OData API is that your user is connected to Cloud Integration and that you have access to an OData client.

Individual actions that you can perform using the OData API are protected by an authorization concept. To find out the required role for the task that you want to perform with the OData API, see [Tasks and Permissions](#).

The OData API covers the following aspects:

OData API Overview	
Section	Description
Integration Content	Get deployed integration content or deploy/undeploy integration artifacts.
Log Files, Neo Environment	Get an overview of HTTP and trace log files.
Message Processing Logs	Get an overview of the messages processed on a tenant and get the details for individual messages.
Message Stores	Get data from the Message Store, such as message payloads, headers, properties, or attachments for processed messages.
Partner Directory	Get, write, or delete data of the Partner Directory. This content can be used to parameterize integration flows.
Security Content	Get, write, or delete security content, for example, keystore entries, user credentials, or certificates to user mappings.
	Access your Edge Integration Cell data even during offline mode when the Operations Cockpit user interface is unavailable. You can retrieve information such as component status, pods, event logs, metrics, and runtime parameters.

The OData API is structured by a number of entities, grouped on SAP Business Accelerator Hub according to the sections listed in the previous table. Each entity (resource) is specified by a number of properties, to which filtering options can also be applied.

You can address the collection of entries or a single entry within a collection.

How the entities are related to each other is described in the entity data model. This model also provides information on how to navigate between entities using the API.

This documentation provides an overview of the available resources and summarizes the supported HTTP methods and query options for each resource.

→ Tip

For more information about the programming model and how the HTTP methods and query options work, see [OData - the Best Way to REST](#) .

OData specification version 2.0 is supported by the Cloud Integration API.

HTTP Calls and URI Components

More information: [HTTP Calls and URI Components](#)

Recommendations to Improve OData Request Performance

Using the OData API allows you to run certain requests on data processed by and stored on SAP BTP. The load that these requests put on the platform depends on the URL pattern and the respective parameters specified in the request, as well as the amount of data available. The latter depends directly on the details of message processing.

⚠ Caution

SAP has set a maximum time for the execution of an OData request. It is the customer's responsibility to design OData requests so that the set of returned data sets remains manageable (as far as this suits the requirements of the client making the request).

To keep the load on the network manageable, we therefore recommend that you use query options as follows:

- Make requests as selective as possible by using the `$filter` option. For example, when you access message processing logs (MPLs) using the API, you can filter in such a way that the request retrieves MPLs in a restricted time interval only, with the log end time between two nearby datetime literals.
- Use the `$select` option to make the API return only those properties that are required by the client. This reduces network load.
- If it is necessary to navigate a large number of data sets (>1000), use the `$top` and `$skip` options to iterate over the result set. Consider designing the client application in such a way that it offers paging options.
- Note that when you use the OData API, you navigate to MessageStore entities starting from the MessageProcessingLog entity. Apply filter options to the MessageProcessingLog rather than the MessageStore entities.

i Note

The DataStore and Variables APIs do not support the following query options:

- - `$filter`
 - `$inlinecount`
 - `$orderby`
 - `$skip`
 - `$top`
 - `$expand`
 - `$select`

Related Information

[Integration Content](#)
[Log Files, Neo Environment](#)
[Message Processing Logs](#)
[Message Stores](#)
[Partner Directory](#)
[Security Content](#)
[Query Options](#)
[Authentication](#)

API Details

The API details correspond to the details shown in the SAP Business Accelerator Hub for : [SAP Cloud Integration API Package](#) 

Related Information

[Integration Content](#)
[Log Files, Neo Environment](#)
[Message Processing Logs](#)
[Message Stores](#)
[Partner Directory](#)
[Security Content](#)

Integration Content

Manage integration artifacts for your tenant.

You can access the OData API at:

[Integration Content](#) 

There, you find the basic operations.

This documentation provides additional information.

Overview

Operating an integration scenario requires certain integration artifacts. Using the OData API you can access integration artifacts.

Integration artifacts contain the information required for the runtime components to process messages in the context of a certain integration scenario. First and foremost, an integration flow artifact contains all information about the connectivity settings and processing steps modeled in an integration flow model with the Web UI (in the [Design](#) section). When an integration developer has finished the modeling tasks, they deploy the integration - as an *integration artifact* - on the tenant and, that way, makes it executable.

There are more kinds of artifacts, as summarized in the following table.

Integration Artifact Types

Artifact Type	Description
Integration flow	An executable integration flow model An integration flow specifies the way how a message is to be processed between different systems. In particular, you define in an integration flow: the senders and receivers of a message, how the senders and receivers are connected to the tenant (adapters), and the steps that define the message processing. You can design an integration flow in the Web UI Design section.
Value mapping	An executable value mapping definition A value mapping allows you to transform one representation of an object to another representation. In most situations when data is exchanged between two systems, two ways of representing the same data need to be translated. For example, one specific person is referred to as a particular customer number in one system and as an employee number in another system. Both numbers refer to the same person. You can design a value mapping in the Web UI Design section. i Note You can create a new Value Mapping artifact from any existing/downloaded Value Mapping project. You can upload either a .zip or a .jar file format.
OData API	An executable OData API model You can use an OData API to expose data sources as OData endpoints. You can design an OData in the Web UI Design section. The system generates from an OData API an integration flow that contains by default an OData sender adapter.

Alternatively, you can access integration content through the OData API.

Permissions

To authorize an API client to access the OData API, perform the steps as described at:

- In the **Cloud Foundry** environment, perform the steps described at: [Setting Up Inbound HTTP Connections \(for API Clients\)](#)
Assign the role template as listed for the desired task (for example, download deployed integration flow) under [Tasks and Permissions](#).
- In the **Neo** environment, perform the steps described at: [Setting Up Inbound HTTP Connections \(for API Clients\)](#), [Neo Environment](#)
Assign one or more roles as listed for the desired task (for example, download deployed integration flow) under [Tasks and Permissions](#).

To be able to maintain custom tags:

- In the **Cloud Foundry** environment, assign the following role template:
WebToolingSettingsProductProfiles.savetenantconfiguration (part of role collection PI_Administrator)
- In the **Neo** environment assign the following role:
WebToolingSettingsProductProfiles.savetenantconfiguration (part of authorization group AuthGroup.Administrator)

Resources

The following table contains a list of resources and their respective descriptions.

Resource	Description
Integration Package - Discover	Represents an integration package in the Cloud Integration Discover section. You can use the <code>CopyIntegrationPackage</code> resource to copy an integration package from the Discover section to the Design section.
Integration Packages - Design	Represents an integration package in the Cloud Integration Design section. You can use resource <code>IntegrationPackages</code> to access (create, update, delete, read) integration packages. You can read an entire integration package, or only an integration flow or you can read all the packages in design time.
Custom Tag Configurations	Represents a set of attributes (custom tags) defined on tenant level. Once defined, the integration package owners need to maintain these attributes when creating new integration packages. The tenant administrator can maintain integration package-related attributes that are also referred to as <i>custom tags</i>. Integration developers can use custom tags to classify their integration packages. If the tenant administrator has classified the custom tag as mandatory, the integration developer has to maintain the attribute when creating an integration package. Otherwise, the integration developer can't save the integration package. Custom tags make it easier for integration developers to organize integration content across a large organization. → Tip Example The tenant administrator can define the mandatory custom tag <code>Author</code>. As a result, for each newly created integration package the integration developer has to specify the author name. This makes it easy for all involved integration developers to keep track of the ownership of the individual integration packages. The tenant administrator can maintain custom tags using one of the following options: - Using the Cloud Integration Settings section. - Using the OData API (Custom Tags interface) More information: Creating Custom Tags SAP Cloud Integration – Custom Tags  (SAP Community blog) i Note For more information on how to access custom tags specified for an individual integration package, see: SAP Cloud Integration – Custom Tags  (SAP Community blog)
Integration Flow	Represents an integration flow model. You can use resource <code>IntegrationDesignTimeArtifacts</code> to read, download, create, upload, deploy, and delete integration flows. i Note - You can't delete content from configure-only package. - You can't delete from a locked artifact.
Configurations of Integration Flow	Represents externalized configuration parameters for an integration flow. You can read or update integration flow configurations. You can access integration flow configurations through the <code>IntegrationDesignTimeArtifacts</code> resource.
Resources of Integration Flow	Represents integration flow resources. You can read, download, add, or update integration flow resources. You can access integration flow resources through the <code>IntegrationDesignTimeArtifacts</code> resource. i Note You can use the <code>ReferencedResourceType</code> parameter if you want to refer to a file type with another file type. For example, you can refer to an xsd file with a wsdl file. In this case, the value of the parameter is wsdl. You can't download resources from configure-only content.

Resource	Description
	i Note For script type resources, the Resource ODATA APIs  only support version v1.
Message Mapping	Represents a message mapping. You can read, create, and delete message mappings. You can access message mappings through the <code>MessageMappingDesignTimeArtifacts</code> resource.
Script Collection	Represents a script collection. You can create and upload a script collection, add resources to a script collection, and deploy a script collection. You can access script collections through the <code>ScriptCollectionDesignTimeArtifacts</code> resource.
Build and Deploy Status	Represents the build and deploy status. This status indicates the deployment status of the artifact that is triggered for deployment. You can access integration flow configurations through the <code>BuildAndDeployStatus</code> resource. For more information on the possible status values, see Build and Deploy Status. For integration artifacts that have reached the worker node, another status becomes relevant, the runtime status. The runtime status indicates if a deployed artifact is ready to operate (see Runtime Status).
MDI Delta Token	Represents the delta token for the SAP Master Data Integration receiver adapter (MDI receiver adapter). See:
Runtime Artifacts	Represents the runtime artifact associated with a design time artifact (for example, an integration flow model or a value mapping). You can use resource <code>IntegrationRuntimeArtifacts</code> to read, deploy, and undeploy integration content. i Note To deploy a runtime artifact, you've to include the artifact that you want to deploy as a binary stream into the body of your POST request. Refer to the OData Protocol specification for details on how to use Media Entities. You can only deploy <code>BUNDLE</code> type integration artifacts (integration flows, value mappings, or OData services).
Error Information of Runtime Artifact	Represents error information related to a runtime artifact. You can read error information related to deployed integration artifacts (such as integration flows, for example). You can access error information through the <code>IntegrationRuntimeArtifacts</code> resource.
Endpoints of Runtime Artifacts	Represents endpoints of deployed integration content (for example, integration flow endpoints). You can use resource <code>ServiceEndpoints</code> to read all endpoints provided for integration flows and to get the number of endpoints.
Value Mappings	Represents value mapping. You can use resource <code>ValueMappingDesignTimeArtifacts</code> to read, download, create, upload, deploy value mappings, and to delete entries from value mappings.
Integration Adapter	Represents an integration adapter (only available in the Cloud Foundry environment). You can use resource <code>IntegrationAdapterDesignTimeArtifacts</code> to import, deploy, or delete an integration adapter.

For general information about query options, see [Query Options](#).

i Note

To access integration content through a user interface, you can use the Cloud Integration [Design](#) application. To access predefined integration packages, you can go to the Cloud Integration [Discover](#) application.

Example Requests

You find various example requests on SAP Business Accelerator Hub at [Integration Content](#) .

For more example requests, see [Integration Content Example Requests](#).

To trigger the modifying actions (POST, PUT, and DELETE), you need to fetch a CSRF token first. You can do that in the following way: First, send a GET call to the API with a header `X-CSRF-Token` with the value `Fetch`. As a response, you get the value of the token in the header `X-CSRF-Token`. When sending the modifying request, add the header `X-CSRF-Token` with the token retrieved from the first call.

On SAP Business Accelerator Hub, you can test API calls against a sandbox tenant or against a custom tenant (to be configured under [API Environment](#)). If you want to perform an API call against your custom tenant using an HTTP client such like Postman, make sure that the request URL is composed in the following way:

This is custom documentation. For more information, please visit [SAP Help Portal](#).

The part https://<host address>/api/v1 is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

You can find the relative resource path for each operation on SAP Business Accelerator Hub.

Build and Deploy Status

Indicates the status of an artifact that is triggered for deployment.

You can access integration flow configurations through the `BuildAndDeployStatus` resource.

For integration artifacts that have reached the worker node, another status becomes relevant, the runtime status. The runtime status indicates if a deployed artifact is ready to operate (see [Runtime Status](#)).

There are the following status values for the build and deploy status.

Status Values

Status	Description
SUCCESS	The artifact is deployed on worker node(s).
FAIL	The artifact isn't deployed on worker node(s), for example, because of a validation error/CMD checks error in the currently deployed integration flow.
DEPLOYING	The artifact is in the process of being deployed.
FAIL_ON_LICENSE_ERROR	The artifact isn't deployed on worker node(s) because the integration flow contains components that require an upgrade to the user's existing license.

Related Information

[Runtime Status](#)
[Deployment Status View](#)

Integration Content Example Requests

You find various example requests on SAP Business Accelerator Hub at <https://api.sap.com/api/IntegrationContent/>🔗.

For more example requests, see:

Runtime artifacts and error information	Runtime Artifacts and Error Information Example Requests
Integration flows and resources	Integration Flow Example Requests Get Runtime Status of Deployed Integration Flow
Service endpoints	ServiceEndpoints Example Requests
Integration packages	Integration Package Example Requests
Custom tags	Get Custom Tags Defined on the Tenant Create New Custom Tags Configuration
Value mappings	Value Mapping Example Requests
Integration adapter	Integration Adapter Example Requests, Cloud Foundry Environment

Get Custom Tags Defined on the Tenant

To get all custom tags defined by the tenant administrator, you need to perform the following call:

Method	Resource Path	Purpose
GET	/CustomTagConfigurations('CustomTags')/\$value	Get all custom tags.

Assume that you like to get all custom tags defined by the tenant administrator.

Send the following GET request:

```
https://<host address>/api/v1/CustomTagConfigurations('CustomTags')/$value
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

If a mandatory custom tag with name **Author** is defined on the tenant, you get the following response:

```
{"customTagsConfiguration":[{"tagName":"Author","isMandatory":true}]}
```

If for the mandatory custom tag with name **Author** also permitted values (for example, Mr. Bean and Ms. Bean) are defined, you get the following response:

```
{"customTagsConfiguration":[{"tagName":"Author","permittedValues":["Mr. Bean, Ms. Bean"],"isMandatory":true}]}
```

Create New Custom Tags Configuration

To create custom tags, you need to perform the following call:

Method	Resource Path	Purpose
POST	/CustomTagConfigurations	Create custom tags.

As request body, provide the following JSON content:

```
{
  "CustomTagsConfigurationContent": "<base64-encoded custom tag configurations content>"
}
```

Assume that you like to create a mandatory custom tag with the name **Owner**.

The related JSON representation is:

```
{"customTagsConfiguration":[{"tagName":"Owner","isMandatory":true}]}
```

After base64 encoding, you get the following string:

```
eyJjdXN0b2lUYWdzQ29uZmJndXJhdGlvbiI6W3sidGFhbnRmZSI6Ik93bmVhIiwiaXNNYW5kYXRvcnkiOnRydWV9XX0=
```

Send the following POST request:

```
https://<host address>/api/v1/CustomTagConfigurations
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

As request body, send:

```
{
  "CustomTagsConfigurationContent": "eyJjdXN0b2lUYWdzQ29uZmJndXJhdGlvbiI6W3sidGFhbnRmZSI6Ik93bmVhIiwiaXNNYW5kYXRvcnkiOnRydWV9XX0="
}
```

If a custom tags configuration is already available on the tenant, add the following query parameter to the request:

```
Overwrite=true
```

Update and Validate Configuration Parameters

Update and validate configuration parameters of an integration flow.

1. Update configuration parameters of the integration flow via an OData API, perform the below call:

Method	Resource Path	Purpose	Response Status
GET	api/v1 /IntegrationDesigntimeArtifacts(Id='{Id}',Version='{Version}')/Configurations	Get configurations of an integration flow by Id and version.	o 200: Retrieval of configurations of an integration flow by Id and version. Exa Collection Integration San { d } o 400: Bad Request structure of parameters o 401: Unauthorized information o 403: Forbidden access the resource o 404: Not found could not be found o 429: Too many requests exceeded. o default: Error Exa odata.error San { error* }

Method	Resource Path	Purpose	Response Status
GET	api/v1/IntegrationDesignTimeArtifacts(Id='{Id}',Version='{Version}')/Configurations/\$count	Get number of configuration parameters.	200: Number of configuration parameters. 400: Bad Request. Invalid structure of parameters. 401: Unauthorized. Missing authentication information. 403: Forbidden. No access to the resource. 404: Not Found. Resource could not be found. 429: Too many requests. Rate limit exceeded. - default: Error. ❖ Example <pre>❏ { "odata.error": { "message": "The address field must begin with '/' and c", "sourceObject": "ComponentMetadataEditor/etype:MessageFl", "resourcePath": "src/main/resources/scenarioflows/integ", "resourceName": "iflw1.iflw" } }</pre>
PUT	api/v1/IntegrationDesignTimeArtifacts(Id='{Id}',Version='{Version}')/\$links/Configurations('parameterKey')	Updates configuration parameter value. i Note For Neo environment, this call performs the validation of integration flow as well.	202

2. Once, all the configuration parameters are updated, validate the integration flow along with the configuration parameters via an OData API, perform the below call::

Method	Resource Path	Purpose	Response Status
POST	api/v1/ValidateIntegrationDesignTimeArtifact?Id='{Id}'&Version='{Version}'	Perform validation checks and return results	202 - Passed: Checkexecutionresult: Passed - Failed: Checkexecutionresult: Failed + detailed error list ❖ Example ❏ Sample Code <pre>Checkexecutionresult: Failed[{ "severity": "Error", "message": "The address field must begin with '/' and c", "sourceObject": "ComponentMetadataEditor/etype:MessageFl", "resourcePath": "src/main/resources/scenarioflows/integ", "resourceName": "iflw1.iflw" }]</pre> 404

Method	Resource Path	Purpose	Response Status
			❖ Example ⌵ Sample Code <pre> { "error": { "code": "Not Found", "message": { "lang": "en", "value": "Integration design time artifact not found." } } } </pre> ○ 400

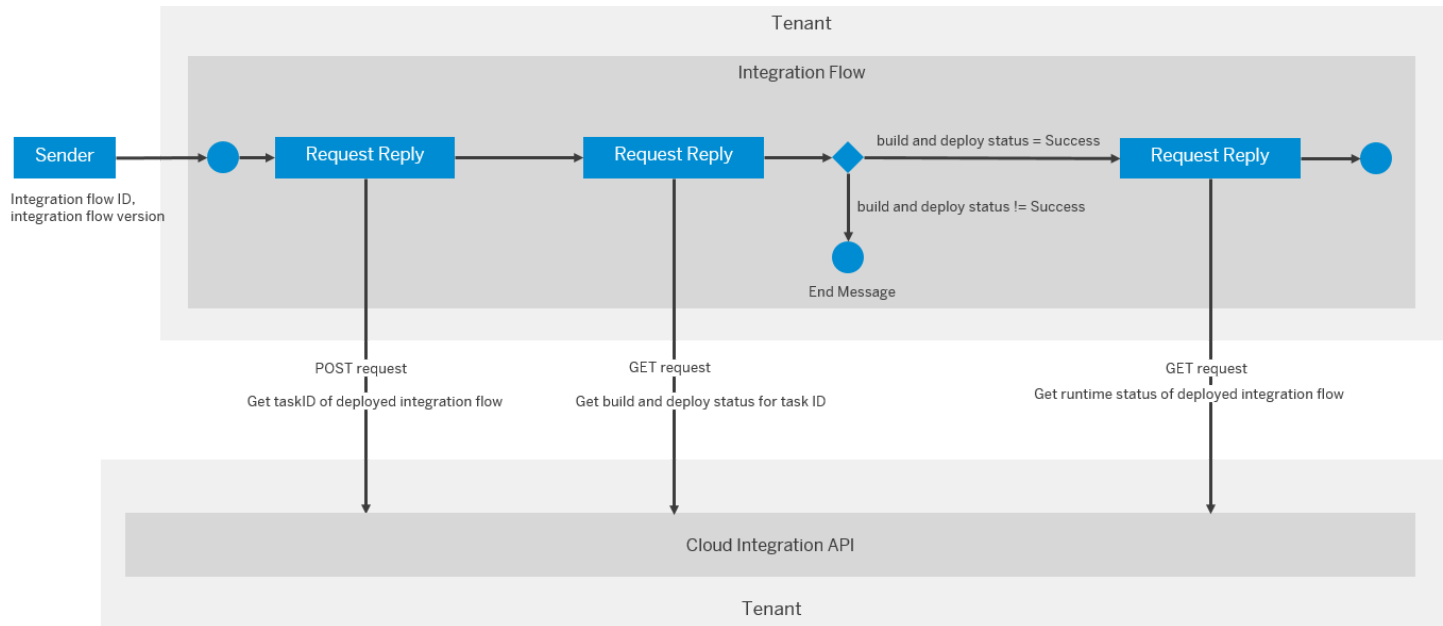
Get Runtime Status of Deployed Integration Flow

Get the runtime status of a currently deployed integration flow.

Perform three subsequent calls:

Method	Resource Path	Purpose
POST	/DeployIntegrationDesignTimeArtifact?Id='<bundleId>'&Version='<version>'	Get taskID of currently deployed integration flow.
GET	/BuildAndDeployStatus(TaskId='<taskId>')	Get build and deploy status of currently deployed integration flow.
GET	/IntegrationRuntimeArtifacts('<bundleId>')	If build and deploy status is Success, get runtime status of currently deployed integration flow.

To set up such a sequence of requests, you can design an integration flow with the following sequence of OData API calls:



The integration flow performs the following steps:

- 1. An HTTP client (for example, Postman) calls the integration flow. The request body contains the Id and version of the integration flow for which you like to request the runtime status.

A content modifier (not shown in the diagram) stores Id and version as exchange properties (let's give them the names: `bundleId` and `version`).

- 2. The first Request Reply step calls the Cloud Integration OData API through an HTTP receiver adapter (for the tenant on which the integration flow in question is deployed).

Cloud Integration calls the OData API with the following URL and query:

```
https://<host address>/api/v1/DeployIntegrationDesignTimeArtifact?
Id='${property.bundleId}'&Version='${property.version}' (POST method)
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

The OData API sends back a response that contains the `taskId`.

A content modifier (not shown in the diagram) stores the `taskId` as exchange property.

- 3. The second Request Reply step calls the OData API (through an HTTP receiver adapter) with the following URL and query:

```
https://<host address>/api/v1/BuildAndDeployStatus(TaskId='${property.taskId}')
```

The OData API provides a response with the following content:

```
<m:properties>
  <d:TaskId>abcd-1234-5678-xyz</d:TaskId>
  <d:Status>Success</d:Status>
</m:properties>
```

A content modifier (not shown in the diagram) stores the status as exchange property.

- 4. A routing step leads to two different routes, depending in the value of the status:
 - The default route leads to a message end event.
 - The route taken when the status provided by the OData API is `Success` contains another Request Reply step that calls the OData API through the HTTP receiver adapter. It specifies a call with the following URL and query:

```
https://<host address>/api/v1/IntegrationRuntimeArtifacts('${property.bundleId}')
```

The response contains attributes for the retrieved integration flow.

The `Status` attribute contains the actual runtime status.

The integration flow sends back the runtime status to the HTTP client.

Related Information

[Integration Content](#)
[Runtime Status](#)

Runtime Artifacts and Error Information Example Requests

Get Runtime Artifacts in Status Error

Perform the following call:

Method	Resource Path
GET	/IntegrationRuntimeArtifacts?\$filter=Status eq 'ERROR'

To get all integration artifacts in status `ERROR`, send the following GET request:

```
https://<host address>/api/v1/IntegrationRuntimeArtifacts?$filter=Status eq 'ERROR'
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

To get error information related to the integration artifact with Id `Mail_Sender_Integration_Flow`, send the following GET request:

https://<host address>/api/v1/IntegrationRuntimeArtifacts('Mail_Sender_Integration_Flow')/ErrorInformation

You get the following response:

```
<entry ...>
  <id>https:<host address>/api/v1/RuntimeArtifactErrorInformations('Mail_Sender_Integration_Flow')</id>
  <title type="text">RuntimeArtifactErrorInformations</title>
  <updated>2021-03-18T11:19:04.598Z</updated>
  <category term="com.sap.hci.api.RuntimeArtifactErrorInformation" scheme="http://schemas.microsoft.com/ado/2007/08/dataserv
  <link href="RuntimeArtifactErrorInformations('Mail_Sender_Integration_Flow')" rel="edit" title="RuntimeArtifactErrorInform
  <link href="RuntimeArtifactErrorInformations('Mail_Sender_Integration_Flow')/$value" rel="edit-media" type="application/oc
  <content type="application/octet-stream" src="RuntimeArtifactErrorInformations('Mail_Sender_Integration_Flow')/$value"/>
  <m:properties>
    <d:Id>Mail_Sender_Integration_Flow</d:Id>
  </m:properties>
</entry>
```

To get the error information in JSON format (for this example), send the following GET request:

https://<host address>/api/v1/IntegrationRuntimeArtifacts('<artifact name>')/ErrorInformation/\$value

Integration Flow Example Requests

You can use resource `IntegrationDesignTimeArtifacts` to read, download, create, upload, deploy, and delete integration flows.

This section provides various example requests and additional information. Also check out the following SAP Community blog: [Cloud Integration – Remote OData API's for Integration Flows](#)👉.

i Note

Update Integration Flow Name

To update an Integration flow name, and save as a draft, send the following call:

Method	Resource Path
PUT	/IntegrationDesignTimeArtifacts(Id='<integration flow ID>',Version='active')

To update an Integration flow with Id `My_Integration_Flow` with the new name `My Integration Flow New`, and save as a draft, send the following PUT request:

https://<host address>/api/v1/IntegrationDesignTimeArtifacts(Id='My_Integration_Flow',Version='active')

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Provide the following request body:

```
{
  "Name": "My Integration Flow New",
  "ArtifactContent": "UESDBBQACAgIAER6cIIAAAAAAAAAAAA ..... vLnByb3BQSwUGAAAAAAYABgDWAQAA8xIAAAAA"
}
```

i Note

For the `ArtifactContent` property, you need to provide the base64-encoded content of the integration flow bundle (for example, of the zip file that contains the integration flow model).

To get the integration flow Id, open the integration flow model and select the **General** tab (ID field).

Update Integration Flow with Specific Version

To copy an Integration flow with a new version, send the following call:

Method	Resource Path
POST	/IntegrationDesignTimeArtifactSaveAsVersion?Id=<integration flow ID>&SaveAsVersion=<new version>

To save Integration flow with Id My_Integration_Flow to version 1.0.0, send the following POST request:

```
https://<host address>/api/v1/IntegrationDesignTimeArtifactSaveAsVersion?
Id='My_Integration_Flow'&SaveAsVersion='1.0.0'
```

i Note

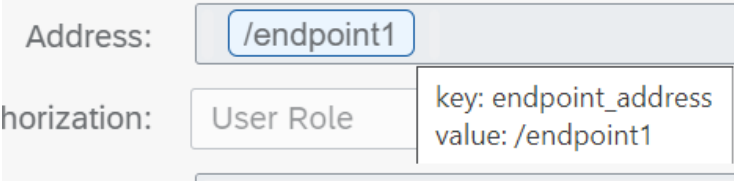
If you would like to update an integration flow artifact with a specific version, the *PUT* method must be executed first.

Update Integration Flow Configuration Parameters

Method	Resource Path
PUT	/IntegrationDesignTimeArtifacts(Id='{Id}',Version='{Version}')/\$links /Configurations('{ParameterKey}')

Assume that you've an integration flow with an externalized parameter for the HTTPS sender adapter address. When defining the HTTPS adapter address as externalized parameter, you've given it the parameter name `endpoint_address` and the initial value `endpoint1`.

The figure shows the externalized parameter in the **Connection** tab of the HTTPS sender adapter.



To change the value of the externalized parameter to `endpoint1`, send the following PUT request:

```
https://<host address>/api/v1/IntegrationDesignTimeArtifacts(Id='My_Integration_Flow',Version='1.0.1')/$links
/Configurations('endpoint_address')
```

Provide the following request body:

```
{
  "ParameterValue": "/endpoint2",
  "DataType": "xsd:string"
}
```

You can also do a batch request, for example:

Sample Code

```
- -batch_34fcd829-64e4-4139-b880-f24aa4ee9235
Content-Type: multipart/mixed; boundary=changeset_5e8e492f-2e3f-490f-9629-a4cd82a7aafd

- -changeset_5e8e492f-2e3f-490f-9629-a4cd82a7aafd
Content-Type: application/http
Content-Transfer-Encoding: binary

PUT IntegrationDesignTimeArtifacts(Id='TestingCustomScheduler',Version='1.0.3')/$links/Configurations('Receiver1_host_33')
Accept: application/json
Content-Type: application/json

{
  "ParameterKey" : "Receiver1_host_33",
  "ParameterValue" : "blr002",
  "DataType" : "xsd:string"
}

- -changeset_5e8e492f-2e3f-490f-9629-a4cd82a7aafd
Content-Type: application/http
```

Content-Transfer-Encoding: binary

PUT IntegrationDesigntimeArtifacts(Id='TestingCustomScheduler',Version='1.0.3')/\$links/Configurations('Sender_host_14') HTTP/1.1

Accept: application/json

Content-Type: application/json

```
{
  "ParameterKey" : "Sender_host_14",
  "ParameterValue" : "wdfsp001",
  "DataType" : "xsd:string"
}
```

--changeset_5e8e492f-2e3f-490f-9629-a4cd82a7aafd--

--batch_34fcd829-64e4-4139-b880-f24aa4ee9235--

Perform Multiple Operations on Resources

You can perform multiple operations on resources in a single call using a batch request:

Sample Code

--batch_abc

Content-Type: multipart/mixed; boundary=changeset_123

--changeset_123

Content-Type: application/http

Content-Transfer-Encoding: binary

POST IntegrationDesigntimeArtifacts(Id='ResourceViewTest',Version='1.0.1')/Resources HTTP/1.1

Content-Type: application/json

```
{
  "Name": "Convert1.xsl",
  "ResourceType": "xslt",
  "ResourceContent": "<Base64 encoded content>"
}
```

--changeset_123

Content-Type: application/http

Content-Transfer-Encoding: binary

POST IntegrationDesigntimeArtifacts(Id='ResourceViewTest',Version='1.0.1')/Resources HTTP/1.1

Content-Type: application/json

```
{
  "Name": "Convert2.xsl",
  "ResourceType": "xslt",
  "ResourceContent": "<Base64 encoded content>"
}
```

--changeset_123--

--batch_abc--

Note

The *Name* and *ResourceType* must not be null.

ResourceContent should be in base64-encoded format.

For more information, check out the following SAP Community blog: [Cloud Integration – Remote OData API's for Integration Flows](#) .

Query Parameters Supported for IntegrationDesigntimeArtifact Requests

IntegrationDesigntimeArtifacts request supports system query parameters for controlling the amount and order of data.

System Query Options are query string parameters that a client specifies to control the amount and order of the data that an OData API returns for the resource identified by the URI. The names of all System Query Options are prefixed with a "\$" character.

Reading DesigntimeArtifact

Query	Description	Example
\$select	You can select response properties based on the requirement.	<code>https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')?\$select=Id,Version</code>

Reading Resources

Query	Description	Example
\$select	You can select response properties based on the requirement.	<code>https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Resources?\$select=Name,ResourceType</code>
\$count	Returns the total count of resources found in the artifact.	<code>https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Resources/\$count</code>
\$orderby	Orders the collection of entries sorted either in ascending or descending order. By default the entries are sorted in ascending order. Use desc to sort all entries returned in descending order.	<code>https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Resources?\$orderby=Name</code> or <code>https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Resources?\$orderby=Name,ResourceType desc</code>
\$filter	Returns a subset of the entries, which matches the filter condition. The following filter conditions are supported: - Eq - ne The following string functions are supported: - substringof - startswith - endswith	<code>https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Resources?\$filter=Name eq 'script1.groovy'</code> <code>https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Resources?\$filter=Name ne 'script1.groovy'</code> <code>https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Resources?\$filter=substringof('script1', Name) eq true</code>
\$format	Response is received either in JSON or Atom XML format. The default format is Atom XML .	<code>https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Resources?\$format=json</code>

Reading Configurations

Query	Description	Example
\$select	You can select response properties based on the requirement.	https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Configurati \$select=ParameterKey,ParameterValue
\$count	Returns the total count of resources found in the artifact.	https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Configurati
\$orderby	Orders the collection of entries sorted either in ascending or descending order. By default the entries are sorted in ascending order. Use desc to sort all entries returned in descending oder.	https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Configurati \$orderby=ParameterKeyorhttps://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Configurati \$orderby=ParameterKey,ParameterValue desc
\$filter	Returns a subset of the entries, which matches the filter condition. The following filter conditions are supported: - Eq - ne The following string functions are supported: - substringof - startswith - endswith	- https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Confi \$filter=ParameterKey eq '<key>' - https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Confi \$filter=ParameterKey ne '<key>' - https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Confi \$filter=substringof('<substring of key>', ParameterKey) eq true
\$format	Response is received either in JSON or Atom XML format. The default format is Atom XML .	- https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Confi - https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Confi

ServiceEndpoints Example Requests

The ServiceEndpoints resource enables you to obtain information about the service endpoints exposed by Cloud Integration on a tenant.

The Service Endpoints also provides you with the adapter-specific protocol information. The protocol is the type of the APIs such as SOAP, REST, and OData. The API protocol includes the protocol version, if applicable.

Adapter	Protocol Type
SOAP	SOAP
IDoc	SOAP
OData V2	ODATAV2
AS2	AS2
AS4	AS4

Adapter	Protocol Type
HTTPS	REST

Specific Example Requests for Service Endpoints

- To return information about all service endpoints with only the endpoints information expanded, send the following GET request:

`https://<host address>/api/v1/ServiceEndpoints?$expand=EntryPoints`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

An endpoint is an array of type `EntryPoint`. Following are the properties of the `EntryPoint` entity:

Property	Type	Description
Name	String	(Required) Name of the entry point.
URL	URL	(Required) Fully qualified endpoint or the base path of the API.
Type	Enumerated String	(Optional) Possible values are DEV, TEST, PROD, SANDBOX.

- To return information about all service endpoints with only the API definitions expanded, send the following GET request:

`https://<host address>/api/v1/ServiceEndpoints?$expand=ApiDefinitions`

An API definition is an array of type `APIDefinition` containing links to the machine-readable API definitions like Open API specification files, OData metadata, and so on. Following are the properties of the `APIDefinition` entity:

Property	Type	Description
URL	URL	(Required) Fully qualified endpoint to the API definitions file.
Type	Enumerated String	(Required) Name of the API definition endpoint. Possible values are oas-yaml, oas-json, raml, edmx, wsd.

- To return the service endpoints only for the integration flow with name `Getting Started Flow`, send the following GET request:

`https://<host address>/api/v1/ServiceEndpoints?$expand=EntryPoints&$filter=Name eq 'Getting Started Flow'`

- To return only the service endpoints information of protocol type SOAP, send the following GET request:

`https://<host address>/api/v1/ServiceEndpoints?$expand=EntryPoints&$filter=Protocol eq 'SOAP'`

Integration Package Example Requests

You can copy integration packages to your [Design](#) space. You can also create, update, delete, and search for integration packages in design time.

Import an Integration Package into the Design Section

To import an integration package into the [Design](#) section, send the following call:

Method	Resource Path
POST	/IntegrationPackages

Provide the following request body:

```
{
  "PackageContent":<base64-encoded package content>
}
```

If the package to be imported is in zip format, it has to be converted into **base 64 encoded** format.

i Note

You can't import a package if there's already a package in the [Design](#) section with the same name. You can use the `Overwrite` parameter to overwrite the existing package. Use the following request:

```
https://<host address>/api/v1/IntegrationPackages?Overwrite=true
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Download an Integration Package

To download an integration package as .zip file from the [Design](#) section, send the following call:

Method	Resource Path
GET	/IntegrationPackages('{Id}')/\$value

i Note

Download fails if the package contains one or more artifacts in draft state.

Value Mapping Example Requests

You can read, query, deploy, download, and upload a Value Mapping artifact using the OData API. You can also read and update the configurations of value mapping design time artifacts

Filter All Value Mappings Based on Source and Target Value for Specific Bidirectional Agency Identifier

To filter all the value mappings based on the source and target value for a specific bidirectional agency identifier, send the following call:

Method	Resource Path
GET	/ValueMappingDesignTimeArtifacts(Id=<Id>,Version=<version>)/ValMapSchema(SrcAgency=<Agency_name>,SrcId=<source_Id>,TgtAgency=<Agency_Name>,TgtId=<target_Id>)/ValMaps?\$filter=Value/SrcValue eq <Source_Value> and Value/TgtValue eq <target_Value>

Create Value Mapping Using Function Import

To create a value mapping using function import [UpsertValMaps](#), send the following call::

Method	Resource Path
POST	/UpsertValMaps?Id=<Id>&Version=<version>&SrcAgency=<Agency_name>&SrcId=<source_Id>&TgtAgency=<Agency_Name>&TgtId=<target_Id>&SrcValue=<value>&TgtValue=<value>&IsConfigured=<true or false>

i Note

- *IsConfigured* is used to change the state of the Bidirectional mapping. If the value is true, then the *ValMapSchema* is configured and if false it is meant for nonconfigured *ValMapSchema*.
- Once a *ValMapSchema* is configured, it can't be changed. The value of *IsConfigured* should always be true for further operations as you cannot edit a configured content.

Value Mapping Batch Requests

Batch operation with multiple queries and changesets:

☰, **Sample Code**

```

--batch
Content-Type: multipart/mixed; boundary=changeset_77162fcd-b8da-41ac-a9f8-9357efbbd621

--changeset_77162fcd-b8da-41ac-a9f8-9357efbbd621
Content-Type: application/http
Content-Transfer-Encoding: binary

POST <operation> HTTP/1.1

--changeset_77162fcd-b8da-41ac-a9f8-9357efbbd621
Content-Type: application/http
Content-Transfer-Encoding: binary

POST <operation> HTTP/1.1

--changeset_77162fcd-b8da-41ac-a9f8-9357efbbd621--

--batch
Content-Type: multipart/mixed; boundary=changeset_77162fcd-b8da-41ac-a9f8-9357efbbd622

--changeset_77162fcd-b8da-41ac-a9f8-9357efbbd622
Content-Type: application/http
Content-Transfer-Encoding: binary

POST <operation> HTTP/1.1

--changeset_77162fcd-b8da-41ac-a9f8-9357efbbd622--

--batch
Content-Type: application/http
Content-Transfer-Encoding: binary

GET <operation> HTTP/1.1

--batch
Content-Type: application/http
Content-Transfer-Encoding: binary

GET <operation> HTTP/1.1

--batch
Content-Type: application/http
Content-Transfer-Encoding: binary

GET <operation> HTTP/1.1

--batch--

```

Integration Adapter Example Requests, Cloud Foundry Environment

i Note

This information is relevant only when you use SAP Cloud Integration in the Cloud Foundry environment.

Deploy an Integration Adapter

To deploy an integration adapter, send the following call:

Method	Resource Path
POST	/api/v1/DeployIntegrationAdapterDesignTimeArtifact? Id='SubsystemSymbolicName1'

- Import an integration adapter with the symbolic name *SubsystemSymbolicName1*.
- Send the request as given in the following example.

❖ Example

For deploying an integration adapter, the URL is given in the following format: `https://<host address>/api/v1/DeployIntegrationAdapterDesignTimeArtifact?Id='SubsystemSymbolicName1'`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

To delete an integration adapter, send the following call:

Method	Resource Path
DELETE	/api/v1/IntegrationAdapterDesignTimeArtifacts(Id='SubsystemSymbolicName1')

- Import the integration adapter with the symbolic name *SubsystemSymbolicName1*.
- Send the request as given in the following example.

 Example

For deleting an integration adapter, the URL is given in the following format: `https://<tenant url>/api/v1/IntegrationAdapterDesignTimeArtifacts(Id='SubsystemSymbolicName1')`

Log Files, Neo Environment

Access technical system logs written during message processing on the worker/runtime node.

i Note

This information is relevant only when you use SAP Cloud Integration in the Neo environment.

You can access the OData API at:

There, you find the basic operations.

<https://api.sap.com/api/LogFiles/>

This documentation provides additional information.

Permissions

To authorize an API client to access the OData API, perform the steps as described at:

In the **Neo** environment, perform the steps described at: [Setting Up Inbound HTTP Connections \(for API Clients\), Neo Environment](#)

Assign the following roles:

`IntegrationOperationServer.read,AuditLog.Read` (see [Tasks and Permissions](#))

Resources

Resource	Description
Log File Archives	Represents a log file. Two kinds of log files are created during message processing: • HTTP access logs Contain information about inbound HTTP requests. You can use this information to analyze authentication and authorization errors for inbound HTTP communication. • Default trace logs Contain technical processing information.
Log Files	Represent a set of log files in a compressed file (zip format).

i Note

The property `NodeScope` identifies the node type associated with the log file.

For example, the following request returns a list of all HTTP access and default trace log files available for all runtime nodes (*worker nodes*) of the tenant cluster:

`https://<host address>/api/v1/LogFiles$filter=NodeScope eq 'worker'`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

i Note

In future, this list probably contains log files from other node types. Therefore, we recommend the use of the variant with the `NodeScope` filter parameter, as it guarantees that only runtime nodes (and no other node types) are considered.

There's a user interface that allows you to access log files. Go to the **Monitor** application under **Access Logs** (only in the Neo environment).

More information: [Access Logs, Neo Environment](#)

For general information about query options, see [Query Options](#).

Example Requests

You find various example requests on SAP Business Accelerator Hub at <https://api.sap.com/api/LogFiles/> .

For more example requests, see [Log Files Example Requests](#).

On SAP Business Accelerator Hub, you can test API calls against a sandbox tenant or against a custom tenant (to be configured under **API Environment**). If you want to perform an API call against your custom tenant using an HTTP client such like Postman, make sure that the request URL is composed in the following way:

`https://<host address>/api/v1/<relative resource path>`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

You can find the relative resource path for each operation on SAP Business Accelerator Hub.

Related Information

[Log Files Example Requests](#)

Log Files Example Requests

You find various example requests on SAP Business Accelerator Hub at:

<https://api.sap.com/api/LogFiles/> .

For more specific example requests, see the following list:

- HTTP access files
 - `https<tmn>/api/v1/LogFileArchives(Scope='all',LogFileType='http',NodeScope='worker')/$value`
Downloads a log file (zip) archive that contains all existing HTTP access log files (specified by `Scope='all'`)
 - `https://<tmn>/api/v1/LogFileArchives(Scope='latest',LogFileType='http',NodeScope='worker')/$value`
Downloads a log file (zip) archive that contains the latest HTTP access log file per runtime node (specified by `Scope='latest'`). The zip archive for the scope `latest` contains only a single log file for each runtime node. Nevertheless, if some runtime nodes have been restarted and appropriate log files still exist for the previous instances of runtime notes, these files are also downloaded.
 - `https://<tmn>/api/v1/LogFileArchives(Scope='all',LogFileType='http',NodeScope='worker')/$value?modifiedAfter=2017-03-25T13:54:51Z`
Downloads a log file (zip) archive that contains all HTTP access log files created after the indicated time (specified by the `modifiedAfter` attribute).

i Note

When filtering using `modifiedAfter`, the API checks if access log files have been modified after the specified time. This means that the files contain entries that were created after the specified time and can contain entries that were already written before the specified time. It doesn't reflect the time when the file has been created.

- Default trace log files
 - `https://<tmn>/api/v1/LogFiles(Name='ljs_trace_a40a97f_2017-12-15.log ',Application='krt01iflmap')`
Returns the metadata of a single default trace log file.

- [https://<tmn>/api/v1/LogFiles\(Name='ljs_trace_a40a97f_2017-12-15.log',Application='krt01lflmap'\)/\\$value](https://<tmn>/api/v1/LogFiles(Name='ljs_trace_a40a97f_2017-12-15.log',Application='krt01lflmap')/$value)
Downloads a single default trace log file (as gzip file).
- [https://<tmn>/api/v1/LogFileArchives\(Scope='all',LogFileType='trace',NodeScope='worker'\)/\\$value](https://<tmn>/api/v1/LogFileArchives(Scope='all',LogFileType='trace',NodeScope='worker')/$value)
Download a log file (zip) archive that contains all existing default trace log files (specified by Scope='all').
- [https://<tmn>/api/v1/LogFileArchives\(Scope='latest',LogFileType='trace',NodeScope='worker'\)/\\$value](https://<tmn>/api/v1/LogFileArchives(Scope='latest',LogFileType='trace',NodeScope='worker')/$value)
Downloads a log file (zip) archive that contains the latest default trace log file per runtime node (specified by Scope='latest').

Message Processing Logs

Get an overview of the messages processed on a tenant and get the details for individual messages. The message processing log (MPL) stores data about the messages processed on a tenant. Furthermore, it stores information about the individual processing steps for each processed message.

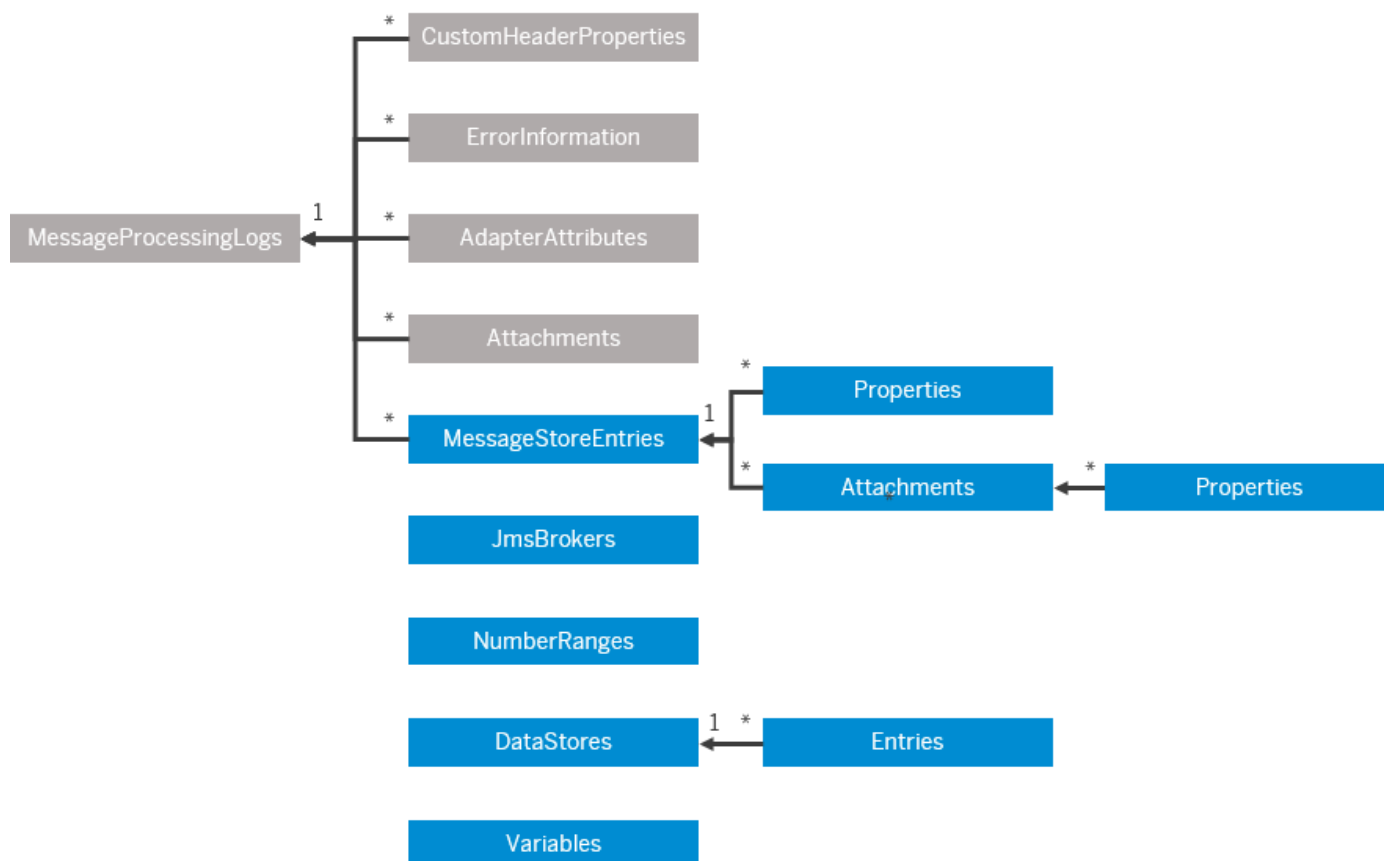
You can access the OData API and learn about its basic operations at:

- [Message Processing Logs](#) for Cloud Integration.
- [Message Processing Logs](#) for Edge Integration Cell

This documentation provides additional information.

Entity Data Model

The following diagram shows how the entities related to `MessageProcessingLog` and `MessageStoreEntry` are related to each other:



Permissions

To authorize an API client to access the OData API:

- In the **Cloud Foundry** environment, perform the steps described in: [Setting Up Inbound HTTP Connections \(for API Clients\)](#)

Assign the following role template:

`MonitoringDataRead` (see [Tasks and Permissions](#))

To access the ID Mapper, assign role template `MessageProcessingLocksRead`.

- In the **Neo** environment, perform the steps described in: [Setting Up Inbound HTTP Connections \(for API Clients\), Neo Environment](#)

Assign the following roles:

`IntegrationOperationServer.read`, `NodeManager.read` (see [Tasks and Permissions](#))

To access the ID Mapper, assign role template `MessageProcessingLocks.Read`.

Resources

Caution

Note the following, if you plan to get message processing logs with status “Discarded”, using the OData API:

- Discarded messages don’t contain error information.
- Calling frequent queries, for error information, in discarded messages, can impose an unnecessary high load on the system.

Therefore, decide the usage for such specific queries, diligently.

Resource	Description
Logs	Represents a message processing logs object. More information: Monitor Message Processing Message Status Message Processing Log
Adapter Attributes	Represents adapter-specific attributes. Each adapter has a defined set of attributes that are described for every adapter type as part of the adapter configuration. The <code>AdapterId</code> property identifies the adapter that logs something (the component ID of the adapter in the component metadata). The <code>AdapterMessageId</code> property uniquely identifies a message in context of a particular adapter.
Attachments	Represents attachments of message processing logs. i Note To be able to view attachments, you need the following role in the NEO environment :<code>esbmessagesstorage.read</code>. In the Cloud Foundry environment, you need the role template <code>MessagePayloadsRead</code>).
Custom Header Properties	Represents custom header properties of the message processing log.  Caution Restriction for the usage of \$orderby Regarding the \$orderby option with the <code>MessageProcessingLogCustomHeaderProperty</code> entity, \$orderby supports only the combinations <code>\$orderby=Name,Value</code> and <code>\$orderby=Name desc,Value desc</code>. If you need additional options, we recommend that you implement these options on the client side. More information: Use Custom Header Properties to Search for Message Processing Logs
Error Information	Represents error information for message processing logs.
ID Mapper	Allows you to get all target IDs for the given source ID (stored in the ID mapper) or to get all source IDs for the given target ID (stored in the ID mapper). i Note Not supported in.
Idempotent Repository	Represents the idempotent repository. i Note Not supported in . i Note The <code>Idempotent Repository</code> resource of the <code>Message Processing Logs</code> API has been deprecated and replaced by a new one. The idempotent repository contains information about files that have already been consumed from the connected server in scenarios where one or more of the following features are used: Aggregation step, FTP, adapter, JMS adapter, SFTP adapter, or XI adapter. Files that are stored in the idempotent repository can be identified by the file name. When Cloud Integration tries to process the file, the

Resource	Description
	system can detect if the file has already been consumed (based on its idempotent repository entry) and that way can prevent it from being consumed a second time from the server. More information: • Define Aggregator • FTP Adapter • JMS Adapter • SFTP Adapter • XI Adapter Certain values (when indicated as <hexaAlias>) need to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string my alias is expressed by the following decimal numbers: 109 121 32 97 108 105 97 115 because, according to the American Standard Code for Information Interchange (ASCII), the letter m is translated to the decimal number 109, and so forth. If you convert each number to a hexadecimal number, you get: 6D 79 20 61 6C 69 61 73. Note that 6D is the hexadecimal representation of 109, and so forth. If you want to specify my alias in an example request, enter 6D7920616C696173.
External Logging (only available in the Cloud Foundry environment.)	Allows you to activate or deactivate external logging. More information: External Logging Cloud Foundry Environment
Data Archiving (only available in the Cloud Foundry environment.)	Allows you to activate data archiving. More information: Archiving Data, Cloud Foundry Environment

For general information about query options, see [Query Options](#).

i Note

To access message processing logs through a user interface, you can use the **Monitor** application (**Monitor Message Processing** section).

Example Requests

You can find various example requests on SAP Business Accelerator Hub at [Message Stores](#) .

For more example requests, see [Message Processing Logs Example Requests](#).

To trigger the modifying actions (POST, PUT, and DELETE), you need to fetch a CSRF token first. You can do that in the following way: First, send a GET call to the API with a header X-CSRF-Token with the value Fetch. As a response, you get the value of the token in the header X-CSRF-Token. When sending the modifying request, add the header X-CSRF-Token with the token retrieved from the first call.

On SAP Business Accelerator Hub, you can test API calls against a sandbox tenant or against a custom tenant (to be configured under **API Environment**). If you want to perform an API call against your custom tenant using an HTTP client such like Postman, make sure that the request URL is composed in the following way:

`https://<host address>/api/v1/<relative resource path>`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

You can find the relative resource path for each operation on SAP Business Accelerator Hub.

i Note

A sandbox environment isn't available for .

Related Information

[Monitor Message Processing](#)

Message Processing Logs Example Requests

You find various example requests on SAP Business Accelerator Hub at [Message Processing Logs](#) .

For more example requests, see:

Logs	Get Message Processing Logs for a Dedicated Time Period
Custom Header Properties	Get Custom Header Properties

Get Message Processing Logs for a Dedicated Time Period

Get the message processing logs for all completed messages processed within a dedicated time period.

Perform the following call:

Method	Resource Path
GET	/MessageProcessingLogs?\$inlinecount=allpages&\$filter=Status eq 'COMPLETED' and LogStart gt datetime'{beginning of time period}' and LogEnd lt datetime'{end of time period}'

Assume that you like to get all message processing logs for completed messages processed in the time period of February 1 to February 14 2021.

Send the following GET request:

```
https://<host address>/api/v1/MessageProcessingLogs?$inlinecount=allpages&$filter=Status eq 'COMPLETED' and LogStart gt datetime'2021-02-01T00:00:00' and LogEnd lt datetime'2021-02-14T00:00:00'
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Get Custom Header Properties

Get custom header properties for a dedicated message ID.

Perform the following call:

Method	Resource Path
GET	/MessageProcessingLogs('{MessageGuid}')/CustomHeaderProperties

You can set custom header properties in a Script step. After processing the integration flow, custom header properties are displayed in the **Monitor** application (under **Monitor Message Processing**). However, you can also use the OData API to retrieve custom header properties.

There's an integration flow design guideline that shows you how to set custom header properties using the Script step (see [Use Custom Header Properties to Search for Message Processing Logs](#)).

When you've deployed and processed this integration flow, the corresponding message processing log shows the custom header property `po_number` with a certain value.

Assume that the message ID is ABCD - 1234 - XYZ. To get the custom header property, send the following GET request:

```
https://<host address>/api/v1/MessageProcessingLogs('ABCD-1234-XYZ')/CustomHeaderProperties'
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

The response contains the following part:

```
<m:properties>
<d:Id>999</d:Id>
<d:Name>po_number</d:Name>
<d:Value>12345</d:Value>
</m:properties>
```

Message Stores

Get data from Message Stores for processed messages and resources of the used JMS queues.

You can access the OData API and learn about its basic operations at:

- [Message Stores](#) for Cloud Integration.
- [Message Stores](#) for

This documentation provides additional information.

Overview on Message Store Resources

Message content is stored by the [Persist](#) step of an integration flow. You can use this API to access the stored message and analyze it at a later point in time.

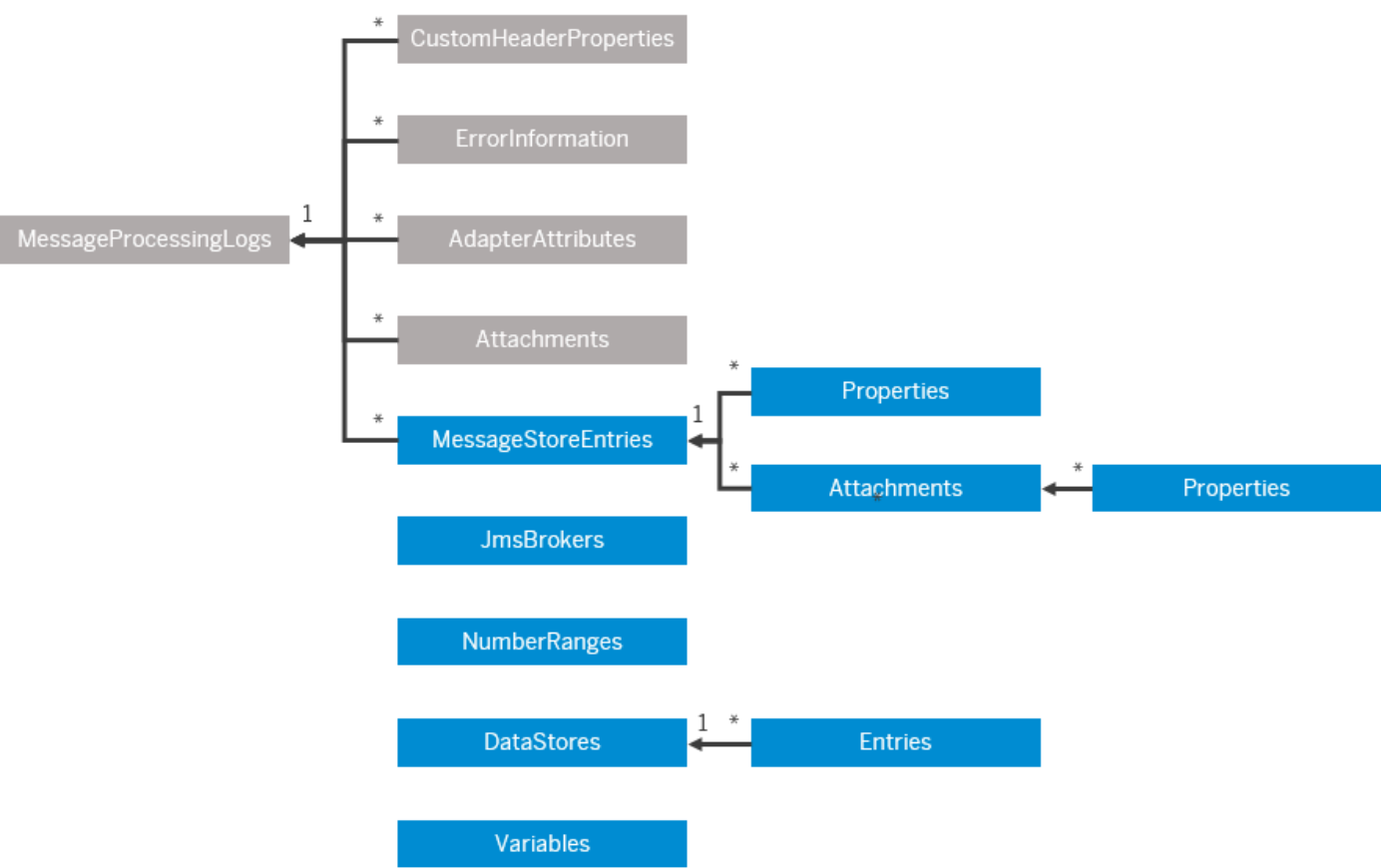
You can also use this API to get information about the JMS resources used in asynchronous messaging scenarios.

You can access the OData API at:

<https://api.sap.com/api/MessageStore/>

Entity Data Model for Message Store Resources

The following diagram shows how the `MessageProcessingLogs` resources (gray) are related to the `MessageStoreEntries` resources (and related resources) of the `Message Stores` API.



Overview on Data Store/Variables Resources

During the processing of an integration flow, you can temporarily store data in the tenant database. This data can be accessed by steps within the same integration flow or another integration flow, deployed on the same tenant.

The following data can be stored:

- Message payloads and headers as data store entries

Data store entries can be accessed by Data Store Select or Data Store Get steps.

- Any data as variable

Variables can be accessed by dedicated integration steps, for example, by a content modifier to store the variable as property or header.

To do that, you use the [Data Store Write](#) step (for data stores and data store entries) or the [Write Variables](#) step (for variables) of an integration flow. With the Data Store Get or Select step, you can read data store content during the processing of an integration flow.

For an overview of the content of the tenant database, see [Using Data Storage Features When Designing Integration Flows](#).

Depending on the data store or variable type, the data is shared by all integration flows (deployed on the tenant) or only by one specific integration flow.

- **Global:** Data is shared across all integration flows deployed on the tenant.
- **Integration Flow:** Data can only be used within 1 integration flow.

Permissions

To authorize an API client to access the OData API:

- In the **Cloud Foundry** environment, perform the steps described in: [Setting Up Inbound HTTP Connections \(for API Clients\)](#).

Depending on the use case, assign the following role templates (see [Tasks and Permissions](#)):

- To view message store content: `MonitoringDataRead`
- To view data store entries: `DataStoresAndQueuesRead`
- To download the message payload (stored as data store entry) and to view variables: `DataStorePayloadsRead`
- To delete data store entries/variables: `DataStoresAndQueuesDelete`
- To view JMS resources: `DataStoresAndQueuesRead`
- In the **Neo** environment, perform the steps described in: [Setting Up Inbound HTTP Connections \(for API Clients\), Neo Environment](#)

Depending on the use case, assign the following roles (see [Tasks and Permissions](#)):

- To view message store content: `IntegrationOperationServer.read, NodeManager.read`
- To view data store entries: `IntegrationOperationServer.read, ESBDDataStore.read`
- To download the message payload (stored as data store entry) and to view variables: `IntegrationOperationServer.read, ESBDDataStore.readPayload`
- To delete data store entries/variables: `IntegrationOperationServer.read, ESBDDataStore.read, ESBDDataStore.delete`
- To view JMS resources: `IntegrationOperationServer.read, ESBDDataStore.read`

Resources

Resource	Description
Entries	Represents a message store entry. A message store entry is created by a Persist step of an integration flow (see Persist Messages).
Entry Attachments	Represents an attachment of a message from the message store A message store entry is created by a Persist step of an integration flow (see Persist Messages).
Entry Attachment Properties	Represents properties of message attachments from the message store. A message store entry is created by a Persist step of an integration flow (see Persist Messages).
Entry Properties	Represents a header of a message from the message store. A message store entry is created by a Persist step of an integration flow (see Persist Messages).
JMS Resources	Represents available resources of the used Java Message Service (JMS) queues. JMS queues are accessed to by integration flows that contain an AS2, a JMS, or an XI adapter (see AS2 Adapter , JMS Adapter , and XI Adapter).
Data Stores	Represents a data store. A data store entry is created by a Data Store Write step of an integration flow (see Define Data Store Write Operations). i Note The DataStore API does not support the following query options: • ◦ <code>\$filter</code> ◦ <code>\$inlinecount</code>

Resource	Description
	◦ \$orderby ◦ \$skip ◦ \$top ◦ \$expand ◦ \$select
Data Store Entries	Represents a data store entry. A data store entry is created by a Data Store Write step of an integration flow (see Define Data Store Write Operations).
Variables	Represents a variable. A variable is created by a Write Variables step of an integration flow (see Define Write Variables). i Note The Variables API does not support the following query options: • ◦ \$filter ◦ \$inlinecount ◦ \$orderby ◦ \$skip ◦ \$top ◦ \$expand ◦ \$select
Number Ranges i Note Not supported in.	Represents a Number Ranges object (see Managing Number Ranges). While sending out a document when using EDI processing, a unique interchange number must be added to each document. To add such an interchange number, you can use the Number Range object. If EDIFACT is used, the outgoing EDIFACT messages have an Interchange Control Reference with a length of 1 to 9 digits.

For general information about query options, see [Query Options](#).

i Note

Cloud Integration doesn't provide a user interface to access the Message Store (data stored by the Persist step). You can only access this data using the OData API.

To access JMS resources through a user interface, you can use the **Monitor** application (**Message Queues** tile under **Manage Stores**).

To access Number Ranges objects through a user interface, you can use the **Monitor** application (**Number Ranges** tile under **Manage Stores**).

Cloud Integration provides a user interface for accessing data stores and variables (go to the **Monitor** application and select the tile **Data Stores** or **Variables** under **Manage Stores**).

Example Requests

You find various example requests on the SAP Business Accelerator Hub at [Message Stores](#) .

For more example requests, see .

- [Message Stores Example Requests](#)
- [Data Store Example Requests](#)
- [JMS Resources Example Requests](#)

To trigger the modifying actions (POST, PUT, and DELETE), you need to fetch a CSRF token first. You can do that in the following way: First, send a GET call to the API with a header X-CSRF-Token with the value Fetch. As a response, you get the value of the token in the header X-CSRF-Token. When sending the modifying request, add the header X-CSRF-Token with the token retrieved from the first call.

On SAP Business Accelerator Hub, you can test API calls against a sandbox tenant or against a custom tenant (to be configured under **API Environment**). If you want to perform an API call against your custom tenant using an HTTP client such like Postman, make sure that the request URL is composed in the following way:

This is custom documentation. For more information, please visit [SAP Help Portal](#).

The part https://<host address>/api/v1 is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

You can find the relative resource path for each operation on SAP Business Accelerator Hub.

i Note

A sandbox environment isn't available for .

Message Stores Example Requests

You find various example requests on SAP Business Accelerator Hub at [Message Stores](#).

For more example requests, see:

Entry Properties	Get Message Store Entry Properties
Number Ranges	Add a Number Ranges Object Update a Number Ranges Object

Get Message Store Entry Properties

Get message store entry properties.

To get message store entry properties, you need to perform two subsequent calls:

Method	Resource Path	Purpose
GET	/MessageProcessingLogs('{MessageGuid}') /MessageStoreEntries	Get all message store entries created during the processing of an integration flow (for a dedicated MessageGuid).
GET	/MessageStoreEntries('{MessageStoreEntryId}')	Get entry properties for an individual message store entry (identified by MessageStoreEntryId).

In detail, perform the following subsequent calls (example).

1. Get all message store entries created during the processing of an integration flow.

Note that each Persist step in an integration flow creates a message store entry.

Each message processing run is identified by a **Message ID** that you can find when selecting the associated integration flow using the **Monitor** application (under **Monitor Message Processing**).

To get message store entries for a dedicated message processing run (for example, with a message ID ABCD1234XYZ), send the following GET request:

https://<host address>/api/v1/MessageProcessingLogs(' ABCD1234XYZ')/MessageStoreEntries

The part https://<host address>/api/v1 is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

As a response, you get a message that contains for each message store entry a passage with the following structure:

```
<m:properties>
  <d:Id>sap-it-abcd-1234-uvw</d:Id>
  <d:MessageGuid>ABCD1234XYZ</d:MessageGuid>
  <d:MessageStoreId>Persist1</d:MessageStoreId>
  <d:TimeStamp>2021-02-25T12:49:48.83</d:TimeStamp>
  <d:HasAttachments>>false</d:HasAttachments>
</m:properties>
```

The property Id uniquely identifies the message store entry.

2. To get entry properties for an individual message store entry, perform a second call. To get properties for the message store entry with Id sap-it-abcd-1234-uvw, you perform the following GET request:

https://<host address>/api/v1/MessageStoreEntries(' sap-it-abcd-1234-uvw')/Properties

As response, you get a message with an entry for each message header with the following part (example for message header myheader):

```
<content type="application/xml">
  <m:properties>
    <d:MessageId>sap-it-abcd-1234-uvw</d:MessageId>
    <d:Name>myheader</d:Name>
    <d:Value>myheader_value</d:Value>
  </m:properties>
</content>
```

Related Information

[Use the Persist Step](#)

Add a Number Ranges Object

Add a **Number Ranges** object to the tenant.

Perform the following call:

Method	Resource Path
POST	/NumberRanges

Example: To add a **Number Ranges** object to your tenant, send the following POST request:

`https://<host address>/api/v1/NumberRanges`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

As request body, provide (example):

≡ Sample Code

```
{
  "CurrentValue": "0",
  "Name": "My NRO Object",
  "MinValue": "0",
  "MaxValue": "9999",
  "Description": " Number Range Object ",
  "Rotate": "true",
  "FieldLength": "4"
}
```

You find the newly created **Number Ranges** object using the **Monitor** application (**Number Ranges** tile under **Manage Stores**).

Update a Number Ranges Object

Perform the following call:

Method	Resource Path
PUT	/NumberRanges(' {objectName}')

For example, to update a Number Ranges object with name **My NRO Object**), send the following PUT request:

`https://<host address>/api/v1/NumberRanges( 'My NRO Object' )`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

In the request body, provide the new properties of the Number Ranges object (example):

≡ Sample Code

```
{
  "CurrentValue": "0",
  "Name": "My NRO Object",
  "MinValue": "0",
  "MaxValue": "9999",
  "Description": " Number Range Object ",
  "Rotate": "true",
  "FieldLength": "4"
}
```

i Note

The **Current Value** returned by the API corresponds to the **Next Value** shown in the Monitoring tab of the UI. The difference is only in terminology.

You find the updated **Number Ranges** object using the **Monitor** application (**Number Ranges** tile under **Manage Stores**).

Data Store Example Requests

Data Stores	Get All Data Stores with Overdue Messages
Data Store Entries	Get Single Data Store Entry Get All Data Store Entries for a Data Store Get All Data Store Entries for a Specific Message ID
Variables	Download a Variable

Get All Data Stores with Overdue Messages

To get data stores that contain overdue messages, perform the following call:

Method	Resource Path	Purpose
GET	/DataStores?overdueonly=true	Gets all data stores that contain overdue messages.

If `overdueonly` is set to `false` or not available, all data stores are shown.

For each retrieved data store entry,the response contains a part with the following structure:

```
<m:properties>
<d:DataStoreName>My_Datastore</d:DataStoreName>
<d:IntegrationFlow>My_Integration_Flow</d:IntegrationFlow>
<d:Type/>
<d:NumberOfMessages>19</d:NumberOfMessages>
<d:NumberOfOverdueMessages>19</d:NumberOfOverdueMessages>
</m:properties>
```

Get Single Data Store Entry

To get a single data store entry, perform the following call:

Method	Resource Path
GET	/DataStoreEntries(Id='{EntryId}',DataStoreName='{DataStoreName}',IntegrationFlow='{IntegrationFlowName}',Type='{Type}')

To get a single entry with entry ID 1234-abcd-4567-xyz from the local data store `MyStore`, perform the following GET request:

```
https://<host address>/api/v1/DataStoreEntries(Id='{1234-abcd-4567-xyz}',DataStoreName='{DataStoreName}',IntegrationFlow='{MyIntegrationFlow}',Type='{Type}')
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

The response contains an entry with the following elements:

```
<m:properties>
  <d:Id>1234-abcd-4567-xyz</d:Id>
  <d:DataStoreName>MyStore</d:DataStoreName>
  <d:IntegrationFlow>MyIntegrationFlow</d:IntegrationFlow>
  <d:Type></d:Type>
  <d>Status>Waiting</d>Status>
  <d:MessageId>ABCD_XYZ</d:MessageId>
  <d:DueAt>2021-04-16T13:53:35.28</d:DueAt>
  <d:CreatedAt>2021-04-14T13:53:35.28</d:CreatedAt>
  <d:RetainUntil>2021-07-13T13:53:35.28</d:RetainUntil>
</m:properties>
```

Get All Data Store Entries for a Data Store

To get data store entries of specific data store, perform the following call:

Method	Resource Path	Purpose
GET	<code>/DataStores(DataStoreName='<data store name>',IntegrationFlow='<integration flow ID>',Type='')/Entries</code>	Gets all entries of a specific data store.

To retrieve all data store entries of data store `MyDataStore` created by integration flow `MyIntegrationFlow`, perform the following request:

```
https://<host address>/api/v1
/DataStores(DataStoreName='MyDataStore',IntegrationFlow='MyIntegrationFlow',Type='')/Entries
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

For each data store entry, the response contains a part with the following structure:

```
<m:properties>
  <d:Id>1234-abcd-7890-xyz</d:Id>
  <d:DataStoreName>MyDataStore</d:DataStoreName>
  <d:IntegrationFlow>MyIntegrationFlow</d:IntegrationFlow>
  <d:Type></d:Type>
  <d>Status>Waiting</d>Status>
  <d:MessageId>ABCD1234XYZ</d:MessageId>
  <d:DueAt>2021-07-08T09:15:05.837</d:DueAt>
  <d:CreatedAt>2021-07-06T09:15:05.837</d:CreatedAt>
  <d:RetainUntil>2021-08-05T09:15:05.837</d:RetainUntil>
</m:properties>
```

Get All Data Store Entries for a Specific Message ID

To get data store entries that are associated with a specific message ID, perform the following call:

Method	Resource Path	Purpose
GET	<code>/DataStoreEntries?messageid=<messageId></code>	Gets all data store entries for a given message ID.

To retrieve all data store entries created during the message processing run identified by the message ID `ABCD1234XYZ`, perform the following request:

```
https://<host address>/api/v1/DataStoreEntries?messageid=ABCD1234XYZ
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

For each data store entry, the response contains a part with the following structure:

```
<m:properties>
<d:Id>1234-abcd-7890-xyz</d:Id>
<d:DataStoreName>My_Datastore</d:DataStoreName>
<d:IntegrationFlow>My_Integration_Flow</d:IntegrationFlow>
<d:Type/>
<d:Status>Overdue</d:Status>
<d:MessageId>ABCD1234XYZ</d:MessageId>
<d:DueAt>2021-02-06T14:24:24.016</d:DueAt>
<d:CreatedAt>2021-02-04T14:24:24.016</d:CreatedAt>
<d:RetainUntil>2021-05-05T14:24:24.016</d:RetainUntil>
</m:properties>
```

Download a Variable

To download a single variable, perform the following call:

Method	Resource Path	Purpose
GET	/Variables(VariableName='{VariableName}',IntegrationFlow='{IntegrationFlowName}')/\$value	Downloads a single variable.

To download variable timestamp for integration flow MyIntegrationFlow, perform the following request:

```
https://<host address>/api/v1/Variables(VariableName='timestamp',IntegrationFlow='MyIntegrationFlow')/$value
```

The part https://<host address>/api/v1 is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

JMS Resources Example Requests

Get information about JMS message queues.

Method	Resource Path	Purpose
GET	/JmsBrokers('Broker1')?\$expand=QueueStates	Get the queue status of each message queue (0 OK, 1 warning, 2 critical).
GET	/JmsBrokers('Broker1')?\$expand=InactiveQueues	Get stopped queues.

You can use this API in an integration flow to set up notifications for critical resource situations. For more information, see the following SAP Community blog:

[Cloud Integration – Automated Notification for Critical or Exhausted JMS Resources](#)

Partner Directory

The Partner Directory contains information on partners that are connected to a tenant in the context of a larger business partner network.

You can access the OData API and learn about its basic operations at:

- <https://api.sap.com/api/PartnerDirectory/> for Cloud Integration.
- [Overview | Partner Directory | SAP Business Accelerator Hub](#) for Edge Integration Cell.

i Note

Read-only access.

This documentation provides additional information.

i Note

There's also a Java API to access the Partner Directory using a programming language like Java Script or Groovy. For more information, check out the JavaDoc linked to at [SDK API](#) (for example, check out the package com.sap.it.api.pd).

 Caution

The data is stored unencrypted in the Partner Directory. Therefore, make sure that the data does not contain any sensitive information (for example, passwords or personal information).

Permissions

To authorize an API client to access the OData API, perform the steps as described at:

- In the **Cloud Foundry** environment, perform the steps described at: [Setting Up Inbound HTTP Connections \(for API Clients\)](#)

Assign the following role collection:

`AuthGroup_TenantPartnerDirectoryConfigurator` (see [Permissions](#))

- In the **Neo** environment, perform the steps described at: [Setting Up Inbound HTTP Connections \(for API Clients\)](#), [Neo Environment](#)

Assign the following authorization group:

`AuthGroup.TenantPartnerDirectoryConfigurator` (see [Permissions](#))

Resources

More information on the concepts: [Partner Directory Concepts](#)

Entity Type	Description
Alternative Partners	Represents an alternative partner. You can access (read, write, delete) an alternative partner. See: Partner Directory Entity Types
Authorized Users	Represents the user with which a partner sender system can log in into SAP Cloud Integration. You can access (read, write, delete) an authorized user. See: Partner Directory Entity Types
Binary Parameters	You can access (read, write, delete) a binary parameter of the Partner Directory (for example, for an XSD document). Here, you can also specify the encoding for xml, xsl, xsd, json, and text (separated by semicolon), for example: <code>xml;encoding=UTF-8</code> See: Partner Directory Entity Types
Partners	You can read all partners or delete a partner from the Partner Directory. A partner is identified by an internal identifier (referred to as <code>Pid</code>) (which is unique within the tenant partner directory and has a maximum length of 60 characters). The PID may consist of alphanumeric characters (A-Z, a-z, 0-9), as well as the following special characters: <code>'-', '!', '!', '!', '~', '<', '>',</code> and <code>'@'</code>. A partner can have parameters (<code>StringParameter</code>, <code>BinaryParameter</code>, or <code>UserCredentialParameter</code>). The parameter is uniquely identified by the <code>Pid</code> of the partner to which the parameter belongs and its <code>Id</code>. The ID of this entity may consist of alphanumeric characters (A-Z, a-z, 0-9), as well as the following special characters: <code>'-', '!', '!', '!', '~', '<', '>',</code> and <code>'@'</code>.
String Parameters	You can access (read, write, delete) a string parameter of the Partner Directory. See: Partner Directory Entity Types
User Credential Parameters	Represents a User Credentials artifact (that is to be used for outbound calls to a partner system). You can access (read, write, delete) a User Credentials artifact. You use this parameter to configure and deploy User Credentials artifacts. The User Credentials parameter contains a user and a password (both maximum length of 200 characters). The ID of this entity may consist of alphanumeric characters (A-Z, a-z, 0-9), as well as the following special characters: <code>'-', '!', '!', '!', '~', '<', '>',</code> and <code>'@'</code>. Note that the user credentials (user and password) relate to the receiver (partner) system that is targeted by an outbound call (where these credentials are used for basic authentication). i Note For user credentials artifacts created by the OData API, an alias with the following naming convention is generated: <code>pd:<Pid>:<Id>:UserCredential</code>

Entity Type	Description
	Pid is the internal identifier of the Partner and Id is the Id of the User Credential Parameter. i Note User credentials artifacts created with the API are displayed in the Web UI under Manage Security (Security Material tile).

For general information about query options, see [Query Options](#).

Example Requests

You find various example requests on SAP Business Accelerator Hub at <https://api.sap.com/api/PartnerDirectory/>.

For more example requests, see [Partner Directory Example Requests](#).

To trigger the modifying actions (POST, PUT, and DELETE), you need to fetch a CSRF token first. You can do that in the following way: First, send a GET call to the API with a header X- CSRF - Token with the value Fetch. As a response, you get the value of the token in the header X- CSRF - Token. When sending the modifying request, add the header X- CSRF - Token with the token retrieved from the first call.

On SAP Business Accelerator Hub, you can test API calls against a sandbox tenant or against a custom tenant (to be configured under [API Environment](#)). If you want to perform an API call against your custom tenant using an HTTP client such like Postman, make sure that the request URL is composed in the following way:

`https://<host address>/api/v1/<relative resource path>`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

You can find the relative resource path for each operation on SAP Business Accelerator Hub.

Caution

Limitations

Be aware of the following limitations when working with the Partner Directory:

Size restrictions for the different entity types:

- Maximum number of StringParameters overall: 3,000,000 (corresponds to 10,000 partners each using 300 StringParameters)
- Maximum number of BinaryParameters overall: 400,000 (corresponds to 10,000 partners each using 40 BinaryParameters)
- Maximum number of AlternativePartners overall: 1,000,000 (corresponds to 10,000 partners each using 100 AlternativePartners)
- Maximum number of AuthorizedUsers overall: 500,000 (corresponds to 10,000 partners each using 50 AuthorizedUsers)

The maximum size of a keystore is 2 MB (when using the Neo environment).

- The 2 MB limit corresponds to around 2000 X.509 certificates.
- A key pair with a chain of three X.509 certificates consumes about 3 KB, so if the keystore only contains key pairs of this type, then you can store around 600 key pairs in the keystore.

Limit for certificate-to-user mapping (when using the Neo environment): 2 MB (corresponds to about 2000 X.509 certificates)

The maximum size of a keystore is 6 MB (when using the Cloud Foundry environment).

- The 6 MB limit corresponds to around 6000 X.509 certificates.
- A key pair with a chain of three X.509 certificates consumes about 3 KB, so if the keystore only contains key pairs of this type, then you can store around 1800 key pairs in the keystore.

If you upload a whole keystore (. j ks file) to the tenant, the maximum keystore size is limited to 2 MB.

For detailed step-by-step descriptions how to use the Partner Directory, see the following blogs:

- [Cloud Integration – Partner Directory – Step-by-Step Example](#)
- [Cloud Integration – Partner Directory – Partner Dependent XML Structures and IDs](#)
- [Cloud Integration – Partner Directory – Sender Partner Connecting with Client Certificate Authentication](#)

- [Cloud Integration – Partner Directory – Partner Dependent User Credential Selection](#)
- [Cloud Integration – Partner Directory – Mass Configuration](#)

Related Information

- [Parameterizing Integration Flows Using the Partner Directory](#)
- [Partner Directory Concepts](#)
- [Partner Directory Example Requests](#)
- [Managing Partner Directory Entries](#)

Permissions

Permissions, Cloud Foundry Environment

The role template `AuthGroup_TenantPartnerDirectoryConfigurator` grants permissions to access (read and write) the Partner Directory.

More information:

- [Personas](#)
- [Tasks and Permissions](#)

Permissions, Neo Environment

The authorization group `AuthGroup.TenantPartnerDirectoryConfigurator` provides permissions to read and write Partner Directory content (`StringParameter`, `BinaryParameter`, `AuthorizedUser`, `AlternativePartnerId`). You can assign this authorization group to the technical user that uses the Partner Directory remote API.

Read access is protected by the role `TenantPartnerDirectory.read`. This role is assigned to the following authorization groups:

- `AuthGroup.Administrator` (Tenant Administrator)
- `AuthGroup.IntegrationDeveloper`
- `AuthGroup.ReadOnly` (Supporter)
- `AuthGroup.SystemDeveloper`
- `AuthGroup.TenantPartnerDirectoryConfigurator`

Write access is protected by the role `TenantPartnerDirectory.write`. This role is assigned to the following authorization groups:

- `AuthGroup.Administrator` (Tenant Administrator)
- `AuthGroup.TenantPartnerDirectoryConfigurator`

Additionally, the authorization group `AuthGroup.TenantPartnerDirectoryConfigurator` contains the roles `NodeManager.deploycredentials` and `NodeManager.readcredentials` for deploying and reading `UserCredentialParameters` via the Partner Directory OData API.

More information:

- [Personas](#)
- [Tasks and Permissions](#)

Partner Directory Example Requests

You find various example requests on SAP Business Accelerator Hub at:

<https://api.sap.com/api/PartnerDirectory/>

For more example requests, see:

Alternative Partner	Get Alternative Partner by Key Fields Get Alternative Partners (Filter with Select) Create Alternative Partner
---------------------	--

	Update an Alternative Partner
Authorized User	Requests for String_Parameter, Binary_Parameter, and Authorized User
String Parameter	
Binary Parameter	
User Credential Parameters	Requests for UserCredentialParameter

See also: [Mass Create/Change/Delete Operations in One Transaction for StringParameter, BinaryParameter, AlternativePartner, and AuthorizedUser](#)

i Note

The charset encoding of the JSON documents in the requests must be UTF-8. The charset encoding of the JSON documents of the responses is also UTF-8.

Get Alternative Partner by Key Fields

Get an alternative Partner through key fields Agency, Scheme, and Id.

Perform the following call:

Method	Resource Path
GET	/AlternativePartners(Hexagency=' {Hexagency}' ,Hexscheme=' {Hexscheme}' ,Hexid=' {Hexid}')

Assume that you like to get the alternative partner associated with agency AgencyA, schema SchemeA, and Id Partner123.

Therefore, you need to send the following GET request providing the hexadecimal values for the key parameters:

```
https://<host
address>/api/v1/AlternativePartners(Hexagency='4167656E637941',Hexscheme='536368656D6541',Hexid='506172746E6572313233')
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

As a response, you get a message that contains the following part:

```
<content type="application/xml">
  <m:properties>
    <d:Hexagency>4167656E637941</d:Hexagency>
    <d:Hexscheme>536368656D6541</d:Hexscheme>
    <d:Hexid>506172746E6572313233</d:Hexid>
    <d:Agency>AgencyA</d:Agency>
    <d:Scheme>SchemeA</d:Scheme>
    <d:Id>Partner123</d:Id>
    <d:Pid>PartnerZ</d:Pid>
    <d:LastModifiedBy><user who modified alternative partner></d:LastModifiedBy>
    <d:LastModifiedTime>2021-03-16T08:25:53.556</d:LastModifiedTime>
    <d:CreatedBy><user who created alternative partner></d:CreatedBy>
    <d:CreatedTime>2020-07-01T11:57:26.096</d:CreatedTime>
  </m:properties>
</content>
```

Certain values (for example, for the agency when indicated in the example request as Hexagency) have to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string `my agency` is expressed by the following decimal numbers: 109 121 32 97 103 101 110 99 121 (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number 109, and so forth). If you convert each number to a hexadecimal number, you get: 6D 79 20 61 67 65 6E 63 79. Note that 6D is the hexadecimal representation of 109, and so forth. That means that, for example, the value `my agency` is given in hexadecimal notation by 6D79206167656E6379. Note that the hex values are built from the string values by first transforming the string value to a UTF-8-encoded byte array and then by calculating a hex string (with uppercase letters 'A', 'B', 'C', 'D', 'E').

Java example for calculating hex values:

```
byte[] bytes = "testValue".getBytes(StandardCharsets.UTF_8);
String hexString = DatatypeConverter.printHexBinary(bytes);
```

However, the properties Hexagency, Hexscheme, and Hexid are not supported in the filter. If you use these properties, you get an empty result.

Get Alternative Partners (Filter with Select)

Get all alternative partners for a dedicated agency, schema, and Id. Furthermore, in the result set show only the Pid parameter.

Perform the following call:

Method	Resource Path
GET	/AlternativePartners?\$filter=Agency eq '{Agency}' and Scheme eq '{Scheme}' and Id eq '{Id}'&\$select=Pid

Assume that you like to get the Pid value for the Partner Directory entry associated with agency `agency1`, schema `schema1`, and Id `Id1`.

Therefore, you need to send the following GET request:

```
https://<host address>/api/v1/AlternativePartners?$filter=Agency eq 'agency1' and Scheme eq 'schema1' and Id eq 'Id1'&$select=Pid
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

As a response, you get a message that contains the following part:

```
<entry>
<id>https://<host address>/api/v1/AlternativePartners(Hexagency='6167656E637931',Hexscheme='736368656D6131',Hexid='496431')</id>
<title type="text">AlternativePartners</title>
<updated>2021-03-16T08:08:41.059Z</updated>
<category term="com.sap.hci.api.AlternativePartner" scheme="http://schemas.microsoft.com/ado/2007/08/dataservices/scheme"/>
<link href="AlternativePartners(Hexagency='6167656E637931',Hexscheme='736368656D6131',Hexid='496431')" rel="edit" title="Alter
<content type="application/xml">
  <m:properties>
    <d:Pid>partnerX</d:Pid>
  </m:properties>
</content>
</entry>
```

Certain values (for example, for the agency when indicated in the example request as Hexagency) are also provided by the API in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string `my agency` is expressed by the following decimal numbers: 109 121 32 97 103 101 110 99 121 (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number 109, and so forth). If you convert each number to a hexadecimal number, you get: 6D 79 20 61 67 65 6E 63 79. Note that 6D is the hexadecimal representation of 109, and so forth. That means that, for example, the value `my agency` is given in hexadecimal notation by 6D79206167656E6379.

However, the properties Hexagency, Hexscheme, and Hexid aren't supported in the filter. If you use these properties, you get an empty result.

Create Alternative Partner

Create an alternative partner.

Perform the following call:

Method	Resource Path
POST	/AlternativePartners

Provide the following request body:

```
{
  "Agency": "string",
  "Scheme": "string",
  "Id": "string",
  "Pid": "string"
}
```

If you set the optional query parameter `user`, this user is used for audit logging instead of the user that is initiating the request.

Assume that you like to create a new alternative partner with the following attributes:

- Agency: myagency
- Scheme: myschema
- Id: myId
- Pid: partner1234

Therefore, you need to send the following POST request:

`https://<host address>/api/v1/AlternativePartners`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Provide the following request body:

```
{
  "Agency": "myschema",
  "Scheme": "myschema",
  "Id": "myId",
  "Pid": "partner1234"
}
```

As a response, you get a message that contains the following part:

```
<m:properties>
  <d:Hexagency>6D79736368656D61</d:Hexagency>
  <d:Hexscheme>6D79736368656D61</d:Hexscheme>
  <d:Hexid>6D794964</d:Hexid>
  <d:Agency>myschema</d:Agency>
  <d:Scheme>myschema</d:Scheme>
  <d:Id>myId</d:Id>
  <d:Pid>partner1234</d:Pid>
  <d:LastModifiedBy><user who modified alternative partner></d:LastModifiedBy>
  <d:LastModifiedTime>2021-03-16T07:31:05.761</d:LastModifiedTime>
  <d:CreatedBy><user who created alternative partner></d:CreatedBy>
  <d:CreatedTime>2021-03-16T07:31:05.761</d:CreatedTime>
</m:properties>
```

Certain values (for example, for the agency when indicated in the example request as Hexagency) are also provided by the API in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string `my agency` is expressed by the following decimal numbers: 109 121 32 97 103 101 110 99 121 (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number 109, and so forth). If you convert each number to a hexadecimal number, you get: 6D 79 20 61 67 65 6E 63 79. Note that 6D is the hexadecimal representation of 109, and so forth. That means that, for example, the value `my agency` is given in hexadecimal notation by 6D79206167656E6379.

Update an Alternative Partner

Update an alternative Partner.

Perform the following call:

Method	Resource Path
PUT	<code>/AlternativePartners(Hexagency='{Hexagency}',Hexscheme='{Hexscheme}',Hexid='{Hexid}')</code>

Provide the following request body (with the new Pid value).

```
{
  "Pid": "string"
}
```

If you set the optional query parameter `user`, this user is used for audit logging instead of the user that is initiating the request.

Assume that you like to update a new alternative partner with the following attributes:

- Agency: AgencyA
- Scheme: SchemaA
- Id: Partner123
- New Pid: PartnerZ

Therefore, you need to send the following PUT request, providing the hexadecimal values for the key parameters:

```
https://<host
address>/api/v1/AlternativePartners(Hexagency='4167656e637941',Hexscheme='536368656d6541',Hexid='506172746e6572313233')
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Provide the following request body:

```
{
  "Pid": "PartnerZ"
}
```

Certain values (for example, for the agency when indicated in the example request as `Hexagency`) have to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string `my agency` is expressed by the following decimal numbers: 109 121 32 97 103 101 110 99 121 (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number 109, and so forth). If you convert each number to a hexadecimal number, you get: 6D 79 20 61 67 65 6E 63 79. Note that 6D is the hexadecimal representation of 109, and so forth. That means that, for example, the value `my agency` is given in hexadecimal notation by 6D79206167656E6379. Note that the hex values are built from the string values by first transforming the string value to a UTF-8-encoded byte array and then by calculating a hex string (with uppercase letters 'A', 'B', 'C', 'D', 'E').

Java example for calculating hex values:

```
byte[] bytes = "testValue".getBytes(StandardCharsets.UTF_8);
String hexString = DatatypeConverter.printHexBinary(bytes);
```

Requests for String Parameter, Binary Parameter, and Authorized User

Create Authorized User

Purpose	Method	Resource Path
Create authorized user.	POST	/AuthorizedUsers

Assume that you like to create an authorized user `MyUser` that is to be associated to partner with Pid `PartnerZ`.

Perform a POST request to the following address:

```
https://<host address>/api/v1/AuthorizedUsers
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Provide the following request body:

```
{
  "User": "MyUser",
  "Pid": "PartnerZ"
}
```

As a response, you get a message that contains the following part:

```
<content type="application/xml">
  <m:properties>
    <d:User>myuser</d:User>
    <d:Pid>PartnerZ</d:Pid>
    <d:LastModifiedBy><modifying user></d:LastModifiedBy>
    <d:LastModifiedTime>2021-03-16T11:46:13.969</d:LastModifiedTime>
    <d:CreatedBy><creating user></d:CreatedBy>
    <d:CreatedTime>2021-03-16T11:46:13.969</d:CreatedTime>
  </m:properties>
</content>
```

i Note

If you want to read `AuthorizedUsers` with a filter expression containing user values, such as `/AuthorizedUser?$filter=User eq 'myuser'`, you must always use lowercase characters (Locale.EnGLISH) for the user value.

Create and Get a String Parameter

Purpose	Method	Resource Path
Create String parameter.	POST	/StringParameters
Get String parameter.	GET	/StringParameters

Assume that you like to create a String parameter `ReceiverAddress` that contains the endpoint address of a receiver system called by Cloud Integration .

To create the String parameter, perform a POST request to the following address:

`https://<host address>/api/v1/StringParameters?user=myuser`

i Note

The custom query option `user` is optional, but recommended if a technical user is used for the communication. The query option `user` is the logged-in user of the system that initiates the call.

Assume that you like to create a String parameter with the following attributes:

- Pid: PartnerZ
- Id: ReceiverAddress
- Value: `http://receiverZ.receiver_org.com`

Provide the following request body:

```
{
  "Pid": "PartnerZ",
  "Id": "ReceiverAddress",
  "Value": "http://receiverZ.receiver_org.com"
}
```

As a response, you get a message that contains the following part:

```
<content type="application/xml">
  <m:properties>
    <d:Pid>PartnerZ</d:Pid>
    <d:Id>ReceiverAddress</d:Id>
    <d:LastModifiedBy>myuser</d:LastModifiedBy>
    <d:LastModifiedTime>2021-03-16T11:25:07.916</d:LastModifiedTime>
    <d:CreatedBy>myuser</d:CreatedBy>
    <d:CreatedTime>2021-03-16T11:25:07.916</d:CreatedTime>
    <d:Value>http://receiverZ.receiver_org.com</d:Value>
  </m:properties>
</content>
```

You've the several options to read a String parameter.

To read String parameters associated with a partner with Pid **PartnerZ**, perform the following GET request:

```
https://<host address>/api/v1/StringParameters?$filter=Pid eq 'PartnerZ'
```

i Note

For better readability, the URI `https://<host address>/api/v1/StringParameters?$filter=Pid eq 'PartnerZ'` isn't URI encoded. The correct URI is `https://appl-tmn.avt.hana.ondemand.com/api/v1/StringParameters$filter=Pid%20eq%20%27PartnerZ%27`.

As a response, you get a message that contains the following part:

```
<content type="application/xml">
  <m:properties>
    <d:Pid>PartnerZ</d:Pid>
    <d:Id>ReceiverAddress</d:Id>
    <d:LastModifiedBy>myuser</d:LastModifiedBy>
    <d:LastModifiedTime>2021-03-16T12:06:08.856</d:LastModifiedTime>
    <d:CreatedBy>myuser</d:CreatedBy>
    <d:CreatedTime>2021-03-16T12:06:08.856</d:CreatedTime>
    <d:Value>http://receiverZ.receiver_org.com</d:Value>
  </m:properties>
</content>
```

To read String parameters with paging and select, you can perform the following GET request:

```
https://<host address>/api/v1/StringParameters?$select=Value
```

As response, you get a message with one item per String parameter. Each item contains a part such like:

```
<entry>
  <id>https://<host address>/api/v1/StringParameters(Pid='PartnerZ',Id='ReceiverAddress')</id>
  <title type="text">StringParameters</title>
  <updated>2021-03-16T12:27:14.51Z</updated>
  <category term="com.sap.hci.api.StringParameter" scheme="http://schemas.microsoft.com/ado/2007/08/dataservices/scheme"
  <link href="StringParameters(Pid='PartnerZ',Id='ReceiverAddress') " rel="edit" title="StringParameter"/>
  <content type="application/xml">
    <m:properties>
      <d:Value>http://receiverZ.receiver_org.com</d:Value>
    </m:properties>
  </content>
</entry>
```

This GET request selects only the value field content of the first 1000 string parameter entities. To get the next 1000 entries, you've to call:

```
https://<host address>/api/v1/StringParameters?$select=Value&$format=json&$skiptoken=1000
```

The paging size for binary parameters is 30.

The following GET request reads the single entity by key values:

```
https://<tmn>/api/v1/StringParameters(Pid='PartnerZ',Id='ReceiverAddress')
```

As response, you get with one item with the following part:

```
<m:properties>
  <d:Pid>PartnerZ</d:Pid>
  <d:Id>ReceiverAddress</d:Id>
  <d:LastModifiedBy>myuser</d:LastModifiedBy>
  <d:LastModifiedTime>2021-03-16T12:06:08.856</d:LastModifiedTime>
  <d:CreatedBy>myuser</d:CreatedBy>
  <d:CreatedTime>2021-03-16T12:06:08.856</d:CreatedTime>
  <d:Value>http://receiverZ.receiver_org.com</d:Value>
</m:properties>
```

For more information, check out the following SAP Community blog: [Cloud Integration – Partner Directory – Step-by-Step Example](#)👉.

Create Binary Parameter

Purpose	Method	Resource Path
Create Binary parameter.	POST	/BinaryParameters

Assume that you like to create a Binary parameter `MyUser` that is to be associated to partner with `Pid PartnerZ`.

Perform a POST request to the following address:

`https://<host address>/api/v1/BinaryParameters`

Provide the following request body:

```
{
  "Pid": "PartnerZ",
  "Id": "Z_XSD",
  "ContentType": "xsd",
  "Value": "PHhz0nNjaGVtYSBhdHRy2R1Y3RzIj4KICAgIDx4czpj . . . . ."
}
```

As `Value`, in our example provide a base64-encoded XSD file.

As response, you get:

```
<m:properties>
  <d:Pid>PartnerZ</d:Pid>
  <d:Id>Z_XSD</d:Id>
  <d:LastModifiedBy>myuser</d:LastModifiedBy>
  <d:LastModifiedTime>2021-03-16T15:29:16.151</d:LastModifiedTime>
  <d:CreatedBy>myuser</d:CreatedBy>
  <d:CreatedTime>2021-03-16T15:29:16.151</d:CreatedTime>
  <d:ContentType>xsd</d:ContentType>
  <d:Value>PHhz0nNjaGVtYSBhdHRyYWJldGVGb3JtRG. . . M6c2NoZWlhPg==</d:Value>
</m:properties>
```

i Note

Note that possible values for `ContentType` are: xml, xsl, xsd, json, text, zip, gz (for GZIP files), zlib, crt (for DER-encoded X.509 certificate).

The maximum length of the `Value` field is restricted to 260 KB.

The PEM-DER string of a certificate must be base64 encoded for the value field of the binary parameter in the POST or PUT request.

For more information on Binary parameters, check out the following SAP Community blog: [Cloud Integration – Partner Directory – Partner Dependent XML Structures and IDs](#).

Update Authorized User

Purpose	Method	Resource Path
Update binary parameter.	PUT	/BinaryParameters(Pid='partner1',Id='bp1')?user=userXYZ

To update a binary parameter associated with user `userXYZ`, perform a PUT request to the following address:

`https://<host address>/api/v1/BinaryParameters(Pid='partner1',Id='bp1')?user=userXYZ`

Provide the following request body:

```
{
  "ContentType": "string",
  "Value": "string"
}
```

Mass Create/Change/Delete Operations in One Transaction for StringParameter, BinaryParameter, AlternativePartner, and AuthorizedUser

You can perform mass processing through an OData batch. Batch processing allows you to group multiple operations. Batch requests with one change set are allowed. One change set is treated as an atomic operation (transaction). Create/change/delete operations can be executed in one change set for the entity types `StringParameter`, `BinaryParameter`, `AlternativePartner`, and `AuthorizedUser`. The following example shows how.

```
POST https://<host address>/api/v1/$batch
Accept: application/json
Content-type: multipart/mixed; boundary=batch_34fcd829-64e4-4139-b880-f24aa4ee9235
--batch_34fcd829-64e4-4139-b880-f24aa4ee9235
Content-Type: multipart/mixed; boundary=changeset_845b320f-7f06-423d-b10a-a24048a27b1c
--changeset_845b320f-7f06-423d-b10a-a24048a27b1c
Content-Type: application/http
Content-Transfer-Encoding: binary

POST StringParameters?user=userAbc HTTP/1.1
Content-Length: 46
Accept: application/json
content-type: application/json

{"Pid":"partner7","Id":"sp11","Value":"sp11v"}
--changeset_845b320f-7f06-423d-b10a-a24048a27b1c
Content-Type: application/http
Content-Transfer-Encoding: binary

POST StringParameters?user=userAbc HTTP/1.1
Content-Length: 46
Accept: application/json
content-type: application/json

{"Pid":"partner7","Id":"sp12","Value":"sp12v"}
--changeset_845b320f-7f06-423d-b10a-a24048a27b1c
Content-Type: application/http
Content-Transfer-Encoding: binary

PUT StringParameters(Pid='partner2',Id='sp3')?user=AriabaUserId HTTP/1.1
Content-Length: 53
Accept: application/json
content-type: application/json

{"Value":"sp3v_modified"}
--changeset_845b320f-7f06-423d-b10a-a24048a27b1c--
--batch_34fcd829-64e4-4139-b880-f24aa4ee9235--
Batch response:
--batch_46440294-4efe-4c78-8d38-6414e44382d0
Content-Type: multipart/mixed; boundary=changeset_8a1989b1-9382-446c-8479-f9330108b1f4
--changeset_8a1989b1-9382-446c-8479-f9330108b1f4
Content-Type: application/http
Content-Transfer-Encoding: binary
HTTP/1.1 201 Created
DataServiceVersion: 2.0
Location: https://<host address>/api/v1/StringParameters(Pid='partner7',Id='sp11')
Content-Type: application/json
Content-Length: 328

{"d":{"__metadata":{"id":"https://<host address>/api/v1/StringParameters(Pid='partner7',Id='sp11')","uri":"https://<host address>/api/v1/StringParameters(Pid='partner7',Id='sp11')"},"uri":"https://<host address>/api/v1/StringParameters(Pid='partner7',Id='sp11')"},"uri":"https://<host address>/api/v1/StringParameters(Pid='partner7',Id='sp11')"}
--changeset_8a1989b1-9382-446c-8479-f9330108b1f4
Content-Type: application/http
Content-Transfer-Encoding: binary
HTTP/1.1 201 Created
DataServiceVersion: 2.0
Location: https://<host address>/api/v1/StringParameters(Pid='partner7',Id='sp12')
Content-Type: application/json
Content-Length: 328

{"d":{"__metadata":{"id":"https://<host address>/api/v1/StringParameters(Pid='partner7',Id='sp12')","uri":"https://<host address>/api/v1/StringParameters(Pid='partner7',Id='sp12')"},"uri":"https://<host address>/api/v1/StringParameters(Pid='partner7',Id='sp12')"},"uri":"https://<host address>/api/v1/StringParameters(Pid='partner7',Id='sp12')"}
--changeset_8a1989b1-9382-446c-8479-f9330108b1f4
Content-Type: application/http
Content-Transfer-Encoding: binary
HTTP/1.1 204 No Content
DataServiceVersion: 2.0
--changeset_8a1989b1-9382-446c-8479-f9330108b1f4--
--batch_46440294-4efe-4c78-8d38-6414e44382d0--
```

For more information, check out the following SAP Community blog: [Cloud Integration – Partner Directory – Mass Configuration](#) .

Requests for UserCredentialParameter

Caution

The charset encoding of the JSON documents in the requests must be UTF-8. The charset encoding of the JSON documents of the responses is also UTF-8.

Create or Update a User Credentials Parameter

Method	Resource Path
POST	/UserCredentialParameters
PUT	

Assume that you like to create a User Credentials parameter with Pid PartnerZ and Id Id1.

Perform a POST request to the following address:

<https://<host address>/api/v1/UserCredentialParameters>

The part <https://<host address>/api/v1> is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Provide the following request body (example):

```
{
  "Pid": "PartnerZ",
  "Id": "Id1",
  "User": "myUser",
  "Password": "Abcd1234"
}
```

As a response, you get a message that contains the following part:

```
<content type="application/xml">
  <m:properties>
    <d:Pid>PartnerZ</d:Pid>
    <d:Id>Id1</d:Id>
    <d:LastModifiedBy>P12345678</d:LastModifiedBy>
    <d:LastModifiedTime>2021-03-17T10:28:31.026</d:LastModifiedTime>
    <d:CreatedBy>P12345678</d:CreatedBy>
    <d:CreatedTime>2021-03-17T10:28:31.026</d:CreatedTime>
    <d:User>myUser</d:User>
    <d>Password m:null="true"/>
  </m:properties>
</content>
```

You can find a new User Credentials artifact in the [Monitor](#) section ([Security Material](#) tile under [Manage Security](#)). The newly created artifact has the name `pd:PartnerZ:Id1:UserCredential`.

The screenshot shows the SAP Cloud Integration interface. The top bar includes the SAP logo and 'Cloud Integration'. The left sidebar has icons for navigation. The main content area is titled 'Overview / Manage Security Material' and shows 'Security Material (52)'. Below this is a table with two columns: 'Name' and 'Type'.

Name	Type
pd:PartnerZ:Id1:UserCredential	User Credentials

Note

The specification of a user query option User (such as `?user=aribaUser`) has no effect. It is always the logged in technical user (in the above example P12345678 that appears in the properties LastModifiedBy and CreatedBy).

You can also use the POST request to update a User Credentials parameter with the same values for PID and Id.

There is no difference between the properties LastModifiedTime and CreatedTime. They always contain the time of the last POST request or the tuple PID and Id.

There is no difference between the properties LastModifiedB and CreatedBy. Both fields contain the logged in user of the last POST request for the tuple PID and Id.

Caution

PUT requests are not supported.

Get User Credentials Parameters

Method	Resource Path
GET	/UserCredentialParameters

Assume that you like to get the User Credentials parameter with Pid PartnerZ.

Perform a GET request to the following address:

`https://<host address>/api/v1/UserCredentialParameters?user=AnyUser&$filter=Pid eq 'PartnerZ'`

If there are two User Credentials parameters with Pid PartnerZ, you get the following response:

```
<feed ...>
...
<entry>
  <id>https://d0262-tmn.hci.eu1.hana.ondemand.com:443/api/v1/UserCredentialParameters(Pid='PartnerZ',Id='Id1')</id>
  <title type="text">UserCredentialParameters</title>
  <updated>2021-03-17T10:42:46.843Z</updated>
  <category term="com.sap.hci.api.UserCredentialParameter" scheme="http://schemas.microsoft.com/ado/2007/08/dataservices" />
  <link href="UserCredentialParameters(Pid='PartnerZ',Id='Id1')" rel="edit" title="UserCredentialParameter"/>
  <content type="application/xml">
    <m:properties>
      <d:Pid>PartnerZ</d:Pid>
      <d:Id>Id1</d:Id>
      <d:LastModifiedBy>P12345678</d:LastModifiedBy>
      <d:LastModifiedTime>2021-03-17T10:28:31.101</d:LastModifiedTime>
      <d:CreatedBy>P12345678</d:CreatedBy>
      <d:CreatedTime>2021-03-17T10:28:31.101</d:CreatedTime>
      <d:User>myUser</d:User>
      <d>Password m:null="true"/>
    </m:properties>
  </content>
</entry><entry>
  <id>https://d0262-tmn.hci.eu1.hana.ondemand.com:443/api/v1/UserCredentialParameters(Pid='PartnerZ',Id='Id2')</id>
  <title type="text">UserCredentialParameters</title>
  <updated>2021-03-17T10:42:46.843Z</updated>
  <category term="com.sap.hci.api.UserCredentialParameter" scheme="http://schemas.microsoft.com/ado/2007/08/dataservices" />
  <link href="UserCredentialParameters(Pid='PartnerZ',Id='Id2')" rel="edit" title="UserCredentialParameter"/>
  <content type="application/xml">
    <m:properties>
      <d:Pid>PartnerZ</d:Pid>
      <d:Id>Id2</d:Id>
      <d:LastModifiedBy>P12345678</d:LastModifiedBy>
      <d:LastModifiedTime>2021-03-17T10:42:20.822</d:LastModifiedTime>
      <d:CreatedBy>P12345678</d:CreatedBy>
      <d:CreatedTime>2021-03-17T10:42:20.822</d:CreatedTime>
      <d:User>myUser</d:User>
      <d>Password m:null="true"/>
    </m:properties>
  </content>
</entry>
</feed>
```

Note

The returned value for the Password property is always null.

You need to specify a filter for property Pid. If you don't specify a filter for this property, you get an error.

If you specify a `User` in the query options (such as `?user=AnyUser` in the example request), this user appears in the audit log.

Assume that you like to get the `User Credentials` parameter with `Pid PartnerZ` and select only the properties `Id` and `CreatedBy`.

Perform a GET request to the following address:

`https://<host address>/api/v1/UserCredentialParameters?user=AnyUser&$filter=Pid eq 'PartnerZ'&$select=Id,CreatedBy`

If there are two `User Credentials` parameters with `Pid PartnerZ`, you get the following response:

```
<feed ...>
...
<entry>
  <id>https://d0262-tmn.hci.eu1.hana.ondemand.com:443/api/v1/UserCredentialParameters(Pid='PartnerZ',Id='Id1')</id>
  <title type="text">UserCredentialParameters</title>
  <updated>2021-03-17T10:50:15.729Z</updated>
  <category term="com.sap.hci.api.UserCredentialParameter" scheme="http://schemas.microsoft.com/ado/2007/08/dataservices" />
  <link href="UserCredentialParameters(Pid='PartnerZ',Id='Id1')" rel="edit" title="UserCredentialParameter"/>
  <content type="application/xml">
    <m:properties>
      <d:Id>Id1</d:Id>
      <d:CreatedBy>P12345678</d:CreatedBy>
    </m:properties>
  </content>
</entry>
<entry>
  <id>https://d0262-tmn.hci.eu1.hana.ondemand.com:443/api/v1/UserCredentialParameters(Pid='PartnerZ',Id='Id2')</id>
  <title type="text">UserCredentialParameters</title>
  <updated>2021-03-17T10:50:15.731Z</updated>
  <category term="com.sap.hci.api.UserCredentialParameter" scheme="http://schemas.microsoft.com/ado/2007/08/dataservices" />
  <link href="UserCredentialParameters(Pid='PartnerZ',Id='Id2')" rel="edit" title="UserCredentialParameter"/>
  <content type="application/xml">
    <m:properties>
      <d:Id>Id2</d:Id>
      <d:CreatedBy>P12345678</d:CreatedBy>
    </m:properties>
  </content>
</entry>
</feed>
```

Assume that you like to get the `User Credentials` parameter with `Pid PartnerZ` and `Id Id1` and select only the properties `User` and `CreatedBy`.

Perform a GET request to the following address:

`https://<host address>/api/v1/UserCredentialParameters(Pid='PartnerZ',Id='Id1')?user=AnyUser&$select=User,CreatedBy`

You get the following response:

```
<entry ...>
...
  <content type="application/xml">
    <m:properties>
      <d:User>myUser</d:User>
      <d:CreatedBy>P12345678</d:CreatedBy>
    </m:properties>
  </content>
</entry>
```

Use Custom Query Option to Get Password in Hash Format

You can use the query option `returnHashedPassword=SHA256` to request that the password is returned in hash format. You get a value in the `password` property with the following format:

`SAHA256.<SHA256 hash value of the password in HEX>`

If you do not use this option, you get a null value for the password.

For example, perform the following GET request:

```
https://<host address>/api/v1/UserCredentialParameters(Pid='PartnerZ',Id='Id1')?
user=AnyUser&returnHashedPassword=SHA256
```

You get the following response:

```
<entry ...>
...
  <content type="application/xml">
    <m:properties>
      <d:Pid>PartnerZ</d:Pid>
      <d:Id>Id1</d:Id>
      <d:LastModifiedBy>P12345678</d:LastModifiedBy>
      <d:LastModifiedTime>2021-03-17T10:28:31.101</d:LastModifiedTime>
      <d:CreatedBy>P12345678</d:CreatedBy>
      <d:CreatedTime>2021-03-17T10:28:31.101</d:CreatedTime>
      <d:User>myUser</d:User>
      <d>Password>SHA256.3f21a8490cef2bfb60a9702e9d2ddb7a805c9bd1a263557dfd51a7d0e9dfa93e</d>Password>
    </m:properties>
  </content>
</entry>
```

Performing Batch Operations

There are restrictions for change sets in batch:

- A change set can only contain exactly one request with the `UserCredentialParameter` entity type.
- If a change set contains a `UserCredentialParameter` request, it cannot contain other requests with different entity types.

POST `https://<host address>/api/v1/$batch`:

```
--batch_06a89345-8056-41c4-91c2-55636fc844fe
Content-Type: multipart/mixed; boundary=changeset_d8938fea-cd05-464a-a7c8-4522e5e7b1e4
--changeset_d8938fea-cd05-464a-a7c8-4522e5e7b1e4
Content-Type: application/http
Content-Transfer-Encoding: binary
POST UserCredentialParameters?user=AribaUser HTTP/1.1
Content-Length: 83
Accept: application/json
Content-Type: application/json
{"Pid":"pid_batch","Id":"id_batch","User":"user_batch","Password":"password_batch"}
--changeset_d8938fea-cd05-464a-a7c8-4522e5e7b1e4--
--batch_06a89345-8056-41c4-91c2-55636fc844fe--
Batch response
--batch_6264c4f6-6117-4159-87e7-82d2e370850e
Content-Type: multipart/mixed; boundary=changeset_0033d380-2e75-4284-8fb1-d7333a40dc9a
--changeset_0033d380-2e75-4284-8fb1-d7333a40dc9a
```

```
Content-Type: application/http
Content-Transfer-Encoding: binary
```

```
Response:
--batch_6264c4f6-6117-4159-87e7-82d2e370850e
Content-Type: multipart/mixed; boundary=changeset_0033d380-2e75-4284-8fb1-d7333a40dc9a
--changeset_0033d380-2e75-4284-8fb1-d7333a40dc9a
Content-Type: application/http
Content-Transfer-Encoding: binary
HTTP/1.1 201 Created
DataServiceVersion: 2.0
Location: https://appl-tmn.avt.hana.ondemand.com:443/api/v1/UserCredentialParameters(Pid='pid_batch',Id='id_batch')
Content-Type: application/json
Content-Length: 527
{"d":{"__metadata":{"id":"https://appl-tmn.avt.hana.ondemand.com:443/api/v1/UserCredentialParameters(Pid='pid_batch',Id='id_batch')",
"uri":"https://appl-tmn.avt.hana.ondemand.com:443/api/v1/UserCredentialParameters(Pid='pid_batch',Id='id_batch')",
"type":"com.sap.hci.api.model.UserCredentialParameter"},
"PId":"pid_batch",
"Id":"id_batch",
"LastModifiedBy":"P12345678",
"LastModifiedTime":"\\/Date(1453721123280)\\/","
"CreatedBy":"P12345678",
"CreatedTime":"\\/Date(1453721123280)\\/","
"User":"user_batch","Password":null}}
--changeset_0033d380-2e75-4284-8fb1-d7333a40dc9a--
--batch_6264c4f6-6117-4159-87e7-82d2e370850e--
```

Security Content

Manage security content on the tenant that is required to configure secure connections with remote systems.

Depending on the kind of connection, the applied authorization and authorization options, and the direction of the request (either inbound or outbound), different kind of security content needs to be created and deployed on the tenant.

You can access the OData API at <https://api.sap.com/api/SecurityContent/> where you can find the basic operations.

This documentation provides additional information.

Permissions

To authorize an API client to access the OData API, perform the steps as described in:

- In the **Cloud Foundry** environment, perform the steps described in: [Setting Up Inbound HTTP Connections \(for API Clients\)](#).

Assign the following role template:

`MonitoringDataRead` (see [Tasks and Permissions](#))

- In the **Neo** environment, perform the steps described in: [Setting Up Inbound HTTP Connections \(for API Clients\), Neo Environment](#)

Assign the following roles:

`IntegrationOperationServer.read`, `NodeManager.read` (see [Tasks and Permissions](#))

Resources

Resource	Description
Certificate	Represents a trusted certificate (keystore entry) (see Managing Keystore Entries and X.509 Certificates).
Key Pair	Represents a key pair of the tenant keystore (see Creating a Key Pair/SSH Key Pair). You can use the API to create, import, or export a private key pair. You can also create a certificate signing request, or import and export the related certificate chain.
Keystore Entry	Represents an entry of the tenant keystore (see Managing Keystore Entries). The keystore contains private/public key pairs and certificates that are used in various scenarios. Typically, a keystore contains keys and certificates for connections in both directions (inbound and outbound). Keys can be used for protecting the communication both on the transport-level and on the message-level. Keystore entries have to be maintained in accordance with the security content owned by the connected sender and receiver systems to ensure that the communication is protected based on public key technology. For example, for scenarios where SAP Cloud Integration receives encrypted messages from a sender system, the tenant keystore needs to contain a private/public key pair (where the private key is used by the Decryptor step of the integration flow). The public key (associated with the private key) needs to be available in the corresponding key storage of the sender system. A keystore typically contains entries that belong to the tenant administrator and entries that are owned by SAP. Each entry is uniquely identified by an alias. Keystore entries have a dedicated lifecycle, which means that they have an expiration date and need to be renewed in a coordinated way. Various properties are available for a keystore entry, for example, alias, key type, key size, the validation period, Subject DN and Issuer DN, and administrative information such as the time when the entry was last modified. You can find a user interface to manage keystore entries in the Keystore tile of the Monitor section under Manage Security .
Keystore	Represents a tenant keystore (see Managing Keystore Entries). You can use this resource to manage the keystore content as a whole. For managing individual keystore entries, use the Keystore Entry and Keystore resources. You can import a keystore (containing multiple keystore entries) or export the public content of the keystore. You can back up keystore entries owned by the tenant administrator and, vice versa, retrieve backed-up entries. You can also trigger a mass deletion of keystore entries. ⚠ Caution You need to use UTF-8 encoding for the charset of the JSON documents in the requests. The charset of the JSON documents of the responses is also UTF-8 encoded. Hex representation is used for the <code>Alias</code> field; the server doesn't allow slashes or backslashes in a URI, even if they're percent encoded. Hex representation of the alias does mean that a UTF-8-encoded byte array is built from the alias string. A hex string is then calculated from this byte array. i Note The maximum size of a keystore is 2 MB (when using the Neo environment).

Resource	Description
	- The 2 MB limit corresponds to around 2000 X.509 certificates. - A key pair with a chain of three X.509 certificates consumes about 3 KB, so if the keystore only contains key pairs of this type, then you can store around 600 key pairs in the keystore. The maximum size of a keystore is 6 MB (when using the Cloud Foundry environment). - The 6 MB limit corresponds to around 6000 X.509 certificates. - A key pair with a chain of three X.509 certificates consumes about 3 KB, so if the keystore only contains key pairs of this type, then you can store around 1800 key pairs in the keystore. If you upload a whole keystore (. j ks file) to the tenant, the maximum keystore size is limited to 2 MB.
Keystore History	Represents the SAP Key History keystore (see Managing the Lifecycle of Keys). This keystore contains old (expired) keys.
User Credentials	Represents a User Credentials artifact (see Deploying a User Credentials Artifact). To set up outbound connections using basic authentication, you need to deploy a User Credentials artifact. The artifact contains a user name and password and, if applicable, other parameters for the user associated with the outbound call.
Secure Parameter	Represents a Secure Parameter artifact (see Deploying a Secure Parameter Artifact). You use this artifact type to deploy confidential data, for example, for custom adapters.
OAuth2 Client Credentials	Represents an OAuth2 Client Credentials artifact (see Deploying an OAuth2 Client Credentials Artifact). When an OAuth 2.0 client credentials grant is implemented, the client gets access to the protected resources in two steps: After presenting a set of client credentials, the client fetches an access token from the token service. In a subsequent step, the client uses the access token to get access to the protected resources.
Certificate-to-User-Mapping (Neo environment)	Represents a Certificate-to-User-Mapping artifact required for inbound calls based on client certificate authentication and User Role authorization (see Managing Certificate-to-User Mappings, Neo Environment). i Note Certificate-to-user mappings are only available when using SAP Cloud Integration in the Neo environment. The sender authenticates itself against Cloud Integration based on a digital client certificate. This certificate is also mapped to a user (based on the information contained in a Certificate-to-User Mapping artifact deployed on the tenant). ! Restriction Be aware of the following restrictions: - Only 1:n relationships between user and certificate are supported. The same user can be used for different certificates. - When using an entity in the context of the Partner Directory, only 290 entries for certificate-to-user mappings are allowed in the Partner Directory. - The user name (user attribute) can have a maximum length of 150 characters. - The key field Id contains the MD5 of the certificate in hex format. Note that certificates with the same MD5 value overwrite each other. The MD5 is calculated from the DER-encoded bytes (not on the PEM format) and the hex format uses lower case characters (maximum length of 32 characters). - The Certificate attribute contains an X.509 certificate in PEM-DER format (maximum length of 8192 characters). - When using an entity in the context of the Partner Directory, note that the Partner Directory creates the necessary user-to-role assignments for these users so that the users can execute integration flows on the runtime node. When deleting a certificate-to-user mapping, there are restrictions for change sets in a batch: - A change set can only contain one request with the CertificateUserMapping entity type. - If a change set contains a CertificateUserMapping request, it can't contain other requests with other entity types.
Access Policies	Access policies allow you to restrict the access to integration artifacts and their associated data. See: Managing Access Policies
Artifact References	Artifact references define for which artifacts (for example, integration flows) an access policy applies. See: Managing Access Policies

For general information about query options, see [Query Options](#).

i Note

This is custom documentation. For more information, please visit [SAP Help Portal](#).

TheCloud Integration **Monitor** application provides a user interface to access these resources:

- For User Credentials and OAuth2 Client Credentials, see the **Security Material** tile under **Manage Security**.
- For Certificate, Key Pair, Keystore Entry, Keystore History, and Keystore, see the **Keystore** tile under **Manage Security**.
- For Certificate-to-User-Mapping (only available in the Neo environment), see the **Certificate-to-User Mappings** tile under **Manage Security**.

Example Requests

You find various example requests on SAP Business Accelerator Hub at [Security Content](#).

For additional example requests, see [Security Content Example Requests](#).

Certain values (for example, for the alias when indicated in the example request as <hexaalias>) need to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string my alias is expressed by the following decimal numbers: 109 121 32 97 108 105 97 115 (because, according to the American Standard Code for Information Interchange (ASCII), the letter m is translated to the decimal number 109, and so forth). If you convert each number to a hexadecimal number, you get: 6D 79 20 61 6C 69 61 73. Note that 6D is the hexadecimal representation of 109, and so forth. If you like to specify my alias in an example request, enter 6D7920616C696173.

To trigger the modifying actions (POST, PUT, and DELETE), you need to fetch a CSRF token first. You can do that in the following way: First, send a GET call to the API with a header X-CSRF-Token with the value Fetch. As a response, you get the value of the token in the header X-CSRF-Token. When sending the modifying request, add the header X-CSRF-Token with the token retrieved from the first call.

On SAP Business Accelerator Hub, you can test API calls against a sandbox tenant or against a custom tenant (to be configured under **API Environment**). If you want to perform an API call against your custom tenant using an HTTP client such like Postman, make sure that the request URL is composed in the following way:

https://<host address>/api/v1/<relative resource path>

The part https://<host address>/api/v1 is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

You can find the relative resource path for each operation on SAP Business Accelerator Hub.

Related Information

- [Manage Security](#)
- [Concepts of Secure Communication](#)
- [Keystore](#)

Security Content Example Requests

You find various example requests on SAP Business Accelerator Hub at [Security Content](#).

For more example requests, see:

Certificate	Export Certificate Import and Update Certificate
Key Pair	Generate a Key Pair
Keystore Entry	Get All Keystore Entries Get Keystore Entry by Alias Rename the Alias of a Keystore Entry Export Public Key in OpenSSH Format
Keystore	Get Keystore Properties and Entries Import/Back up a Keystore Trigger Mass Deletion of Keystore Entries
Keystore History	Restore an Entry from the SAP Key History Keystore to the New SAP Keys Keystore
Certificate-to-User Mapping	Create a Certificate-to-User Mapping, Neo Environment Get Certificate-to-User Mappings with Specific User, Neo Environment

Export Certificate

Export a certificate from the keystore.

Perform the following call:

Method	Resource Path
GET	/KeystoreEntries('{Hexalias}')/Certificate/\$value

Assume that your tenant keystore contains a certificate with alias `smtp.mail.yahoo.com` (server certificate for the Yahoo mail server) and you like to export it. The hexadecimal value of the alias is: `736d74702e6d61696c2e7961686f6f2e636f6d`.

Certain values (for example, for the alias when indicated in the example request as `<hexalias>`) need to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string `my alias` is expressed by the following decimal numbers: `109 121 32 97 108 105 97 115` (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number `109`, and so forth). If you convert each number to a hexadecimal number, you get: `6D 79 20 61 6C 69 61 73`. Note that `6D` is the hexadecimal representation of `109`, and so forth. If you like to specify `my alias` in an example request, enter `6D7920616C696173`.

Therefore, you need to send the following GET request:

`https://<host address>/api/v1/KeystoreEntries('736d74702e6d61696c2e7961686f6f2e636f6d')/Certificate/$value`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

As a result, you get the certificate in the following form (content type: `application/pkix-cert`):

```
-----BEGIN CERTIFICATE-----
MIITDCCBzSgAwIBAgIQDBxHETUvPf18xKE7JdDoNjANBgkqhkiG9w0BAQsFADBw
MQswCQYDVQQGEwJVUzEVMBMGA1UEChMMRGlnaUNlcnQgSW5jMRkwFwYDVQQLExB3
d3cuZGlnaWNlcnQyY29tMS8wLQYDVQQDEyZEaWdpQ2VydCBTSEEyIEhpZ2ggQXNz

...

F8jCgaR+bTz+hYoLtfN9mow1kr5jNgc68HALdKF8SluCyZS5jFbD0M4HQay16pU
J++y5pvlBmtzQ57107itrSQ32praT6whMLCBy9pAnJIKzVM4IDM5H1drHc6shhuo
+XiCLYEJRHstbLVQYFwB2w==
-----END CERTIFICATE-----
```

Import and Update Certificate

Import and update a certificate.

Perform the following call:

Method	Resource Path
PUT	/CertificateResources('{Hexalias}')/\$value

Caution

Although an HTTP PUT method is used, a new OData entity is created with the sample request.

You like to import a certificate to the tenant keystore (with an alias `mycertificate`). The hexadecimal value of the alias is `6d796365727469666963617465`.

Certain values (for example, for the alias when indicated in the example request as `<hexalias>`) need to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string `my alias` is expressed by the following decimal numbers: `109 121 32 97 108 105 97 115` (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number `109`, and so forth). If you convert each number to a

hexadecimal number, you get: 6D 79 20 61 6C 69 61 73. Note that 6D is the hexadecimal representation of 109, and so forth. If you like to specify my alias in an example request, enter 6D7920616C696173.

Send the following PUT request:

https://<host address>/api/v1/CertificateResources('6d796365727469666963617465')/\$value

The part https://<host address>/api/v1 is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Add the certificate content with the request body (example content):

```
[-----BEGIN CERTIFICATE-----
MIIDdzCCA1+gAwIBA...
....
R9I4LtD+gdwyah617jzV/0eBHRnDJELqYzmp
-----END CERTIFICATE-----]
```

Generate a Key Pair

Generate a key pair.

Perform the following call:

Method	Resource Path
POST	/KeyPairGenerationRequests

Input Properties

The following table indicates whether the properties are mandatory or not and also contains the default values:

Property	Mandatory	Type	Default Value	Description
Hexalias	X	String		Any dummy hex value System calculates the correct hex value from the specified Alias property.
Alias	X	String		Keystore entry alias
KeyType		String	"RSA"	Key type Possible values are "RSA", "DSA", "EC". If not specified, the default value is used.
SignatureAlgorithm		String	"SHA-512/RSA"	Signature Algorithm Possible values depend on the KeyType value. If KeyType="RSA", possible values are: "SHA-512/RSA", "SHA-256/RSA", "SHA-384/RSA", "SHA-224/RSA", "SHA-1/RSA". If KeyType="DSA", possible values are: "SHA-256/DSA", "SHA-224/DSA", "SHA-1/DSA". If KeyType="EC", possible values are: "SHA-512/ECDSA", "SHA-256/ECDSA", "SHA-1/ECDSA".
KeySize		Integer	4096	KeySize must be specified for KeyType="RSA" and "DSA"; in the case of KeyType="EC", either this property or the property "KeyAlgorithmParameter" must be specified. For KeyType="EC", the value must be either null or from 112 through 571.
KeyAlgorithmParameter		String	null	Currently only used for KeyType="EC": Either this property or the property "KeySize" must then be specified. This

Property	Mandatory	Type	Default Value	Description
				property can contain the standard name of the Elliptic Curve domain parameters. Possible values: secp160k1,secp160r1,secp160r2,secp192k1,secp192r1,NIST P-192,X9.62 prime192v1,secp224k1,secp224r1,NIST P-224,secp256k1,secp256r1,NIST P-256,X9.62 prime256v1,secp384r1,NIST P-384,secp521r1,NIST P-521,X9.62 prime192v2,X9.62 prime192v3,X9.62 prime239v1,X9.62 prime239v2,X9.62 prime239v3,sect163k1, NIST K-163,sect163r1,sect163r2,NIST B-163,sect193r1,sect193r2,sect233k1,NIST K-233,sect233r1,NIST B-233,sect239k1,sect283k1,NIST K-283,sect283r1,NIST B-283,sect409k1,NIST K-409,sect409r1,NIST B-409,sect571k1,NIST K-571,sect571r1,NIST B-571,X9.62 c2tnb191v1,X9.62 c2tnb191v2,X9.62 c2tnb191v3,X9.62 c2tnb239v1,X9.62 c2tnb239v2,X9.62 c2tnb239v3,X9.62 c2tnb359v1,X9.62 c2tnb431r1 We recommend using the following curves. • Curves over F_p – 192 bits: secp192k1 and secp192r1. – 224 bits: secp224k1 and secp224r1. – 256 bits: secp256k1 and secp256r1. – 384 bits: secp384r1. – 521 bits: secp521r1. • Curves over F_{2^m} – 163 bits: sect163k1, sect163r1, and sect163r2. – 233 bits: sect233k1 and sect233r1. – 239 bits: sect239k1. – 283 bits: sect283k1 and sect283r1. – 409 bits: sect409k1 and sect409r1. – 571 bits: sect571k1 and sect571r1.
CommonName	X	String		Common name of the subject distinguished name of the certificate
OrganizationUnit		String		Organization unit of the subject distinguished name of the certificate
Organization		String		Organization of the subject distinguished name of the certificate
Locality		String		Locality of the subject distinguished name of the certificate
State		String		State/province of the subject distinguished name of the certificate
Country	X	String		Country or region of the subject distinguished name of the certificate, two characters required.
Email		String		E-mail of the subject distinguished name of the certificate
ValidNotBefore		Date	Current time	Lower boundary of the validity period of the certificate
ValidNotAfter		Date	Current time + 3 years	Upper boundary of the validity period of the certificate

When the KeyType "EC" is used together with a KeySize value, the NIST-recommended elliptic curve domain parameters will be used according to the following table:

KeySize	Parameter Used	Domain Parameter	keysize
≤ 192	P-192	192	≤ 224
$192 < \text{keysize} \leq 224$	P-224	224	$< \text{keysize} \leq 256$
$224 < \text{keysize} \leq 256$	P-256	256	$< \text{keysize} \leq 384$
$256 < \text{keysize} \leq 384$	P-384	384	$< \text{keysize} \leq 521$
$384 < \text{keysize} \leq 521$	P-521	521	

Certain values (for example, for the alias when indicated in the example request as <hexalias>) need to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number

instead. For example: The string `my alias` is expressed by the following decimal numbers: 109 121 32 97 108 105 97 115 (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number 109, and so forth). If you convert each number to a hexadecimal number, you get: 6D 79 20 61 6C 69 61 73. Note that 6D is the hexadecimal representation of 109, and so forth. If you like to specify `my alias` in an example request, enter 6D7920616C696173.

Request Example

For example, you can use the following request body:

```
{
  "Hexalias": "<any dummy hex value",
  "Alias": "cust_alias",
  "KeyType": "RSA",
  "SignatureAlgorithm": "SHA256/RSA",
  "KeySize": 2048,
  "CommonName": "cpi.test.wdf.sap.com",
  "OrganizationUnit": "My Organization Unit",
  "Organization": "My Organization",
  "Locality": "My Location",
  "State": "My State",
  "Country": "DE",
  "Email": "mymail@myorganization.com",
  "ValidNotBefore": "/Date(1469685029780)/",
  "ValidNotAfter": "/Date(1469775029780)/"
}
```

Get All Keystore Entries

Get all entries of the tenant keystore.

Perform the following call:

Method	Resource Path
GET	/KeystoreEntries

Assume that your tenant keystore contains a keystore entry with alias `smtp.mail.yahoo.com` (server certificate for the Yahoo mail server).

Send the following GET request:

`https://<host address>/api/v1/KeystoreEntries`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

As a response, you get a message that contains for each keystore entry a passage with the following structure (assuming that your tenant keystore contains a keystore entry with alias `smtp.mail.yahoo.com`):

```
<m:properties>
  <d:Hexalias>736D74702E6D61696C2E7961686F6F2E636F6D</d:Hexalias>
  <d:Alias>smtp.mail.yahoo.com</d:Alias>
  <d:KeyType>RSA</d:KeyType>
  <d:KeySize>2048</d:KeySize>
  <d:ValidNotBefore>2019-10-04T00:00:00Z</d:ValidNotBefore>
  <d:ValidNotAfter>2020-04-01T12:00:00Z</d:ValidNotAfter>
  ....
</m:properties>
```

You can design an integration flow that evaluates the validity of each keystore entry and sends out a notification if the validity of certain entries soon expires.

For an example scenario (checking for the validity of certificate-to-user mappings), check out the following SAP Community blog: [Cloud Integration – Automated Notification for Client Certificates Reaching Expiry](#) 📄.

Get Keystore Entry by Alias

Get a single keystore entry by alias.

Perform the following call:

Method	Resource Path
GET	/KeystoreEntries('{Hexalias}')

Assume that your tenant keystore contains a keystore entry with alias `smtp.mail.yahoo.com` (server certificate for the Yahoo mail server) and you like to export it. The hexadecimal value of the alias is: `736d74702e6d61696c2e7961686f6f2e636f6d`

Certain values (for example, for the alias when indicated in the example request as `<hexalias>`) need to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string `my alias` is expressed by the following decimal numbers: `109 121 32 97 108 105 97 115` (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number `109`, and so forth). If you convert each number to a hexadecimal number, you get: `6D 79 20 61 6C 69 61 73`. Note that `6D` is the hexadecimal representation of `109`, and so forth. If you like to specify `my alias` in an example request, enter `6D7920616C696173`.

Therefore, you need to send the following GET request:

```
https://<host address>/api/v1/KeystoreEntries('736d74702e6d61696c2e7961686f6f2e636f6d')
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

As a response, you get a message for the keystore entry that contains the following part:

```
<m:properties>
  <d:Hexalias>736D74702E6D61696C2E7961686F6F2E636F6D</d:Hexalias>
  <d:Alias>smtp.mail.yahoo.com</d:Alias>
  <d:KeyType>RSA</d:KeyType>
  <d:KeySize>2048</d:KeySize>
  <d:ValidNotBefore>2019-10-04T00:00:00Z</d:ValidNotBefore>
  <d:ValidNotAfter>2020-04-01T12:00:00Z</d:ValidNotAfter>
  ....
</m:properties>
```

You can design an integration flow that evaluates the validity of the keystore entry and sends out a notification if the entry soon expires.

For an example scenario (checking for the validity of certificate-to-user mappings), check out the following SAP Community blog: [Cloud Integration – Automated Notification for Client Certificates Reaching Expiry](#)👉.

Rename the Alias of a Keystore Entry

Rename an alias of a tenant keystore entry.

Perform the following call:

Method	Resource Path
PUT	/KeystoreEntries('{Hexalias}')

The request body must contain a property of the `KeystoreEntry` entity type with a null value. Otherwise, you get an exception.

Make sure that you specify the value for the old alias in hexadecimal format. The new alias must be URL-encoded.

Assume that your tenant keystore contains a keystore entry with alias `mykeypair` and you like to rename it to `mykeypair-new`. The hexadecimal value of `mykeypair` is: `6d796b657970616972`

Certain values (for example, for the alias when indicated in the example request as `<hexalias>`) need to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string `my alias` is expressed by the following decimal numbers: `109 121 32 97 108 105 97 115` (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number `109`, and so forth). If you convert each number to a hexadecimal number, you get: `6D 79 20 61 6C 69 61 73`. Note that `6D` is the hexadecimal representation of `109`, and so forth. If you like to specify `my alias` in an example request, enter `6D7920616C696173`.

Therefore, you need to send the following PUT request:

```
https://<host address>/api/v1/KeystoreEntries('6d796b657970616972')?renameAlias=mykeypair-new
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Export Public Key in OpenSSH Format

Export a public key in OpenSSH format.

Perform the following call:

Method	Resource Path
GET	/KeystoreEntries(' {Hexalias}')/Sshkey/\$value

The following keys are supported: rsa-encrypted and dsa-encrypted keys. Other algorithms (for example, elliptic curve (EC)) aren't supported.

Assume you like to get the public key of keystore entry `baltimore cybertrust root` (hexadecimal value: `62616c74696d6f7265206379626572747275737420726f6f74`) in OpenSSH format, send the following GET request:

`https://<host address>/api/v1/KeystoreEntries(' 62616c74696d6f7265206379626572747275737420726f6f74 ')/Sshkey/$value`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Certain values (for example, for the alias when indicated in the example request as `<hexalias>`) need to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string `my alias` is expressed by the following decimal numbers: `109 121 32 97 108 105 97 115` (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number `109`, and so forth). If you convert each number to a hexadecimal number, you get: `6D 79 20 61 6C 69 61 73`. Note that `6D` is the hexadecimal representation of `109`, and so forth. If you like to specify `my alias` in an example request, enter `6D7920616C696173`.

As response, you get:

```
ssh-rsa AAAAB3NzaC1yc2E ..... dKoanTZDcJxo5 public key for alias baltimore cybertrust root
```

Get Keystore Properties and Entries

Get the time when the keystore was modified last (LastModifiedTime) and the keystore entries of the **Current** keystore.

Perform the following call:

Method	Resource Path
GET	/Keystores(' system')? \$expand=Entries&\$select=LastModifiedTime,Entries

Import/Back up a Keystore

Import/back up a keystore.

Perform the following call:

Method	Resource Path
POST	/KeystoreResources

In the request body, provide the following properties:

Properties		
Property	Possible Values	Description
Name	system	Key field name of the keystore Currently, only <code>system</code> is allowed (SAP supports only one keystore).

Property	Possible Values	Description
Resource	<base64-encoded JKS or JCEKS keystore>	Keystore in JKS or JCEKS format
password	password	Keystore password and password of all private keys in the keystore  Caution All private keys must have the same password.

i Note

If the imported keystore contains entries that don't represent an X.509 certificate entry or a key-pair entry with a certificate chain of X.509 certificates, these entries are ignored during the import. The returned keystore entries don't contain an entry for these ignored original entries.

i Note

Only if the custom query option `returnKeystoreEntries` was `true`, the response contains the `KeystoreEntry` instances of the merged keystore, as well as the nonimported and removed entries (HTTP response code 200). A returned `KeystoreEntry` instance contains information about whether the instance was newly created, updated, unchanged, not imported, or removed in the property status. If the entry wasn't imported, information on the reason for this behavior is also provided in the `status` property.

Trigger Mass Deletion of Keystore Entries

Delete several entries owned by the tenant administrator from the keystore.

Perform the following call:

Method	Resource Path
PUT	<code>/KeystoreResources('system')?deleteEntries=true</code>

The aliases of the entries that you want to delete must be specified in the `Aliases` property of the request body.

For example, send the following request:

`https://<host address>/api/v1/KeystoreResources('system')?deleteEntries=true`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

In the request body, provide the aliases of the entries to be deleted.

```
{"Aliases": "alias1;alias2;alias3"}
```

i Note

- If an alias contains the character `'`, it must be escaped.

Example:

`"Company A; Company B" -> "Company A\; Company B";`

- If an alias contains the character `\`, it must be escaped.

Example:

`Alias1="Company\A", Alias2="CompanyB" -> "Aliases": "Company\\A\;CompanyB"`

Restore an Entry from the SAP Key History Keystore to the New SAP Keys Keystore

Restore an entry from the [SAP Key History](#) keystore to the [New SAP Keys](#) keystore.

Perform the following call:

Method	Resource Path
PUT	/HistoryKeystoreEntries('{Hexalias}')

You can specify an optional destination alias (URL-encoded).

If no destination alias is specified, the original alias contained in the metadata of the `KeyHistory` entry is used as destination alias.

Make sure to specify the alias as hexadecimal number.

Certain values (for example, for the alias when indicated in the example request as `<hexalias>`) need to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string `my alias` is expressed by the following decimal numbers: 109 121 32 97 108 105 97 115 (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number 109, and so forth). If you convert each number to a hexadecimal number, you get: 6D 79 20 61 6C 69 61 73. Note that 6D is the hexadecimal representation of 109, and so forth. If you like to specify `my alias` in an example request, enter 6D7920616C696173.

Send the following PUT request:

`https://<host address>/api/v1/HistoryKeystoreEntries('{Hexalias}')?copy=true&destinationAlias={URL-encoded destination alias}`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Create a Certificate-to-User Mapping, Neo Environment

Create a certificate-to-user mapping.

i Note

This information is relevant only when you use SAP Cloud Integration in the Neo environment.

Perform the following call:

Method	Resource Path
POST	/CertificateUserMappings

i Note

Make sure that the PEM-DER string of the certificate is base64-encoded for the `Certificate` field.

Even if the ID specified in the POST request isn't correctly calculated, the POST response returns the correct ID.

The user supplied in the query part of the URI is only used for audit logging. It doesn't appear in the `CreatedBy` or `LastModifiedBy` property of the entity. The logged-in 'technical user' is used for the `CreatedBy` and `LastModifiedBy` properties. This behavior is different to the other entity types `StringParameter`, `BinaryParameter`, `AuthorizedUser`, and `AlternativePartnerId`.

You can execute the POST request with the same certificate several times. Unlike the other entity types, no duplicate key exception is raised. You can use the POST request to update the user.

You like to create a certificate-to-user mapping with a user `myuser` and a base64 encoded certificate that is represented by a string like the following one `LS0tLS1CRUdJTiBDRVJUV2T0Y2 . . . . AotLS0tLUV0RCBDRVJUSUZJQ0FURS0tLS0t` (shortened example string).

Send the following POST request:

`https://<host address>/api/v1/CertificateUserMappings`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Add the following request body (example content):

```
{
  "User": "myuser",
  "Certificate": "LS0tLS1CRUdJTiBDRVJUV2T0Y2 . . . . AotLS0tLUV0RCBDRVJUSUZJQ0FURS0tLS0t"
}
```

If the request was successful, you get a response that contains the following part:

```
<m:properties>
  <d:Id>abcd123...xyz</d:Id>
  <d:User>myuser</d:User>
  <d:Certificate>LS0tLS1CRUdJTiBDRVJUV2T0Y2...AotLS0tLUVORCBDRVJUSUZJQ0FURS0tLS0t</d:Certificate>
  <d:LastModifiedBy>your_user</d:LastModifiedBy>
  <d:LastModifiedTime>2021-02-23T08:57:25.021</d:LastModifiedTime>
  <d:CreatedBy>your_user</d:CreatedBy>
  <d:CreatedTime>2021-02-23T08:57:25.021</d:CreatedTime>
  <d:ValidUntil m:null="true"/>
</m:properties>
```

Get Certificate-to-User Mappings with Specific User, Neo Environment

Get certificate-to-user mappings with a specific user.

i Note

This information is relevant only when you use SAP Cloud Integration in the Neo environment.

Perform the following call:

Method	Resource Path
GET	/CertificateUserMappings

Request Example

You like to get all certificate-to-user mappings with user `myUser`.

Send the following GET request:

```
https://<host address>/api/v1/CertificateUserMappings?$filter=User eq 'myUser'
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

If the request was successful, you get a response that contains one or multiple parts with the following attributes (example):

```
<m:properties>
  <d:Id>abcd123...xyz</d:Id>
  <d:User>myuser</d:User>
  <d:Certificate>LS0tLS1CRUdJTiBDRVJUV2T0Y2...AotLS0tLUVORCBDRVJUSUZJQ0FURS0tLS0t</d:Certificate>
  <d:LastModifiedBy>your_user</d:LastModifiedBy>
  <d:LastModifiedTime>2021-02-23T08:57:25.021</d:LastModifiedTime>
  <d:CreatedBy>your_user</d:CreatedBy>
  <d:CreatedTime>2021-02-23T08:57:25.021</d:CreatedTime>
  <d:ValidUntil>1747094340000</d:ValidUntil>
</m:properties>
```

Read Expiration Date of a Client Certificate, Neo Environment

Read the expiration date of a client certificate associated with a certificate-to-user mapping.

i Note

This information is relevant only when you use SAP Cloud Integration in the Neo environment.

Perform the following call:

Method	Resource Path
GET	/CertificateUserMappings?\$select=Id,User,ValidUntil

The following example request reads the expiration date (`ValidUntil`) of the client certificate associated with all deployed certificate-to-user mappings (together with `Id` and target user of the certificate-to-user mapping).

```
https://<host address>/api/v1/CertificateUserMappings?$select=Id,User,ValidUntil
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

If the request was successful, you get a response that contains one or multiple parts with the following attributes (example):

```
<m:properties>
<d:Id>abcd123...xyz</d:Id>
<d:User>myuser</d:User>
<d:ValidUntil>1747094340000</d:ValidUntil>
</m:properties>
```

Using such a request, you can design an integration flow that evaluates the expiration dates of all client certificates (associated with certificate-to-user mappings) and sends out a notification if the entry soon expires.

For an example scenario, check out the following SAP Community blog: [Cloud Integration – Automated Notification for Client Certificates Reaching Expiry](#) .

B2B Scenarios

Use the **B2B Scenarios OData API** to access and manage data related to inter-enterprise business integration scenarios.

With this API, you can securely query business documents and this way support efficient data exchange and business process collaboration between enterprises.

Find the OData API and the basic operations at [B2B Scenarios](#)  on SAP Business Accelerator Hub. Note that the **Try out** feature is currently unavailable.

Permissions

To authorize an API client to access the OData API, perform the steps described in [Setting Up Inbound HTTP Connections \(for API Clients\)](#) in the Cloud Foundry environment.

Then, assign the following role templates:

- MonitoringDataRead
- B2bInterchangeOperate
- B2bInterchangePayloadRead
- B2bInterchangeRead

See also [Tasks and Permissions](#).

Resources

The following resources are included in the API:

Resource	Description
Orphaned Interchanges	Retrieve a list of all orphaned interchanges available in the system.
Business Documents	Retrieve a list of all inter-enterprise business documents available in the system.
Business Document Additional Field	Retrieve business document notes.
Sender Technical Acknowledgment	Retrieve sender technical acknowledgment.
Receiver Technical Acknowledgment	Retrieve receiver technical acknowledgment.
Sender Functional Acknowledgment	Retrieve sender functional acknowledgment.
Receiver Functional Acknowledgment	Retrieve receiver functional acknowledgment.
Custom Objects	Retrieve custom objects.
Error Details	Retrieve error details.
Business Document Processing Event	Retrieve processing events.
Business Document Payloads	Retrieve documents payloads.
Business Document Protocol Headers	Retrieve protocol headers.
Business Document Notes	Retrieve business document notes.

Resource	Description
Technical Acknowledgment	Retrieve technical acknowledgments.
Functional Acknowledgment	Retrieve functional acknowledgments.
Data Archiving	Allows you to activate data archiving. See Archiving Data .

For general information about query options, see [Query Options](#).

Example Requests

To trigger the modifying actions (POST, PUT, and DELETE), you need to fetch a CSRF token first, as described in the following:

1. Send a GET call to the API with a header X-CSRF-Token with the value Fetch.
2. As a response, you get the value of the token in the header X-CSRF-Token.
3. When sending the modifying request, add the header X-CSRF-Token with the token retrieved from the first call.

On SAP Business Accelerator Hub, you can test API calls against a sandbox tenant or against a custom tenant, which you configure under [API Environment](#). If you want to perform an API call against your custom tenant using an HTTP client such as Postman, make sure that the request URL is composed according to the following pattern:

https://<host address>/api/v1/<relative resource path>

i Note

https://<host address>/api/v1 is the service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

You can find the relative resource path for each operation on SAP Business Accelerator Hub. For example requests, see [B2B Scenarios](#) .

Get Orphaned Interchanges for a Dedicated Time Period

In the following example, you want to get the orphaned interchanges within a dedicated time period. In this case, assume that the time period is 2025-10-07T11:23:17.152 to 2025-10-08T11:23:17.152.

1. Perform a GET call with the following resource path:

OrphanedInterchanges/\$count?\$filter=(Date ge datetime'2025-10-07T11:23:17.152' and Date le datetime'2025-10-08T11:23:17.152')
2. Then, send the following GET request:

https://<host address>/api/v1/ OrphanedInterchanges/\$count?\$filter=(Date ge datetime'2025-10-07T11:23:17.152' and Date le datetime'2025-10-08T11:23:17.152')

Get Document Payloads

In the following example, you want to retrieve business document payloads for a specific business document.

1. Perform a GET call with the following resource path:

/BusinessDocuments(' {Id} ')/BusinessDocumentPayloads

With this API, you can retrieve all business document payloads associated with a specific business document. It's typically used to view all raw data content (such as attachments or messages) related to a business document.
2. Assume that the document ID is 9b12fbad477049ff907ac7f5a8411528. To get the custom header property, send the following GET request:

https://<host address>/api/v1/BusinessDocuments(' 9b12fbad477049ff907ac7f5a8411528 ')/BusinessDocumentPayloads

Authentication

The API is protected by basic authentication and OAuth.

Related Information

[Setting Up Inbound HTTP Connections \(for API Clients\)](#)

HTTP Calls and URI Components

An HTTP call is composed of a **service root URI**, **resource path**, and **query options**.

To find the **resource path** and the supported **query options**, go to SAP Business Accelerator Hub and select the API. For example, to find the resource paths, the supported operations and query options for message processing logs, check out the [Message Processing Logs](#) page on SAP Business Accelerator Hub.

How to get the service root URI, depends on the environment:

- SAP BTP, Cloud Foundry environment:

Open SAP BTP cockpit, go to the subaccount and choose **Instances and Subscriptions**. View the details of the service key that you have created when setting up inbound authentication for API access, and find the `url` parameter (see [Setting Up Inbound HTTP Connections \(for API Clients\)](#)). Throughout this documentation, we refer to this part of the address also as **host address**.

i Note

If you have set up inbound API access with basic authentication of an IdP user (according to [Basic Authentication of an IdP User for API Clients](#)), no service key is in place. In this case, create a service key (with plan api as described at [Creating Service Instance and Service Key for Inbound Authentication](#)), get the `url` parameter and delete the service key after you have retrieved the service root URI.

The service root URI then is derived by appending `/api/v1`.

The service root URI is then: `<url parameter from service key>/api/v1`.

For example, to get all message processing logs with status **Completed**, use the following call:

```
https://12345abc.it-xyz789.cfapps.us10-001.hana.ondemand.com/api/v1/MessageProcessingLogs?$filter=Status eq 'COMPLETED'
```

In the example, these are the URI components:

- Service root URI:

```
https://12345abc.it-xyz789.cfapps.us10-001.hana.ondemand.com/api/v1 (where https://12345abc.it-xyz789.cfapps.us10-001.hana.ondemand.com is the url parameter of the service key)
```

- Resource path:

```
MessageProcessingLogs
```

- Query option:

```
$filter=Status eq 'COMPLETED'
```

- Neo environment:

Check out the address of your Cloud Integration user interface (Web UI). It ends with `/itspaces/shell/integration`.

Remove the `/itspaces/shell/integration` and replace it by `/api/v1`.

The service root URI is then: `<Cloud Integration user interface address without /itspaces/shell/integration>/api/v1`.

For more information about the service root URI, resource path, and query options, see [URI Conventions \(OData Version 2.0\)](#).

⚠ Caution

The browser translates space characters to `%20`. If you paste a URL from this documentation to the browser, the OData call therefore works. The string `%20` is left out of the example URLs to make them easier to read.

Query Options

Query options allow you to control the amount and order of the data that a data service returns for the resource identified by the URI.

OData System Query Options Supported by the Cloud Integration API

OData system query options are provided by the OData framework and documented in detail in the OData specification at <http://www.odata.org/documentation/odata-version-2-0/uri-conventions>.

This section provides a summary of the supported query options.

 Caution

Not all query options listed below are always supported by each entity type of the Cloud Integration OData API.

For example, The DataStore and Variables APIs do not support the following query options:

- - `$filter`
 - `$inlinecount`
 - `$orderby`
 - `$skip`
 - `$top`
 - `$expand`
 - `$select`

For more information on the supported query options, check out the description of the individual resources on SAP Business Accelerator Hub.

Supported System Query Options

System Query Option	... Does the Following
<code>\$count</code>	Returns the number of records according to the resource path section of the URI (or, if a filter is applied, the number of records matching the filter).
<code>\$expand</code>	Returns related entities for a given navigation property in line with the entities being retrieved. For example, adding the query <code>\$expand=AdapterAttributes</code> to an OData request addressing the <code>MessagingProcessingLog</code> entity returns adapter-specific attributes in line with the message processing log data.
<code>\$filter</code>	Returns a subset of records according to the resource path section of the URI and to the filter expression.
<code>\$inlinecount</code>	Specifies that the response contains a count of the number of records in the collection of entries identified by the resource path section of the URI.
<code>\$metadata</code>	Returns the data model (which is the structure of all resources).
<code>\$orderby</code>	Specifies an expression for determining which values are used to order the collection of records identified by the resource path section of the URI.  Caution Limitations for the usage of <code>\$orderby</code> Regarding the <code>\$orderby</code> option with the <code>MessageProcessingLogCustomHeaderProperty</code> entity, note the following: <code>\$orderby</code> supports only the combinations: <code>\$orderby=Name, Value</code> as well as <code>\$orderby=Name desc, Value desc</code> . In case you need additional options, we recommend you to implement these at the client side.
<code>\$select</code>	Returns a limited set of information on the entities identified by the resource path section of the URI. i Note Using the <code>\$select</code> option can have a positive impact on the performance of a request.
<code>\$skip</code>	Identifies a subset of records (according to the resource path section of the URI), where the subset is defined by seeking n entries and selecting only the remaining entries (starting with entry n+1).
<code>\$top</code>	Identifies a subset of records (according to the resource path section of the URI), where the subset is defined by selecting only the first n items of the set.
<code>\$value</code>	When applied with the entity <code>RuntimeArtifactErrorInformation</code> , this query returns the error message in JSON format.

Filter System Query Option (`$filter`)

Examples for the `$filter` query option:

 Caution

The browser translates space characters to %20. If you paste a URL from this documentation to the browser, the OData call therefore works. The string %20 is left out of the example URLs to make them easier to read.

- `https://<tmn>/api/v1/MessageProcessingLogs?$filter=MessageGuid eq 'AFQaBF-k2zSgMgIYekzBzk07Bs-T'`

Addresses the message processing log with the MessageGuid AFQaBF-k2zSgMgIYekzBzk07Bs-T. This is equivalent to the query `https://<tmn>/api/v1/MessageProcessingLogs('AFQaBF-k2zSgMgIYekzBzk07Bs-T')`.

- `https://<tmn>/api/v1/MessageProcessingLogs?$filter=ApplicationMessageId eq 'applicationID'`

Addresses all message processing logs with the ApplicationMessageId applicationID.

- `https://<tmn>/api/v1/MessageProcessingLogs?$filter=ApplicationMessageType eq 'application-message-type'`

Addresses all message processing logs with the ApplicationMessageType application-message-type

- `https://<tmn>/api/v1/MessageProcessingLogs?$filter=LogEnd gt datetime'2014-09-01T00:00:00' and LogEnd lt datetime'2014-09-02T00:00:00'`

Addresses all message processing logs with log end time between the two datetime literals 2014-09-01T00:00:00Z and 2014-09-02T00:00:00.

You can also use `https://<tmn>/api/v1/MessageProcessingLogs?$filter=LogEnd gt datetime'2014-09-01T00:00:00' and LogStart lt datetime'2014-09-02T00:00:00'` to get all Message Processing Logs inside the chosen time interval.

Currently, parsing of time zone information in datetime literals is not supported. All datetime literals are interpreted as UTC (Universal Time Coordinated). Therefore, a zone offset cannot be interpreted. For example, a call with the following syntax will not be interpreted correctly:

`https://<tmn>/api/v1/MessageProcessingLogs?$filter=LogEnd gt datetime'2014-09-01T00:00:00Z' and LogEnd lt datetime'2014-09-02T00:00:00Z'`.

- `https://<tmn>/api/v1/MessageProcessingLogs?$filter=Sender eq 'sender'`

Returns all message processing logs, where the associated messages were sent by sender.

- `https://<tmn>/api/v1/MessageProcessingLogs?$filter=Receiver eq 'receiver'`

Returns all message processing logs, where the associated messages were addressed to receiver.

- `https://<tmn>/api/v1/MessageProcessingLogs?$filter=CustomStatus eq 'ERROR'`

Returns all message processing logs, where the associated messages are in CustomStatus, here ERROR.

- `https://<tmn>/api/v1/MessageProcessingLogs?$filter=Status eq 'ERROR'`

Returns all message processing logs, where the associated messages are in ERROR status.

- `https://<tmn>/api/v1/MessageProcessingLogs?$filter=CorrelationId eq 'AbCd-jshkdgsz23jds-T'`

Returns all messages that are correlated by the CorrelationId AbCd-jshkdgsz23jds-T.

→ Tip

You can combine other filters into one OData request, but it is recommended to use filters containing MessageGuid, CorrelationID and ApplicationMessageID separately from other filters.

Server-Side Paging (\$skiptoken System Query Option)

The Cloud Integration API provides 1000 entries for a call maximum. This feature also protects the Cloud Integration runtime environment if unfiltered queries enforce the return of huge amount of data.

If a query returns more than 1000 entries, the response will contain a *Next* link, which can be used to initiate the return of the additional entries.

Example:

```
<feed ...>
  <id><tmn>/api/v1/MessageProcessingLogs</id>
  <title type="text">MessageProcessingLogs</title>
  ...
  <link href="https://<tmn>/api/v1/MessageProcessingLogs?$skiptoken=1000" rel="next" />
</feed>
```

Filtering Message Processing Logs for Custom Header Properties

You can set custom header properties to filter message processing logs. The **Monitor** application shows these custom header properties in the filtering section of the message processing log screen.

When requesting message processing logs (MPLs) using the OData API, you can use additional query options to filter MPLs along custom header properties as well as to get information on the available custom header properties.

You can combine filter for custom properties with other query parameters such like `filter`, `format`, `orderby`, and `top`, for example.

Let's assume that you have MPLs with the custom header property `po_number` which can have different values (12345, for example).

If you like to return MPLs with given custom header properties and values (for the example above), perform one of the following API calls:

```
/MessageProcessingLogs?filterCustomHeaderProperties='po_number' eq '12345'
```

```
/MessageProcessingLogs?$format=json&...&$filter=<different-filters>&filterCustomHeaderProperties='po_number' eq '12345'
```

The response contains the MPL details.

Let's assume that you don't know which custom properties and values are available and you like to get a list of all custom header properties used in different MPLs, perform the following API call:

```
/MessageProcessingLogCustomHeaderProperties
```

The response contains one or many of the following sections with the custom header properties and their values (for example):

```
<content type="application/xml">
  <m:properties>
    <d:Id>1234</d:Id>
    <d:Name>po_number</d:Name>
    <d:Value>12345</d:Value>
  </m:properties>
</content>
```

You can use following request to get information about MPLs using a specific custom header property:

```
/MessageProcessingLogCustomHeaderProperties('1234')/Log
```

To return information about a specific custom header property, perform the following call:

```
/MessageProcessingLogCustomHeaderProperties?$filter=Name eq 'custom_property_name' and Value eq 'custom_Property_value'
```

For example, assume that there are MPLs with the custom header property `po_number` and you perform the following API call:

```
/MessageProcessingLogCustomHeaderProperties?$filter=Name eq 'po_number' and Value eq '12345'
```

The response contains the following section with the custom header property, its value, and an Id:

```
<content type="application/xml">
  <m:properties>
    <d:Id>1234</d:Id>
    <d:Name>po_number</d:Name>
    <d:Value>12345</d:Value>
  </m:properties>
</content>
```

Now that you know the custom header property Id, you can use following request to get information about MPLs associated with the custom header property with this Id:

```
/MessageProcessingLogCustomHeaderProperties('1234')/Log
```

The response contains the MPL details.

→ Tip

For more information on how to set custom header properties, see [Use Custom Header Properties to Search for Message Processing Logs](#).

Support Components

If you find an issue with the OData API, you can raise a ticket.

The following table lists the relevant support components for the different resources of the API.

API	Resource	Support Component
Integration Content	CopyIntegrationPackage	LOD-HCI-PI-WT-IFL
Integration Content	IntegrationPackages	LOD-HCI-PI-WT-IFL
Integration Content	IntegrationRuntimeArtifacts	LOD-HCI-PI-OP-NM
Integration Content	IntegrationDesigntimeArtifacts	LOD-HCI-PI-WT-IFL
Integration Content	IntegrationDesigntimeArtifactsSaveAsVersion	LOD-HCI-PI-WT-IFL
Integration Content	DeployIntegrationDesigntimeArtifacts	LOD-HCI-PI-WT-IFL
Integration Content	ServiceEndpoints	LOD-HCI-PI-WT-IFL
Integration Content	ValueMappingDesigntimeArtifacts	LOD-HCI-PI-MAP
Integration Content	DeployValueMappingDesigntimeArtifacts	LOD-HCI-PI-MAP
Integration Content	ValueMappingDesigntimeArtifactSaveAsVersion	LOD-HCI-PI-MAP
Integration Content	UpsertValMaps	LOD-HCI-PI-MAP
Integration Content	UpdateDefaultValMap	LOD-HCI-PI-MAP
Integration Content	DeleteValMaps	LOD-HCI-PI-MAP
Integration Content	IntegrationAdapterDesigntimeArtifacts	LOD-HCI-PI-WT-IFL
Integration Content	DeployIntegrationAdapterDesigntimeArtifacts	LOD-HCI-PI-WT-IFL
Integration Content	CustomTagConfigurations	LOD-HCI-PI-WT-IFL
Integration Content	MessageMappingDesigntimeArtifacts	LOD-HCI-PI-MAP
Integration Content	DeployMessageMappingDesigntimeArtifact	LOD-HCI-PI-MAP
Integration Content	MessageMappingDesigntimeArtifactSaveAsVersion	LOD-HCI-PI-MAP
Integration Content	BuildAndDeployStatus	LOD-HCI-PI-WT-IFL
Integration Content	ScriptCollectionDesigntimeArtifacts	LOD-HCI-PI-WT-IFL
Integration Content	DeployScriptCollectionDesigntimeArtifact	LOD-HCI-PI-WT-IFL
Log Files	LogFiles	LOD-HCP-PI-OP-SRV
Log Files	LogFileArchives	LOD-HCP-PI-OP-SRV
Message Processing Logs	MessageProcessingLogs	LOD-HCP-PI-OP-SRV
Message Processing Logs	IdMapTolds	LOD-HCP-PI-OP-SRV
Message Processing Logs	GenericIdempotentRepositoryEntries	LOD-HCP-PI-OP-SRV
Message Processing Logs	activateArchivingConfiguration	LOD-HCP-PI-OP-SRV
Message Processing Logs	ArchivingConfigurations	LOD-HCP-PI-OP-SRV
Message Processing Logs	ArchivingKeyPerformanceIndicators	LOD-HCP-PI-OP-SRV
Messsage Stores	MessageProcessingLogs	LOD-HCP-PI-OP-SRV
Messsage Stores	MessageStoreEntries	LOD-HCI-PI-CON-SOAP
Messsage Stores	MessageStoreEntryAttachments	LOD-HCI-PI-CON-SOAP
Messsage Stores	MessageStoreEntryAttachmentProperties	LOD-HCI-PI-CON-SOAP
Messsage Stores	JMSBrokers	LOD-HCI-PI-CON-SOAP
Messsage Stores	NumberRanges	LOD-HCI-PI-CON-SOAP
Messsage Stores	DataStores	LOD-HCI-PI-CON-SOAP
Messsage Stores	DataStoreEntries	LOD-HCI-PI-CON-SOAP

API	Resource	Support Component
Message Stores	Variables	LOD-HCI-PI-CON-SOAP
Partner Directory	AlternativePartners	LOD-HCI-PI-CON-SOAP
Partner Directory	AuthorizedUsers	LOD-HCI-PI-CON-SOAP
Partner Directory	BinaryParameters	LOD-HCI-PI-CON-SOAP
Partner Directory	Partners	LOD-HCI-PI-CON-SOAP
Partner Directory	StringParameters	LOD-HCI-PI-CON-SOAP
Partner Directory	UserCredentialParameters	LOD-HCI-PI-CON-SOAP
Security Content	Certificate Resources	LOD-HCI-PI-CON-SOAP
Security Content	KeystoreEntries	LOD-HCI-PI-CON-SOAP
Security Content	KeyPairGenerationRequests	LOD-HCI-PI-CON-SOAP
Security Content	SSHKeyGenerationRequests	LOD-HCI-PI-CON-SOAP
Security Content	KeyPairResources	LOD-HCI-PI-CON-SOAP
Security Content	CertificateChainResources	LOD-HCI-PI-CON-SOAP
Security Content	Keystores	LOD-HCI-PI-CON-SOAP
Security Content	KeystoreResources	LOD-HCI-PI-CON-SOAP
Security Content	HistoryKeystoreEntries	LOD-HCI-PI-CON-SOAP
Security Content	UserCredentials	LOD-HCI-PI-CON-SOAP
Security Content	OAuth2ClientCredentials	LOD-HCI-PI-MAP
Security Content	CertificateUserMappings	LOD-HCI-PI-CON-SOAP

Addressing the CISUsers Entity (for User Creation)

If you want to connect your Cloud Integration account to a separate Cloud Identity service, you can use the OData API to create users that a client of the Cloud Integration tenant can use to send HTTP messages via basic authentication to the runtime nodes.

The OData API allows you to create, update, read, and delete users within the Cloud Identity service that is connected to your account.

Create User

https://<tmn>/api/v1/CISUsers

HTTP Method: POST

HTTP Headers:

Name	Value	Description
Content-Type	application/json;charset=utf-8	Content type must be JSON and the charset UTF-8.
Accept	application/json	If not set, the body of the HTTP response will contain XML data. If set, the response will be JSON data.

HTTP request body (example):

```
{
  "Email": "xxx@sap.com"
}
```

HTTP response, when Accept "application/json" is set and creation was successful (HTTP response code 201):

```
{
  "d": {
    "__metadata": {
      "id": "https://<account>-tmn.hci.hana.ondemand.com:443/api/v1/CISUsers('P000051')",

```

```
    "uri": "https://<account>-tmn.hci.hana.ondemand.com:443/api/v1/CISUsers('P000051')",
    "type": "com.sap.hci.api.CISUser"
  },
  "Email": "xxx@sap.com",
  "CisId": "P000051"
}
```

You have to specify the `CisId` property as well as the `Email` property because `CisId` is a key field.

Properties in the Request Body

Name	Mandatory	Description
Email	Yes	E-mail address can only be used once to create a user.
CisId	No	CisId will contain the internal user ID after a user has been created. CisId will be ignored within an HTTP POST request body.
LoginName	No	Can be used to set a login name for the user.
Type	No	Not used yet
Password	No	You can use this property to set an initial password. When a password is set, the account is activated, but an e-mail is still sent to the specified e-mail address.
FamilyName	No	You can use this property to set the family name of the user.
GivenName	No	You can use this property to set the given name of the user.

Response Status and Error Codes

Response Code	Response Description
201 OK	User successfully created.
400 Bad Request	Wrong properties used.
401 Unauthorized	User of the call to the OData API does not exist in the Cloud Identity service or the password is wrong.
403 Forbidden	User not assigned the appropriate role. Assign either the <code>AuthGroup.TenantPartnerDirectoryConfigurator</code> or <code>AuthGroup.Administrator</code> authorization group or the <code>CloudIdentityService.Write</code> role. Compare the user name in the role assignment (case-sensitive).
500 Internal Server Error	Creation of user failed. For more details, check the Java trace files. A possible reason for the error is that the e-mail address is already used.

Read User

```
https://<tmn>/api/v1/CISUsers('<CisId>')
```

Example:

```
https://<tmn>/api/v1/CISUsers('P000095')
```

HTTP Method: GET

HTTP Headers:

Name	Value	Description
Accept	application/json	If not set, the body of the HTTP response will contain XML data. If set, the response will be JSON.

HTTP response, when Accept "application/json" is set and creation was successful (HTTP response code 200):

```
{
  "d": {
    "__metadata": {
```

```
{
  "id": "https://<account>-tmn.hci.hana.ondemand.com:443/api/v1/CISUsers('P000095')",
  "uri": "https://<account>-tmn.hci.hana.ondemand.com:443/api/v1/CISUsers('P000095')",
  "type": "com.sap.hci.api.CISUser"
},
"Email": "test@test.com",
"CisId": "P000095",
"LoginName": "franz",
"Type": null,
"Password": null,
"FamilyName": "Smith",
"GivenName": "John"
}}
```

The returned value for the Password property will always be null even if a password is set.

You can also specify the properties that are returned in the query part of the HTTP request URI, for example:

```
https://<tmn>/api/v1/CISUsers('P000095')?$select=FamilyName,GivenName
```

This request only returns the FamilyName and GivenName property:

```
{
  "d": {
    "__metadata": {
      "id": "https://<account>-tmn.hci.hana.ondemand.com:443/api/v1/CISUsers('P000095')",
      "uri": "https://<account>-tmn.hci.hana.ondemand.com:443/api/v1/CISUsers('P000095')",
      "type": "com.sap.hci.api.CISUser"
    },
    "FamilyName": "Smith",
    "GivenName": "John"
  }
}
```

Response Status and Error Codes

Response Code	Response Description
200 OK	CISUser was found and successfully returned.
400 Bad Request	Wrong properties used.
401 Unauthorized	User of the call to the OData API does not exist in the Cloud Identity service or the password is wrong.
403 Forbidden	User not assigned the appropriate role. Assign either the AuthGroup.TenantPartnerDirectoryConfigurator, AuthGroup.Administrator, AuthGroup.IntegrationDeveloper, AuthGroup.SystemDeveloper, or AuthGroup.ReadOnly, authorization group or the CloudIdentityService.Read role.
404 Not Found	No CISUser was found for the specified CisId.
500 Internal Server Error	Read failed. For more details, check the Java trace files.

Change User

```
https://<tmn>/api/v1/CISUsers('<CisId>') )
```

Example:

```
https://<tmn>/api/v1/CISUsers('P000095')
```

HTTP Method: PATCH

We recommend using the HTTP method PATCH:

In OData 2.0 specification, the HTTP method PATCH has merge semantics, whereas the HTTP method PUT is used for an update of all properties of an OData entity (property values are replaced either by taking the values indicated in the request body, or reset to their default values if not mentioned in the request). The HTTP PUT method is not supported.

Some HTTP clients do not support the PATCH method. In this case, you can try to use theX-HTTP-Method-Override request propertyy

HTTP Headers:

Name	Value	Description
Content-Type	application/json;charset=utf-8	Content type must be JSON and the charset UTF-8.

Name	Value	Description
Accept	application/json	If not set, the body of the HTTP response will contain XML data. If set, the response will be JSON.

HTTP request body (example):

```
{"Email":"test3@test.com", "LoginName":"SmithJ", "FamilyName":"Smith", "GivenName":"John", "Password":"password1234"}
```

The properties provided in the request are merged with the existing properties. Therefore, you can only specify a subset of the properties. For example, if you only want to change the password, then you only specify the Password property.

HTTP request body (example of changing the password only):

```
{"Password":"Pass1234"}
```

Properties whose values are null in the request body are ignored during the merge.

Example:

```
{"LoginName":null}
```

If you want to initialize a property, use the empty value, for example:

```
{"LoginName":""}
```

Empty values are not possible for the properties Email and Password.

Properties in the request body:

Properties in the Request Body

Name	Mandatory	Description
Email	No	E-mail address must be unique.
LoginName	No	Login name of the user
Type	No	Not used yet
Password	No	
FamilyName	No	
GivenName	No	You can use this property to set the given name of the user.

Response Status and Error Codes

Response Code	Response Description
204 No Content	User was merged using the provided properties.
400 Bad Request	Wrong properties used.
401 Unauthorized	User of the call to the OData API does not exist in the Cloud Identity service or the password is wrong.
403 Forbidden	User not assigned the appropriate role. Assign either the <code>AuthGroup.TenantPartnerDirectoryConfigurator</code> or <code>AuthGroup.Administrator</code> authorization group or the <code>CloudIdentityService.Write</code> role.
500 Internal Server Error	Changing the user properties failed. For more details, check the Java trace files. Possible reasons for the error are: The user with the specified Cisd no longer exists. In this case, the response body contains a message such as <code>Update of the user with ID P000095 failed. HTTP response code of Identity Service is: 400 - User with [id] not found.</code> The provided password is insufficient, because it does not fulfill the requirements of a valid password.

Response Code	Response Description
	The provided Email value is already used with another CISUser.

Delete User

https://<tmn>/api/v1/CISUsers('<CisId>'))

Example:

https://<tmn>/api/v1/CISUsers('P000095')

HTTP Method: DELETE

HTTP response code for successful deletion: 200:

Response Status and Error Codes	
Response Code	Response Description
200 OK	CISUser was successfully deleted.
400 Bad Request	Wrong properties used.
401 Unauthorized	User of the call to the OData API does not exist in the Cloud Identity service or the password is wrong.
403 Forbidden	User not assigned the appropriate role. Assign either the AuthGroup.TenantPartnerDirectoryConfigurator or AuthGroup.Administrator authorization group or the CloudIdentityService.Write role.
500 Internal Server Error	User deletion failed. If the returned body contains a message such as No CISUser found for CisId P000095, deletion was not possible because a user with the specified CisId does not exist. For more details, check the Java trace files.

Related Information

Local OData API Access for Edge Integration Cell

For Edge Integration Cell users, it is possible to configure local OData API access to call monitoring APIs directly from Edge Integration Cell.

You can configure local OData API access to call monitoring APIs directly from your Edge Integration Cell runtime, without requiring intervention from the Cloud Integration UI.

The available APIs cover the following:

- [Message Processing Logs](#)
- [Message Stores](#)
- [Partner Directory](#)
-

These APIs are published as a package on the SAP Business Accelerator Hub at [Overview | SAP Edge Integration Cell | SAP Business Accelerator Hub](#).

For a more detailed version of this topic, see [Local OData API Access](#).